

Instituto Politécnico Nacional
"La Técnica al Servicio de la Patria"

INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE INGENIERÍA MECÁNICA Y ELÉCTRICA UNIDAD CULHUACAN

SECCIÓN DE ESTUDIOS DE POSGRADO E INVESTIGACIÓN

SOMN_IA: Sistema Inteligente para Detectar la Somnolencia y Distracción en Conductores

T E S I S

QUE PARA OBTENER EL TÍTULO DE:

Maestro en Ciencias en Ingeniería en Microelectrónica

P R E S E N T A :

Flores Monroy Jonathan Mauricio



DIRECTORES DE TESIS:

DRA. MARIKO NAKANO MIYATAKE
DR. HECTOR MANUEL PEREZ MEANA

Contenido

Contenido 1
Índice de Figuras 4
Índice de Tablas 8
Agradecimientos 10
Resumen 11
Abstract 13
Problemática 15
Referencias 16
Justificación 17
Objetivos 18
1. Objetivo General 18
2. Objetivos Específicos 18
Contribución 19
Referencias 21
Alcance 23
Organización De La Tesis 24
Capítulo 1: Marco Teórico 25
1.1 Accidentes Automotrices 26
1.2 Fatiga, Somnolencia y Distracción 33
1.2.1 Fatiga 33
1.2.2 Somnolencia 34
1.2.3 Distracción 35
1.3 Conclusiones del capítulo 36
1.4 Referencia del capítulo 37
Capítulo 2: ESTADO DEL ARTE 39
2.1 Introducción 40
2.2 Detección de somnolencia y distracción en los conductores 40
2.3 Investigaciones previas 41
2.4 Patentes/prototipos previos 43
2.5 Conclusiones del capítulo 46
2.6 Referencias del capítulo 47
Capítulo 3: Método propuesto y resultados 49
3.1 Introducción 50
3.1.1 Algoritmo general (Detección de somnolencia y distracción en los conductores) 50
3.2 Construcción del algoritmo general 51
3.2.1 Obtención de video en tiempo real 51
3.2.2 Separación del video de entrada a fotogramas 53
3.2.3 Detección de la región a analizar 56
3.2.4 Implementación de la red neuronal convolucional propuesta (primera versión) 63
3.2.5 Análisis de resultados consecutivos 79
3.2.6 Activación de alarmas pertinentes 86

3.2.7 Resultados obtenidos del subcapítulo 92
3.3 Mejoramiento del Sistema General 97
3.3.1 Mejoramiento del reconocimiento facial 97
3.3.2 Mejoramiento de la RNC-PP propuesta 104
3.3.3 Análisis de resultados consecutivos 115
3.3.4 Activación de alarmas pertinentes 117
3.3.5 Resultados obtenidos del subcapítulo 121
3.4 Red Neuronal Convolucional Propuesta vs Redes neuronales Convolucionales Populares 126
3.4.1 Configuración de transferencia de aprendizaje en la arquitectura MobileNet-V2 129
3.4.2 Configuración de transferencia de aprendizaje en la arquitectura VGG16 136
3.4.3 Configuración de transferencia de aprendizaje en la arquitectura ResNet-50V2 138
3.4.4 Configuración de transferencia de aprendizaje en la arquitectura InceptionV3 141
3.4.5 Configuración de transferencia de aprendizaje en la arquitectura Xception 144
3.4.6 Resultados obtenidos del subcapítulo 148
3.5 Implementación del algoritmo detección de distracción y somnolencia en los teléfonos inteligentes con sistema operativo Android 152
3.5.1 Puntos importantes para tomar en cuenta 152
3.5.2 Primeros parámetros para la implementación en teléfonos móviles 155
3.5.3 Implementación del detector de rostros en los teléfonos móviles 156
3.5.4 Implementación del sistema detector de somnolencia y distracción en los conductores 161
3.5.5 Resultados obtenidos del subcapítulo 164
3.6 Automatización de umbral para la detección de Somnolencia y Distracción . 165
3.6.1 Algoritmo Universal 165
3.7 Corrección de predicción en tiempo real 167
3.7.1 Resultados obtenidos del subcapítulo 169
3.8 Conclusiones del capítulo 171
3.9 Referencia del capítulo 173
Capítulo 4: SOMN_IA 177
4.1 Introducción 178
4.2 Construcción de hardware externo 178
4.3 Composición interna de SOMN_IA 181
4.4 Diagrama de conexión 183
4.5 Prototipo en físico 185
4.6 Ejemplos de funcionamiento 187
4.7 Implementación del software en los diferentes SO. 190
4.8 Conclusiones del capítulo. 190
Capítulo 5: CONCLUSIONES GENERALES Y TRABAJOS FUTUROS 191
5.1 Conclusiones generales 192

5.2 Trabajos futuros. 193
ANEXO 194

Índice de Figuras

Capítulo 1.- MARCO TEÓRICO.

FIGURA 1. 1: DIFERENTES TIPOS DE COLISIONES POR ALCANCE 26
FIGURA 1. 2: COLISIÓN FRONTAL 27
FIGURA 1. 3: COLISIÓN EMBESTIDA O FRONTO-LATERAL 27
FIGURA 1. 4: COLISIÓN LATERAL POR RASPADO NEGATIVO 28
FIGURA 1. 5: COLISIÓN LATERAL POR RASPADO POSITIVO 28
FIGURA 1. 6: ATROPELLAMIENTO WRAP PROJECTION 29
FIGURA 1. 7: ATROPELLAMIENTO FORWARD PROJECTION 30
FIGURA 1. 8: ATROPELLAMIENTO FENDER VAULT 30
FIGURA 1. 9: ATROPELLAMIENTO ROOF VAULT 30
FIGURA 1. 10: PÉRDIDA DE CONTROL 31
FIGURA 1. 11: FATIGA 34
FIGURA 1. 12: CONDUCTOR QUEDÁNDOSE DORMIDO POR SOMNOLENCIA 35
FIGURA 1. 13: CONDUCTOR DISTRAÍDO POR EL USO DE UN TELÉFONO MÓVIL 36

Capítulo 3.- MÉTODO PROPUESTO Y RESULTADOS.

FIGURA 3. 1: ALGORITMO GENERAL PROPUESTO PARA LA DETECCIÓN DE SOMNOLENCIA Y DISTRACCIÓN EN UN CONDUCTOR 51
FIGURA 3. 2: PARTES QUE CONFORMAN A LA CÁMARA IR IMX219-77IR 51
FIGURA 3. 3: PROCESO DE OBTENCIÓN DE IMÁGENES EN LAS DIFERENTES CONDICIONES DE ILUMINACIÓN. 52
FIGURA 3. 4: PROCESO DE OBTENCIÓN DE IMÁGENES CON ESPECTRO VISIBLE. 53
FIGURA 3. 5: PROCEDIMIENTO DE FRAGMENTACIÓN 54
FIGURA 3. 6: FILTROS HAAR-LIKE 56
FIGURA 3. 7: FILTROS HAAR CARACTERÍSTICOS 57
FIGURA 3. 8: EJEMPLO DEL PROCESAMIENTO DE LA IMAGEN INTEGRAL 58
FIGURA 3. 9: PROCESO DE LA GENERACIÓN DE CLASIFICADORES DÉBILES HASTA LA OBTENCIÓN DE UN CLASIFICADOR FUERTE 58
FIGURA 3. 10: PROCESO DE LOS CLASIFICADORES DE CASCADA 60
FIGURA 3. 11: CONVERSIÓN DE LA IMAGEN DE ENTRADA BGR A ESCALA DE GRISES 61
FIGURA 3. 12: REPRESENTACIÓN PIRAMIDAL DE LA PERDIDA DE INFORMACIÓN DE UNA IMAGEN DIGITAL 61
FIGURA 3. 13: PROCESO DE UBICACIÓN DEL ROSTRO DEL USUARIO DESDE PUNTO DE VISTA DE LOS VECINOS CERCANOS 62
FIGURA 3. 14: 4. DETECCIÓN DEL ROSTRO UTILIZANDO MÁXIMO Y MÍNIMO TAMAÑO DEL CUADRO DELIMITADOR 63
FIGURA 3. 15: PATRONES DE UN ROSTRO SIN RASGOS DE SOMNOLENCIA 65
FIGURA 3. 16: PATRONES DE UN ROSTRO CON RASGOS DE SOMNOLENCIA 66
FIGURA 3. 17: ALGUNOS FOTOGRAMAS UTILIZADOS EN LA FASE DE ENTRENAMIENTO DEL SISTEMA PROPUESTO 68
FIGURA 3. 18: DIAGRAMA DE DISTRIBUCIÓN DE LAS IMÁGENES DE ENTRENAMIENTO Y PRUEBA 68
FIGURA 3. 19: ARQUITECTURA DE LA RNC-PP PROPUESTA (PRIMERA VERSIÓN) 71
FIGURA 3. 20: EJEMPLO DE NORMALIZACIÓN INTERNO 72
FIGURA 3. 21: EJEMPLOS DE DIVERSOS RANGOS DE CORTE 73
FIGURA 3. 22: EJEMPLOS DE DIVERSOS RANGOS DE ZOOM 73
FIGURA 3. 23: EJEMPLO DEL PROCESO DE CONVOLUCIÓN 75
FIGURA 3. 24: EJEMPLO DEL PROCESO DE LA PRIMERA CAPA CONVOLUCIONAL DE LA RNC-PP PROPUESTA (PRIMERA VERSIÓN) 76

FIGURA 3. 25: EJEMPLO DEL APLANAMIENTO DE INFORMACIÓN 77
FIGURA 3. 26: PROCESO DE ACOPLAMIENTO DE LA ETAPA CONVOLUCIONAL A LA ETAPA DE IMPLEMENTACIÓN A LAS CAPAS OCULTAS DE LAS REDES NEURONALES 78
FIGURA 3. 27: ALGORITMO DEL PROCESO DE RECONOCIMIENTO FACIAL Y SUS DIVERSAS ACCIONES CUANDO SE DETECTA O NO EL ROSTRO A ESTUDIAR 79
FIGURA 3. 28: PROCESO DE EXTRACCIÓN DEL ÁREA A ANALIZAR Y LO MOSTRADO EN GUI 80
FIGURA 3. 29: PROCESO DE DETECCIÓN Y FRAGMENTACIÓN DEL ROSTRO DEL CONDUCTOR 80
FIGURA 3. 30: PROCESO DE REDUCCIÓN UTILIZANDO INTER_AREA 82
FIGURA 3. 31: PROCESO DE REDUCCIÓN UTILIZANDO INTER_AREA EN EL PROCESO REAL EN SOMN_IA 83
FIGURA 3. 32: ALGORITMO PROPUESTO PARA EL RECONOCIMIENTO DE SOMNOLENCIA EN EL CONDUCTOR 85
FIGURA 3. 33: PROCESO DE DETECCIÓN DE PARPADEO EN EL CONDUCTOR 86
FIGURA 3. 34: EJEMPLO DE LOS ESTADOS DETECTADOS POR SOMN_IA EN CONDICIONES REALES 86
FIGURA 3. 35: PROCESOS DE ACTIVACIÓN Y DESACTIVACIÓN DE LAS ALARMAS VISUALES EN LOS DISPOSITIVOS DE LABORATORIO EN DONDE SE IMPLEMENTA EL SOFTWARE DE DETECCIÓN DE SOMNOLENCIA Y DISTRACCIÓN PROPUESTO 88
FIGURA 3. 36: PROCESO DE ACTIVACIÓN DE SAL (SUBSISTEMA DE ALERTA LUMÍNICA) 89
FIGURA 3. 37: PROCESOS DE ACTIVACIÓN Y DESACTIVACIÓN DE SAL EN LOS DIFERENTES DISPOSITIVOS MÓVILES EN DONDE SE IMPLEMENTA EL SOFTWARE DE DETECCIÓN DE SOMNOLENCIA Y DISTRACCIÓN PROPUESTO 89
FIGURA 3. 38: PROCESOS DE ACTIVACIÓN Y DESACTIVACIÓN DE LAS ALARMAS SONORAS EN LOS DISPOSITIVOS DE LABORATORIO EN DONDE SE IMPLEMENTA EL SOFTWARE DE DETECCIÓN DE SOMNOLENCIA Y DISTRACCIÓN PROPUESTO 90
FIGURA 3. 39: PROCESO DE ACTIVACIÓN DE SAS (SUBSISTEMA DE ALERTA SONORA) 91
FIGURA 3. 40: PROCESOS DE ACTIVACIÓN Y DESACTIVACIÓN DE SAS EN LOS DIFERENTES DISPOSITIVOS MÓVILES EN DONDE SE IMPLEMENTA EL SOFTWARE DE DETECCIÓN DE SOMNOLENCIA Y DISTRACCIÓN PROPUESTO 91
FIGURA 3. 41: GRÁFICA DE PERDIDA Y PRECISIÓN EN EL ENTRENAMIENTO DE LA RNC PARA LA DETECCIÓN DE SOMNOLENCIA. 93
FIGURA 3. 42: EJEMPLOS OPERACIONALES DEL SISTEMA DE DETECCIÓN SE SOMNOLENCIA Y DISTRACCIÓN EN TIEMPO REAL 97
FIGURA 3. 43: EJEMPLO DE LA PERDIDA DE REGIÓN A ESTUDIAR POR ROTACIÓN DEL ROSTRO 98
FIGURA 3. 44: EJEMPLO DE FALSOS POSITIVOS EN LOS VIDEOS DE PRUEBA 99
FIGURA 3. 45: EJEMPLO DE NO RECONOCIMIENTO FACIAL POR USO DE ACCESORIOS EXTERNOS AL ROSTRO 100
FIGURA 3. 46: EJEMPLO DE ERRORES OCASIONADOS POR EL POSICIONAMIENTO DEL SENSOR ÓPTICO 101
FIGURA 3. 47: EJEMPLO DE NO DETECCIÓN DE LA REGIÓN A ESTUDIAR POR ACCIONES INVOLUNTARIAS DEL CONDUCTOR 101
FIGURA 3. 48: PUNTOS DE REFERENCIA PARA PODER DETECTAR UN ROSTRO DE ACUERDO CON MEDIAPIPE FACE DETECTION 102
FIGURA 3. 49: PATRONES ESCOGIDOS PARA GENERAR LA NUEVA BASE DE DATOS 105
FIGURA 3. 50: DIAGRAMA DE DISTRIBUCIÓN DE LAS IMÁGENES DE ENTRENAMIENTO Y PRUEBA 106
FIGURA 3. 51:ARQUITECTURA DE LA NUEVA RNC-PP PROPUESTA 107
FIGURA 3. 52: EJEMPLO DE LA IMPLEMENTACIÓN DEL RANGO DE CAMBIO DE ANCHURA 108
FIGURA 3. 53: EJEMPLO DE LA IMPLEMENTACIÓN DEL RANGO DE CAMBIO DE ALTURA 108
FIGURA 3. 54:EJEMPLO DE LA IMPLEMENTACIÓN DEL RANGO DE ROTACIÓN 109
FIGURA 3. 55: EJEMPLO DE LA IMPLEMENTACIÓN DEL VOLTEADO HORIZONTAL 109
FIGURA 3. 56: IMPLEMENTACIÓN DE LA VALIDACIÓN CRUZADA PROPUESTA 111

FIGURA 3. 57: PROCESO DE IMPLEMENTAR LA VALIDACIÓN CRUZADA A LA ARQUITECTURA CONVOLUCIONAL Y A LA RED NEURONAL PROPUESTA 112
FIGURA 3. 58: ALGORITMO PARA EL RECONOCIMIENTO DE DISTRACCIÓN (NUEVA VERSIÓN). 115
FIGURA 3. 59: DETECCIÓN DE DISTRACCIÓN POR CLASIFICACIÓN 116
FIGURA 3. 60: EJEMPLO DE LAS DIFERENTES DETECCIONES DETECTADAS POR SOMN_IA APLICANDO LA NUEVA RNC-PP 117
FIGURA 3. 61:PROCESO DE ACTIVACIÓN Y DESACTIVACIÓN DEL SAL MODIFICADA AL MOMENTO DE DETECTAR LA DISTRACCIÓN 118
FIGURA 3. 62: PROCESO DE ACTIVACIÓN Y DESACTIVACIÓN DEL SAS MODIFICADA AL MOMENTO DE DETECTAR LA DISTRACCIÓN 119
FIGURA 3. 63: PROCESO DE ACTIVACIÓN Y DESACTIVACIÓN DEL SAL MODIFICADA AL MOMENTO DE DETECTAR LA SOMNOLENCIA 120
FIGURA 3. 64: PROCESO DE ACTIVACIÓN Y DESACTIVACIÓN DEL SAL MODIFICADA AL MOMENTO DE DETECTAR LA SOMNOLENCIA 120
FIGURA 3. 65: MATRICES DE CONFUSIÓN [49] EN LAS DIFERENTES PRUEBAS UTILIZANDO LA BASE DE DATOS NTHU-DDD 124
FIGURA 3. 66: DIFERENTES TIPOS DE TRANSFERENCIA DE APRENDIZAJE UTILIZANDO LA ARQUITECTURA VGG16 COMO EJEMPLO 127
FIGURA 3. 67: GRÁFICAS DE LAS CAPAS REENTRENADAS DE MOBILENET V2 CON RESPECTO A SU PRECISIÓN Y EL NÚMERO DE PARÁMETROS ENTRENADOS. SE REALIZARON 18 REENTRENAMIENTOS. 149
FIGURA 3. 68: GRÁFICAS DE LAS CAPAS REENTRENADAS DE VGG16 CON RESPECTO A SU PRECISIÓN Y EL NÚMERO DE PARÁMETROS ENTRENADOS. SE REALIZARON 5 REENTRENAMIENTOS. 149
FIGURA 3. 69: GRÁFICAS DE LAS CAPAS REENTRENADAS DE RESNET-50V2 CON RESPECTO A SU PRECISIÓN Y EL NÚMERO DE PARÁMETROS ENTRENADOS. SE REALIZARON 5 REENTRENAMIENTOS. 150
FIGURA 3. 70: GRÁFICAS DE LAS CAPAS REENTRENADAS DE INCEPTIONV3 CON RESPECTO A SU PRECISIÓN Y EL NÚMERO DE PARÁMETROS ENTRENADOS. SE REALIZARON 8 REENTRENAMIENTOS. 150
FIGURA 3. 71: GRÁFICAS DE LAS CAPAS REENTRENADAS DE XCEPTION CON RESPECTO A SU PRECISIÓN Y EL NÚMERO DE PARÁMETROS ENTRENADOS. SE REALIZARON 14 REENTRENAMIENTOS. 151
FIGURA 3. 72:DIFERENTES SENSORES ÓPTICOS QUE SE PUEDEN O PRIMA UTILIZAR EN LA APP SOMN_IA 155
FIGURA 3. 73: VISTA DEL FOTOGRAMA EN LOS DIFERENTES LENGUAJES DE PROGRAMACIÓN 156
FIGURA 3. 74: ALGORITMO DE ACELERACIÓN PARA RECONOCIMIENTO FACIAL CON IMAGEN DE ENTRADA Y SALIDA ACEPTABLE 157
FIGURA 3. 75: PERMUTA DE EN LOS CANALES RGB A BGR DE LA IMAGEN DE ENTRADA 157
FIGURA 3. 76: DETECCIÓN DE ROSTROS IMPLEMENTADO EN EL TELÉFONO MÓVIL, CON UNA VELOCIDAD DE RECONOCIMIENTO LENTO 158
FIGURA 3. 77: IMPLEMENTACIÓN DE PROCESAMIENTO DE DISMINUCIÓN DE LA IMAGEN DE ENTRADA 158
FIGURA 3. 78: PROCESO DE ADECUACIÓN DE LA IMAGEN ANALIZADA PARA SU MUESTRA EN LA GUI 160
FIGURA 3. 79:COMPARACIÓN ENTRE LA PRIMERA VELOCIDAD DE RECONOCIMIENTO FACIAL Y EL RECONOCIMIENTO EMPLEANDO EL ALGORITMO DE ACELERACIÓN DE MONITOREO PROPUESTO 161
FIGURA 3. 80: ALGORITMO GENERAL DE RECONOCIMIENTO DE SOMNOLENCIA Y DISTRACCIÓN IMPLEMENTANDO EN TELÉFONOS MÓVILES 161
FIGURA 3. 81: ALGORITMO DEL RECONOCIMIENTO DE SOMNOLENCIA IMPLEMENTADO LA AUTOMATIZACIÓN DE UMBRAL AUTO GENERATIVA 166

FIGURA 3. 82: ALGORITMOS PARA EL RECONOCIMIENTO DE DISTRACCIÓN IMPLEMENTADO LA AUTOMATIZACIÓN DE UMBRAL AUTO GENERATIVA 167
EN LA FIGURA 3. 82 A Y B PODEMOS OBSERVAR CÓMO QUEDARÍA EL ALGORITMO GENERAL DE DETECCIÓN DE SOMNOLENCIA Y DISTRACCIÓN, IMPLEMENTANDO LAS AUTOMATIZACIONES DE UMBRALES AUTO GENERATIVAS (*DDT* Y *DD*)..... 167
FIGURA 3. 83: ERROR DE PREDICCIÓN EN FOTOGRAMAS SECUENCIALES 168
FIGURA 3. 84: ALGORITMO CORRECCIÓN DE ERROR DE PREDICCIÓN 168
FIGURA 3. 85: COMPARACIÓN DE RENDIMIENTO ENTRE EL SISTEMA ANTERIOR (A) Y LA VERSIÓN MODIFICADA (B) EN LA DETECCIÓN DE SOMNOLENCIA DEL CONDUCTOR 169
FIGURA 3. 86: COMPARACIÓN DE RENDIMIENTO ENTRE EL SISTEMA ANTERIOR (A) Y LA VERSIÓN MODIFICADA (B) EN LA DETECCIÓN DE DISTRACCIONES DEL CONDUCTOR..... 170

Capítulo 4.- SOMN_IA.

FIGURA 4. 1: VISTA ISOMÉTRICA DEL DISPOSITIVO SOMN_IA 179
FIGURA 4. 2: VISTA ISOMÉTRICA-TRASERA DE SOMN_IA 181
FIGURA 4. 3: VISTA INFERIOR DE SOMN_IA 181
FIGURA 4. 4: DIAGRAMA A BLOQUES DE LA INTERCOMUNICACIÓN DE LOS MÓDULOS INTERNOS DE SOMN_IA..... 182
FIGURA 4. 5: DIAGRAMA DE CONEXIÓN LOCAL UTILIZANDO LA CÁMARA WEB..... 183
FIGURA 4. 6: DIAGRAMA DE CONEXIÓN LOCAL UTILIZANDO LA CÁMARA IR 183
FIGURA 4. 7: DIAGRAMA DE CONEXIÓN MÓVIL UTILIZANDO LA CÁMARA WEB 184
FIGURA 4. 8: DIAGRAMA DE CONEXIÓN MÓVIL UTILIZANDO LA CÁMARA IR..... 184
FIGURA 4. 9: VISTA ISOMÉTRICA EN FÍSICO DEL PROTOTIPO DE SOMN_IA 185
FIGURA 4. 10: VISTA FRONTAL DEL PROTOTIPO DE SOMN_IA MOSTRANDO LAS DOS CÁMARAS SELECCIONADAS..... 185
FIGURA 4. 11: VISTA ISOMÉTRICA TRASERA DEL PROTOTIPO DE SOMN_IA 186
FIGURA 4. 12: VISTA INFERIOR DEL PROTOTIPO DE SOMN_IA 186
FIGURA 4. 13: DETECCIÓN DE SOMNOLENCIA Y DISTRACCIÓN EN CONDUCTORES UTILIZANDO LA CÁMARA WEB. 187
FIGURA 4. 14: DETECCIÓN DE SOMNOLENCIA Y DISTRACCIÓN DEL CONDUCTOR UTILIZANDO LA CÁMARA IR (VERSIÓN CON ILUMINACIÓN) 187
FIGURA 4. 15: DETECCIÓN DE DISTRACCIÓN Y ACTIVACIÓN DE LAS ALARMAS PERTINENTES. 188
FIGURA 4. 16: DETECCIÓN DE SOMNOLENCIA Y ACTIVACIÓN DE LAS ALARMAS PERTINENTES..... 188
FIGURA 4. 17: EJEMPLO DEL MONITOREO EN TIEMPO REAL. 189
FIGURA 4. 18:BLOQUEO DE SOMN_IA E GUI PARA SU DESBLOQUEO 189

Índice de Tablas

Capítulo 1.- MARCO TEÓRICO.

TABLA 1. 1: INEGI; ACCIDENTES DE TRÁNSITO TERRESTRE EN ZONAS URBANAS Y SUBURBANAS; ACCIDENTES POR TIPO DE ACCIDENTE [13] 32
TABLA 1. 2: INEGI; ACCIDENTES DE TRÁNSITO TERRESTRE EN ZONAS URBANAS Y SUBURBANAS; ACCIDENTES POR CAUSA [14] 33

Capítulo 3.- MÉTODO PROPUESTO Y RESULTADOS.

TABLA 3. 1: ESPECIFICACIONES TÉCNICAS DE LA CÁMARA IR IMX219-77IR 52
TABLA 3. 2: SENSORES ÓPTICOS CON PERIFÉRICO USB TIPO A-MACHO UTILIZADOS EN ESTA INVESTIGACIÓN. 53
TABLA 3. 3: TABLA COMPARATIVA EN EL PROCESO DE FRAGMENTACIÓN DE VIDEOS EN TIEMPO REAL EN LOS DIFERENTES DISPOSITIVOS UTILIZADOS EN ESTA INVESTIGACIÓN 55
TABLA 3. 4: TABLA COMPARATIVA EN EL PROCESO DE FRAGMENTACIÓN DE VIDEOS PREGRABADOS EN LOS DIFERENTES DISPOSITIVOS UTILIZADOS EN ESTA INVESTIGACIÓN 55
TABLA 3. 5: CARACTERÍSTICAS DE LA BASE DE DATOS *"NTHU-DDD"* 67
TABLA 3. 6: PARÁMETROS PROPUESTOS PARA LA ARQUITECTURA CONVOLUCIONAL 74
TABLA 3. 7: ARQUITECTURA GENERAL DE LA RNC-PP 92
TABLA 3. 8: COMPARACIÓN DE RENDIMIENTO ENTRE VARIOS SISTEMAS DE DETECCIÓN DE SOMNOLENCIA DEL CONDUCTOR EN TIEMPO REAL 93
TABLA 3. 9: TABLA COMPARATIVA EMPLEADO EL SISTEMA GENERAL DE DETECCIÓN SOMNOLENCIA Y DISTRACCIÓN EN TIEMPO REAL EN LOS DIFERENTES DISPOSITIVOS UTILIZADOS EN ESTA INVESTIGACIÓN 94
TABLA 3. 10: TABLA COMPARATIVA EN EL PROCESO DE FRAGMENTACIÓN DE VIDEOS PREGRABADOS EN LOS DIFERENTES DISPOSITIVOS UTILIZADOS EN ESTA INVESTIGACIÓN 95
TABLA 3. 11: COSTOS POR UNIDAD UTILIZADA EN ESTAS PRUEBAS 96
TABLA 3. 12: ENLACE DE LOS VIDEOS DE FUNCIONAMIENTO CON UNA PEQUEÑA DESCRIPCIÓN 97
TABLA 3. 13: TABLA COMPARATIVA DE PRECISIÓN DEPENDIENDO LA POSICIÓN DEL SENSOR ÓPTICO 100
TABLA 3. 14: ARQUITECTURA DE BLAZEFACE 102
TABLA 3. 15: TABLA COMPARATIVA DE RENDIMIENTO Y PRECISIÓN ENTRE EL RECONOCIMIENTO FACIAL PROPUESTO POR V&J Y MEDIAPIPE FACE DETECTION 103
TABLA 3. 16: PARÁMETROS PROPUESTOS PARA LA ARQUITECTURA DE LA NUEVA RNC-PP 112
TABLA 3. 17: ARQUITECTURA GENERAL DE LA RNC-PP (NUEVA VERSIÓN) 122
TABLA 3. 18: TABLA COMPARATIVA ENTRE LAS DOS VERSIONES DE RNC-PP CREADAS 123
TABLA 3. 19: TABLA COMPARATIVA EMPLEADO EL SISTEMA GENERAL ACTUALIZADO Y OPTIMIZADO DE DETECCIÓN SOMNOLENCIA Y DISTRACCIÓN EN TIEMPO REAL EN LOS DIFERENTES DISPOSITIVOS UTILIZADOS EN ESTA INVESTIGACIÓN 124
TABLA 3. 20: TABLA COMPARATIVA EN EL PROCESO DE FRAGMENTACIÓN DE VIDEOS PREGRABADOS EN LOS DIFERENTES DISPOSITIVOS UTILIZADOS EN ESTA INVESTIGACIÓN 125
TABLA 3. 21: ENLACE DE LOS VIDEOS DE FUNCIONAMIENTO CON UNA PEQUEÑA DESCRIPCIÓN 126
TABLA 3. 22:DIFERENTES FINE TUNING IMPLEMENTADAS A LA RNC MOBILENET-V2 131
TABLA 3. 23: DIFERENTES FINE TUNING IMPLEMENTADAS A LA RNC VGG16 138
TABLA 3. 24: DIFERENTES FINE TUNING IMPLEMENTADAS A LA RNC RESNET-50V2 140
TABLA 3. 25: DIFERENTES FINE TUNING IMPLEMENTADAS A LA RNC INCEPTIONV3 142
TABLA 3. 26: DIFERENTES FINE TUNING IMPLEMENTADAS A LA RNC XCEPTION 146
TABLA 3. 27: TABLA COMPARATIVA DE LOS MEJORES MODELOS REENTRENADOS DE ACUERDO CON SU MEJOR PRECISIÓN. 151

TABLA 3. 28: TABLA DE VELOCIDAD UTILIZANDO DIVERSOS PREPROCESAMIENTOS DE REDIMENSIONAMIENTO DE IMÁGENES. 159
TABLA 3. 29: EN ESTA TABLA REPRESENTAMOS EL NIVEL DE PERDIDA DE INFORMACIÓN MIENTRAS MÁS SE REDUCE LA IMAGEN UTILIZANDO EL REDIMENSIONAMIENTO INTE_AREA 159
TABLA 3. 30: TABLA DE RENDIMIENTO EN LOS DIVERSOS DISPOSITIVOS DE PRUEBA..... 164
TABLA 3. 31: ENLACE DE LOS VIDEOS DE FUNCIONAMIENTO..... 165
TABLA 3. 32: COMPARACIÓN DE RENDIMIENTO ENTRE EL SISTEMA ANTERIOR Y LA VERSIÓN MODIFICADA PROPUESTA 171

Agradecimientos

Para la creación de la actual tesis, me gustaría dar acrecimiento a las siguientes personas y entidades:

- ***AL INSTITUTO POLITÉCNICO NACIONAL***, Por ser mi alma mater la cual por más de 10 años ha estado presente, apoyando mi carrera tanto económica como intelectualmente.
- ***A LA ESCUELA SUPERIOR DE INGENIERÍA MECÁNICA Y ELÉCTRICA UNIDAD CULHUACÁN***, por darme la oportunidad de cursar el grado de maestría, así como también, por haberme dado todo el cocimiento necesario para terminar de la mejor manera este ciclo de aprendizaje.
- ***AL CONACYT,*** por haberme apoyado económicamente, ya que, sin ese apoyo, este grado no habría sido posible.
- ***A MIS TUTORES***; a la Dra. Mariko Nakano Miyatake quien, por dos años, siempre estuvo apoyándome tanto intelectual como personalmente, sin usted, sin su conocimiento y sin sus virtudes, este trabajo seria inalcanzable, al Dr. Hector Pérez Meana, por siempre contar con su apoyo, en los diversos requerimientos necesarios para la implementación y terminación de esta tesis y al Dr. Enrique Escamilla Hermandades, que sin su apoyo en el área técnica y de soporte, sería imposible la implementación de este proyecto así como su realización como tal.
- ***A MIS COMPAÑEROS***, por haberme asesorado, ayudado y apoyado en momentos en los cuales nos sabía que hacer, sus sabios consejos fueron de gran ayuda para la terminación de este trabajo.
- ***A MIS AMIGOS***, los cuales me ayudaron mucho, siendo parte indirecta del proyecto.
- ***A MIS PADRES***, que, sin ustedes, nada de esto sería posible, a ustedes que me han ayudado desde que tengo memoria, a ustedes que, en los momentos duros, siempre han estado ahí, a ustedes que se esfuerzan cada día para poder hacer que mi vida sea un poco más fácil de sobrellevar, hoy que termino mis estudios de posgrado, les dedico este gran paso.
- A los que no eh mencionado, porque por ellos, este presente trabajo que directa o indirectamente fueron de gran ayuda para su realización.

Muchas gracias.

Resumen

Actualmente, los accidentes automovilísticos van en aumento cada año, siendo este una de las principales causas de muertes por accidentes en el mundo, de acuerdo con estadísticas del INEGI, las emergencias automovilísticas aumentaron un 34% en 2021, con respecto al año anterior; la mayoría de los siniestros ocurridos, son los denominados ***ACCIDENTES DIRECTOS***, es decir, son incidentes que se pueden evitar, estos son principalmente causados por la negligencia del conductor, de estos, los causados por personas que no prestan atención al camino o bien, haberse quedado dormido mientras conducen, se han convertido en unos de los más comunes.

De acuerdo con la Clínica del Sueño, tres de cada diez accidentes automovilísticos en México son causados por personas que se han quedado dormidas tras el volante, por otro lado, los incidentes causados por distracción al momento de conducir, se han convertido en la principal causa de siniestros automotrices, debido a que en 2017, el 33% de las emergencias relacionadas con vehículos, fueron causados por conductores que se distrajeron por algún motivo al momento de conducir, como hablar por teléfono, no ver el camino, distraerse con dispositivos electrónicos, etc.

Sabiendo lo anterior, en esta investigación proponemos un aparato capaz de detectar los dos estados más peligrosos al momento de conducir; SOMN_IA es un dispositivo capaz de determinar cuando el conductor se ha quedado dormido, así como también comprueba si el piloto está prestando o no atención al camino, avisando si es necesario con alertas lumínicas, informativas y sonoras que el usuario está conduciendo de una manera imprudente.

Si llegase el caso extremo en que el usuario pase los umbrales propuestos y no reacciona o no vuelve a conducir de una forma correcta, se activa un modo que hemos denominado ***RASTREO***, el cual automáticamente SOMN_IA se pondrá en contacto ya sea con una persona de confianza, a una central de monitoreo, agencias de viajes, y demás, para poder comunicarse con el conductor y así impedir una tragedia.

SOMN_IA está pensado para poder implementarse en cualquier dispositivo tanto móvil como fijo, por lo cual, se ha desarrollado diversos softwares y ejecutables, los cuales se han optimizado para su ejecución en diversos hardwares, tales como Workstation, Laptops, PC de escritorio, tarjetas de desarrollo dedicadas y teléfonos inteligentes; haciendo el sistema tan universal que se puede monitorear la somnolencia y la distracción en los conductores en casi cualquier dispositivo, sin las limitaciones naturales, como memoria, consumo energético, SOs, etc.

SOMN_IA es capaz de detectar los estados del conductor debido a que se ha implementado diversas técnicas de la rama de la inteligencia artificial, los cuales son:

1) ***Visión por computadora:*** La cual es utilizada como el sistema base para obtener información del exterior, en este caso, es implementada para poder obtener las

regiones y rasgos necesarios del conductor para poder analizar los patrones que den como resultado, la somnolencia o la distracción.

2) ***Reconocimiento de patrones:*** Esta técnica es utilizada para poder obtener elementos característicos de un conductor cansado o distraído. El reconocimiento de patrones que nosotros utilizamos en esta investigación es la del reconocimiento supervisado por aprendizaje profundo.

3) ***Aprendizaje profundo:*** Esta técnica es la más importante, debido a que es la encargada de arrojarnos las predicciones dadas por las máquinas, al momento de analizar las regiones obtenidas por la etapa de la visión por computadora, en este proceso, usamos una técnica muy conocida en el aprendizaje profundo, la cual está enfocada en modelos de clasificación, este procesos se denomina redes neuronales convolucionales (RNC), las cuales son las encargadas de poder clasificar los 2 estados principales del conductor, la primera y la cual se divide en dos secundarias clasificaciones es la que hemos denominado como ***ESTADO NORMAL DEL CONDUCTOR*** o también la denominamos como ***ATENCIÓN*** en donde la RNC clasifica los patrones del usuario en ***"ojos abiertos"*** y ***"ojos cerrados"***, y la segunda es la denominada como ***DISTRACCIÓN***.

   La configuración que generamos para este tipo de red neuronal, está específicamente diseñada para poder tener la mejor ***"precisión o accuracy"*** con relación al ***"coste/beneficio"***, teniendo una exactitud del 95.77%; cabe mencionar que se hicieron diversas pruebas utilizando transferencia de aprendizaje, en donde se utilizaron arquitecturas populares de RNC, tales como Mobilenet V2, VGG16, ResNet50V2, InceptionV3 y Xception, en donde su mejor acierto fue de 96.75%, 97.77%, 97.60%, 96% y 97.13% respectivamente. Si bien, los resultados de las diversas configuraciones de RNC utilizadas son superiores a la que se usa en SOMN_IA, su número de parámetros para entrenar son mayores de 1 millón, así como también su peso (en disco) y tiempo de procesamiento es mucho mayor.

4) ***Sistema en tiempo real:*** Si bien, esta no es una técnica de inteligencia artificial, es requerida, debido a la naturaleza que tiene nuestro sistema, se es necesario para poder correr los softwares y ejecutables de la mejor manera posible, tomando en cuenta las limitantes que tienen los dispositivos embebidos, en donde SOMN_IA se encuentra empleado.

Finalmente, SOMN_IA es un dispositivo que trabaja en conjunto con diversos sistemas, los cuales, aseguran una conducción segura.

Abstract

Currently, car accidents are increasing every year, this being one of the main causes of deaths from accidents in the world, according to INEGI statistics, car emergencies increased by 34% in 2021, compared with the previous year; Most of the accidents that occur are called ***DIRECT ACCIDENTS***, that means, they are incidents that can be avoided, these are mainly caused by the negligence of the driver, of these, those caused by people who do not pay attention to the road or have fallen asleep while driving, have become one of the most common.

According to the "Clinica del Sueño", three out of ten car accidents in Mexico are caused by people who have fallen asleep behind the wheel, on the other hand, incidents caused by distraction while driving have become the main cause of auto accidents, because in 2017, 33% of vehicle-related emergencies were caused by drivers who were distracted for some reason while driving, such as talking on the phone, not seeing the road, being distracted by electronic devices, etc.

Knowing the above, in this research we propose a device capable of detecting the two most dangerous states when driving; SOMN_IA is a device capable of determining when the driver has fallen asleep, as well as checking whether or not the driver is paying attention to the road, warning, if necessary, with light, information and sound alerts that the user is driving recklessly .

If the case occurs in which the user passes the proposed thresholds and does not react or does not drive correctly again, a mode that we have called ***TRACKING*** is activated, which automatically SOMN_IA will contact either a trusted, person , to a monitoring center, travel agencies, and others, to be able to communicate with the driver and thus prevent a tragedy.

SOMN_IA is designed to be implemented on any device, both mobile and fixed, for which various software and executables have been developed, which have been optimized for execution on various hardware, such as Workstations, laptops, desktop PCs, memory cards, etc., dedicated development and smart phones; making the system so universal that you can monitor driver drowsiness and distraction on almost any device, without natural limitations such as memory, power consumption, OSs, etc.

SOMN_IA is capable of predicting the states of the driver due to the implementation of various techniques from the branch of artificial intelligence, which are:

1) ***Computer vision:*** Which is used as the base system to obtain information from the outside, in this case, it is implemented to obtain the necessary regions and features of the driver to analyze the patterns that result in drowsiness or distraction.

2) ***Pattern recognition:*** This technique is used to obtain characteristic elements of a tired or distracted driver. The pattern recognition that we use in this research is unsupervised recognition by deep learning.
3) ***Deep learning:*** : This technique is the most important, because it is in charge of throwing us the predictions given by the machines, when analyzing the regions obtained by the computer vision stage, in this process, we use a well-known technique in the deep learning, which is focused on classification models, this process is called convolutional neural networks (CNN), which are responsible for being able to classify the 2 main states of the driver, the first and which is divided into two secondary ones is the which we have called **N*ORMAL STATE OF THE DRIVER***, where the RNC classifies the user patterns into ***"open eyes"*** and ***"closed eyes"***, and the second is called ***DISTRACTION***.

   The configuration that we generate for this type of neural network is specifically designed to be able to have the best ***"accuracy"*** in relation to the ***"cost/benefit"***, having an accuracy of 95.77%; It is worth mentioning that they did various tests using fine tuning, where popular CNN configurations were used, such as Mobilenet V2, VGG16, ResNet50V2, InceptionV3 and Xception, where their best success was 96.75%, 97.77%, 97.60%, 96 % and 97.13% respectively. Although the results of the various CNN configurations used are superior to the one used in SOMN_IA, its number of parameters to train is greater than 1 million, as well as its weight (on disk) and processing time is much greater.
4) ***Real-time system:*** Although this is not an artificial intelligence technique, it is required, due to the nature of our system, it is necessary to be able to run the software and executables in the best possible way, considering the limitations of embedded devices, where SOMN_IA is used.

Finally, SOMN_IA is a device that works together with various systems, which ensure safe driving.

Problemática

Actualmente, alrededor del mundo, los accidentes automovilísticos se han vuelto un tema controversial en los últimos años, desde accidentes indirectos en donde la causa principal de dichos siniestros son factores externos al conductor [1], tales como condiciones inapropiadas en la carretera, fallas mecánicas en los vehículos, hasta imprevistos lamentables como el cruce inesperado de objetos, animales o hasta personas en lugares donde su circulación no está permitida, por otro lado, están los accidentes directos [1], en donde dichos accidentes son causados por la imprudencia del conductor, como conducir en estado de ebriedad, por distracción al momento de manipular artefactos ajenos al mismo automóvil o bien, por la somnolencia o quedarse dormido al momento de conducir.

Los accidentes relacionados con quedarse dormido mientras se conducen, son enormes, muchos estudios revelaron que entre 10% y el 30% de los accidentes se deben a la conducción somnolienta [2,3]. La fatiga tiene efectos costosos sobre la seguridad, la salud y la calidad de vida. De acuerdo con datos del Consejo Nacional para Prevención de Accidentes (CONAPRA) (2013) [4], en México cada año mueren en promedio 16,500 mexicanos por percances viales además que los accidentes viales le cuestan al país alrededor de 150 mil millones de pesos que representan el 1.7% del Producto Interno Bruto (PIB), sumando costos directos e indirectos, a esto se sabe que 3 de cada 10 accidentes automovilísticos en México son causados por personas que se quedan dormidas al volante, sobre todo durante las fiestas navideñas, alertaron especialistas de la Clínica del Sueño[5].

Por otra parte, los accidentes causados por distracciones por parte del conductor son la principal causa de incidentes, debido a que el 33% de las emergencias relacionadas con vehículos, fueron causados por conductores que se distrajeron por algún motivo al momento de conducir, como hablar por teléfono, no ver el camino, distraerse con dispositivos electrónicos, etc. [6]

De acuerdo con proyecciones hechas por [7], ANASEVI y AT&T, de 2020 a 2024 los accidentes relacionados por conductores que no prestan atención al camino por estar distraídos dejarían un total de 86,988 personas fallecidas, victimas por la negligencia de estos usuarios.

Lamentablemente, los dos casos más grandes de accidentes automotrices (comentados anteriormente) son si bien, propiciados casi siempre por errores o negligencias humanas, también, son los accidentes más factibles de prevenir.

Referencias

[1] INEGI, "Accidentes de tránsito terrestre en zonas urbanas y suburbanas", [en línea] https://www.inegi.org.mx/sistemas/olap/consulta/general_ver4/MDXQueryDatos.asp?#Regreso&c=13159

[2] Traffic Safety Culture Index, Foundation for Traffic Safety, Junio 2020, https://aaafoundation.org/2019-traffic-safety-culture-index.

[3] Chacon-Murguia, M., Prieto-Resendiz C. Detecting driver drowsiness: A survey of system designs and technology. IEEE Consum. Electron. Mag. 2015 Abril; PP: 107-119.

[4] Páez M., Abarca E.: Tools for security in movements, Predictive models of driver's drowsiness, [en línea] https://imt.mx/resumen-boletines.html?IdArticulo=449&IdBoletin=168.

[5] Barboza G., Accidentes viales también son causados por quedarse dormido, Diciembre 2019, [en línea] https://www.rastreator.mx/seguros-de-auto/noticias/accidentes-viales-tambien-son-causados-por-quedarse-dormido#:~:text=Se%20considera%20que%20tres%20de,duerme%20seis%20horas%20en%20promedio.

[6] Noticias-Seguridad Vial, 30% de accidentes viales son causados por distracciones, Agosto 2016, [en línea] https://prevencionar.com.mx/2016/08/16/30-accidentes-viales-causados-distracciones/.

[7] Zuñiga E, Distracciones en conducción causaría cerca de 87,000 muertes para 2024, Julio 2021, [en línea] https://www.tyt.com.mx/nota/conduccion-distraida-victimas-accidentes-vehiculos-estudio.

Hipótesis

Crear un sistema con ayuda de nuevas tecnologías e implementarlos en diversos dispositivos tecnológicos, como tarjetas de desarrollo, sistema dedicados, y demás, que no sólo detecte la somnolencia y la distracción en los conductores sino que además pueda alertar al conductor con alarmas visibles y sonoras, que adicionalmente tenga un monitoreo en tiempo real, tanto interno (estudio permanente de los patrones del conductor) como externo (compartir ubicaciones en tiempo real a personas o entidades externas), pueden ayudar a reducir este tipo de incidentes, bajando así los decesos y gastos indirectos ocasionados por descuidos o negligencias al momento de controlar un vehículo.

Justificación

Si bien, los accidentes automovilísticos han sido un tema que se ha abordado mucho en los últimos años, las técnicas de reconocimiento no son muy empleadas actualmente debido a que muchos fabricantes implementan sistemas complementarios con este tipo de reconocimiento en sus vehículos, particularmente, son añadidos en vehículos de gran coste, por tal motivo, no es muy habitual su implementación en dispositivos externos, por otra parte, la implementación de la gran mayoría de investigaciones y/o patentes que se han buscado o encontrado, casi siempre, tienen diversas limitaciones, las cuales propician que no sean innovaciones tan profundas; normalmente, la detección de somnolencia se detecta de forma aislada a la detección de distracción, haciendo difícil encontrar sistemas que los unifiquen.

Existen pocos sistemas o investigaciones que combinan dichas detecciones (somnolencia y distracción), los cuales tiene limitaciones tanto físicas como tecnológicas, en donde si bien pueden detectar la fatiga y la falta de atención al momento de conducir, su implementación en el vehículo es muy compleja y nada factible al momento de la detección en tiempo real. SOMN_IA por otra parte es capaz de detectar los estados mencionados anteriormente, pensando en la universalidad, la cual ayuda a que sea un dispositivo tan amigable, que no importa las condiciones, el sistema puede trabajar.

Objetivos

1. Objetivo General

Crear e implementar un dispositivo tecnológico que pueda detectar los dos estados más peligrosos al momento de conducir (somnolencia y distracción), utilizando técnicas de nueva generación y empleando una interfaz gráfica, así como elementos pasivos y activos para poder comunicarle al usuario de que está manejando de una manera peligrosa y poder evitar así una tragedia.

2. Objetivos Específicos

- Crear un dispositivo móvil capaz de detectar la somnolencia y la distracción.
- Implementar diversos algoritmos de reconocimiento facial, la cual será la región necesaria para poder estudiar los patrones de somnolencia y distracción; adicionalmente, aplicar métodos de procesamiento de imágenes para facilitar el estudio y la clasificación mencionada en los puestos posteriores a este.
- Crear una base de datos optimizada y especialmente diseñada para dispositivos móviles a partir de base de datos publicas enfocadas a este tipo de investigaciones.
- Desarrollar un modelo de clasificación liviano capaz de poder diferenciar 3 estados específicos del conductor los cuales son ***"ojos abiertos", "ojos cerrados"*** y ***"distracción"*** con ayuda de técnicas de inteligencia artificial y utilizando la base de datos mencionada en el punto anterior, dicho modelo debe estar optimizado y generado para poder correrlo en diversos dispositivos con limitaciones físicas como memoria, poder de procesamiento, entre otros así como también, en máquinas físicas que no tengan estos problemas (Workstation, laptops, PC, etc.).
- Comparar el modelo clasificatorio propuesto con modelos populares y mejorarlo al grado de ser la mejor opción con respecto al coste/beneficio.
- Crear un algoritmo base, en donde unifique el modelo clasificatorio creado en los anteriores puntos, el cual será implementado en diversos SO's (ANDROID, Windows, Linux), así como también en diversos dispositivos (teléfonos inteligentes, laptops, PC's y tarjetas de desarrollo dedicadas).
- Automatizar el algoritmo base en todos los dispositivos.
- Crear un algoritmo automático que corrija los errores dados por el modelo de la RNC propuesta al momento del monitoreo en tiempo real.
- Crear un sistema de alarmas sonoras y visuales las cuales serán activadas cuando sea necesario, esto lo decidiría el algoritmo implementado mencionado en los puntos anteriores.

- Implementar un sistema de rastreo en tiempo real del conductor, el cual será accionado como última opción en caso de que el usuario no sea capaz de conducir por cuestiones directas o indirectas de este.

Contribución

Actualmente, la mayoría de los sistemas comerciales enfocados para la predicción de somnolencia, así como de distracción en los conductores, son técnicas empleadas en conjunto con otros sistemas, lo que hace que su funcionamiento dependa de varios parámetros, haciendo la detección, un subsistema de alto coste computacional, además que, casi todos los detectores vienen implementados desde fábrica, lo que hace que la universalidad no sea posible [1,2].

Los sistemas que se pueden instalar de forma externa son dispositivos que llegan o tienden a ser muy difíciles de implementar en el automóvil, debido a que utilizan diversos sensores para poder determinar la fatiga o distracción al momento de conducir, como, por ejemplo: sensores de movimiento, de aceleración, detectores de movimiento de cabeza que normalmente se detectan con sensores giroscópicos, cámaras de monitoreo, etc. [3-22], que en conjunto pueden determinar los estados previamente comentados. Otras técnicas de detección y predicción son sistemas de estudio en laboratorio, debido que utilizan sensores que leen señales fisiológicas del conductor, dichos módulos son invasivos, los cuales los hacen muy incomodos al momento de conducir y muy poco prácticos [23,24].

Por todo lo mencionado anteriormente , el sistema que se propone en esta tesis, es un dispositivo universal que a diferencia de los que hay en el mercado, así como en las diferentes patentes y/o métodos de investigación, su implementación es tan simple como conectar un plug-in (convertidor DC-DC BUCK [25]) en el encendedor del automóvil, se utiliza una cámara (de espectro visible, infrarrojo o ambos espectros) como único sensor y es de fácil instalación, debido a que puede situar en cualquier parte del parabrisas (recomendable a un máximo de 30º grados ya sea a la izquierda o derecha del punto medio de rostro del usuario y finalmente SOMN_IA que es donde se encuentra el cerebro, la interfaz gráfica, la electrónica y demás; tiene diversos soportes para instalar el sistema ya sea en el cristal frontal del automóvil, al igual que el dispositivo de video mencionado anteriormente o en cualquier parte del habitáculo (se recomienda lo más cercano al usuario, como por ejemplo, en alguna parte del salpicadero).

Un punto interesante para este tipo de innovación, es que el sistema no sólo se queda implementado en un hardware específico como en SOMN_IA (hardware propuesto), sino que también, es un software individual, el cual está adaptado para poder funcionar en diversos dispositivos, en donde no se tiene limitaciones de memoria, energía, o poder computacional, pero es utilizado de forma no móvil, tales como

Workstation, Laptops, PC, y adicionalmente está optimizado para su implementación en dispositivos móviles en donde si se encuentran las limitaciones anteriormente mencionadas, estos pueden ser el propio SOMNI_IA y tarjetas de desarrollo dedicadas. Haciendo aún más su universalidad; cabe mencionar que el software esta optimizado para trabajar en Sistemas Operativos diferentes, estos son: Linux, Windows y Android.

Hablando más específicamente del software, se proponen diversas adecuaciones mejoras y modelos que hacen su uso más factible y de fácil implementación en diversos dispositivos tecnológicos, tales como:

- ***Modelo clasificatorio utilizando técnicas de aprendizaje profundo, tipo Red Neuronal Convolucional (RNC) de bajo coste computacional [26]***: Esta red es una propuesta muy diferente a muchos modelos de RNC que hay en el mercado, debido a que se emplea un set de datos creado por nosotros, dicho set no es mayor a 5,000 imágenes en total; esta base de datos que generamos se divide en 75% entrenamiento y validación, el 25% restante se divide en imágenes de prueba, lo que hace que a diferencia de las RNC populares, estas 5,000 imágenes crean un número de parámetros entrenables menor a 400,000, que por otro lado las RNC populares requieren más de 1,000,000 [27-31] de parámetros para poder obtener una clasificación buena, adicionalmente, esta configuración es apta para su implementación en dispositivos móviles.
- ***Automatización de umbrales en tiempo real:*** Se propone una normalización y automatización de umbrales los cuales se adaptan de forma automática a la velocidad promedio del reconocimiento de somnolencia y distracción de cada dispositivo, lo que hace que no importe a cuantos fotogramas por segundo (fps) este corriendo el programa, el tiempo de detección y alertas siempre serán el mismo.
- ***Corrección de error de la clasificación dada por la RNC***: Si bien, las RNC tienden a tener una precisión de predicción alta (mayores del 95% en las pruebas de rendimiento), siempre existe un error, el cual para nuestra propuesta y cualquier otro sistema, es sumamente peligroso, por tal motivo, se propone un algoritmo que detecta esta predicción errónea y la corrige en tiempo real, esto hace que el sistema no tenga que volver a reconocer patrones de sueño o distracción peligrosos desde cero, aumentando la seguridad al momento de conducir.

Referencias

[1] Ford. Sistema De Mantenimiento De Carril Con Detector De Fatiga. [en línea] https://www.ford.cl/tecnologia/sistema-mantenimientode-carril/

[2] Volkswagen de México. ¿Sabes cómo funciona el detector de cansancio?, Julio 2020. [video en línea] https://www.youtube.com/watch?v=h24fCx5KCsk

[3] Benedikt S, Hoefle S y Loeffelmann L. Method And Control Unit For Detecting Drowsiness Of A Driver For A Driver Assistance System For A Vehicle. Patente EE.UU. US11292478B2. 5 de Abril de 2022.

[4] Martin-Alvarez AR. Device To Prevent Accidents In Case Of Drowsiness Or Distraction Of The Driver Of A Vehicle. Patente EE.UU. US20080143504A1. 19 de Junio de 2008.

[5] Mimar T. Driver Distraction And Drowsiness Warning And Sleepiness Reduction For Accident Avoidance, Patente EE.UU. US9460601B2, 4 de Octubre de 2016.

[6] Manotas J,JR. Method Of Detecting Drowsiness Of A Vehicle Operator. Patente EE.UU. US20100214105A1. 26 de Agosto de 2010.

[7] Zhang H, Witt GJ, Smith MR. Drowsiness Detection System And Method. Patente EE.UU US7202792B2. 10 de Abril de 2007.

[8] Wu R, Turner J, Browning C, Brummett T. Drowsy Driver Detection System. Patente EE.UU. US8957779B2. 17 de Febrero de 2015.

[9] Eskandarian A, Mortazavi A. Unobtrusive Driver Drowsiness Detection System And Method.Patente EE.UU. US8519853B2. 27 de Agosto de 2013.

[10] Deruyck J, McLaughlin BR. Detection Of Driver Behaviors Using In-Vehicle Systems And Methods. Patente EE.UU. US20160046298A1. 18 de Febrero de 2016.

[11] 成波王珂李家文. Driver Fatigue Detection Method Based On Face Video Analysis. Patente China. CN102254151B. 16 de Enero de 2013.

[12] 谢剑斌刘通李沛秦闫玮. Method For Detecting Fatigue State Of Driver Based On Digital Video. Patente China. CN101950355B. 05 de Septiembre de 2012.

[13] Aluf E, Troyansky L. Detection Of Cognitive State Of A Driver. Patente EE.UU. US20220095975A1. 31 de Marzo de 2022.

[14] Lakhani H, Schaaf T, Nicolich – Henkin LR, DeMatos R, Yu M. Vehicle Voice User Interface. Patente EE.UU. US11273778B1. 15 de Marzo de 2022.

[15] Stefan M, Priya S, Joseph V, Evgeny B. Techniques For Detecting Acknowledgement From A Driver Of A Vehicle. Patentes EE.UU. 08 de MArzo de 2022.

[16] Aleksandar P. Image Capture Device And Associated System For Monitoring A Driver. Patente EE.UU. US20220048386A1. 17 de Febrero de 2022.

[17] Giralt A, Petrov M. Method For Determining A Drowsiness Level Of A Motor Vehicle Driver. Patente EE.UU. US20220027646A1. 27 de Enero de 2022.

[18] YAMAUCHI K, SAWAI S, SANO M, SAKATA T, NEMOTO H. Drowsiness Sign Notification System, Drowsiness Sign Notification Method, And Drowsiness Sign Notification Program. Patente EE.UU. US20220005339A1. 06 de Enero de 2022.

[19] PARK Ge O. System And Method For Monitoring Driver. Patente EE.UU. US20220032923A1. 03 de Febrero de 2022.

[20] Alvarez IJ, Carranza M, Graefe R, Guim bernat F, Martinez-spessot C, Oliver D, Panneer S, Paulitsch M, Rosales R. Driver Monitoring System (Dms) Data Management. Patente EE.UU. US20210403004A1. 30 de Diciembre de 2021.

[21] Delahaye V. Device And Method For Detecting The Distraction Of A Driver Of A Vehicle. Patente EE.UU. US20220076040A1. 10 de Marzo de 2022.

[22] Blacutt S, Flores D, Flores R, Soto M. System And Method For Determining A Level Of Operator Fatigue. Patente EE.UU. US9198575B1. 01 de Diciembre de 2015.

[23] Wu BF, Chen YH, Yeh CH, Li YF. Reasoning-based framework for driving safety monitoring using driving event recognition. IEEE Trans. Intell. Transp. Syst, Septiebre 2013, PP: 1231-1241.

[24] Vidente V, Laguna P, Bartra A, Bailón R. Drowsiness detection using heart rate variability, Med. Biol. Eng. Comput. Junio 2016, PP: 827-937.

[25] Pérez MA. Electronica Industrial. Universidad Técnica Federico Santa María. 2016, PP: 2-15.

[26] Lei F, Liu C, Dai Q, Ling BW-K. Shallow Convolutional Neural Network For Image Classifcation. Diciembre 2019, PP: 1-3.

[27] Sandler M., Howard A., Zhu M., Zhmoginov A., Chen L.: MobileNetV2: Inverted residuals and linear bottlenecks. In: Proc. the IEEE Conference on Computer Vision and Pattern Recognition, pp. 4510-4520. IEEE, Salt Lake City (2018).

[28] Simonyan K., Zisserman A.: Very deep convolutional networks for large-scale image recognition. In: International Conference on Learning Representations (ICLR), San Diego, (2015).

[29] He. K., Zhang X., Ren S., Sun J.: Deep residual learning for image recognition. In: Proc. the IEEE Computer Society Conf. on Computer Vision and Pattern Recognition, pp. 770-778. IEEE, Las Vegas (2016).

[30] Szegedy C., Liu W., Jia Y., Sermanet P., Reed S., Anguelov D., Erhan D., Vanhouckel V., Rabinovich A.: Going Deeper with Convolutions. In: Proc. IEEE Conf. on Computer Vision and Pattern Recognition, pp. 1-9, IEEE, Boston (2015).

[31] Chollet F.: Xception: Deep Learning with Depthwise Separable Convolutions. In: Proc. IEEE Conf. on Computer Vision and Pattern Recognition, pp. 1800-1807, IEEE, Honolulu (2017).

Alcance

El sistema propuesto SOMN_IA tiene como alcance poder detectar si los conductores o usuarios pilotos de un vehículo tienen los patrones principales de fatiga o distracción, con esta información poder activar alarmas sonoras, visuales e informativas, las cuales reduzcan significativamente los accidentes vinculados por estas causas; utilizando un único sensor y que no dependa de muchos parámetros pero que se tenga la misma eficacia, así como también, independizar el sistema y no englobarlo en un sólo dispositivo, haciendo que su uso sea tan práctico, que sea capaz de utilizarse en cualquier situación y/o condición.

Organización De La Tesis

La presente Tesis se ha decidido organizar de la siguiente manera:

- **Capítulo 1.-** MARCO TEÓRICO.
  En este capítulo abordaremos de forma general algunos puntos importantes que se tomaron a consideración para la creación de esta investigación, así como también, se tendrá la base teórica de todo lo que se abordará en esta tesis.
- **Capítulo 2.-** ESTADO DEL ARTE.
  En este capítulo se mencionan algunas investigaciones, prototipos, patentes y demás, que han servido como base en la creación de nuestra propuesta de innovación.
- **Capítulo 3.-** MÉTODO PROPUESTO Y RESULTADOS.
  Este capítulo es el más extenso de toda la tesis, aquí abordaremos desde la creación de la idea general del sistema, pasando por su primera versión, posteriormente se desarrolla su mejoramiento y la implementación en teléfonos inteligentes, finalmente se explica el proceso que se llevó para poder automatizar el sistema, así como la optimización necesaria para su correcto funcionamiento. Cada proceso se va analizando y justificando la razón del porqué se diseñó de cierta manera.
- **Capítulo 4.-** SOMN_IA.
  En este capítulo mostramos el resultado de la fabricación, ensamble y ejecución del prototipo móvil que proponemos para detectar la somnolencia y la distracción en los conductores y a su vez se explica el modo de instalación del software general en los dispositivos de laboratorio (Workstation, PC de escritorio, Laptops, etc.), en las tarjetas dedicadas y finalmente la instalación de la aplicación creada.
- **Capítulo 5.-** CONCLUSIONES GENERALES Y TRABAJOS FUTUROS.
  En este capítulo enlistamos todo el mejoramiento, la investigación e implementación que se trabajará en un futuro y finalmente damos por finalizado esta investigación con las conclusiones generales.

Capítulo 1: Marco Teórico

1.1 Accidentes Automotrices

Los accidentes vehiculares también llamados accidentes automotrices, de tránsito, colisiones o siniestros, son acontecimientos involuntarios que ocurren cuando un vehículo, en este caso automotriz terrestre (automóvil, motocicleta, camiones, tráileres, etc.) se encuentra y ocasiona un choque contra diversos elementos viales tales como otros vehículos, peatones, bardas, postes, entre otros [1]. Los accidentes automotrices [2], por lo general o en su gran mayoría son eventos atribuidos a factores humanos que se producen por irresponsabilidad o negligencia de estos, por ende, estos tipos de accidentes se pueden evitar; si bien, los incidentes son casi siempre por errores humanos, otros elementos los cuales pueden ocasionar estos siniestros son fallas mecánicas, condiciones climatológicas, mala construcción o imperfecciones en los caminos o carreteras, señalización, entre otros.

Existen diversos accidentes vehiculares, por tal motivo, se han clasificado en los siguientes:

- ***Accidentes por colisiones (entre dos vehículos)***: Las colisiones vehiculares son producidas cuando dos o más vehículos se encuentran y ocasionan un accidente, esto es diferente a los choques, debido a que estos se producen con objetos estáticos; este tipo de accidentes, normalmente son ocasionados por negligencias o irresponsabilidades de conductor, como distracción, no respetar los límites de velocidad o las señalizaciones, conducir en estados no aptos, etc. Existen diversos tipos de colisiones los cuales son:
  - ***Colisiones por alcance[3]:*** Este tipo de colisiones son las más comunes, se refiera a un incidente que ocurre cuando un conductor golpea la parte trasera de otro vehículo, esto debido a que no pudo frenar a tiempo, ocasionando daños a un tercero, normalmente este tipo de accidentes tienen como consecuencia en ambos conductores, lesiones en cuellos, espalda, caderas, etc., así como también perdidas monetarias y/o de patrimonio.



Figura 1. 1: Diferentes tipos de colisiones por alcance[1]

[1] Figura tomada de ACCIDENTES DE TRÁFICO, Boiso Paco, 2007, http://pantha-rei.blogspot.com/2007/02/colisiones-por-alcance-grfico-4.html

- ***Colisión Frontal[1,4]:*** Este es una de las colisiones más graves y mortales, debido a que es un impacto delantero entre dos conductores, los cuales llevan una velocidad, lo cual hace que se más aparatosos, por su encuentro de lleno entre dos vehículos, esto aumenta la posibilidad de que el conductor tenga consecuencias graves como lesiones muy severas y tienden a tener un índice de mortalidad alto.



Figura 1. 2: Colisión Frontal[2]

- ***Colisión Lateral[3]:*** Este tipo de incidentes son causados cuando un conductor golpea alguna parte de la lateral del otro vehículo, propiciando diversas acciones por una acción-reacción, como derrapes, disparos en dirección del auto que impacta, hasta volcaduras. En este tipo de colisiones, se divide en 4 principales:
  1. ***Colisión embestida o fronto-lateral[4]:*** Este tipo de colisiones se producen cuando un vehículo embiste con la parte frontal o trasera a otro en alguno de sus laterales, dicho percance crea un grado de 90º.

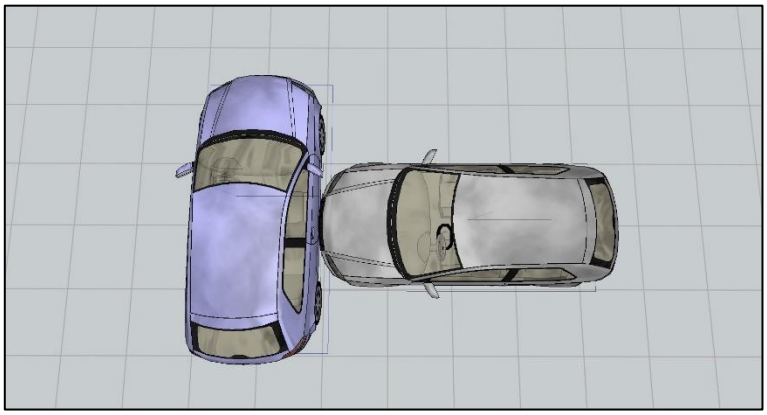


Figura 1. 3: Colisión embestida o fronto-lateral[3]

[2] Figura tomada de Imagen Vectorial Choque frontal del coche, depositphotos, https://mx.depositphotos.com/416116614/stock-illustration-frontal-car-collision-trouble-situation.html
[3] Figura tomada de Píldoras De Reconstrucción De Accidentes De Tráfico Iv- Tipos De Colisión Entre Vehículos En Accidentes De Tráfico, IPSUM, 2019, https://reconstruccionaccidentestrafico.com/tipos-de-colision-entre-vehiculos-en-accidentes-trafico/

2. ***Colisión lateral por raspado[4]:*** Se denomina así, a la colisión entre los laterales de los vehículos participes en este tipo de accidentes.
   - ***Colisión lateral por raspado negativo[4,5]:*** Es cuando los vehículos circulan en el mismo sentido.

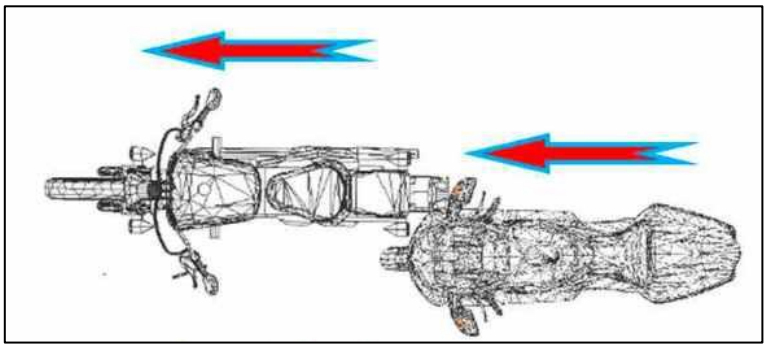


Figura 1. 4: Colisión lateral por raspado negativo[4]

   - ***Colisión lateral por raspado positivo[4,5]:*** Es cuando los vehículos circula en sentido contrario.

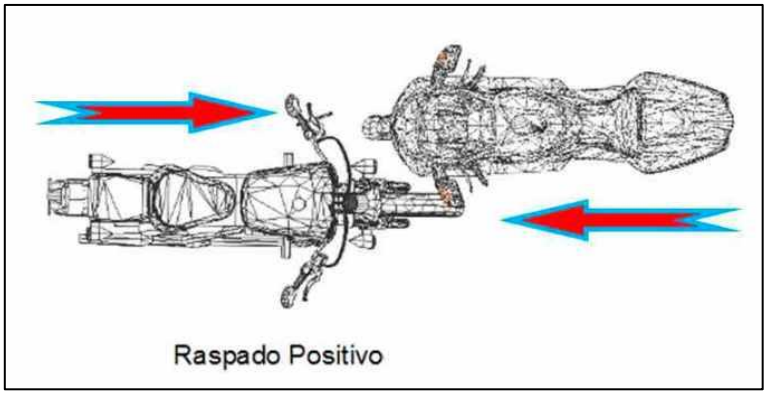


Figura 1. 5: Colisión lateral por raspado positivo[5]

Las causas por las cuales estos tipos de colisiones son provocados, principalmente son por descuidos del conductor, aunque en ocasiones, son provocados por condiciones meteorológicas o errores mecánicos.

- ***Atropellos o Arrollamientos [4]:*** Se define como atropellamiento o arrollamiento, a la acción que sucede cuando un conductor golpea, embiste

[4] Figura tomada de Sobre La Definición De Las Distintas Clases De Accidentes De Tráfico Por El Modo En Que Se Producen, Acinves, 2012, https://acinves.wordpress.com/2012/03/01/sobre-la-definicion-de-las-distintas-clases-de-accidentes-de-trafico-por-el-modo-en-que-se-producen/
[5] Figura tomada de Sobre La Definición De Las Distintas Clases De Accidentes De Tráfico Por El Modo En Que Se Producen, Acinves, 2012, https://acinves.wordpress.com/2012/03/01/sobre-la-definicion-de-las-distintas-clases-de-accidentes-de-trafico-por-el-modo-en-que-se-producen/

o pasa por encima con el vehículo a uno o varios peatones; en dichos accidentes, el que sale menos dañado es el conductor debido a que el automóvil lo protege en gran medida, en cambio, el que llega a resultar con heridas graves o hasta mortales, son las personas quienes recienten esta acción. En la gran mayoría, al igual que en las colisiones, este tipo de accidentes se puede evitar y son principalmente ocasionadas por negligencias de los conductores. Existen diversas clasificaciones para los atropellos o arrollamientos:

1. ***Wrap Projection [4]:*** Es el tipo de arrollamiento más común, en resumen, es cuando el vehículo golpea al peatón con la parte frontal del vehículo, dependiendo la velocidad en que el automóvil viaja, es la reacción que se tiene, si va a baja velocidad y logra detenerse el automotor, será proyectado directamente hacia adelante, si el vehículo va a gran velocidad y no logra detenerse, el peatón quedara por debajo del automóvil.



Figura 1. 6: Atropellamiento Wrap Projection[6]

2. ***Forward Projection [4]:*** Este tipo de accidentes es muy parecido a ***Wrap Projection,*** la diferencia radica en el tipo de vehículo que lo ocasiona, en ***Wrap Projection*** es ocasionado por autos relativamente pequeños (sedanes, hatchbacks, SUV, etc.), pero en ***Forward Projection*** lo ocasiona vehículos de perfil plano, como camiones o tráileres.

[6] Figura tomada de ¿Conoces la mecánica del atropello? Según caes, así te dieron, PERITOS DE ACCIDENTES, 2017, https://www.peritosdeaccidentes.com/conoces-mecanica-del-atropello/



Figura 1. 7: Atropellamiento Forward Projection[7]

3. ***Fender Vault [4]:*** Este tipo de arrollamiento se refiere a accidentes en donde el peatón se impacta directamente con algún perfil del automóvil, dependiendo la velocidad, será la gravedad de este tipo de accidente; hay muy poca probabilidad en que el conductor salga proyectado hacia arriba o se quede por debajo del vehículo.

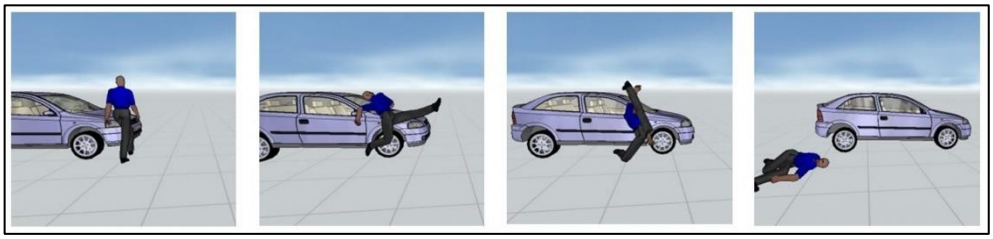


Figura 1. 8: Atropellamiento Fender Vault[8]

4. ***Roof Vault [4]:*** Este tipo de colisión entre vehicular-peatón es muy común, y tiene los mismo principio que ***Wrap Projection***, la diferencia radica en que aquí, la velocidad es alta, y los peatones salen proyectados hacia arriba, haciendo uno de los accidentes viales más peligrosos.



Figura 1. 9: Atropellamiento Roof Vault[9]

[7] Figura tomada de Mecánica Del Atropello. Pistas Que Ayudan Al Perito En La Reconstrucción Del Accidente De Tráfico, IPSUM, 2015, https://reconstruccionaccidentestrafico.com/mecanica-del-atropello-pistas-que-ayudan-al-perito-en-la-reconstruccion-del-accidente-de-trafico/
[8] Figura tomada de Mecánica Del Atropello. Pistas Que Ayudan Al Perito En La Reconstrucción Del Accidente De Tráfico, IPSUM, 2015, https://reconstruccionaccidentestrafico.com/mecanica-del-atropello-pistas-que-ayudan-al-perito-en-la-reconstruccion-del-accidente-de-trafico/
[9] Figura tomada de Video registra brutal atropellamiento a motociclista en Monterrey, Casas David, 2018, https://www.elsoldemexico.com.mx/republica/sociedad/video-registra-brutal-atropellamiento-a-motociclista-en-monterrey-1100679.html

- ***Pérdida de control[6]:*** Este tipo de accidentes son en los que intervienen más elementos para que pueda ser ocasionado, pero no está exento a que muchas veces son causados por negligencias del conductor, como no respetar las señalizaciones de velocidad, distracciones, quedarse dormido, etc., si no que muchas veces son ocasionados por condiciones meteorológicas como perdida de fricción en el camino por lluvia o nieve, debido a que en estas condiciones, se disminuye el amarre de las llantas a la carretera; otro factor importante, son las fallas mecánicas.



Figura 1. 10: Pérdida de control[10]

[10] Figura tomada de Qué hacer si pierdes el control de tu coche, Seat, 2018, https://www.lavanguardia.com/motor/actualidad/20180801/451082816116/que-pierdes-control-coche.html#foto-1

Tabla 1. 1: INEGI; Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas; Accidentes por tipo de accidente [13]

Variable	Entidad	2016	2017	2018	2019	2020
Total de eventos	Nacional	360,051.0	367,789.0	365,281.0	362,729.0	301,678.0
Colisión con vehículo	Nacional	65.5%	65.6%	65.2%	63.5%	59.6%
Colisión con peatón	Nacional	3.7%	3.6%	3.5%	3.5%	3.1%
Colisión con animal	Nacional	0.4%	0.3%	0.3%	0.3%	0.3%
Colisión con objeto fijo	Nacional	13.1%	12.9%	12.8%	12.7%	14.0%
Volcadura	Nacional	2.4%	2.3%	2.4%	2.4%	2.8%
Caída de pasajero	Nacional	0.6%	0.5%	0.5%	0.5%	0.5%
Salida del camino	Nacional	2.2%	2.3%	2.4%	2.6%	2.9%
Incendio	Nacional	0.1%	0.1%	0.1%	0.1%	0.1%
Colisión con ferrocarril	Nacional	0.1%	0.1%	0.1%	0.1%	0.1%
Colisión con motociclista	Nacional	9.1%	9.5%	10.2%	11.8%	13.5%
Colisión con ciclista	Nacional	1.4%	1.3%	1.2%	1.3%	1.2%
Otros accidentes	Nacional	1.3%	1.5%	1.3%	1.2%	1.9%

Los accidentes vehiculares son un gran problema para el mundo, pero específicamente para México, es alarmante, debido a que el país ocupa el séptimo lugar a nivel mundial y en tercero en Latinoamérica por muertes provocados por accidentes vehiculares, de acuerdo con el Instituto Nacional de Salud Pública [7]. Se sabe que entre 2013 y 2015, el 80% de los accidentes son causado por negligencias o irresponsabilidades del conductor, el 9% por agentes naturales, el 7% por problemas mecánicos, y el restante por fallas en el camino, todo esto de acuerdo con los reportes generales de la Policía federal[8].

Existen diversas causantes de los accidentes vehiculares, los cuales de acuerdo con la Comisión Nacional de Seguridad (CNS) [8], las principales son:

- Conducir bajo los efectos del alcohol, medicina o estupefacientes.
- Realizar maniobras imprudentes y de omisión por parte del conductor
- Conducción a exceso de velocidad.

- Condición física del conductor.
- Conducir con somnolencia, cansancio o con sueño.

Según varios informes estadísticos, como [10,11], más del 30% de los accidentes de tránsito son causados por somnolencia (33%) o distracción del conductor (33%). Varias circunstancias, como la falta de sueño del conductor, la fatiga física o mental del conductor, el estado monótono de la carretera, etc., se consideran factores principales de la somnolencia del conductor. Por otro lado, la generalización del uso del teléfono celular durante la conducción provoca la distracción del conductor y luego la falta de alerta a la situación que lo rodea. En México ocurren aproximadamente 16,500 accidentes mortales al año, cuya carga económica llega a ser de unos 150 mil millones de pesos mexicanos, según el informe del ***CONAPRA*** [12].

Tabla 1. 2: INEGI; Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas; Accidentes por causa [14]

Variable	Entidad	2016	2017	2018	2019	2020
Total, de eventos	Nacional	360,051.0	367,789.0	365,281.0	362,729.0	301,678.0
Conductor	Nacional	91%	91%	92%	92%	95%
Peatón o pasajero	Nacional	1%	1%	1%	1%	1%
Falla del Vehículo	Nacional	1%	1%	1%	1%	1%
Mala condición del camino	Nacional	3%	3%	3%	3%	2%
Otras causas	Nacional	4%	4%	3%	3%	1%

1.2 Fatiga, Somnolencia y Distracción

1.2.1 Fatiga[15]

La fatiga clínicamente es un trastorno por cansancio extremo por falta de energía y de motivación, puede ser una respuesta consecuente del esfuerzo físico, al estrés emocional, al aburrimiento o a la falta de sueño.

La fatiga es uno de los síntomas más comunes en medicina clínica en donde se manifiestan diversos síndromes sistemáticos, neurológicos y psiquiátricos.

Si bien, la fatiga puede ser considerado como un trastorno consecuente del cansancio, no debe confundirse con la somnolencia, debido a que en muchas ocasiones, la somnolencia es directamente propiciada por la fatiga; existen diversas causas que provocan la fatiga, tales como:

- ***Anemia[14,15]:*** La anemia, sobre todo la crónica progresiva tiende a manifestar fatiga, en donde en ocasiones son muy relacionadas con taquicardias. Las bajas

concentraciones de séricas de ferritina en ausencia de la anemia pueden causar fatiga que se corrigen administrando dosis de hierro.

- ***Depresión[14,15]:*** Los trastornos mentales a menudo son causantes de un estrés psicológico constante en el individuo, dicho trastorno puede aumentar el cansancio de tal manera que este puede evolucionar a fatigas crónicas, por la acumulación de esfuerzo continuo.
- Medicamentos, sedantes o antidepresivos. [14,15]
- Trastornos del sueño, como insomnio, apnea obstructiva o narcolepsia: [14,15]



Figura 1. 11: Fatiga[11]

1.2.2 Somnolencia [17]

La somnolencia es la tendencia de sentirse cansado o con mucho sueño durante todo el día; las personas que tienden a sentirse muy agotados o en todo caso soñolientos pueden quedarse dormidas en situaciones o momentos inapropiados, otra definición podría ser, que la somnolencia es la tendencia del ser humano a quedarse dormido, es una necesidad biológica, la cual es necesaria para su transición de la vigilia al sueño.

Actualmente la somnolencia es un gran problema, debido al crecimiento exponencial de los fenómenos digitales, los cuales son una de las causas en donde la privación del sueño se ve más a menudo, debido a la gran demanda de tiempo que los usuarios emplean, las redes sociales, son una de las principales privaciones de sueño en los jóvenes; algunos estudios revelan que entre el 0.3% y el 25% de la población mundial sufren de somnolencia.

Por otra parte, la somnolencia es la segunda fase de un sueño normal, denominada ***SUEÑO NO MOR [18]***; que es cuando el usuario está teniendo patrones de cansancio

[11] Figura tomada de Por qué no debería reprimir su próximo bostezo, Salud, Nutrición y Bienestar, https://www.saludnutricionbienestar.com/por-que-no-deberia-reprimir-su-proximo-bostezo/

y empieza la fase denominada sueño ligero, en esta fase, el cansancio se encuentra evidente, los parpados se sienten pesados y los músculos se empiezan a relajar, en esta etapa es muy fácil de despertar al individuo.

Las causas principales del a somnolencia es la privación del sueño, como consecuencia de narcóticos o medicamentos como antidepresivos, así como trastornos como la fatiga, la depresión o la rinitis alérgica; adicionalmente la carga de trabajo en la actualidad, apuntan como una de las principales causas por las cuales la somnolencia en las personas va en aumento.



Figura 1. 12: Conductor quedándose dormido por somnolencia[12]

1.2.3 Distracción [19]

Existen diversas definiciones sobre distracción, en general, la distracción se puede delimitar como la no atención a una cosa, acción o tarea; en el ámbito automotriz, la acción de distracción es cuando el conductor deja de prestar atención al camino, porque está haciendo algo en paralelo, como voltear hacia algún lado en un tiempo excesivo, manipular objetos, artefactos electrónicos, hacer llamadas, platicar con alguien, prestar atención a sucesos externos, etc.

[12] Figura tomada de 9 tips para no dormirse conduciendo, MejorconSalud, https://mejorconsalud.as.com/9-tips-no-dormirse-conduciendo/



Figura 1. 13: Conductor distraído por el uso de un teléfono móvil[13]

1.3 Conclusiones del capítulo

Los accidentes automovilísticos son uno de los principales motivos de decesos en el mundo y para México es un problema a tener en cuenta, ya que se ocupa el tercer lugar a nivel mundial en incidentes terrestres y aunque si bien, muchos son causados de forma indirecta, ya sea por imperfecciones en el camino, fallas mecánicas o bien condiciones climatológicas no favorables de acuerdo con la INEGI, la mayoría de estas peripecias se pueden evitar (alrededor del 90%); partiendo de esta premisa, poder detectar y prevenir los incidentes con mayor tasa de causantes es fundamental, por ello combatir la somnolencia y la distracción que son la causantes más comunes ya que son las responsables de más del 60% de estos, se es una enorme aportación, por lo tanto, tener las características principales, de la somnolencia, así como también de la distracción, nos da la pauta y el principio para poder prevenirlas y así poder reducir en gran medida los siniestros descritos con anterioridad.

[13] Figura tomada de ¿Cómo Evitar las Distracciones al Conducir? Sigue estos Consejos, CONECTATE.com.do, 2021, https://www.conectate.com.do/articulo/como-evitar-las-distracciones-al-conducir/

1.4 Referencia del capítulo

[1] INGETAX, [en línea] https://www.ingetax.com/colisiones-entre-vehiculos-que-tipos-existen/.

[2] Pérez-Porto J, Gardey A. Accidente De Tránsito. 2015-2016. [en línea] https://definicion.de/accidente-de-transito/

[3] Giles E. Accidentes Automovilísticos ¿Cómo evitarlos?. Mayo 2017. [en línea] https://www.crabi.com/blog/consejos-prevenir-accidentes-automovilisticos.

[4] IPSUM. Píldoras De Reconstrucción De Accidentes De Tráfico Iv- Tipos De Colisión Entre Vehículos En Accidentes De Tráfico. Diciembre 2019. [en línea] https://reconstruccionaccidentestrafico.com/tipos-de-colision-entre-vehiculos-en-accidentes-trafico/.

[5] Acinves. Sobre La Definición De Las Distintas Clases De Accidentes De Tráfico Por El Modo En Que Se Producen. Marzo 2012. [en línea] https://acinves.wordpress.com/2012/03/01/sobre-la-definicion-de-las-distintas-clases-de-accidentes-de-trafico-por-el-modo-en-que-se-producen/

[6] Wikipedia. Accidente de tránsito. Mayo 2022. [en línea] https://es.wikipedia.org/wiki/Accidente_de_tr%C3%A1nsito#Tipos_de_incidentes_de_tr%C3%A1nsito.

[7] Reglamento de tránsito de la CDMX. [en línea] https://tramites.cdmx.gob.mx/cursos/public/curso-intermedio/0/4#:~:text=De%20acuerdo%20con%20el%20Instituto,en%20muertes%20por%20hechos%20viales.

[8] Comisión Nacional de Seguridad. Accidentes Y Sus Factores. 2010. [en línea] http://www.cns.gob.mx/portalWebApp/appmanager/portal/desk?_nfpb=true&_pageLabel=portals_portal_page_m2p1p2&content_id=830068&folderNode=830052&folderNode1=810277

[9] Promocar. Causas Principales de los Accidentes Automovilísticos. Abril 2022. [en línea] https://promocar.com.mx/causas-principales-accidentes-automovilisticos/#:~:text=En%20cuanto%20a%20que%20los,naturales%20y%204%25%20al%20camino.

[10] Traffic Safety Culture Index, Foundation for Traffic Safety, Junio 2020, https://aaafoundation.org/2019-traffic-safety-culture-index.

[11] Chacon-Murguia, M., Prieto-Resendiz C. Detecting driver drowsiness: A survey of system designs and technology. IEEE Consum. Electron. Mag. 2015 Abril; PP: 107-119.

[12] Páez M., Abarca E.: Tools for security in movements, Predictive models of driver's drows-iness, [en línea] https://imt.mx/resumen-boletines.html?IdArticulo=449&IdBoletin=168.

[13] INEGI, "Accidentes de tránsito terrestre en zonas urbanas y suburbanas", [en línea] https://www.inegi.org.mx/sistemas/olap/consulta/general_ver4/MDXQueryDatos.asp?#Regreso&c=13159

[14] INEGI. Accidentes de Tránsito Terrestre en Zonas Urbanas y Suburbanas; Accidentes por causa. [en línea] https://www.inegi.org.mx/sistemas/olap/consulta/general_ver4/MDXQueryDatos.asp?proy=atus_accidentes

[15] Instituto Nacional del Cáncer. Fatiga. [en línea] https://www.cancer.gov/espanol/publicaciones/diccionarios/diccionario-cancer/def/fatiga

[16] Gelfand JM, Douglas VC. Principio de Medicina Interna. Harrison. Cap 29. PP: 1-5.

[17] Mayor ER, Rey de Castro Mujica J. Somnolencia: Qué es, qué la causa y cómo se mide. 2010. PP: 137-139.

[18] Carrillo-Mora P, Barajas-Martínez KG, Sánchez-Vázquez I, Rangel-Caballero MF. Trastornos del sueño: ¿qué son y cuáles son sus consecuencias?. Febrero 2018, PP: 2-5

[19] Montes SA, Ledesma RD, Poó FM. Estudio y prevención de la distracción e inatención en la conducción, Octubre 2013, PP: 1-3

Capítulo 2: ESTADO DEL ARTE

2.1 Introducción

Los accidentes automovilísticos cada año aumentan, esto hace que los sistemas de monitoreo se conviertan en una necesidad, más que un lujo, por ello, en la actualidad, existen diversas formas de prevenir los incidentes vehiculares, por lo que va desde la construcción segura de las carreteras, hasta sistemas enfocados específicamente para poder detectar y prevenir un cierto tipo de causante, por tal motivo, en este capítulo, se abordaran todo aquella investigación o desarrollo que se ha implementado en la actualidad para reducir el índice de mortalidad de las peripecias terrestres.

2.2 Detección de somnolencia y distracción en los conductores

En la actualidad existen diversas metodologías las cuales son utilizadas para poder detectar cuando un conductor presta o no atención al camino, como también poder determinar el nivel de somnolencia que tiene el usuario cuando se está conduciendo. Investigando, encontramos diversas innovaciones, sistemas o subsistemas las cuales pueden ayudar en el monitoreo o en la reducción de accidentes por esto motivos, adicionalmente se encontraron elementos que de forma indirecta pueden combatir los incidentes vehiculares por estas causas, por tal motivo, hemos decidido clasificar este tipo de prevenciones en dos principales.

1. ***Prevención indirecta:*** Las prevenciones indirectas, son técnicas que se utilizan para minimizar los accidentes vehiculares, por lo general son consecuencias de un mejoramiento o de una invención que no tiene como propósito principal el monitoreo o la detección propia de la somnolencia o la de la fatiga, las cuales son las siguientes:
    - ***Prevenciones indirectas externas:*** Se refiere a todo lo que sea externo o que no esté integrado al vehículo, más específicamente, a las construcciones de las carreteras o caminos, debido a que muchas de estas, en la actualidad están pensadas y diseñadas para reducir el cansancio y el aburrimiento en grandes recorridos, por tal motivo, se evita tener tramos rectos muy prologados, además que en gran parte de estas se agregan alertas tales como iluminaciones, vibradores, casetas o tramos curvos, los cuales ponen en alerta al conductor.
    - ***Prevención indirecta interna:*** Las prevenciones que particularmente entran esta sección, son todas aquellas mejoras he innovaciones que constantemente están presenten en los automóviles, los cuales podrían ser los mejoramientos de sistemas de transmisión[1,2], de dirección[3,11], confort en el habitáculo, sistema interno se monitoreo en carros de alta gama, tales como los Sistemas Avanzados de Asistencia al Conductor (ADAS) [12] que proporcionan varias funcionalidades para ayudar al

usurario, como sensores de aparcamiento, advertencia de cambio de carril, reconocimiento de señales de tráfico, sistema de información de ángulo muerto, etc. Todos esto subsistemas que si bien ayudan al usuario a operar de mejor manera los vehículos, no tiene como principal motivo solo detectar la somnolencia o distracción, sino que es una consecuencia de la constante actualización y desarrollo de las grandes empresas.

2. Prevención directa: Este tipo de prevenciones, las hemos denominado así, debido a que son innovaciones, invenciones o sistemas que tiene como propósito principal el estudio o monitoreo de la somnolencia, distracción o ambos, por lo que, considerando estas problemáticas, se han propuesto varios enfoques los cuales se clasifican en tres categorías que dependen de la información de entrada utilizada para el proceso de detección.
   - La primera categoría es el sistema basado en el comportamiento del vehículo o basado en el patrón de conducción, en el que los datos obtenidos del vehículo, como la velocidad, la aceleración, el ángulo de dirección y la desviación de carril, se utilizan para detectar la condición anormal del conductor [4, 5].
   - La segunda categoría utiliza las señales fisiológicas del conductor [6, 7], como electrooculograma (EOG), electrocardiograma (ECG), electromiograma (EMG) y electroencefalograma (EEG). Aunque estas señales pueden relacionarse directamente con el grado de condición fisiológica del conductor, como la fatiga y la distracción, el conductor debe ponerse los sensores portátiles para la adquisición de la señal, que es invasiva para el conductor y un obstáculo para conducir.
   - La tercera categoría es el sistema basado en la visión, en el que la cámara de video captura la cara del conductor o la imagen del ojo del conductor, y se mide la frecuencia de parpadeo, la duración del cierre de los párpados, la frecuencia de bostezos y movimientos de cabeza, etc. para determinar la somnolencia del conductor [8-10].

   Para evitar accidentes de tránsito causados por la somnolencia del conductor, en la construcción de la autopista, se evita intencionalmente un camino plano y recto.

2.3 Investigaciones previas

A continuación describiremos diversas investigaciones que sirvieron como base, para el desarrollo de la prepuesta general.

1. **A Hybrid Driver Fatigue and Distraction Detection Model using AlexNet based on Facial Features[13]:** En términos generales, se propone un sistema el cual utiliza aprendizaje profundo, con una arquitectura libre para extraer

características de imágenes dadas por los videos de entrada. El autor generó dos diferentes bases de datos.

I. La primera base de datos contiene imágenes con tres diferentes posiciones del rostro tales como: "Sin rostro", "Rostro volteando" y "Rostro estático".

II. La segunda es una base de datos con otras tres características diferentes pero ahora, el autor se enfocó en la boca del conductor, tales como: "boca cerrada", "hablando o riendo" y "bostezando".

Usando aprendizaje profundo y, con ayuda de la base de datos de la cabeza, el autor clasifica 3 estados del conductor que se denominan como: "cansancio", "distracción" y "no determinado". Las imágenes clasificadas como "no determinado" se le aplica un segundo algoritmo de aprendizaje profundo y con ayuda de las imágenes de la segunda base de datos de la boca, el autor determina si se está "despierto" o "cansado"

2. ***Driver Drowsiness Detection by Applying Deep Learning Techniques to Sequences of Images [14]:*** En este artículo se proponen dos sistemas, en donde ambos contienen dos módulos de procesamiento de información, análisis de patrones y resultados; finalmente la activación de una alarma.

- Primer Sistema:
  - Procesamiento: Implementación de algoritmos para la eliminación de ruido, detección de rostro por medio de HOG y SVM, modificación de dimensiones y aclaración de la imagen de entrada por medio de la igualación de histograma.
  - Análisis: Transferencia de aprendizaje profundo a una arquitectura de acceso libre de para la clasificación de "somnoliento" y "despierto".
  - Activación de alarma: Comparación del grado de somnolencia con un valor umbral designado para activación de una alarma.
- Segundo Sistema:
  - Procesamiento: Se utilizaron "Puntos de referencia en el rostro" para calcular el número de parpadeos, bostezos, micro sueños, duración de bostezo, duración de parpadeos.
  - Análisis: Inferencias difusas basadas en reglas difusas.
  - Activación de alarma: Dependiendo del resultado de inferencia difusa, el sistema determina si activar o no la alarma.

3. ***Estimation of driver vigilance status using real-time facial expression and deep learning[15]:*** El sistema propuesto por el autor utiliza algoritmos de reconocimiento facial para poder identificar la región a estudiar (el rostro del conductor), dicho algoritmo es un extractor de caracterizas por patrones Haar. Cuando se detecta la región, dicho fragmento de la imagen pasa por un algoritmo de aprendizaje profundo para detectar la somnolencia del usuario.

4. ***Driver Safety Development: Real-Time Driver Drowsiness Detection System based on Convolutional Neural Network[16]:*** El sistema que propone el autor, detecta la posición de los ojos utilizando "Puntos de referencia en el rostro" y se segmenta la región la región de ambos, a la cual se le implementa un algoritmo de aprendizaje profundo para detectar la somnolencia de conductor.

2.4 Patentes/prototipos previos

A continuación, describiremos diversas patentes o proyectos que sirvieron como base, para el desarrollo de la prepuesta general.

1. ***Method and control unit for detecting drowsiness of a driver for a driver assistance system for a vehicle[17]:*** Es un método para detectar la somnolencia de un conductor empleando un sistema de asistencia, incluye la lectura de al menos una primera señal indicadora que representa un primer parámetro de cansancio del usuario determinado por un primer dispositivo y un segundo indicador que representa un segundo parámetro determinado por un segundo dispositivo de detección y opcionalmente una tercera señal indicadora que representa un tercer parámetro de letargo del conductor determinado por un tercer dispositivo de detección de somnolencia del vehículo.

   Todo los parámetros y resultantes que anteriormente se mencionan determinan la validez de las señales indicadoras y con ello se determina la somnolencia.

2. ***Device to prevent accidents in case of drowsiness or distraction of the driver of a vehicle[18]:*** La invención se refiere a un dispositivo para la prevención de accidentes en el caso de somnolencia del conductor de un vehículo. El dispositivo de la invención comprende una serie de sensores que están implementados en el volante del vehículo para detectar el agarre por parte del conductor y el pulso del conductor. Los mencionados sensores están conectados a una unidad de control que está equipada con la programación y/o circuitería necesaria para activar un indicador sonoro en caso de que el volante se suelte con ambas manos y/o el pulso del conductor caiga por debajo del umbral. El dispositivo emplea un interruptor de apagado.

3. ***Driver distraction and drowsiness warning and sleepiness reduction for accident avoidance[19]:*** La presente invención se refiere a un dispositivo telemático de vehículos para la supervisión del conductor para evitar accidentes por somnolencia y condiciones de distracción. La distracción y la somnolencia se detectan mediante el procesamiento facial del rostro del conductor y el seguimiento de la pose en función de la velocidad y la distancia de viaje máxima permitida, y emitiendo una alerta al conductor cuando se detecta una condición de somnolencia o distracción. La mitigación incluye alertas audibles, así como otros

métodos como una luz azul tenue para animar al conductor (imagen mostrada en una pantalla). También se realiza el centro de adaptación de la dirección de la mirada del conductor y el tiempo máximo permitido para un conductor dado y el desplazamiento del ángulo de la cámara, así como el desplazamiento temporal para las curvas para cambiar el punto de fuga y otras condiciones.

4. ***Method of detecting drowsiness of a vehicle operator[20]:*** Es un método para rectificar la somnolencia del conductor de un vehículo que incluye capturar una secuencia de imágenes del conductor. Se determina, en base a las imágenes, si la cabeza del conductor se inclina alejándose de una orientación vertical en una dirección sustancialmente lateral hacia un hombro del conductor. El conductor se despierta con estímulos sensoriales solo si se determina que la cabeza se inclina alejándose de una orientación vertical en una dirección sustancialmente lateral hacia un hombro del conductor.
5. ***Drowsiness detection system and method[21]:*** La invención se refiere a un método y dispositivo que utiliza un sensor optoelectrónico en combinación con una unidad electrónica instalada en el interior de un vehículo automóvil. El sensor se ajusta en línea con la cabeza del conductor sentado en dicho vehículo automóvil al mismo tiempo que el espejo retrovisor interior, que comprende un espejo unidireccional detrás del cual se coloca el sensor. Tras detectar la presencia de un conductor sentado en el vehículo automóvil encuadran primero todo el rostro y luego los ojos del conductor en la señal de vídeo laser emitida por el sensor asistido por la unidad electrónica y luego determinan las duraciones sucesivas de los episodios de parpadeo, que se comparan con un valor límite situado entre una duración típica de una persona despierta y otra duración típica de una persona somnolienta.
6. ***Drowsy driver detection system[22]:*** Es un método para detectar el deterioro de un conductor en un vehículo.

   El método incluye detectar, usando un sensor, una posición de la cabeza del conductor en una pluralidad de puntos de tiempo; se determina utilizando un microprocesador los cambios en la posición de la cabeza del usurario entre la pluralidad de puntos de tiempo; se evalúa utilizando el mismo microprocesador, si los cambios en la posición de la cabeza entre la pluralidad de puntos de tiempo exhiben al menos uno de un patrón periódico y cuasi periódico; el sistema propuesta determina si el piloto no está apto para la conducción basándose en el patrón de los cambios en la posición del rostro; y si se está incapacitado, se alertará mediante una alarma sonora.
7. ***Unobtrusive driver drowsiness detection system and method[23]:*** Es un sistema y método de detección de somnolencia utiliza el procesamiento de señales de

descomposición de modo empírico (EMD) para detectar si el conductor de un vehículo está somnoliento.

El sistema utiliza un componente descompuesto de la señal del volante para extraer características específicas que representan las fases de degradación del control de dirección, posteriormente se clasifican las características medidas en estado de alerta o somnolencia.

El sistema de detección es independiente de la geometría de la carretera y compensa automáticamente la variabilidad del rendimiento del control de la dirección entre los conductores.

El sistema es preciso en la detección de períodos de somnolencia y cambios de carril relacionados con la somnolencia.

El sistema de detección es discreto y se puede aplicar en línea.

8. ***Detection of driver behaviors using in-vehicle systems and methods[24]:*** Es un sistema que incluye un sensor de movimiento configurado para su despliegue en la cabina de un vehículo y para generar, sustancialmente en tiempo real, un mapeo digital del movimiento del conductor durante la operación del vehículo. Se configura una computadora para su implementación en la cabina y se acopla al sistema de sensor de movimiento. Un detector de comportamiento del conductor está acoplado a la computadora y configurado para detectar un evento de distracción del conductor utilizando el mapeo de movimiento del conductor. Un dispositivo de comunicación en el vehículo está configurado para comunicar un mensaje de alerta a uno o ambos de un dispositivo de interfaz de usuario en la cabina y un sistema remoto en respuesta al evento de distracción del conductor detectado.
9. ***Driver fatigue detection method based on face video analysis[25]:*** La invención se refiere a un método de detección de fatiga del conductor basado en análisis de vídeo facial, perteneciente al campo de la seguridad activa de vehículos y reconocimiento de modo. El método comprende los pasos de: llevar a cabo la detección de rostros en una imagen de rostro a detectar para posicionar principalmente órganos locales, tales como ojos, nariz y boca; obtener además puntos característicos de la cara colocados con precisión; y finalmente, sobre la base de un resultado de posicionamiento preciso de múltiples cuadros de puntos característicos de la cara a detectar, llevar a cabo una descripción de cuantificación en una característica de movimiento de la cara y obtener el resultado de la detección de fatiga en un conductor a detectar de acuerdo con un movimiento de la cara indicador estadístico. El método de detección de fatiga del conductor tiene una alta confiabilidad, no tiene requisitos especiales en el hardware, no implica una operación compleja, y tiene buena instantaneidad para ser aplicado en tiempo real en la práctica. El método de detección de fatiga del conductor tiene una mayor

solidez en los aspectos del uso de anteojos, ángulos de la cara, iluminación no uniforme y similares, y puede ser adecuado para varios tipos de vehículos para recordar al conductor que está en un estado de fatiga para evitar accidentes de tráfico.

10. ***Method for detecting fatigue state of driver based on digital video[26]:*** La invención divulga un método para detectar un estado de fatiga de un conductor basado en un video digital. El método comprende los siguientes pasos: en primer lugar, se realiza la detección y posicionamiento de un rostro humano en base a imágenes adquiridas por una cámara; después de posicionar el rostro humano, se busca en el área de los ojos y el área de la boca; y basándose en el área de los ojos y el área de la boca encontrados, se juzgan el estado de los ojos y el estado de la boca para juzgar si el conductor está en estado de conducción fatigado o no. En la invención, se adoptan el algoritmo de detección de rostros de alta precisión y el algoritmo de detección de fatiga de alta fiabilidad, de modo que se puede realizar la función de detección del estado de fatiga de alta precisión y fiabilidad.

2.5 Conclusiones del capítulo

Prevenir accidentes terrestres se ha convertido en un tema de investigación sumamente importante, en donde no sólo entra el ámbito tecnológico, sino que también, se empieza a abrir la puerta para la creación y el mejoramiento de diversos protocolos o diseños externos al automóvil y por consiguiente al conductor, para que con ello se pueda combatir este enorme problema, sin embargo, en la actualidad, todos estos esfuerzos no han sido suficientes, debido a que si bien, aumentar la seguridad tanto en los caminos así como mejorar cada vez más los niveles de conducción de los automóviles son elementos extraordinarios, cada año va en aumento los accidentes causados por la somnolencia y la distracción principalmente, por lo tanto, seguir investigando he innovado sistemas que monitoreen estos elementos específicamente, son cada vez es más necesarios, convirtiéndolos en sistemas indispensables al momento de conducir un vehículo.

Aun que existen en la actualidad, diversos sistemas que tienen como propósito principal poder prevenir los accidentes causados por la somnolencia y la distracción como se mostró en los diversas investigaciones y patentes en este capítulo, la mayoría solo se enfoca en una problemática, aislando por consiguiente uno de las dos problemáticas, esto si bien podría parecer buena idea, realmente lo que ocasiona es que aún no sea tenga una inclusión general, por tal motivo, crear y desarrollar un sistema que tenga ambas propiedades, es sumamente necesario, por ende, a lo largo de este trabajo de investigación, abordaremos los métodos necesarios para poder implementar las dos detecciones y con ellos, generalizar las investigaciones previas en un sólo

dispositivo, teniendo claro, un sistema innovador que una lo mejor de todos los proyectos anteriormente descritos.

2.6 Referencias del capítulo

[1] ONSUN, [en línea] https://osunautomotive.wordpress.com/2019/04/05/ventajas-y-desventajas-de-la-transmision-manual-y-automatica/

[2] AUTOLAND. Transmisión manual y automática: Ventajas y Desventajas. Diciembre 2020. [en línea] https://autoland.com.pe/transmision-manual-y-automatica-ventajas-y-desventajas/

[3] Buendia R. Conoce los diferentes tipos de direcciones para saber cuál te conviene más. Agosto 2016. [en línea] https://www.motorpasion.com.mx/tecnologia/los-diferentes-tipos-de-direcciones-y-cual-me-conviene-mas

[4] Wang J, Zhu S, Gong Y. Driving safety monitoring using semisupervised learning on time series data. 2010, PP: 728-737.

[5] Wu B, Chen Y, Yeh C, Li Y. Reasoning-based framework for driving safety monitoring using driving event recognition. 2013. PP: 1231-1241.

[6] Kokonogi A, Michail E, Chouvarda I, Maglaveras N. A study of heart rate and brain system complexity and their interaction in sleep-deprived subjects. 2008, PP: 969-971.

[7] Zhang C, Wang H, Fu R.: Automated detection of driver fatigue based on entropy and complexity measures. 2014. PP: 168-177.

[8] Zhao G, He Y, Yang H, Tao Y. Research on fatigue detection based on visual features. 2020. PP: 1-20.

[9] Phan A, Nguyen N, Trieu T, Phan T. An efficient approach for detecting driver drowsi-ness based on deep learning. 2021.

[10] Flores-Monroy J, Nakano-Miyatake M, Perez-Meana H, Sanchez-Perez G. Visual-based real time driver drowsiness detection system using CNN. 2021.

[11] Velasco-Sánchez E, Oliva-Meyer MA, Sánchez-Lozano M. El sistema de dirección. Mayo 2015, PP: 2-15.

[12] Mateos-Aparicio JP. ¿Qué son los sistemas ADAS?. Marzo 2020. [en línea] https://www.autofacil.es/tecnologia/son-sistemas-adas/184460.html

[13] Anber S, Alsaggaf W, Shalash W. A Hybrid Driver Fatigue and Distraction Detection Model Using AlexNet Based on Facial Features. Enero 2022.

[14] Magán E, Paz-Sesmero M, Alonso-Weber JM, Sanchis A. Driver Drowsiness Detection by Applying Deep Learning Techniques to Sequences of Images. Enero 2022.

[15] Tamanani R, Muresan R, Al-Dweik A. Estimation of Driver Vigilance Status Using Real-Time Facial Expression and Deep Learning. Julio 2017.

[16] Hashemi D, Mirrashid A, Beheshti-Shirazi A. Driver Safety Development: Real-Time Driver Drowsiness Detection System based on Convolutional Neural Network. Mayo 2021.

[17] Benedikt S, Hoefle S y Loeffelmann L. Method And Control Unit For Detecting Drowsiness Of A Driver For A Driver Assistance System For A Vehicle. Patente EE.UU. US11292478B2. 5 de Abril de 2022.

[18] Martin-Alvarez AR. Device To Prevent Accidents In Case Of Drowsiness Or Distraction Of The Driver Of A Vehicle. Patente EE.UU. US20080143504A1. 19 de Junio de 2008.

[19] Mimar T. Driver Distraction And Drowsiness Warning And Sleepiness Reduction For Accident Avoidance, Patente EE.UU. US9460601B2, 4 de Octubre de 2016.

[20] Manotas J,JR. Method Of Detecting Drowsiness Of A Vehicle Operator. Patente EE.UU. US20100214105A1. 26 de Agosto de 2010.

[21] Zhang H, Witt GJ, Smith MR. Drowsiness Detection System And Method. Patente EE.UU US7202792B2. 10 de Abril de 2007.

[22] Wu R, Turner J, Browning C, Brummett T. Drowsy Driver Detection System. Patente EE.UU. US8957779B2. 17 de Febrero de 2015.

[23] Eskandarian A, Mortazavi A. Unobtrusive Driver Drowsiness Detection System And Method.Patente EE.UU. US8519853B2. 27 de Agosto de 2013.

[24] Deruyck J, McLaughlin BR. Detection Of Driver Behaviors Using In-Vehicle Systems And Methods. Patente EE.UU. US20160046298A1. 18 de Febrero de 2016.

[25] 成波王珂李家文. Driver Fatigue Detection Method Based On Face Video Analysis. Patente China. CN102254151B. 16 de Enero de 2013.

[26] 谢剑斌刘通李沛秦闫玮. Method For Detecting Fatigue State Of Driver Based On Digital Video. Patente China. CN101950355B. 05 de Septiembre de 2012.

Capítulo 3: Método propuesto y resultados

3.1 Introducción

3.1.1 Algoritmo general (Detección de somnolencia y distracción en los conductores)

A continuación, veremos una idea general del sistema propuesto, el cual se irá desglosando cada punto en las secciones posteriores a este.

Para poder combatir la somnolencia y la distracción en los conductores, hemos propuesto un sistema que logra detectar cuando un conductor se ha quedado dormido mientras conduce en tiempo real o todo caso que no esté prestando atención al camino, dicho sistema está compuesto por 4 etapas, las cuales han sido optimizadas para poder trabajar en diversos dispositivos, tanto con recursos limitados (dispositivos móviles), así como también en dispositivos en donde los recursos no sea un problema.

I. Primera etapa ***"Detección de la región a analizar":*** En esta etapa lo que generamos es un detector de rostros de bajo costo computacional, el cual es utilizado para poder identificar la región facial del conductor, dicha región será utilizada para analizar los estados que posteriormente se clasificarán.

II. Segunda etapa ***"Implementación de una Red neuronal Convolucional poco profunda (RNC-PP)"***: Al tener la región a analizar, se implementa una red neuronal convolucional poco profunda (RNC-PP) creada por nosotros, la cual está específicamente diseñada para minimizar el costo computacional y poder clasificar 3 estados principales del conductor, "Ojos abiertos", "Ojos cerrados" y "Distracción".

III. Tercera etapa ***"Análisis de resultados consecutivos":*** En esta parte del sistema, se analizan los resultados obtenidos por la RNC-PP, a lo cual, el sistema puede diferenciar entre una ***"Conducción normal"*** (ojos abiertos y parpadeos normales); ***"Letargo"***, que es cuando el conductor tiene los ojos cerrados por mucho tiempo, sobrepasando un umbral propuesto (1.5s) el cual es ideal para diferenciar entre un parpadeo normal a un micro sueño o somnolencia como tal, y finalmente ***"Distracción"***, en donde el sistema monitorea al usuario para poder determinar si presta o no atención al camino.

IV. Cuarta etapa **"Activación de alarmas pertinentes":** De acuerdo con los resultados obtenidos de la etapa anterior, si pasan de los umbrales propuestos, el sistema es capaz de generar tres tipos de alarmas, la primera y más importante, se activa una alarma sonara, el cual advierte al usuario de que está conduciendo de forma imprudente o peligrosa (es la activación de un sonido molesto), la segunda es una alarma visual, la cual activara ciertos sistemas lumínicos lo cual avisará y despertará al conductor y finalmente las alarmas informativas que sólo son alertas descritas en la interfaz gráfica del sistema.

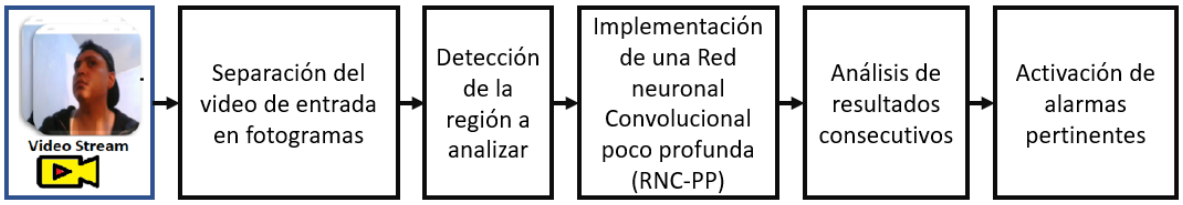


Figura 3. 1: Algoritmo general propuesto para la detección de somnolencia y distracción en un conductor

3.2 Construcción del algoritmo general

3.2.1 Obtención de video en tiempo real

El primer paso del algoritmo, si bien es la ***"Detección de la región a analizar"*** (Punto 3.23) como tal, hay que tener en cuenta, que antes de esto, se tiene que obtener y preprocesar los datos que se necesitan, para esto hacemos uso de un sensor capaz de interactuar con el medio ambiente, tomando y recopilando datos bidimensionales (imágenes digitales[1]), las cuales pueden ser de dos tipos, esto dependerá puramente del sistema que se desee emplear.

a) ***Implementación del sistema en monitoreo de 24 horas y utilización del dispositivo SOMN_IA (el sistema propuesto por nosotros para la versión móvil) o algún dispositivo compatible con cámaras CSI:*** Para este punto se utilizó la cámara IMX219-77IR[2] como dispositivo de video en tiempo real, debido a su capacidad de integrar dos capturas de imágenes en un sólo dispositivo. IMX219-77IR contine 2 sensores fotorresistores LDR (Light Dependent Resistor), los cuales captan el nivel de iluminancia en el ambiente [3] y dependiendo de esta, será el tipo de video que se digitalice, pudiendo ser de espectro visible [4] o espectro infrarrojo [5](Figura 3.2).

Este tipo de cámara es diferente a las convencionales (por comunicación Bus universal Serial USB) [6] debido a que su comunicación es la denominada CSI (Interfaz Serie para Cámaras) [7], para esta versión en específico, se debe utilizar CSI de segunda generación.

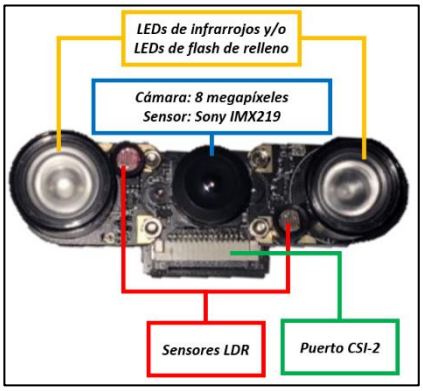


Figura 3. 2: Partes que conforman a la cámara IR IMX219-77IR

Tabla 3. 1: Especificaciones técnicas de la cámara IR IMX219-77IR

Parámetro.	Característica.
Nombre	IMX219-77IR
Resolución	3280 × 2464
Sensor	Sony IMX219
Megapíxeles	8
Voltaje de funcionamiento	3,3 V / 5 V
Especificaciones del sensor	• CMOS: ¼ pulgada • Apertura (F): 2.35 • Longitud focal: 3.15 mm • Ángulo de visión (diagonal) 160° • Distorsión <14.3%
Dimensiones	25mm × 24mm
No. de LEDs de infrarrojos y/o LEDs de flash de relleno	4

En la Figura 3.2, podemos observar las características principales de la cámara IMX219-77IR, así como su apariencia física, en la Tabla 3.1, observamos las características generales de la Video Cámara [2].

En la Figura 3.3 observamos cómo se comporta de forma ilustrativa la secuencia de cambio de obtención de video, dependiendo el nivel de luz.

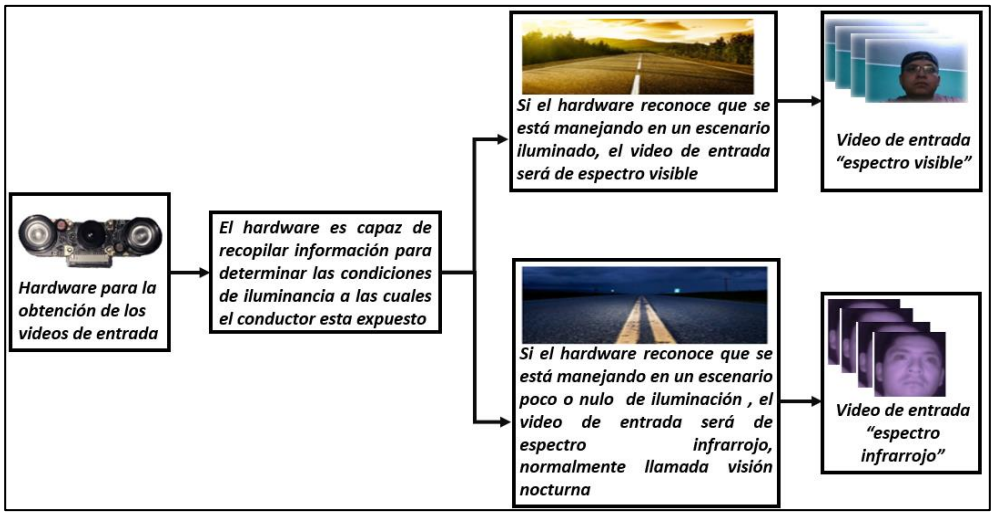


Figura 3. 3: Proceso de obtención de imágenes en las diferentes condiciones de iluminación.

b) Implementación del sistema en entornos controlados o en condiciones con buena iluminación, utilizando el dispositivo SOMN_IA (el sistema propuesto por nosotros para la versión móvil) o algún otro dispositivo con puerto USB: Para el monitoreo de somnolencia y distracciones con estas características,

hacemos uso de una video cámara digital con un periférico USB tipo A (Bus universal Serial) [6], el cual sólo obtendrá espectros visibles como secuencia de fotogramas de entrada, para este punto, se utilizó una video cámara web "C1" (características completas del dispositivo de video en la Tabla 3.2). La obtención de los videos de entrada lo podemos observar en la Figura 3.4 y como vemos, el proceso es básicamente igual al de la Figura 3.3, con la excepción de que esta no es capaz de obtener datos estudiables en ambientes obscuros, una ventaja de utilizar este tipo de filmadoras es que pueden ser intercambiables, ya que nuestro sistema puede detectar las somnolencia y distracción [8,9] en diversas calidades de video.



Figura 3. 4: Proceso de obtención de imágenes con espectro visible.

Tabla 3. 2: Sensores ópticos con periférico USB tipo A-macho utilizados en esta investigación.

Cámaras utilizadas en diversas pruebas	Nombre	Características.
C1 [10]	Cámara Web HD C505	Resolución máxima: 1280 x 720/30 fps Número Efectivo de Pixeles: 2.0 MP Campo visual diagonal (dFoV): 60° Voltaje de funcionamiento: 5 V
C2 [11]	Cámara genérica	Resolución: 1920 x 1080 / 30 fps Campo de visión: 360grados Tipo de sensor: CMOS
C3 [12]	Video Cámara integrada Huawei D16	Resolución máxima: 640 x 480/30 fps

3.2.2 Separación del video de entrada a fotogramas

El funcionamiento de una videocámara digital es simple, en resumen, lo que hace el dispositivo es tomar muchas imágenes a gran velocidad, cuando el usuario o algún programa termina el proceso de grabar, el dispositivo es capaz de unir dichas imágenes

tomadas y correrlas a una velocidad de transición establecida por el fabricante (normalmente corren a 24 imágenes por segundo o 24 fps) [13], debido a que los cambios de fotogramas son tan rápidos, se da la ilusión óptica del video o el movimiento de las imágenes, ya que el ojo humano no es capaz de percibir dichas transiciones.

Sabiendo lo anterior, hacemos uso de librerías enfocadas al procesamiento de imágenes [14], en donde se omite la etapa de unión, procesamiento y guardado del video y se analiza cada fotograma, en la figura 3.5, vemos el proceso de la fragmentación del video a fotogramas; cabe mencionar, que nuestro sistema tiene sólo la intención de estudiar las imágenes de entrada, por tal motivo, en ninguna parte del proceso hasta el momento, se hacen métodos de guardar datos o videos.

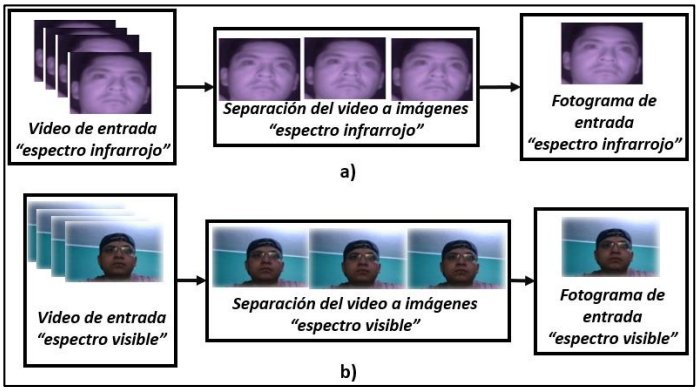


Figura 3. 5: Procedimiento de fragmentación

Como observamos en la Figura 3.5, el proceso de separación no es complicado pero si hay que tener en cuenta una cosa, cada vez que mostramos el fotograma de entrada por separado, el algoritmo no se queda en este proceso, finaliza la visualización de la imagen y continúa con el siguiente fotograma, esto es importante saberlo, debido a que la velocidad de fragmentación de video es directamente proporcional al nivel de procesamiento computacional del dispositivo en donde se implementa y la video cámara usada; en la Tabla 3.3 y 3.4 vemos las velocidades de segmentación de los videos en tiempo real y pregrabados en los diversos dispositivos en donde se implementó el sistema, esto es importante saberlo, debido a que estos datos nos dan una pauta para poder obtener una visualización del poder computacional que estamos trabajando, y así determinar antes si el dispositivo que vamos a utilizar para el análisis y monitoreo de somnolencia y distracción que posteriormente implementaremos (la velocidad se reduce hasta un 50% con los procesos siguientes).

Tabla 3. 3: Tabla comparativa en el proceso de fragmentación de videos en tiempo real en los diferentes dispositivos utilizados en esta investigación

Dispositivo	Velocidad de fragmentación de video en tiempo real a fotogramas por segundo(fps)			
	IMX219-77IR	Cámara Web HD C505	Cámara genérica	Video Cámara integrada Huawei D16
3070TI GPU, 8GB Memory, 6144 cores, 1.770MHz.	No Posee Puerto CSI	30.62	30.62	----------
Huawei D16 Laptop, AMD Ryzen 5 4600H	No Posee Puerto CSI	29.4	30	29.4
Nvidia Jetson AGX Xavier	No Posee Puerto CSI	15.15	14.70	----------
Nvidia Jetson Nano 4GB	30.61	7.42	8.37	----------

Tabla 3. 4: Tabla comparativa en el proceso de fragmentación de videos pregrabados en los diferentes dispositivos utilizados en esta investigación

Dispositivo	Velocidad de fragmentación de video a fotogramas por segundo(fps)			
	Video 1 Resolución: 1920 x1080	Video 2 Resolución: 1080 x 1920	Video 3 Resolución: 1280 x 720	Video 4 Resolución: 640 x 480
3070TI GPU, 8GB Memory, 6144 cores, 1.770MHz.	136.36	166.66	309	65.21
Huawei D16 Laptop, AMD Ryzen 5 4600H	78.95	115.38	231.75	100
Nvidia Jetson AGX Xavier	35.71	38.46	66.21	187.5
Nvidia Jetson Nano 4GB	20.27	18.29	48.79	125

3.2.3 Detección de la región a analizar

Como tal, este es el primer paso que se necesita para poder detectar la somnolencia y distracción en los conductores, teniendo ya la imagen de entrada, hemos propuesto la utilización de un detector de objetos, enfocados en los patrones principales de rostros, debido a que esta región será la que utilizaremos para poder emplear las predicciones descritas en los siguientes apartados de esta tesis.

El algoritmo utilizado para el reconocimiento facial es el siguiente:

- ***Detección de objetos mediante clasificadores en cascada basado en funciones Haar***: Este es uno de los detectores faciales más populares, normalmente se le atribuye su creación a los autores Paul Viola y Michael Jones por su artículo [15], debido a que ellos implementaron esta técnica y agregaron una mejora al algoritmo Boosting en donde aceleraron su velocidad de clasificación en 2001; se trata de una algoritmo de aprendizaje automático de tipo clasificatorio, para poder determinar si hay o no un rostro en una imagen digital, para esto se usó una base de datos muy amplia, la cual están etiquetados sólo dos parámetros:
  1. Imágenes positivas: Son aquellos fotogramas que contiene un rostro.
  2. Imágenes negativas: Son aquellos fotogramas que no contienen rostros

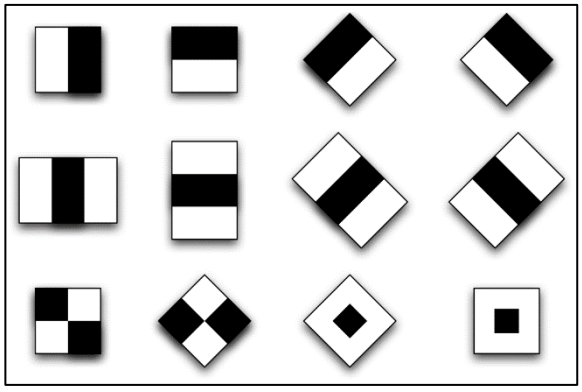


Figura 3. 6: Filtros HAAR-like[14]

El reconocimiento facial utilizando este tipo de algoritmo se basa en entrenar imágenes positivas y negativas para poder clasificar los patrones del rostro, estos se identifican comparando las intensidades lumínicas de regiones rectangulares utilizando imágenes base denominadas Características de filtros HAAR-Like (Figura 3.6), estos filtros tienen las características te tener un tamaño promedio de 24x24 pixeles; algunos puntos importantes de las características Haar son:

[14] Figura obtenida de Detección de Rostros basado en Filtros Haar + Adaboost, Pardo Herrera Carlos Julio, Vision Por COmpuitadora, https://carlosjuliopardoblog.wordpress.com/2017/05/12/filtros-haar-deteccion-de-rostros/

1. Características de dos rectángulos cuyo valor es la diferencia entre las sumas de los píxeles contenidos en ambos rectángulos. Las regiones tienen la misma área y forma y son adyacentes (Figura 3.7 a y b).
2. Características de tres rectángulos que calculan la diferencia entre los rectángulos exteriores y el interior multiplicado por un peso para compensar la diferencia de áreas (Figura 3.7 c).
3. Características de cuatro rectángulos que computan la diferencia entre pares diagonales de rectángulos (Figura 3.7 d).

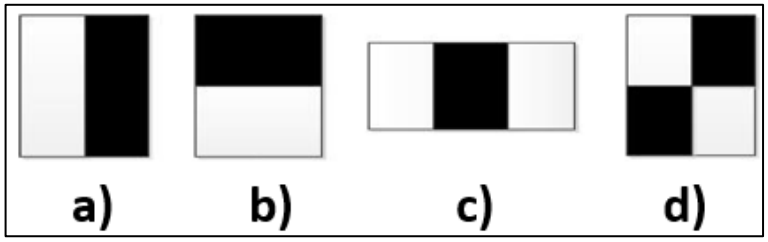


Figura 3. 7: Filtros HAAR característicos

Teniendo las características que servirán como parámetros para la comparativa, los autores utilizan imágenes integrales, las cuales, son un tipo de procesamiento de imágenes en donde se transforma una imagen normal en la suma de los pixeles situados a la izquierda y superior (x, y), en la ecuación 1 podemos observar de manera matemática el proceso para la conversión de la imagen normal a una imagen integral.

$$\boldsymbol{ii}(\boldsymbol{x},\boldsymbol{y}) = \sum_{\boldsymbol{x'}\leq\boldsymbol{x},\boldsymbol{y'}\leq\boldsymbol{y}} \boldsymbol{i}(\boldsymbol{x'},\boldsymbol{y'}) \tag{1}$$

En donde $i(x, y)$ es nuestra imagen original y $ii(x, y)$ es la imagen integral resultante. Viola y Jones plantean una forma para el cálculo de imágenes integrales por un sólo paso, reduciendo el tiempo de análisis, por tal motivo se proponen las recurrencias de las ecuaciones 2 y 3.

$$\boldsymbol{s}(\boldsymbol{x},\boldsymbol{y}) = \boldsymbol{s}(\boldsymbol{x},\boldsymbol{y}-\boldsymbol{1}) + \boldsymbol{i}(\boldsymbol{x},\boldsymbol{y}) \tag{2}$$

$$\boldsymbol{ii}(\boldsymbol{x},\boldsymbol{y}) = \boldsymbol{ii}(\boldsymbol{x}-\boldsymbol{1},\boldsymbol{y}) + \boldsymbol{s}(\boldsymbol{x},\boldsymbol{y}) \tag{3}$$

Dode s(x,y) es la acumulacion de filas por lo tanto $s(x, -1) = 0$ y $ii(-1, y) = 0$ en la primera posicion. Para poder ejemplificar esto, en la Figura podemo ver un ejemplo de esta imeplementacion.

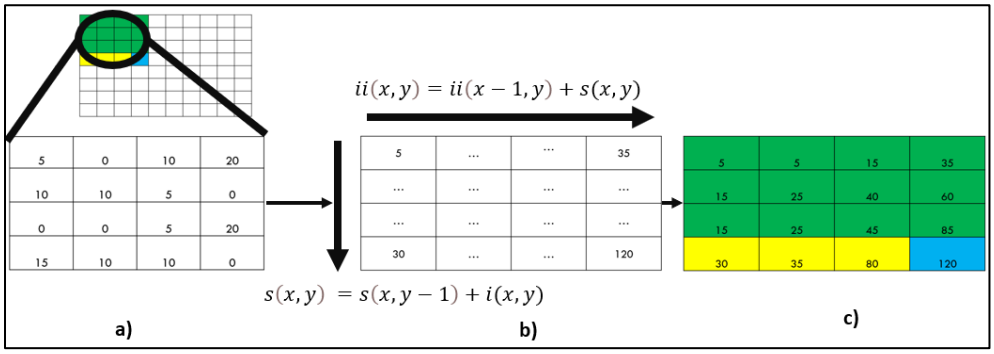


Figura 3. 8: Ejemplo del procesamiento de la imagen integral

El siguiente paso es la implementación del entrenamiento del modelo para el reconocimiento facial, en este proceso, se hace uso de clasificadores débiles que posteriormente se convertirán en un clasificador fuerte, en donde se utiliza un meta-algoritmo denominado AdaBoost. El propósito de AdaBoost es obtener patrones o rasgos propios de un rostro con la ayuda de muchas imágenes de entrenamiento, tales características pueden ser: ojos, boca, nariz, cejas, etc., con estos datos, los primeros clasificadores empezarán a catalogar de manera consecutiva y adaptativa dichos parámetros, como se muestra en la Figura 3.9

.

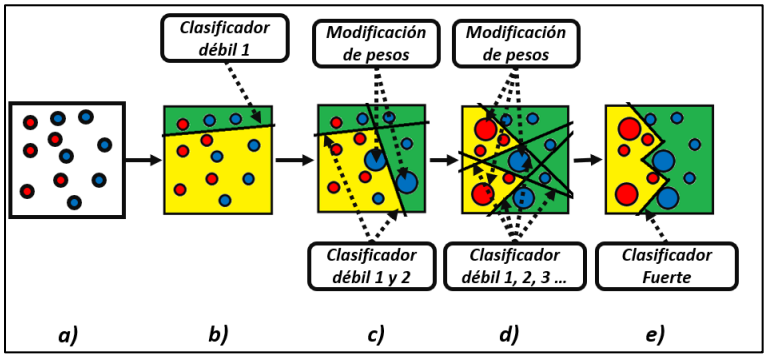


Figura 3. 9: Proceso de la generación de clasificadores débiles hasta la obtención de un clasificador fuerte

El funcionamiento como tal del algoritmo AdaBoost es el siguiente:

1. Supongamos que tenemos un set de imágenes (x_1 , y_1)--- (x_n , y_n) las cuales el 50% son imágenes negativas y el otro 50% son positivas, donde las etiquetas quedaría como y_i=0,1 respectivamente. A este set lo que haremos por consiguiente será parametrizarlo en patrones específicos como se muestra en figura 3.9 a.
2. El siguiente paso es inicializar los pesos, por tal motivo hacemos uso de:

$$w_{1,j} = \frac{1}{2m}, \frac{1}{2l} \; para \; y_i = 0,1 \qquad (4)$$

Después de la inicializar, tenemos que puntualizar los patrones, por lo tanto lo que se hace es normalizarlos con ayuda de:

$$w_{t,i} \leftarrow \frac{w_{t,i}}{\sum_{j=1}^{n} w_{t,j}} \quad (5)$$

Donde W_t es la distribución probabilística, t es igual a t=1,…,T y las características j se entrenan usando un clasificador h_j el cual por condiciones dadas por los autores, se limita a uno solo.

3. El siguiente paso es calcular el error el cual se evalúa con respecto a w_t, por lo tanto:

$$\epsilon_j = \sum_i w_i |h_j(x_i) - y_i| \quad (6)$$

4. Cuando tengamos ya los pesos y el cálculo del error del clasificador h_t, se debe escoger el que menos rango de error genere ϵ_t (Figura 3.9 b).
5. El paso siguiente es la modificación y actualización de los pesos con:

$$w_{t+1,i} = w_{t,i} \beta_t^{1-e_i} \quad (7)$$

En donde e_i=0 si el patrón de la imagen de prueba x_i es clasificada correctamente, de lo contrario e_i=1 y β_t es igual a $\epsilon_t/(1-\epsilon_t)$ (Figura 3.9 c y d)

6. Finalmente obtendremos el ultimo clasificador, que es la unión de todos los clasificadores débiles, los autores lo denominas como “clasificador fuerte” (Figura 3.9 e).

$$h(x) = \begin{cases} 1 & \sum_{t=1}^{T} \alpha_t h_t(x) \geq \frac{1}{2} \sum_{t=1}^{T} \alpha_t \\ 0 & \textit{de lo contrtario} \end{cases} \quad (8)$$

Donde $\alpha_t = log \frac{1}{\beta_t}$

Si bien las imágenes integrales, las características Haar-like y el meta-algoritmo AdaBoost son buenos para la clasificación, la mayoría de los datos que existen en un fotograma son elementos que no pertenecen a un rostro, por lo tanto, son denominados, datos basura; de modo que si queremos determinar si en una imagen nueva (que no contenga nuevos rostros) haya o no un rostro y le aplicamos el modelo pre-entrenado, por la propia naturaleza del algoritmo, este tratará de reconocer patrones que sean iguales o similares al del objeto; AdaBoost por lo tanto, aplicara su algoritmo, modificando sus pesos, hasta concluir que no existe dicho objeto, esto se traduce a una pérdida de tiempo y procesamiento fundamental en el reconocimiento en tiempo real; los autores vieron esta desventaja y emplearon un concepto denominado ***“Clasificadores de cascada”,*** en donde proponen, que en lugar de depender de los autoajustes de AdaBoost por cada imagen que en promedio son de 38 etapas y más de 6000 características, se propone que tengan un sistema de discriminación, en el cual básicamente, el total de características se dividan en ventanas o bloques, estos bloques que contienen

10 características cada 1, estarán estudiando y desestimando, como se muestra en la figura 3.10.

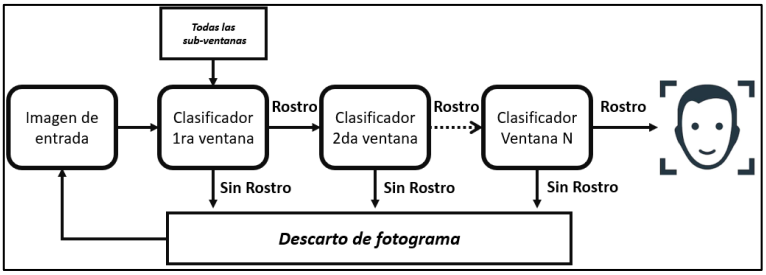


Figura 3. 10: Proceso de los clasificadores de cascada

Como se observa en la figura 3.10, el sistema estudia la imagen de entrada, si detecta patrones de rostro en al menos una característica, el algoritmo continua con la siguiente ventana y así consecutivamente, hasta terminar con todo el proceso de Adaboost y poder encontrar el objeto, de lo contrario, si en alguna ventana, esta no detecta ningún patrón de rostro, el algoritmo se rompe y pasa a la siguiente imagen, así se reducimos considerablemente el tiempo de estudio y detección; un punto importante es que el clasificador más débil se debe pone al inicio para así lograr rechazar la mayoría de ventanas que no contengan un rostro, evitando así un gasto innecesario computacional, el nivel de complejidad del clasificador por ende va aumentan, mientras más se aproxime al final del algoritmo.

Al momento del entrenamiento para la clasificación se deben tener 3 características importantes en los clasificadores:

A. El número de etapas de los clasificadores.
B. El número de características de cada etapa.
C. El umbral (rigurosidad de la detección) de cada etapa, el cual definirá la complejidad de cada etapa.

- **Implementación del detector de rostros[16]**: La implementación si bien, es básicamente la ejecución de lo comentado anteriormente, existen bibliotecas que ayudan a su implementación, reduciendo el tiempo de programación, así como la seguridad de que dichos modelos ya están optimizados para su mejor rendimiento, pero hay que tener en cuenta algunos puntos:

  1. ***Imagen de entrada:*** Para poder implementar de forma correcta el detector de rostros propuesto por V&J, nosotros convertimos las imágenes de entrada que normalmente pueden venir en los canales RGB,BGR, CMYK, etc., a su canal promedio, en otras palabras, las imágenes de espectro visible (Figura 3.11 a) las debemos convertir a su igual en escala de grises (Figura 3.12 b), con el fin de reducir el costo computacional al momento de hacer el proceso de detección de la región a utilizar.



Figura 3. 11: Conversión de la imagen de entrada BGR a escala de grises

2. ***Factor de escalamiento:*** El factor de escalamiento es utilizado para reducir el tamaño de la imagen de entrada, esto ayuda a que los elementos que generen ruido o imperfecciones a las imágenes disminuyan o en todo caso desaparezcan, además el tiempo de detección será menor, pero hay que tener en cuenta una cosa, cuanto más grande sea el factor, el reconocimiento de rostro puede ser poco o nulo, en cambio, si el factor es muy pequeño , existirán los llamados falsos positivos o en otras palabras, se detectarán rostros en lugares donde no hay rostros, además de que la velocidad de reconocimiento aumenta; la escala va de 1.01 a 1.99 (dependiendo el valor derecho después del punto decimal, será el porcentaje de reducción de la imagen, por ejemplo: 1.1 reduce la imagen 10%, 1.5 reduce la imagen 50% y 1.9 reduce la imagen un 90%); los valores recomendados para este factor es de entre el 10% al 30% (los autores manejaron el 25% ósea 1.25), el sistema que proponemos utiliza el **factor 1.1**.

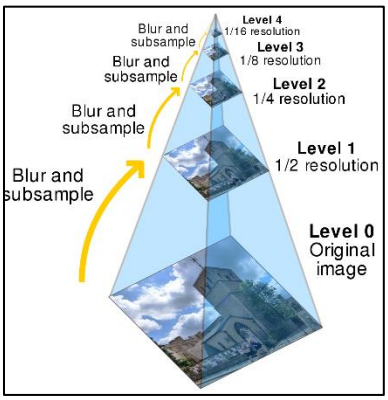


Figura 3. 12: Representación piramidal de la perdida de información de una imagen digital[15]

[15] Figura tomada de Pyramid (image processing), WIKIPEDIA, 2015, https://en.wikipedia.org/wiki/Pyramid_(image_processing)#/media/File:Image_pyramid.svg

3. ***Mínimo de vecinos:*** Este valor representa cuantas detecciones de la región del rostro son necesarias para poder determinar que ahí existe un rostro, puesto que el detector es un poco sensible a pequeñas translaciones y diferencias de escala, se suelen producir múltiples detecciones alrededor de cada cara.

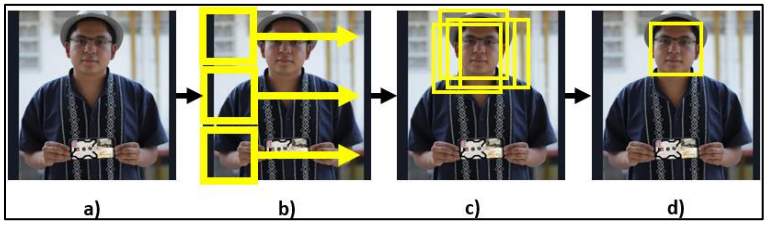


Figura 3. 13: Proceso de ubicación del rostro del usuario desde punto de vista de los vecinos cercanos

En la Figura 3.13, podemos observar de forma gráfica, cómo funciona la detección de la región del rostro; en la Figura 3.13 b, vemos el procesos del recorrido que hace Adaboost con ayuda de los filtros base, en la Figura 3.13 c, vemos el valor mínimo de las regiones vecinas para poder determinar si hay o no un rostro, aquí se tiene que tomar en cuenta diversos parámetros; el primero radica en que mientras más vecinos estipules, el promedio de detección de rostro disminuye así como el tiempo de análisis aumenta, en cambio, sí se pone un número muy bajo de vecinos, el tiempo de análisis será mucho menor, el sistema encontrará más fácil el rostro, pero incrementa por mucho la posibilidad de que detecte falsos positivos y finalmente en la Figura 3.13 d, vemos como sólo al final, el algoritmo muestra el rostro detectado.

Los valores del mínimo vecino van desde 1 hasta N, debido a que podrías poner un número muy grande y en teoría te lo debe aceptar, lo recomendable es que no sea mayor a 20, en nuestro sistema proponemos **un mínimo de 5 vecinos**.

4. ***Máximo y mínimo tamaño del cuadro delimitador:*** El cuadro delimitador que se genera cuando se encuentra el objeto-rostro, es propio del sistema, por lo tanto nosotros podemos delimitar que tan grande o pequeña sea el rostro localizado, en la Figura 3.14 a vemos un ejemplo de una localización de un rostro pequeño y en Figura 3.14 b de un rostro grande; esto hace referencia a que tan lejos o cerca este el usuario de la sensor (video cámara), en nuestro sistema el usuario está en un movimiento constante, y el cuadro delimitador se va moviendo y a su vez auto ajustando, por tal motivo los valores de rostros muy pequeños o muy grandes no nos es de utilidad, para la detección de somnolencia y distracción, **la región estudiada que**

proponemos tiene un mínimo de 150×150 pixeles y un máximo de 250×250px.

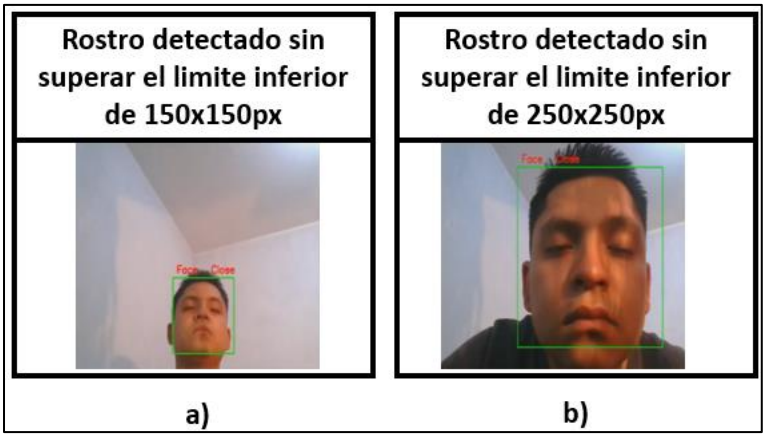


Figura 3. 14: 4. Detección del rostro utilizando máximo y mínimo tamaño del cuadro delimitador

3.2.4 Implementación de la red neuronal convolucional propuesta (primera versión)

Actualmente, las técnicas para poder clasificar patrones con ayuda de la inteligencia artificial (IA) son cada vez más recurrente, debido a su gran exactitud, así como su facilidad de interpretarlo en código. La IA como tal engloba muchas áreas de estudio [17], como el procesamiento del lenguaje natural, visión artificial, sistemas expertos, entre otros; para nuestro caso en especial nos enfocaremos en un área en específico, la denominada aprendizaje automático o ML (Machine Learnig) por sus siglas en inglés [18].

Al presente el ML se ha hecho muy pulpar por su gran variedad de técnicas para poder agrupar [19], clasificar [20], predecir, y demás con ayuda de fundamentos matemáticos como la regresión lineal [20], regresión logística [21], KNN [22], SVM [23], entre otros, en la resolución de muchos problemas que antes eran muy difícil de calcular o de procesar. Hoy en día el ML se ha vuelto una herramienta fundamental debido a que con ayuda de esta base matemática, se han creado algoritmos muy populares en donde cada uno es utilizado para específicamente la resolución de una tarea y hasta algunos pueden realizar varias; entre todas estas, existe un área en específico que en los últimos años ha tomado mucha fuerza ya que es usada para la resolución de una infinidad de problemas, reduciendo los errores que como tal el ML llega a tener con sus algoritmos clásicos; nos referimos al denominado aprendizaje profundo o DL (Deep Learning) por sus siglas en inglés [24].

La principal característica del DL es que en esta área se hace uso de una técnica de aprendizaje basado en el aprendizaje humano, las denominadas redes neuronales artificiales (RNA) [25]. Como tal las RNA son una representación abstracta de las redes

neuronales biológicas, su función es básicamente similar, en estas se les da de comer elementos o datos, posteriormente pasan por un proceso de aprendizaje y al final nos devuelven un resultado, como lo haría una neurona o una red natural. Existen diversas aplicaciones, como predicciones, clasificaciones, agrupaciones, etc.

Para nuestro caso, las redes neuronales que nos interesan son las capaces de clasificar imágenes, este tipo son las denominadas redes neuronales convolucionales (RNC) [26]. Para poder hacer uso de este tipo de aprendizaje profundo supervisado, se necesitan diversos parámetros, condiciones, y matemática especifica que en este apartado se abordará.

1) ***Estados de predicción:*** Las RNA son capaces de hacer muchas cosas, entre todas ellas, la de clasificar y predecir datos es uno de las más importantes, por lo tanto, poder decidir que queremos catalogar es fundamental. Los estados necesarios de un conductor que necesitamos conocer en esta investigación en particular es la de somnolencia y distracción. La distracción es fácil predecirla con ayuda de nuestro detector de rostros, por lo tanto, este estado por el momento lo omitiremos, entonces nos queda el estado de somnolencia.

   La somnolencia como tal no es un estado que se pueda clasificar de una imagen a otra, por lo tanto, nuestra propuesta es poder clasificar los principales patrones del rostro de un conductor, el cual muestren una conducción correcta y una incorrecta o peligrosa (que el conductor se haya quedado dormido o que tenga signos de somnolencia).

   Después de hacer diversas pruebas y de ver diversos videos de personas conduciendo, concluimos que los patrones característicos de una persona que se ha quedado dormida o tiene rasgos de sueño a una que está en un estado normal de conducción, son los siguientes:

   - Rostro sin rasgos de somnolencia:
     I. ***Ojos (Figura 3.15 b-1)*** : La posición de los ojos son los patrones por excelencia para poder diferenciar a una persona que presta atención o en todo caso, se ha quedado dormida, en este caso los ojos abiertos representan a las personas que están presentando una conducción correcta.
     II. ***Posición de las cejas(Figura 3.15 b-2):*** La posición de las cejas, van muy relacionadas con la posición de los ojos, estos los podemos utilizar como un referente secundario, cuando una persona sin rasgos de sueño presenta un área menor entro los limites inferiores de las cejas y superiores de los parpados, la persona esta despierta y por lo tanto presentan una conducción correcta.
     III. ***Líneas de expresión (Figura 3.15 b-3 y b-6):*** Después del estudio de varias pruebas (videos) de conducción, decidimos agregar como

patrones diferenciales con la somnolencia las líneas de expresión generales, debido a que estas se presenta cuando un conductor entrecierra los ojos; en ocasiones, si sólo empelamos las características tan cerradas como ojos cerrados y abiertos, las RNC puede confundirlo fácilmente con el patrones más cercanos, que este caso seria los ojos cerrados, pero esto no quiere decir que no esté prestando atención, estas líneas y patrones de expresión facial mayormente presentan la concentración o el enfoque a elementos externos del vehículo que en ocasiones no se pueden observar bien, debido a complicaciones del tiempo.

IV. ***Posición del rostro (Figura 3.15 b-4):*** Al tener un sólo sensor, nos limitamos un poco debido a que no podemos hacer un monitoreo completo de todo el cuerpo del usuario, esto lo corregimos debido a patrones en específicos, en este caso, la posición de la nariz del conductor se no es muy útil, debido a que este será que supla algunos sensores de profundidad, reduciéndolos costoso tanto computacionales como de fabricación.

La posición norma de un conductor en el estado normal de conducción, el eje X del punto medio de la nariz se mantienen en un conste movimiento (izquierda a derecha), pero el eje Y no debería modificarse tanto, eso se traduce a que si una persona mueve las posiciones horizontales demasiado, el usuario presenta señales de somnolencia.

V. ***Posición de la boca (Figura 3.15 b-5):*** Normalmente la gesticulación de la boca representa una gran característica al momento de poder determinar si una persona está o no prestando atención al camino; en este caso, los patrones que nos interesan de la boca es cuando el usuario canta, sonríe y habla, esto debido a que si presenta alguno de los movimientos, es muy difícil que el conductor se valla a quedar dormido.

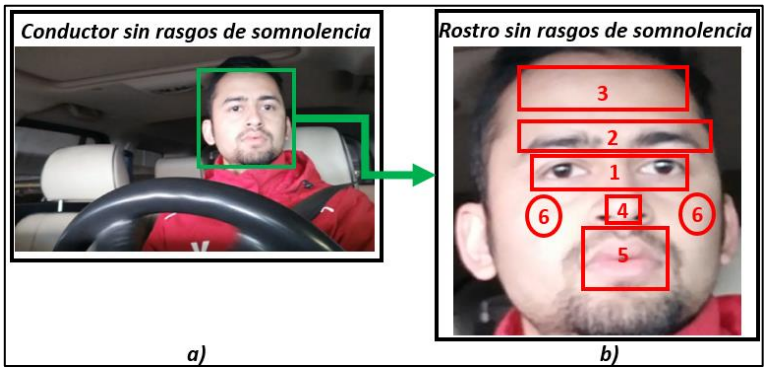


Figura 3. 15: Patrones de un rostro sin rasgos de somnolencia

- Rostro con rasgos de somnolencia:
  1. ***Ojos (Figura 3.16 b-1):*** La diferencia con los patrones anteriores, en esta característica, los ojos deben estar cerrados, clasificando principalmente que una persona se ha quedado dómida.
  2. ***Posición de las cejas(Figura 3.16 b-2):*** En este patrón, mientras más sea el área entre las cejas y los parpados, mayor la probabilidad de que la persona este durmiendo, esto va muy relacionado con la clasificación del estado de los ojos.
  3. ***Posición del rostro (Figura 3.16 b-3):*** Cuando el movimiento del rostro es horizontal de arriba a hacia abajo, y el usuario se mantiene en una posición negativa con respecto al normal de una persona despierta, esto quiere decir que nuestro sujo se ha quedado dormido.
  4. ***Posición de la boca(Figura 3.16 b-4):*** Esta característica al igual que la posición de los ojos, es fundamental para poder determinar si una persona tiene rasgos de sueño, la principal diferencia es le bostezo, cuando esto sucede, es inevitable que pensar que el usuario tiene patrones de sueño.

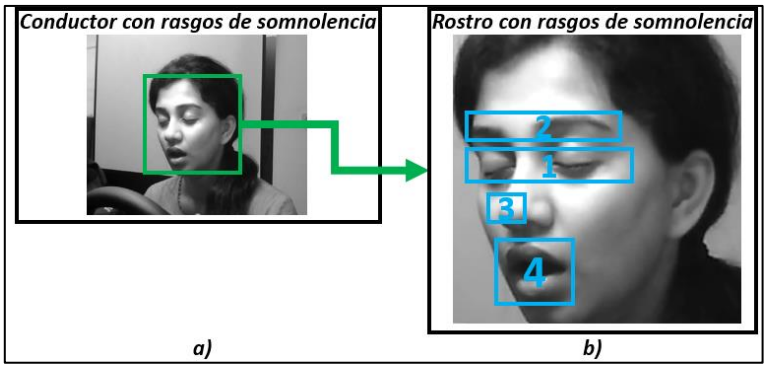


Figura 3. 16: Patrones de un rostro con rasgos de somnolencia

Teniendo como base los principales patrones de somnolencia se ha decidido crear una base de datos que contengan imágenes de rostros de diversas personas con estos patrones, por lo tanto, las etiquetas que estaremos manejando serán ***"ojos abiertos"*** y ***"ojos cerrados",*** esto sólo como una generalización de los estados, haciendo más fácil la comprensión al momento de pasar al análisis de resultados (3.25).

2) ***Base de datos:*** Las RNC son técnicas de clasificación de imágenes [26], la característica principal de estos clasificadores, es que estudia patrones de una imagen y al final nos devuelve una respuesta probabilística, por lo tanto se necesita crear una base de datos que alimente el aprendizaje de esta red; teniendo ya en cuenta lo anterior , la idea de que parámetros vamos a utilizar, los patrones que vamos a clasificar y las etiquetas en las que vamos a agrupar, nuestro siguiente

paso es obtener imágenes de rostros con estas características, por ende hacemos uso de una base de datos libre creada por la National Tsing Hua University (NTHU) que es muy utilizada para este tipo de investigaciones, el nombre del set de datos es Driver Drowsiness Detection Dataset o comúnmente denominada ***"NTHU-DDD"*** [27]. Algunas de las características de esta base de datos son:

Tabla 3. 5: Características de la base de datos *"NTHU-DDD"*

Parámetro.	Descripción.
Tipo de base de datos.	Video formato AVI.
Características de los sujetos de prueba.	La base de datos está conformada por sujetos masculinos y femeninos, de diferentes etnias y rasgos físicos.
Tipo de iluminación	Luz visible y luz infrarroja.
Conjunto de datos de entrenamiento.	36 sujetos con 5 escenarios diferentes: • Sujetos con lentes e iluminación visible. • Sujetos sin lentes e iluminación visible. • Sujetos con lentes e iluminación infrarroja. • Sujetos sin lentes e iluminación infrarroja. • Sujetos con lentes de sol u obscuros e iluminación visible.
Duración de los videos.	Aproximadamente un promedio de 1.5 minutos a una velocidad de 20 30 fps.
Característica del video o imagen de entrada.	Los videos tiene una resolución de 640 x 480 a 3 canales.
Patrones característicos.	• Parpadeos. • Somnolencia. • Conducción normal. • Distracción. • Bostezo.

La base de datos ***"NTHU-DDD"*** tiene la característica principal en que es un set de videos, por lo tanto, nuestra siguiente tarea es la creación de una base de datos propia con las características que nosotros necesitamos (los descritos en el párrafo anterior), para ellos, hicimos uso del algoritmo de fragmentación de video a fotogramas (3.22) y el detector de rostros (3.23) para poder detectar la región que a nosotros nos interesa estudiar, posteriormente utilizamos una técnica de procesamiento de imágenes para poder crear una sub imagen en donde únicamente tenga la región facial del usuario (Figura 3.17), este fotograma creado se guarda en una carpeta para su posterior filtración (en el punto 3.25- ***Recorte de la región del rostro*** se explica a fondo el proceso de recorte y creación de las sub imágenes).



Figura 3. 17: Algunos fotogramas utilizados en la fase de entrenamiento del sistema propuesto

Cuando se finalizó el proceso de extracción de rostros de cada uno de los 18 sujetos de prueba, se pasó al proceso de selección y filtración de imágenes con las características previamente mencionadas, por lo que la base de datos quedo organizada de la siguiente manera:

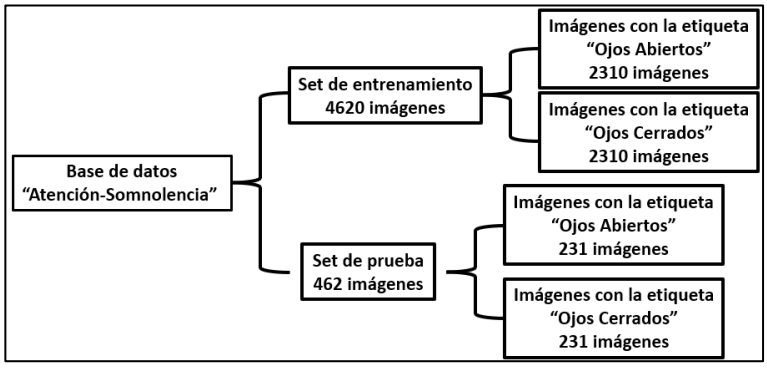


Figura 3. 18: Diagrama de distribución de las imágenes de entrenamiento y prueba

El set de datos que creamos tiene las siguientes características:

I. ***Set de entrenamiento:*** En este apartado están agrupados la mayoría de los datos necesarios para el entrenamiento, el cual en el caso de las RNC, serán el mayor número de imágenes que necesitamos clasificar. En esta carpeta están ya los datos depurados, filtrados y etiquetados.

II. ***Set de prueba:*** En esta carpeta, está un número menor de datos, los cuales tienen una característica especial, al momento de entrenar la RNC, se necesita poner a prueba el nivel de precisión del clasificador, por lo tanto, el sistema compara el entrenamiento que se lleva al momento con imágenes que nunca ha visto (set de prueba) y dependiendo el resultado es la acción posterior que se generará. Al igual que el set de entrenamiento, en esta carpeta están ya los datos depurados, filtrados y etiquetados.

Poder detectar la somnolencia en los conductores es fundamental para esta investigación, por tal motivo se decidió implementar una RNC. Para poder

entrenar el clasificador, hacemos uso de una base de datos la cual previamente hemos montado sus bases, como los patrones de imágenes necesarios para que se clasifiquen de la mejor manera, las etiquetas y hasta los sets necesarios para el futuro entrenamiento, pero nos falta algo fundamental ¿Cuántas imágenes se va a entrenar?; si bien, una característica que se tiene muy en cuenta al momento de utilizar algoritmos de aprendizaje profundo, son que mientras más datos se tenga, en este caso más imágenes tengamos, la RNC aprenderá mucho mejor, los índices de errores se reducirán mientras más datos de entrenamiento se tengan; pero para nuestro caso en particular, esto es una gran problemática, porque si bien, la fiabilidad de la red aumenta cuantos más datos se tengan, esto también se traduce a que el modelo predictivo tendrá muchos más parámetros entrenados, haciendo que la RNC demande un mayor poder computacional, así como su peso en disco sea mayor, esto tal vez no afecte a dispositivos del laboratorio pero perjudica de forma considerable el rendimiento en los que tienen limitación de hardware como en SOMN_IA, en las tarjetas de desarrollo, en dispositivos móviles, etc., por tal motivo, para poder solucionar este problema, nosotros hemos propuesto lo siguiente:

1. ***Número de imágenes de entrenamiento (Figura 3.18)***: Se propone un set de entrenamiento no mayor a 5000 imágenes, las cuales serán cuidadosamente seleccionadas y filtradas, tomando en cuenta cada patrón característico a cada etiqueta, así como también un número balanceado de imágenes promedio entre cada individuo (18 usuarios), por lo que quedo de la siguiente manera:
   - El set de entrenamiento tiene un total de 4620 imágenes.
   - El set de entrenamiento se dividió en dos sets secundarios, uno con la etiqueta ***"Ojos Abiertos"*** y el otro con la etiqueta ***"Ojos Cerrados"*** los cuales, en cada carpeta contienen 2310 imágenes.
   - Los sets secundarios contienen entre 128 y 129 imágenes por cada sujeto de prueba.
2. ***Número de imágenes de prueba (Figura 3.18)***: Al igual que el apartado de imágenes de entrenamiento, aquí se propone una cifra pequeña de imágenes, pero en este caso, se plantea que el set contenga un valor de fotogramas igual al 10% con respecto del set total de entrenamiento, esto es inferior a lo que habitualmente se utiliza en problemas de clasificación (usualmente se utilizan entren 20% al 25%), por lo que queda de la siguiente manera:
   - El set de prueba contiene un total de 462 imágenes.
   - El set de prueba, al igual que el de entrenamiento se dividió en dos sets secundarios, uno con la etiqueta ***"Ojos Abiertos"*** y el otro con la

etiqueta ***"Ojos Cerrados"*** los cuales, en cada carpeta contienen 231 imágenes.

- Los sets secundarios contienen entre 12 y 13 imágenes por cada sujeto de prueba.

Otros puntos para tomar en cuenta para la creación de la base de datos, cuidando mucho el tema de costo computacional, es el tamaño de las imágenes de entrenamiento, así como el número de canales que se van a utilizar, por tal motivo:

I. ***Normalización del tamaño de imágenes:*** Las imágenes utilizadas tienen un tamaño normalizado de 150×150 pixeles con una reducción dimensional de su imagen original utilizando interpolación por área (en el punto 3.25-***Redimensionamiento de la imagen fragmentada*** se explica de forma detallada este proceso).

II. ***Canales de la imagen de entrenamiento***: El número de canales de entrenamiento se queda en 3 canales en el orden: Canal Azul, Canal Verde y Canal Rojo.

3) ***Red Neuronal Convolucional Poco Profunda (RNC-PP):*** Al tener ya como base nuestros estados que vamos a clasificar, la base de datos creada, normalizada, balanceada y etiquetada, lo que nos falta es la configuración de la red neuronal convolucional; para poder utilizar este tipo de redes, hay que tener en cuanta diversos parámetros específicos, primero que nada, al ser una técnica de predicción, necesitamos un modelo que previamente se haya entrenado, por ende, se deben tomar diverso puntos en cuenta, uno de estos es el peso del modelo, debido a que este parámetros se es directamente proporcional al nivel del procesamiento computacional requerido, por lo tanto, la configuración propuesta, debe tener 3 características principales.

I. ***El número de parámetros entrenados deben ser los menos posibles:*** Estas características en especial, se tomó en cuanta desde la creación de la base de datos, por tal motivo, debemos crear una RNC capaz de ser entrenada teniendo los mejores resultados pero a su vez poseyendo un set de datos relativamente pequeño.

II. ***El nivel de complejidad debe ser reducido:*** Mientras más compleja sea la arquitectura de una red neuronal, el procesamiento computacional, el peso en disco, así como el número de parámetros entrenables debe ser mayor, esto es en cierto punto, perjudicial, al momento de poder implementar las RNC en los dispositivos móviles.

III. ***Universalidad***: Como vamos a diseñar desde cero nuestra arquitectura de red neuronal, debemos optar por una configuración así como también un preprocesado de las imágenes de entrada y salida, no tan complejos de replicar en dispositivos alternos al del entrenamiento, esto debido a que

muchas redes convencionales o populares como por ejemplo, la arquitectura de Mobilenet [28] o Xception [29], tienen procesamientos digitales muy específicos, los cuales los hacen difíciles de implementar de forma manual en terminales ajenas a la raíz, por tal motivo, tener parámetros universales, así como simplificados, es de gran importancia al momento de exportar el modelo creado.

4) ***Arquitectura de la RNC-PP propuesta:*** La red neuronal que proponemos tiene la siguiente arquitectura de configuración:

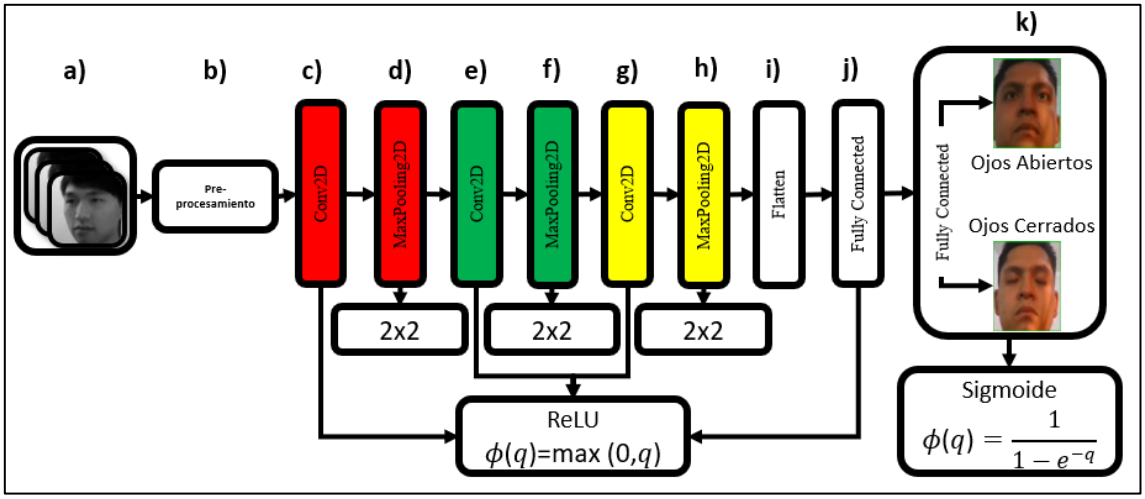


Figura 3. 19: Arquitectura de la RNC-PP propuesta (primera versión)

I. ***Imágenes de entrada (Figura 3.18 a)***: La primera parte de nuestra arquitectura está conformada por las imágenes de entrada de los sets que previamente habíamos seleccionado (punto 2 ***Base de datos***).

II. ***Preprocesamiento (Figura 3.18 b):*** Al tener ya nuestro set de imágenes cargado en la memoria, lo siguiente en cuestión es un preprocesamiento necesario para que se extraigan los mejores patrones clasificatorios de cada imagen, con esto aumentamos considerablemente el número de patrones los cuales la RNC extraerá, entrenará y clasificará posteriormente; las modificaciones y procesamiento digital de las imágenes son las siguientes:

1. ***Re-escalamiento[30]:*** Esto se debe hacer normalmente para poder generalizar los datos de entrada, si bien, ya lo habíamos hecho en los pasos anteriores, es recomendable normalizar una vez más en los parámetros que la RNC toma como estándares, en cuestión, lo único que realizamos es una regulación de los pixeles de la imagen a un rango de 0 a 1, por lo tanto, hacemos uso de:

$$\boldsymbol{P_{normalizado}(x,y) = 1/P_{\text{sin } normalizar}(x,y)} \qquad (9)$$

Donde:

$P_{normalizado}$: Es el nuevo valor del píxel normalizado de la imagen de entrada en turno.

x,y: Son las coordenadas del píxel actual de la imagen de entrada en turno.

$P_{sin_normalizar}$: Es el valor antiguo del píxel normalizado de la imagen en turno.

En la Figura 3.20 vemos un ejemplo de esto:

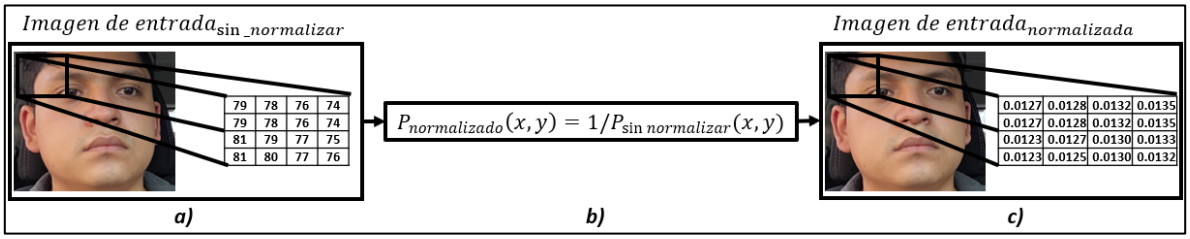


Figura 3. 20: Ejemplo de normalización interno

Como se mencionó previamente, nuestra propuesta de RNC es para una imagen de 3 canales, por lo tanto, el proceso que se muestra en la Figura 3.20 se debe hacer para cada canal, en este caso, sólo ejemplificamos el canal B (azul).

2. ***Rango de corte[31]:*** Normalmente, si se quiere sólo implementar un entrenamiento simple, con los datos de la normalización o el re-escalamiento seria todo, pero al tener muy pocas imágenes para el entrenamiento, hemos decidido implementar otros parámetros, como por ejemplo rango de corte, en este parámetro lo que se realiza es una distorsión del fotograma, a diferencia de una rotación como tal, en este proceso, se toma un lado de la imagen el cual se convierte en el ángulo 0 o de origen y posteriormente se modifica las dimensiones según sea el ángulo que se asignó, esto es muy utilizado para poder modificar las imágenes y así como también, tenemos diversas imágenes de cómo el ser humano puede ver diversos imágenes en distintos ángulos, es básicamente como si viéramos la misma imagen de diversos perfiles, en la Figura 3.20, podemos ver un ejemplo de esto; **el valor del rango de corte que seleccionamos es de 0.2**.

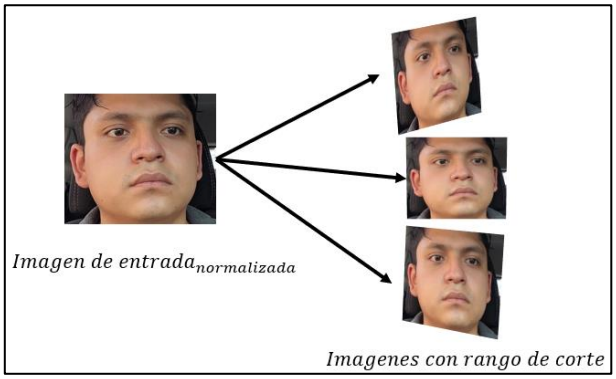


Figura 3. 21: Ejemplos de diversos rangos de corte

De acuerdo con la documentación, el valor que utilicemos para determinar el rango de corte se evalúa en el rango de corte en radianes y flotantes.

3. ***Rango de zoom[31]***: Para finalizar el procesamiento de las imágenes, decidimos implementar otro parámetro, el cual su función es ampliar y disminuir la imagen de entrada, en esta opción, el parámetro que utilizamos es 0.2, debido a que este rango amplifica la imagen un 80% y la disminuye un 120%, em la Figura 3.22 podemos ver un ejemplo:

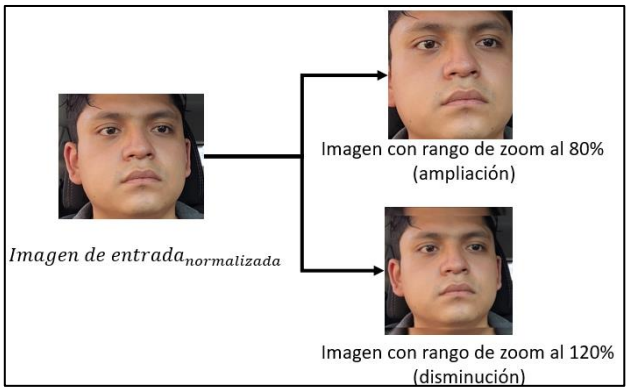


Figura 3. 22: Ejemplos de diversos rangos de zoom

Los diversos preprocesamientos de las imágenes que hemos seleccionado son aleatorios y sumatorios, esto quiere decir que de una imagen de entrada, la ampliación y el rango de corte no lo controlamos nosotros, si no que la computadora hace un proceso interno el cual selecciona de forma random las imágenes y se le aplica estos procesamientos, además que estas nuevas imágenes creadas se irán anexando y sumando al set original.

III. ***Extracción y mapeo de características:*** En esta etapa de la arquitectura, empieza el análisis característico de este tipo de redes, por lo tanto, tomando en cuenta que necesitamos una RNC no tan compleja además de que sea poco profunda [48], hemos propuesto la configuración de 3 capas

convolucionales simples, así como su respectivo MaxPooling (Figura 3.19 c, d, e, f, g, h), por lo que la configuración quedo de la siguiente manera:

Tabla 3. 6: Parámetros propuestos para la arquitectura convolucional

No. de capa	Parámetro	Valor
Primera capa	Capa convolucional (No. de filtros)	16
	Tamaño de kernel	3×3
	Activación	ReLU
	MaxPooling	2×2
Segunda capa	Capa convolucional (No. de filtros)	32
	Tamaño de kernel	3×3
	Activación	ReLU
	MaxPooling	2×2
Tercera Capa	Capa convolucional (No. de filtros)	16
	Tamaño de kernel	3×3
	Activación	ReLU
	MaxPooling	2×2

A continuación veremos las 4 características tomadas en cuenta para la construcción de nuestra etapa de extracción de características, dichos rasgos son utilizados en cada capa, como se muestra en la Tabla 3.6:

1. ***Capa convolucional[32]:*** El termino convolución relacionado a imágenes, es en pocas palabras, la implementación de una función matricial en paralelo a nuestro fotograma que queremos analizar, como su homologo matemático en donde se implementan funciones simultáneamente, la diferencia es que en este proceso hacemos uso de filtros.

   Los filtros son la unión de pequeñas matrices NxN los cuales se denominan kerneles; cada kernel tiene una característica específica, como por ejemplo: extraer texturas, bordes, formas y demás, en la Figura 3.23 vemos un ejemplo de la implementación de un kernel que extrae bordes.

   Cabe mencionar que los filtros son implementados en cada canal para nuestra propuesta, si fueran imágenes en escala de grises, solo se aplicaría una vez, al contrario que las imágenes RGB que se implementa 3 veces además que, la imagen resultante después de la convolución pasa a ser de tres canales a uno sólo.

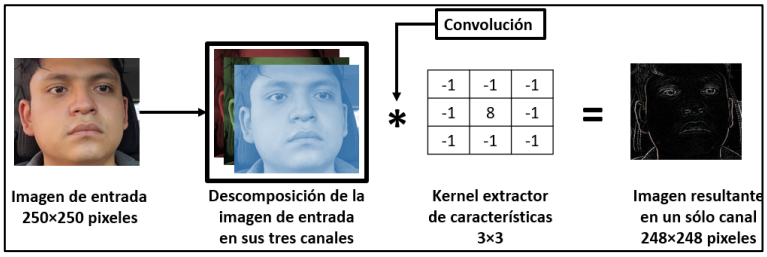


Figura 3. 23: Ejemplo del proceso de convolución

2. ***Número de filtros[33-37]:*** Un rasgo característico que podemos efectuar al momento de entrenar, es que podemos decidir cuantos filtros podemos implementarle a una imagen de entrada, por ejemplo, si decidimos utilizar 10 filtros, obtendremos 10 imágenes resultantes con diferentes patrones, esto a su vez es directamente proporcional a cuantas características requerimos por imagen, cabe mencionar que al implementar dichos filtros, la imágenes resultantes serán de una dimensión menor (MxN) con respecto a la original, dependiendo el tamaño del Kernel (2×2, 3×3,11×11, etc.), en la Figura 3.23, podemos observar un ejemplo de cómo el fotograma original de tamaño 250×250 pixeles se redujo en 2 pixeles de ancho y largo dando como resultado un frame de 248×248 pixeles.
3. ***Linealización[33-37]:*** Los resultados que nos arrojan los valores dados por los filtros, vienen en mapas de características no lineales, por lo tanto, estudiar estos valores sería muy complicado entenderlos para el algoritmo posteriormente, por lo tanto se necesita una función matemática denominada ***activación*** que pueda linealizar estos resultados.
4. ***Muestreo[33-37]:*** El último parámetro que deseamos implementar para la arquitectura de nuestra RNC-PP, es la reducción de dimensiones espaciales (ancho x alto), el cual reducirá el número de parámetros que se entrenarán al momento de ingresar dichos patrones al perceptrón multicapa, pero no afectando las características espaciales por cada imagen de entrada.

   Al tener ya conocimiento previo de los puntos importantes al momento de crear la primera parte de la RNC-PP (extracción de características), en nuestra propuesta hemos optado en utilizar la siguiente configuración (Tabla 3.6):

   4.1 ***Primera capa convolucional:*** Hemos decidido implementar una primera capa convolucional de 16 filtros aleatorios con un Kernel de 3×3, la cual nos genera 16 imágenes a partir de la imagen principal de entrada con una reducción de 2 pixeles donde nuestra imagen de entrada pasa de tener 150×150 pixeles a 148×148 pixeles, a su vez, la función de activación que hemos decidido utilizar es a la unidad lineal

rectificada (Figura 3.19 d) ϕ(q)=max (0,q) [37] propia de la librería utilizada, así como también la reducción de dimensiones espaciales utilizada fue la implementación de la operación MaxPoolig [38] con una ventana de 2×2, el cual encuentra el valor máximo de pixel en la región de muestreo y lo pasa como pixel único a la nueva imagen, creando 16 imágenes de salida con un tamaño final de 74×74 pixeles.

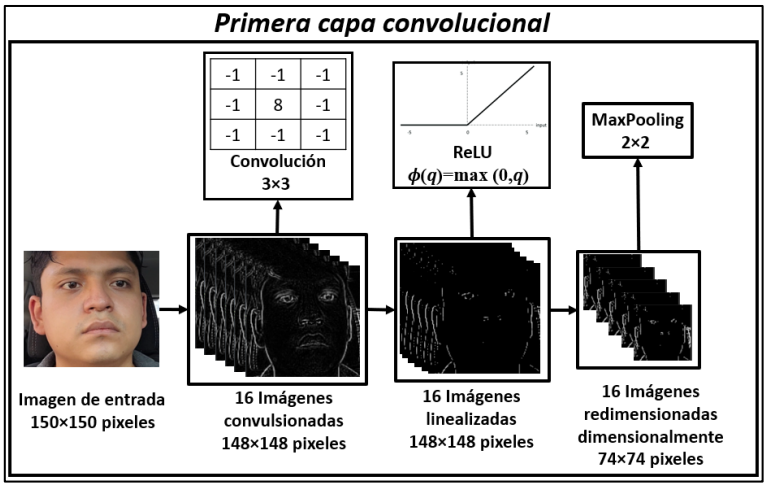


Figura 3. 24: Ejemplo del proceso de la primera capa convolucional de la RNC-PP propuesta (primera versión)

4.2 Segunda capa convolucional: En la segunda capa convolucional, implementamos 32 filtros a cada 1 de las 16 imágenes creadas con la primera capa convolucional, por lo que tenemos 32 imágenes resultantes por cada una de las 16 imágenes de la primera capa, con una dimensión reducida de 74×74 pixeles a 72×72 pixeles, posteriormente, al igual que la primera capa, hemos decidido utilizar a ReLU (ϕ(q)=max (0,q)) como activación, así como también, implementamos de MaxPooling como reductor de dimensiones espaciales utilizando una ventana 2×2, teniendo como resultado 32 imágenes de salida por cada 16 de entrada con un tamaño final de 36×36 pixeles cada una, dando un resultado de 512 imágenes nuevas.

4.3 Tercera capa convolucional: En esta última capa convolucional, repetimos el proceso de la capa anterior, implementamos 16 diferentes filtros a cada una de las 512 imágenes, por lo que tenemos 8,192 imágenes resultantes, con una dimensión reducida de 36×36 pixeles a 34×34 pixeles, posteriormente, al igual que la primera y segunda capa convolucional, hemos decidido utilizar a ReLU (ϕ(q)=max (0,q)) como activación, así como también, la implementación de MaxPooling como reductor de dimensiones espaciales utilizando una

ventana 2×2, teniendo como resultado 16 imágenes de salida por cada 512 imágenes de entrada con un tamaño final de 17×17 pixeles cada una.

5) ***Aplanamiento[35]:*** Teniendo ya las imágenes resultantes después de la etapa de extracción de características, necesitamos convertir estos datos a unos que entiendan las redes neuronales o el perceptrón multicapa, por lo tanto, lo único que debemos realizar, es un simple redimensionamiento de la matriz M×N resultante a un vector de características, a este proceso se le denomina Flattten (Figura 3.19 i), en la Figura 3.25 vemos un ejemplo de este proceso.

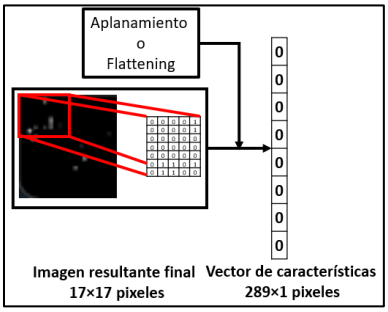


Figura 3. 25: Ejemplo del aplanamiento de información

6) ***Completamente conectada[34-36]:*** Finalmente nos toca implementar la parte de las redes neuronales, en este proceso sólo haremos uso de dos capas ocultas, las cuales tendrán la siguiente configuración:

1. ***Primera capa oculta:*** En esta parte haremos uso de 128 neuronas convencionales totalmente conectadas las cuales estarán acopladas directamente al vector de características resultante del aplanamiento, adicionalmente se utilizó una linealización de ReLU ($\phi(q)=\max(0,q)$).
2. ***Segunda capa oculta:*** Finalmente hacemos uso de una última capa de redes neuronales, con un número de neuronas igual a 2, las cuales extraerán los últimos patrones resultantes de la capa anterior y los clasificaren en dos estados probabilistamente, si la persona tiene rasgos de un estado normal de conducción nos da como resultado un valor cercano a 1, de lo contrario, si la red determina que la persona que tiene patrones de somnolencia el valor que nos estará arrojando será cercano 0; en este proceso, la activación ya no será por ReLU si no que haremos uso de una nueva función llamada sigmoide $\phi_q = \frac{1}{1+e^{-q}}$, esta función fue seleccionada debido a que después de diversas pruebas, fue la que mejor rendimiento obtuvo al momento del entrenamiento.

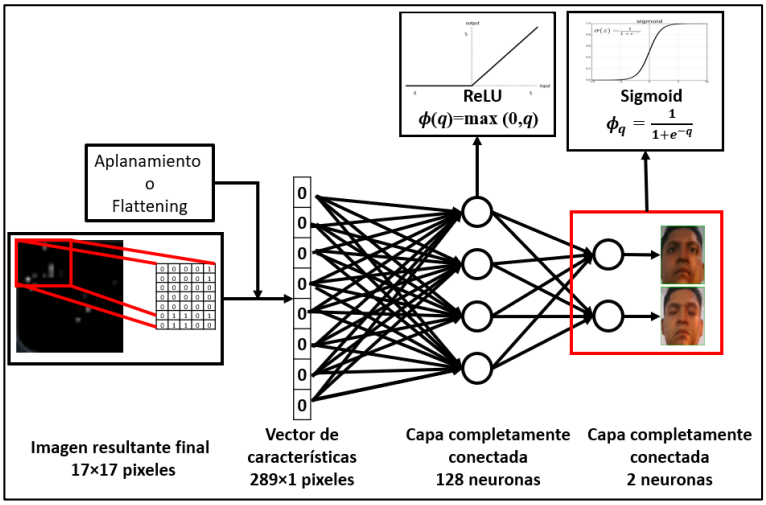


Figura 3. 26: Proceso de acoplamiento de la etapa convolucional a la etapa de implementación a las capas ocultas de las redes neuronales

7) ***Optimizador[34-36]:*** En la etapa de entrenamiento de la RNC-PP propuesta, utilizamos el optimizador Adaptive Moment Estimation (Adam) [39] para minimizar la función de pérdida de entropía cruzada binaria. Para la RNC-PP propuesta, el optimizador Adam muestra un mejor rendimiento en comparación con otros optimizadores. El optimizador de Adam y la función de pérdida están dados por (x) y (y).

$$\boldsymbol{w_{t+1}} = \boldsymbol{w_t} - \boldsymbol{\alpha}\left(\frac{\boldsymbol{m_t}}{\sqrt{\boldsymbol{v_t} + \boldsymbol{eps}}}\right) \tag{10}$$

Donde:

- $\boldsymbol{w_{t+1}} = \beta_1 m_t + (1 - \beta_1)(\boldsymbol{w_t} - \boldsymbol{w_{t-1}})$
- $v_{t+1} = \beta_2 v_t + (1 - \beta_1) G_t$
- $G_t = (\boldsymbol{w_t} - \boldsymbol{w_{t-1}})^2$
- eps es una máquina de épsilon que evita la división entre 0.
- Nosotros usamos a :
- $\beta_1 = 0.9$
- $\beta_2 = 0.999$
- Tasa de aprendizaje $\boldsymbol{\alpha}$ = 0.001

$$\mathbf{loss} = -\frac{\mathbf{1}}{\mathbf{n}}\sum_{\mathbf{i=1}}^{\mathbf{n}}(\mathbf{y_i}\mathbf{log}\,(\hat{\mathbf{y}}_\mathbf{1}) + (\mathbf{1} - \mathbf{y_i})\mathbf{log}\,(\mathbf{1} - \hat{\mathbf{y}}_\mathbf{1})) \tag{11}$$

En donde: y_i es la salida real y $\hat{y}_1$ es la salida deseada, finalmente N es el número del tamaño del lote que en nuestro caso fue de 64.

Para finalizar este apartado, la RNC-PP que proponemos nos arroja dos clasificaciones probabilísticas, las cuales en 3.25 haremos su análisis y su caracterización.

3.2.5 Análisis de resultados consecutivos

Como observamos en el punto 3.23, la idea principal del algoritmo de reconocimiento facial por medio de clasificadores de cascada es determinar si existe o no un rostro, por tal motivo, es fácil determinar que:

A. Si existe un rostro a estudiar se pondrá una etiqueta denominada ***"Si"***, esta será utilizada en el algoritmo de la Figura 3.27 que posteriormente se explicará.

B. Si no existe un rostro a estudiar o el algoritmo empleado para la detección de rostros no es capaz de reconocer alguno, obtendremos una etiqueta denominada ***"No"***, que de igual manera será utilizada en el algoritmo de la Figura 3.27 que posteriormente se explicará.

Teniendo como premisa estos dos estados ***"No"*** y ***"Si"***, vemos que las salidas como resultado del análisis de este proceso, son binarias, 1 y 0 respectivamente, por ende, nosotros proponemos el siguiente algoritmo.

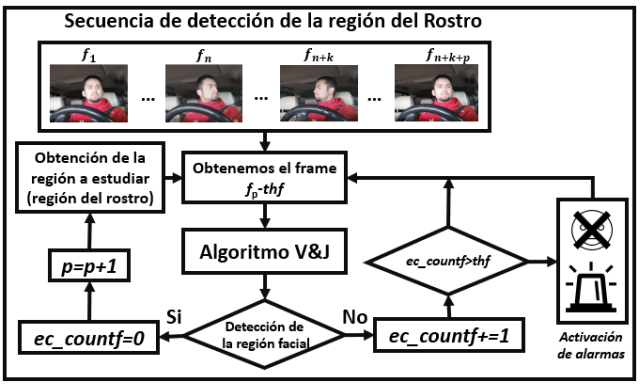


Figura 3. 27: Algoritmo del proceso de reconocimiento facial y sus diversas acciones cuando se detecta o no el rostro a estudiar

En la Figura 3.27 mostramos el proceso que se lleva para poder determinar si existe un rostro, el cual posteriormente se analizará haciendo uso del algoritmo de la Figura 3.32 . Este proceso también es implementado para poder prevenir la distracción en los conductores, debido a que si no existe un rostro o el algoritmo no lo detecta pero como tal si existe, hemos decidido tomarlo como un elemento de distracción, para ello hacemos uso de 3 estados resultantes que se pueden presentar:

1. ***Detección de la región a estudiar:*** Este estado es el que debería estar presente en todo el proceso de análisis, o bien, en su gran mayoría, esta parte del algoritmo propuesto funciona de la siguiente manera:
   1.1 Obtendremos la imagen de entrada (la cual es el resultado de la fragmentación del video en tiempo real comentado en el punto 3.22), la cual se le aplica el detector de reconocimiento facial propuesto por Viola y Jones (explicado en el punto 3.23), al detectar un rostro, hacemos uso de un contador que hemos denominado ***"ec_countf"*** el cual lo igualaremos a 0,

debido a que este es testigo para la detección de distracción o la no detección de la región a estudiar (explicado en los próximos puntos).

1.2 Posteriormente localizaremos el área del rostro en donde consecutivamente se realizan dos procesos en paralelo:

A) El primer proceso consta de la extracción del rostro localizado de la imagen original (Figura 3.28 b) para su estudio en el proceso siguiente.

B) El segundo proceso es la señalización digital de la región del rostro en la imagen mostrada por la interfaz del usuario (Figura 3.28 c).

1.3 Finalmente termina el análisis del fotograma actual f_p y continua con la siguiente imagen del video fragmentado f_{p+1}.

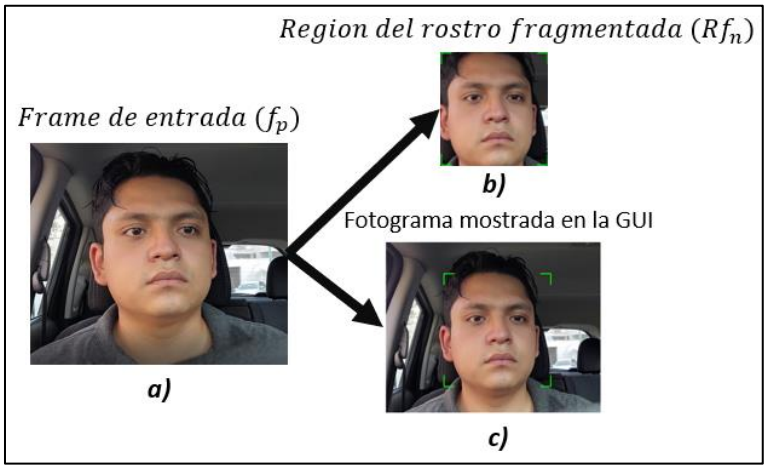


Figura 3. 28: Proceso de extracción del área a analizar y lo mostrado en GUI

En el proceso de la fragmentación del rostro, hacemos uso de 3 técnicas de procesamiento digital de imágenes:

a) ***Recorte de la región del rostro***: Este procesamiento se realiza obteniendo las coordenadas (x_1,y_1;… ;x_2,y_2) de la región del objeto detectado (Figura 3.29 b), posteriormente creamos una imagen (imagen copia) replicando solamente los pixeles y los canales internos de la zona del rostro(Figura 3.29 c), esta nueva imagen la denominaremos "rf_n"

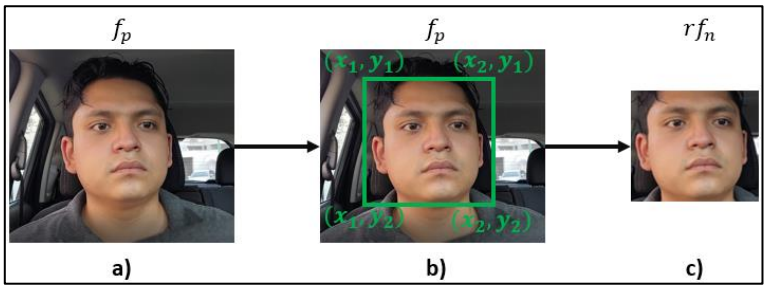


Figura 3. 29: Proceso de detección y fragmentación del rostro del conductor

b) ***Redimensionamiento de la imagen fragmentada:*** En este proceso realizamos una reducción de tamaño de la imagen rf_n, utilizando una técnica de procesamiento de imágenes denominada muestreo usando relación de área de píxeles o interpolación por área, comúnmente llamada ***INTER_AREA***, esta técnica de modificación de dimensiones tiene la característica de que es la más usada para poder reducir el tamaño de una imagen de acuerdo con [40].

El funcionamiento de ***INTER_AREA*** es el siguiente[40]:

I. Implementación de la ecuación de ***INTER_AREA***: Como se comentó, nuestro propósito es reducir a rf_n teniendo la mejor calidad para su mejor estudio, cuidando también la velocidad de procesamiento, por lo tanto hacemos uso de:

$$\boldsymbol{rf'_n(x,y) = \frac{1}{mn}\sum_{i=0}^{m}\sum_{j=0}^{n} rf_n(x_i, y_j)} \qquad (12)$$

Donde:

- rf'_n es el valor del área expresado en un píxel de 8 bits (teniendo un rango de 0 a 255) de la imagen resultante reducida actual.
- [x, y] son las coordenadas del píxel actual calculado.
- m es el ancho del bloque de pixeles que se van a procesar y reducir.
- n es lo largo del bloque de pixeles que se van a procesar y reducir.
- rf_n es la imagen recortada original.
- [x_i , y_j] son las coordenadas de los pixeles que se irán sumando consecutivamente (i=0,...,i=m y j=0,...,j=n).

II. Teniendo en mente la ecuación de ***INTER_AREA***, lo que sigue es determinar a **m** y **n**, en este caso pondremos como ejemplo a la Figura 3.30 a, donde obtenemos a nuestro fotograma de prueba (rf_n), con un tamaño inicial de 250×250 pixeles, posteriormente, queremos reducirlo a un tamaño de 125×125 pixeles que será rf'_n , matemáticamente hablando lo podemos representar como:

$$\boldsymbol{m,n = \frac{ancho\ de\ rf_n}{ancho\ de\ rf'_n}, \frac{largo\ de\ rf_n}{ancho\ de\ rf'_n}} \qquad (13)$$

$$\boldsymbol{m,n = \frac{250\ pixeles}{125\ pixeles}, \frac{250\ pixeles}{125\ pixeles}} \qquad (14)$$

por lo tanto m= 2 y n = 2, debido a que la compresión de los pixeles se reduce a la mitad tanto el ancho como el largo de rf_n.

III. En este paso sólo aplicamos la reducción de pixeles por cada bloque **m, n** en la imagen rf_n haciendo uso de la ecuación de ***INTER_AREA*** (Figura 3.30 b) y vamos guardando el valor resultante de cada pixel $rf'_n(x,y)$ en una imagen nueva rf'_n.

IV. Finalmente tenemos la imagen reducida rf'n (Figura 3.30 c), como podemos observar rf_n se redujo al 50%.

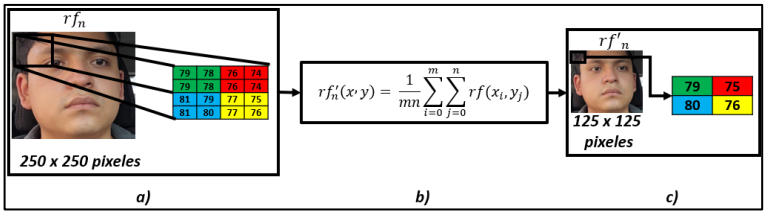


Figura 3. 30: Proceso de reducción utilizando INTER_AREA

NOTA: Cabe mencionar que en el ejemplo de la Figura 3.30, sólo se tomó como representación un sólo canal, más específicamente el azul (B), hay que recordar que una imagen digital está dividida en 3 canales normalmente (Canal Rojo R, Canal Verde G y Canal azul B) [13], por lo tanto, el procedimiento para la reducción de rf_n es el mismo, sólo que se tendría que hacer para cada canal y al final únicamente se unen los tres canales creando la imagen rf'_n a color, si se quiere utilizar la imagen promedio o en escala a grises, la imagen rf_n tendría que pasar un procesamiento para su conversión y posteriormente aplicar el proceso de la Figura 3.30, en este caso, si sería idéntico al mostrado en la Figura 3.30.

Para nuestro caso real, las dimensiones (ancho y alto) de las imágenes rf_{n+1} se van modificando constantemente, por tal motivo como se aclara en el punto ***3.23-Detección de objetos mediante clasificadores en cascada basado en funciones Haar -Máximo y mínimo tamaño del cuadro delimitador,*** nuestra imagen mínima de detección de rostro será de 150×150 pixeles, para que no entre en conflicto con el algoritmo de reducción que en este punto se aborda, así como también por ser la normalización para el análisis de la imagen con la RNC-PP que proponemos.

Un punto importante a tener en cuenta es que *rf'n* será una imagen permanente de 150x150 pixeles y por otro lado rf_n, no tiene un valor de ancho y largo constante, por lo que **m** y **n** siempre estarán

cambiando debido a las múltiples variaciones que se tiene de la imagen recortada, entonces, ¿Qué pasará cuando **m** y **n** no tengan un valor entero?; bueno, eso lo corrige la biblioteca utilizada y el proceso consiste en:

- El proceso de obtener **m** y **n** es igual, por lo tanto se hace uso de la ecuación (12) y supongamos que queremos reducir la imagen de prueba rf_n (Figura 3.31 a) a un tamaño de rf'_n de 150×150 pixeles, por lo que nos queda como m=1,66… y n=1,66….
- Para poder obtener el valor del bloque que vamos a procesar en **m**×**n** para obtener el píxel $rf'_n(x,y)$; lo primero que se tomará será el valor entero, por lo tanto **m** y **n** en este caso valdrán 1, y el 0,666… restante lo tomará del nivel de color del píxel siguiente (en %), por ejemplo, si el cuarto píxel tiene un nivel de color de 100, se restará el 66.66…% quedando así el cuarto pixel fragmentado en 67 y 23, el 67 se tomara para el cálculo del primer bloque y el 9 para el segundo; y así consecutivamente hasta terminar de reducir toda la imagen, en la Figura 3.31 vemos una representación a color de cómo se comportaría este proceso.

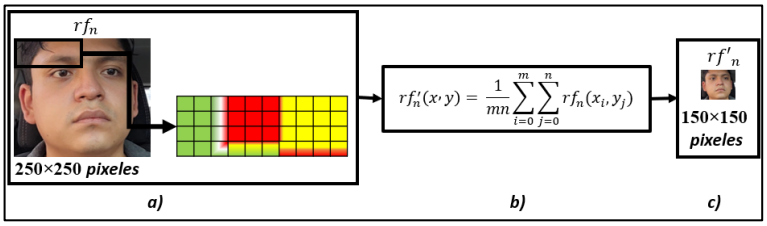


Figura 3. 31: Proceso de reducción utilizando INTER_AREA en el proceso real en SOMN_IA

Como observamos en la Figura 3.31 a, los colores representan las variaciones de los bloques **m**×**n**, esto se hace para poder ajustarse a los diversos tamaños de imágenes de entrada y normalizando las imágenes de salida a nuestro tamaño requerido (Figura 3.31 c).

Una alternativa para ***INTER_AREA*** es la interpolación del vecino más cercano (**INTER_NEAREST**) [41] o la interpolación lineal (**INTER_LINEAR**) [41], anqué si son eficientes y técnicamente tiene una mayor velocidad de cálculo computacional, su calidad de imagen se pierde, dando como resultado la perdida de información fundamental para el estudio de somnolencia al momento de aplicarle la **RNC-PP propuesta.**

2. ***Detección de no atención o no reconocimiento de rostros***: Este es el proceso alterno si el algoritmo de detección de rostro no logra detectar la región de estudio, para esto hacemos uso del contador que previamente se había creado, ***"ec_countf"*** se incrementa en una unidad cada vez que el sistema no es capaz de reconocer la región de la cara del usuario, si ***"ec_countf"*** supera un umbral predeterminado (*thf=N*), se activa una alarma que advierte al conductor que preste atención a la carretera y/o advierte que el sistema no puede detectar el rostro del conductor (Figura 3.34 c); el proceso de ejecución de alarmas pertinentes se ve en el apartado 3.25.

   El valor del umbral *thf* debe ser consistente a la velocidad de monitoreo del dispositivo en el que se implementa el algoritmo de detección de somnolencia y distracción, por ende, en este apartado sólo tomaremos como *thf*=38 fotogramas debido a que únicamente haremos las pruebas de rendimiento en el dispositivo "***Dis_1***" (3070TI GPU, 8GB Memory, 6144 cores, 1.770MHz); en el punto **3.6 Automatización de umbral para la detección de Somnolencia y Distracción** proponemos un mejoramiento que es la automatización para este umbral en todos los dispositivos tanto móviles como fijos.

   El valor de *thf* que proponemos es equivalente a 1.8 segundos, debido a que en ***Dis_1***, la velocidad promedio de reconocimiento en tiempo real y aplicando la ***RNC-PP*** propuesta es de 21 fps. El motivo por el cual se decidió delimitar el umbral a 1.8 segundos radica a la distancia que el conductor puede recorrer con una velocidad promedio de 80Km/h, esto se puede ejemplificar fácilmente con lo siguiente: Si una persona conduce a una velocidad promedio de 80 km/h y el usuario se distrae durante 2s, esto hace que el conductor haya recorrido un total de 44,44m sin prestar atención, esto es aproximadamente el ancho de una cancha de futbol profesional (45m), por lo tanto, tener un valor menor a 2 segundos es fundamental para evitar accidentes.
3. ***Ruptura del ciclo de alarma:*** La alarma estará activa siempre y cuando el sistema no pueda detectar la región del rostro de forma consecutiva, por lo tanto, si el sistema en algún punto detecta la zona de estudio, automáticamente ***"ec_countf"*** se reinicia (***ec_countf = 0***), y pasa por el primer proceso que anteriormente se mencionó, desactivando la alerta consigo y finalmente el sistema si cicla infinitamente o hasta que el sistema termine de monitorear al conductor.

La siguiente parte del análisis de resultados, es el proceso para la determinación de somnolencia, parpadeo y conducción normal con ayuda de las predicciones dadas por la ***RNC-PP*** propuesta. LA ***RNC-PP*** propuesta nos arroja dos resultados los cuales los hemos etiquetado como:

- Si la ***RNC-PP*** estima que el usuario tiene patrones de una conducción normal (valores cercanos a 1 de acuerdo con los resultados probabilísticos de la RNC-PP propuesta) entonces su etiqueta será ***"ojos abiertos".***
- Si la ***RNC-PP*** estima que el usuario tiene patrones de somnolencia (valores cercanos a 0 de acuerdo con los resultados probabilísticos de la RNC-PP propuesta) su etiqueta será ***"ojos cerrados"***.

Teniendo la base de estas predicciones, hemos implementado el siguiente algoritmo para poder determinar los estados ***estado normal***, ***parpadeo*** y ***somnolencia***:

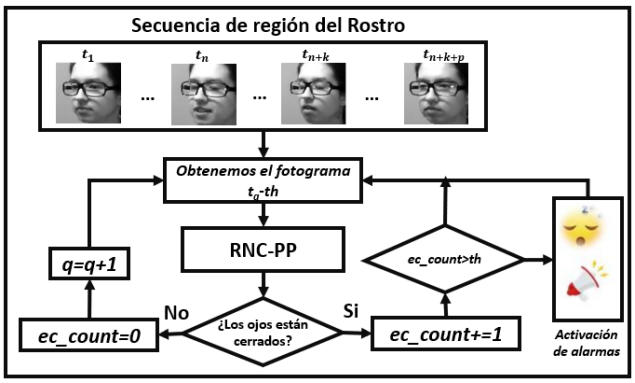


Figura 3. 32: Algoritmo propuesto para el reconocimiento de somnolencia en el conductor

En la Figura 3.32, podemos observar el proceso general para la determinación de somnolencia en los conductores, hay que tener en cuenta que la imagen de entrada será la imagen fragmentada y reducida por el proceso anterior, por lo tanto, a esta imagen la denominaremos como t_q (t_q en este caso es la imagen rf'_n del proceso anterior); teniendo esto en cuenta, en el algoritmo propuesto pueden suceder los estados:

1) ***Estado normal:*** Cuando la ***RNC-PP*** propuesta establece que la imagen t_q se clasifica como ***"ojos abiertos"***, entonces el conductor está prestando atención al camino y no existe ninguna anomalía o peligro al trasladarse, es una conducción correcta, por lo tanto, igualamos un contador que hemos denominado ***"ec_count"*** a cero, debido que este será el testigo para poder diferenciar los dos estados siguientes que podrían suceder (estado 2 ***"Parpadeo"*** y estado 3 ***"Somnolencia"***); posteriormente termina el proceso de este estado y sigue analizando la siguiente imagen t_{q+1}.
2) ***Parpadeo:*** Este estado es uno de los principales que hay que poder diferenciar; un parpadeo es el cierre de ojos por un tiempo N, dicha proceso es involuntario y en todo caso es natural, por lo tanto nos dimos a la tarea de poder generar una diferenciación entre dormir y parpadear. De acuerdo con [42], un parpadeo normal de un adulto dura alrededor de 290 a 750 milisegundos y la velocidad de nuestro sistema en promedio es de 21 fps, por ende decidimos poner a nuestra variable *th* igual a 36 (*th* es nuestro umbral de diferenciación entre parpadeo y somnolencia);

por lo tanto nuestro contador testigo ***"ec_count"*** se irá incrementando en una unidad, cada vez que la ***RNC-PP*** propuesta clasifique a t_q como ***"ojos cerrados"*** ; si ***"ec_count"*** es supera a *th*, el estado 3 ***"Somnolencia"*** entra en acción; en el caso de que t_q se clasifique como ***"ojos cerrados"*** un menor tiempo y la ***RNC-PP*** detecte el cambio de estado a ***"ojos abiertos"***, ***"ec_count"*** se igualará a cero, y el estado 1 entrara en acción como se observa en la Figura 3.33

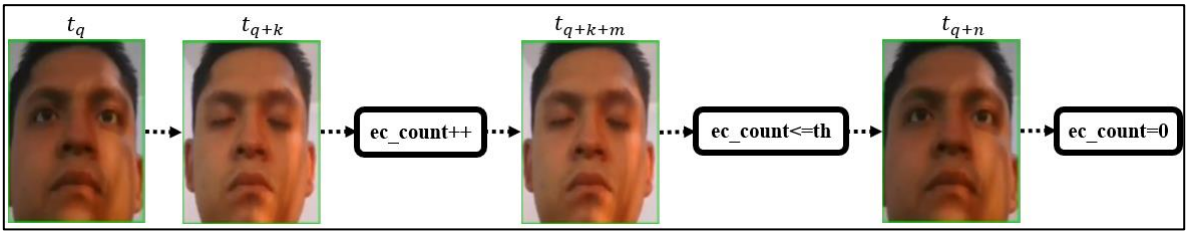


Figura 3. 33: Proceso de detección de parpadeo en el conductor

3) ***Somnolencia***: Detectar este estado fundamental, por tal motivo, si "***ec_count"*** sobrepasa a *th*, entonces esto quiere decir que el conductor se ha quedado dormido, llegado a este punto, las alertas pertinente se activan (en el punto 3.25 se explica el proceso) y estas no se desactivaran hasta que ***"ec_count"*** se iguale a cero, en otras palabras, la única forma en que se apaguen las alarmas es que el usuario se despierte, después de esto, el sistema se cicla infinitamente hasta el final de la conducción o hasta que el algoritmo de detección de rostro deje de mandar imágenes t_q.

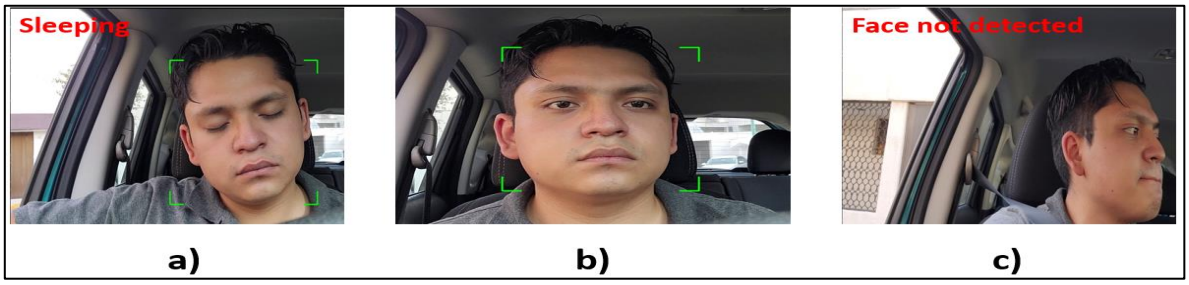


Figura 3. 34: Ejemplo de los estados detectados por SOMN_IA en condiciones reales

3.2.6 Activación de alarmas pertinentes

Las alarmas que nosotros manejamos en el algoritmo de detección de somnolencia y distracción en los conductores así como en SOMN_IA, son configuraciones un tanto diferentes debido a la naturaleza de los dispositivos utilizados, por un lado, cuando aplicamos el monitoreo en terminales fijas y con un poder de procesamiento elevado como Laptops, PC, Workstation, etc.; tenemos la facilidad de utilizar los recursos y el hardware que vienen incluidos de fábrica (Monitor y bocinas); por el otro, cuando utilizamos a SOMNI_IA o alguna tarjeta de desarrollo dedicada, nos vemos en la necesidad de utilizar sistemas externos los cuales harán la tarea de alarmas externas,

esto lo hacemos para reducir los recursos limitados que estos tienen, haciendo más fluido y certero el monitoreo. En ambas condiciones, las alarmas son del mismo tipo:

I. ***Alarma Visual***: Este tipo de alarma tiene la característica de utilizar elementos lumínicos, los cuales son principalmente usados para poder despertar a los conductores; cuando lo aplicamos en ***dispositivos de laboratorio*** (PC, laptops, Workstation, etc.,) su funcionamiento se queda un poco limitado debido al limitantes del sistema, por otro lado, cuando se aplica en dispositivos móviles (SOMN_IA, tarjeta de desarrollo, etc.) el funcionamiento se mejora considerablemente, debido al trabajo en conjunto con un sistema externo enfocado puramente para alarmas lumínicas (este proceso se explica en el punto ***1. Alarma visual – b) Configuración de la alarma visual en el dispositivo móvil***). Esta alarma tiene mejores resultados en el monitoreo con bajo nivel de iluminación.

II. ***Alarma Sonora:*** Este tipo de alarma hace uso de los elementos acústicos, cuando se emplea el sistema en dispositivos de laboratorio, se utilizan las bocinas internas de la terminal empleada, en el caso de utilizarse en los módulos móviles, hacemos uso de un sensor piezoeléctrico, el cual funciona por pulsos eléctricos (este proceso se explica en el punto ***2. Alarma Sonora – b) Configuración de la alarma sonora en el dispositivo móvil***).

III. ***Alarma informativa:*** El propósito fundamental de este tipo de alarma, sólo es avisarle al conductor el porqué las alarmas se activaron, puede ser que el sistema no detecte el rostro así que el usuario debe prestar atención o bien informar que presenta señales de sueño y por tal motivo se activaron las alarmas (Figura 3.34 c, a respectivamente).

Las configuraciones de las alarmas son las siguientes:

1. Alarma visual

a) Configuración de la alarma visual en el dispositivo de laboratorio:

I. ***Alarma visual por distracción***: Esta alarma se activa cuando el contador ***"ec_countf"*** sobrepasa el umbral *thf*, por lo tanto, el sistema automáticamente muestra una imagen donde advierte al conductor que no está prestando atención al camino (Figura 3.35 b), cuando el conductor vuele a prestar atención al camino o el algoritmo detecta su rostro, ***"ec_countf"*** se iguala a cero y la imagen de alertar se autodestruye, continuando con el ciclo normal del proceso (Figura 3.35 c).

II. ***Alarma Visual por somnolencia***: Esta alarma se activa cuando el contador ***"ec_count"*** sobrepasa el umbral *th*, por lo tanto, el sistema automáticamente muestra una imagen con alto contraste donde advierte al conductor que se está quedando dormido o en todo caso que se ha quedado dormido (Figura 3.35 d), cuando el conductor

vuelve a tener patrones de una conducción normal, ***"ec_count"*** se iguala a cero y la imagen de alertar se autodestruye, continuando con el ciclo normal (Figura 3.35 e).

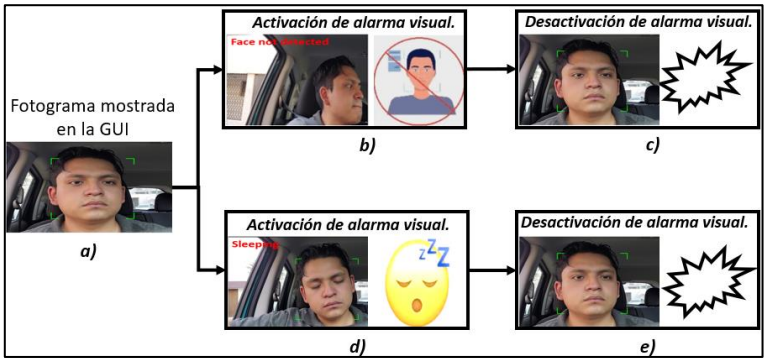


Figura 3. 35: Procesos de activación y desactivación de las alarmas visuales en los dispositivos de laboratorio en donde se implementa el software de detección de somnolencia y distracción propuesto

b) ***Configuración de la alarma visual en el dispositivo móvil****:* Como comentamos en puntos anteriores, los dispositivos móviles tienen limitaciones tanto internas como externas, por lo tanto, hemos creado una configuración que tenga la tarea de alertar a los conductores que no está prestando atención o se esté quedando dormido; la funciona de la siguiente manera:

1. Cuando el sistema detecta el estado de emergencia (distracción o somnolencia), manda una señal a un microcontrolador externo al sistema, en donde está codificado un algoritmo.
2. Al detectar la señal de alerta, el microcontrolador activa o enciende N tiras de leds ultrabrillantes las cuales harán la función de alerta.

NOTA: N es el valor tiras de LED's ultrabrillantes que se van a activar, SOMN_IA está equipada con 4.

Con estos simples pasos, si bien no son los más complejos, ayudan mucho al sistema, debido a que el algoritmo principal se depura, y este disminuye los procesos necesarios para poder despertar o alertar al conductor, además que al no tener la carga de las imágenes (como en la configuración previa a esta), el ejecutable se hace más ligero.

Otro punto que hay que destacar, es que el sistema ya no tendrá que esperar a terminar de mostrar las imágenes para continuar el monitoreo, si no que seguirá estudiándolo de forma constante, lo que hace que requiramos menos poder computacional. A este proceso lo hemos denominado ***"Subsistema de Alerta Lumínica"*** (SAL)



Figura 3. 36: Proceso de activación de SAL (Subsistema de Alerta Lumínica)

I. ***Alarma visual por distracción***: Esta alarma se activa cuando el contador ***"ec_countf"*** sobrepasa el umbral *thf*, por lo tanto, automáticamente entra en acción ***SAL***, donde N será de 2 solamente, esto es suficiente para advertir al conductor que no está prestando atención al camino (Figura 3.37 b), cuando el conductor vuelve a prestar atención al camino o el algoritmo detecta su rostro, ***"ec_countf"*** se iguala a cero y SOMN_IA o el dispositivo móvil en turno manda una señal diferente para que el ***SAL*** se desactive (Figura 3.37 c) continuando así con el ciclo normal.

II. ***Alarma Visual por somnolencia***: Esta alarma se activa cuando el contador ***"ec_count"*** sobrepasa el umbral *th* y automáticamente entra en acción ***SAL***, donde N será en este caso 4, esta acción es suficiente para avisar al conductor que se está quedando dormido o en todo caso que se ha quedado dormido(Figura 3.37 d), cuando el conductor vuelve a tener patrones de una conducción normal, ***"ec_count"*** se iguala a cero y SOMN_IA o el dispositivo móvil en turno manda una señal diferente para que el ***SAL*** se desactive y apague todas las tiras de LED's ultrabrillantes, continuando así con el ciclo normal (Figura 3.37 e).

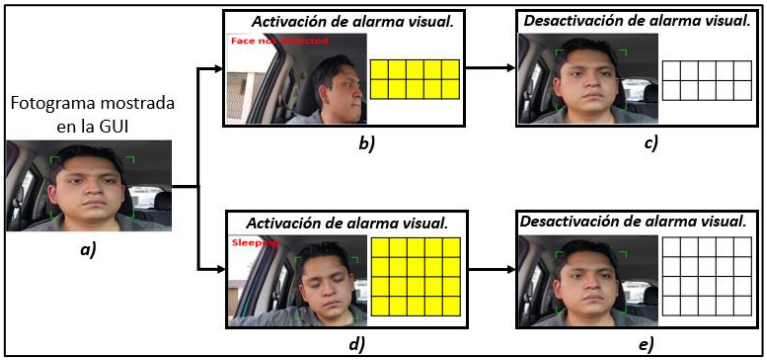


Figura 3. 37: Procesos de activación y desactivación de SAL en los diferentes dispositivos móviles en donde se implementa el software de detección de somnolencia y distracción propuesto

2. Alarma Sonora

a) Configuración de la alarma sonora en el dispositivo de laboratorio:

I. ***Alarma sonora por distracción***: Al igual que la alarma visual, esta se activará cuando ***"ec_countf"*** sobrepasa el umbral *thf*, por lo tanto, el

sistema automáticamente reproducirá un sonido de alerta (**https://www.youtube.com/watch?v=ZJ4ds4Z-iYw&t=1778**) donde advierte al conductor que no está prestando atención al camino (Figura 3.38 b), cuando el conductor vuelve a prestar atención al camino o el algoritmo detecta su rostro, ***"ec_countf"*** se iguala a cero y el sonido automáticamente se detiene, continuando con el ciclo normal del proceso (Figura 3.38 c).

II. ***Alarma sonora por somnolencia***: Al igual que la alarma visual, esta alerta se activará cuando ***"ec_count"*** sobrepasa el umbral *th*, por lo tanto, el sistema automáticamente reproducirá un sonido de alerta muy molesto y a gran volumen (**https://www.youtube.com/watch?v=ZJ4ds4Z-iYw&t=1778**) donde el propósito es despertar al conductor (Figura 3.38 b), por tal motivo, el sistema seguirá sonando hasta que el sistema detecte el cambio de estado, cuando el conductor vuelve a tener patrones de una conducción normal, ***"ec_count"*** se iguala a cero y el sonido automáticamente se detiene, continuando con el ciclo normal del proceso (Figura 3.38 c).

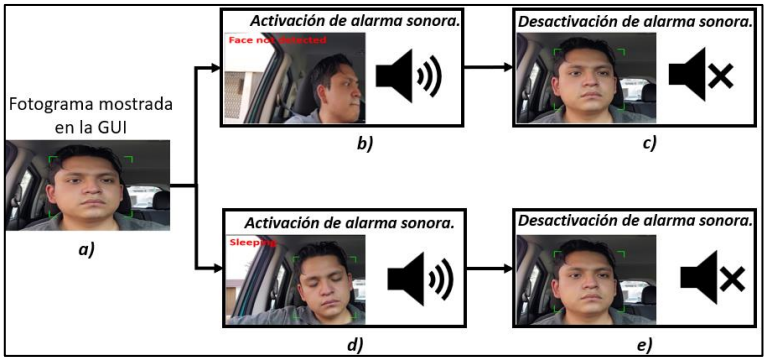


Figura 3. 38: Procesos de activación y desactivación de las alarmas sonoras en los dispositivos de laboratorio en donde se implementa el software de detección de somnolencia y distracción propuesto

b) ***Configuración de la alarma sonora en el dispositivo móvil:*** Al igual que las alertas lumínicas para dispositivos móviles, en este caso hemos creado un subsistema denominado "***Subsistema de Alerta Sonora"*** (SAS), el cual funciona de la siguiente manera:

1. Cuando el sistema detecta el estado de emergencia (distracción o somnolencia), manda una señal a un microcontrolador externo al sistema, en donde está codificado un algoritmo.

2. Al detectar la señal de alerta, el microcontrolador activa o enciende un buzzer o zumbador piezoeléctrico normalmente activado, el cual

empieza a sonar una cadena de pitidos, dicha secuencia dependerá del estado de alerta que se presente.



Figura 3. 39: Proceso de activación de SAS (Subsistema de Alerta Sonora)

I. ***Alarma sonora por distracción***: Esta alarma se activa cuando el contador ***"ec_countf"*** sobrepasa el umbral *thf*, por lo tanto, automáticamente entra en acción **SAS**, con la secuencia ***I***, esto es suficiente para advertir al conductor que no está prestando atención al camino (Figura 3.40 b), cuando el conductor vuelve a prestar atención al camino o el algoritmo detecta su rostro, **"ec_countf"** se iguala a cero y SOMN_IA o el dispositivo móvil en turno manda una señal diferente para que el **SAS** se desactive y con ellos automáticamente silenciar a buzzer (Figura 3.40 c) continuando así con el ciclo normal.

II. ***Alarma sonora por somnolencia***: Esta alarma se activa cuando el contador ***"ec_count"*** sobrepasa el umbral *th* y automáticamente entra en acción ***SAS***, con la secuencia ***II***, esta acción es suficientemente desagradable y molesta de escuchar pero eficiente para avisarle al conductor que se está quedando dormido o en todo caso que se ha quedado dormido(Figura 3.40 d), cuando el conductor vuelve a tener patrones de una conducción normal, ***"ec_count"*** se iguala a cero y SOMN_IA o el dispositivo móvil en turno manda una señal diferente para que el ***SAS*** se desactive y con ello pare el sonido molesto, continuando así con el ciclo normal (Figura 3.40 e).

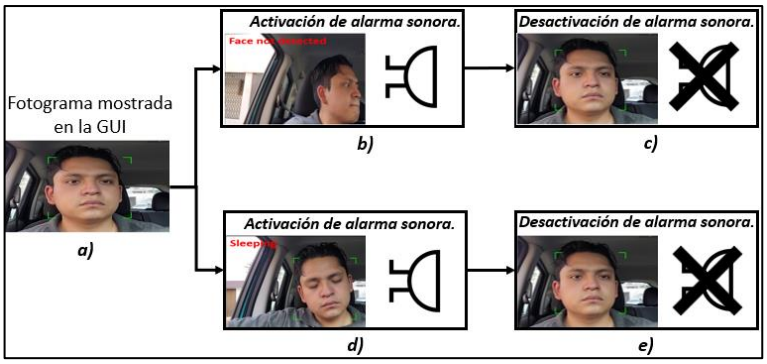


Figura 3. 40: Procesos de activación y desactivación de SAS en los diferentes dispositivos móviles en donde se implementa el software de detección de somnolencia y distracción propuesto

3. Alarma Informativa.

Esta alarma a diferencia de las dos anteriores, no tiene diferencia de activación, es igual tanto para dispositivos de laboratorio como en dispositivos móviles, por lo tanto:

I. ***Alarma informativa por distracción***: Esta alarma se activa cuando el contador ***"ec_countf"*** sobrepasa el umbral *thf*, por lo tanto, envía a la GUI una leyenda **"*Face not detected*"**, para avisarle el motivo al conductor del porque las alertas sonoras y lumínicas se han activado (Figura 3.34 c), cuando el conductor vuelve a prestar atención al camino o el algoritmo detecta su rostro, **"ec_countf"** se iguala a cero y la leyenda desaparece automáticamente (Figura 3.34 b) continuando así con el ciclo normal.

II. ***Alarma sonora por somnolencia***: Esta alarma se activa cuando el contador ***"ec_count"*** sobrepasa el umbral *th* y automáticamente en la GUI aparece la leyenda ***"Sleeping"*** (Figura 3.34 a), cuando el conductor vuelve a tener patrones de una conducción normal, ***"ec_count"*** se iguala a cero y la leyenda desaparece, continuando así con el ciclo normal (Figura 3.34 b).

3.2.7 Resultados obtenidos del subcapítulo

1. **Análisis de resultados de la RNC-PP propuesta:** En la Tabla 3.7 podemos observar la arquitectura general de la RNC-PP que proponemos, algo que hay que tomar muy en cuenta, es que nuestra configuración dio un total de casi 602,000 parámetros de entrenamiento, que si bien, son una gran cantidad, para el análisis de imágenes, este valor es muy reducido, ya que arquitecturas convencionales como Mobilenet[28], VGG16[43], Xception[29], etc., tienen más de un millón de parámeros entrenados, por lo tanto nuestra propuesta fue un rotundo éxito.

   Tomando en cuenta el valor de los parámetros entrenados, así como la cantidad de imágenes de la base de datos, la precisión que tenemos resultante es sumamente excepcional, ya que al tener sólo el 1% de error, nos aseguramos de que el monitoreo será sumamente seguro, esto consolida que la integridad de los conductores así como de terceros será sobre guardada de la mejor manera.

Tabla 3. 7: Arquitectura general de la RNC-PP

ESTRUCTURA PROPUESTA DE RNC-PP		
Nombre de la capa	Tamaño del mapa de características	Número de parámetros
Conv2D	148 × 148 × 16	448
MaxPooling2D	74 × 74 × 16	0
Conv2D	72 × 72 × 32	4640
MaxPooling2D	36 × 36 × 32	0
Conv2D	34 × 34 × 16	4624

MaxPooling2D	17 × 17 × 16	0
Flatten	4624	0
FC	128	592000
FC	1	129
Total de parámetros : 601,841 Parámetros entrenables: 601,841 Parámetros no entrenables: 0 Precisión= 98.95 [49]		

La Figura 3.41 se muestra el comportamiento del valor de pérdida y la precisión de los conjuntos de entrenamiento y validación. A partir de esta figura, podemos observar el buen comportamiento de entrenamiento sin sobreajuste[44].

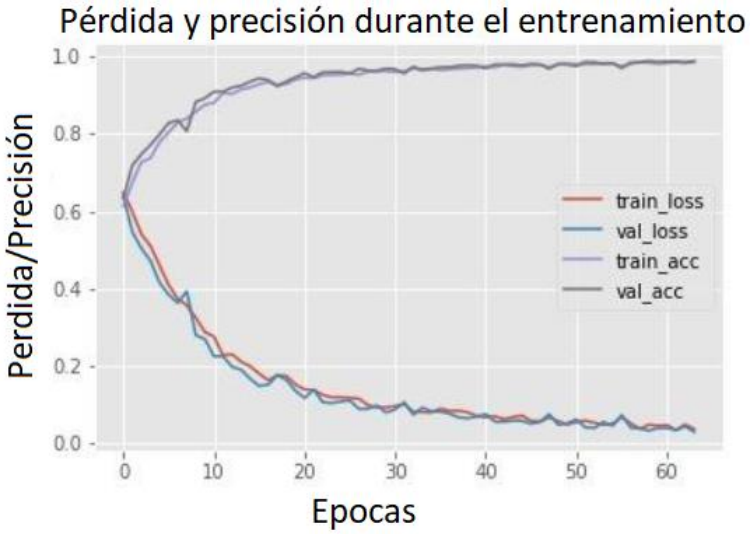


Figura 3. 41: Gráfica de perdida y precisión en el entrenamiento de la RNC para la detección de somnolencia.

El rendimiento de la RNC-PP propuesta se comparó con diversas investigaciones que utilizan el mismo conjunto de datos NTHU-DDD. En la Tabla 3.8 se muestra la comparación de rendimiento en podemos observar que el sistema que propones y por ende, la arquitectura que empleamos, supera a los diversos sistemas de somnolencia del conductor en tiempo real propuesto anteriormente.

Tabla 3. 8: Comparación de rendimiento entre varios sistemas de detección de somnolencia del conductor en tiempo real

Arquitecturas propuestas	Año	Precisión
Jabbar [47]	2018	80.93%
Hashemi [46]	2020	98.15%
Ayachi [45]	2021	96.05%
Propuesto	2021	98.95%

2. **Análisis de rendimiento del sistema propuesto :** En la Tabla 3.9 podemos observar la velocidad de monitoreo en los diversos dispositivos que se emplearon en esta parte de la tesis; como se muestra, el monitoreo empezó a bajar de velocidad conforme se van reduciendo el poder computacional, esto en cierto punto es se puede obviar, por lo tanto, a partir de este momento el siguiente proceso, es el mejoramiento del de la RNC-PP, así como la optimización para poder correr de forma más fluida el sistema en los diversos dispositivos en donde se implemente el detector de somnolencia y distracción.

Tabla 3. 9: Tabla comparativa empleado el sistema general de detección somnolencia y distracción en tiempo real en los diferentes dispositivos utilizados en esta investigación

Dispositivo	Velocidad de monitoreo en tiempo real a fotogramas por segundo(fps)			
	IMX219-77IR	Cámara Web HD C505	Cámara genérica	Video Cámara integrada Huawei D16
3070TI GPU, 8GB Memory, 6144 cores, 1.770MHz.	No Posee Puerto CSI	20	20	----------
Huawei D16 Laptop, AMD Ryzen 5 4600H	No Posee Puerto CSI	15	15	15
Nvidia Jetson AGX Xavier	No Posee Puerto CSI	6	6	----------
Nvidia Jetson Nano 4GB	7	4	5	----------
SOMN_IA	7	4	5	----------

En la Tabla 3.10, se muestra las velocidades de monitoreo que se alcanzaron en los diferentes dispositivos empleados en las pruebas de monitoreo en videos pre guardado; se realizaron un total 40 pruebas, en donde tomaron 10 videos por cada resolución y se sacó un promedio de la rapidez de monitoreo.

Cabe mencionar que la velocidad de reconocimiento tiende a aumentar si el detector de rostro no detecta alguna región a estudiar, por lo tanto, una característica de estos videos es que se seleccionaron cuidadosamente todos aquellos en donde la tasa de reconocimiento facial fuese cerca o mayor del 95% en todo el proceso y la cantidad de fotogramas estudiados en este test fue de 1500 fotogramas, por video.

De igual manera podemos notar que la velocidad de monitoreo tiende a reducirse cuando se trata de videos pre guardados en comparación al monitoreo en tiempo real.

Tabla 3. 10: Tabla comparativa en el proceso de fragmentación de videos pregrabados en los diferentes dispositivos utilizados en esta investigación

Dispositivo	Velocidad de reconocimiento en video a fotogramas por segundo(fps)			
	Video 1 Resolución: 1920 x1080	Video 2 Resolución: 1080 x 1920	Video 3 Resolución: 1280 x 720	Video 4 Resolución: 640 x 480
3070TI GPU, 8GB Memory, 6144 cores, 1.770MHz.	19.48	18.29	21.55	23.07
Huawei D16 Laptop, AMD Ryzen 5 4600H	17.44	16.30	19.72	23.43
Nvidia Jetson AGX Xavier	6.05	6.78	7.35	9.31
Nvidia Jetson Nano 4GB	3.26	2.9	3.44	4.48

Por último hacemos, en la Tabla 3.11, podemos ver una comparativa entre los costos netos de cada unidad utilizada en estas pruebas, ya que a partir de estos, podemos determinar, cual es la mejor opción al momento de implementar el sistema de detección de somnolencia y distracción.

Si bien, se tiene una tasa de recomiendo superar mientras más poderoso sea la unidad de cómputo, este es directamente proporcional al costo monetario que este representa; de igual manera podemos ultimar en este apartado, que el mejor distintivo en este momento y a su vez con el mejor rendimiento partiendo de la premisa de la precisión/precio, es la laptop Huawei D16 Laptop, AMD Ryzen 5 4600H y por otro lado, el peor sería la Nvidia Jetson AGX Xavier, debido a que aunque es un dispositivo móvil, el cual tiene un embebido compacto y muy fácil de adecuar, tiene un costo casi del 50% super al del dispositivo de laboratorio (laptop); de igual manera, vemos que la Nvidia Jetson Nano 4GB tuvo el peor rendimiento al momento del monitorio pero a su vez, su velocidad es cercana a la de la Nvidia Jetson AGX Xavier, el cual tiene un valor casi 4 veces mayor.

Como punto final de este apartado, podemos concluir que los elementos con menores ventajas computacionales son los óptimos de utilizar, dado esto y como se comentó en el punto anterior, en los siguientes apartados se abordará el proceso de mejoramiento de rendimiento y optimización para aumentar la tasa de reconocimiento en todos los dispositivos de prueba.

Tabla 3. 11: Costos por unidad utilizada en estas pruebas

Dispositivo	Costo
3070TI GPU, 8GB Memory, 6144 cores, 1.770MHz.	$78,000
Huawei D16 Laptop, AMD Ryzen 5 4600H	$21,000
Nvidia Jetson AGX Xavier	$30,149
Nvidia Jetson Nano 4GB	$8,500

3. **Ejemplos operacionales:** En la Figura 3.42 podemos observar algunos ejemplos del sistema en operación; en la Figura 3.42 a mostramos en la GUI la región del rostro a analizar así como la predicción dada por la RNC-PP, en la Figura 3.42 b, podemos observar la activación de la alarma lumínica (en los dispositivos de laboratorio), la predicción dada por la red neuronal y finalmente la alarma infamativa en la GUI, en la en la Figura 3.42 c y d, mostramos como el sistema no es capaz de detectar el rostro, por lo tanto no existe una región a analizar y finalmente se activan las alarmas pertinente respectivamente; finalmente en la Figura 3.42 e, vemos como el usuario está utilizando un dispositivo innecesario en la conducción y por consiguiente en la en la Figura 3.42 f, se activan las alarmas pertinentes para avisarle al conductor que no está prestando atención al camino.

En la Tabla 3.12 dejamos el enlace de algunos videos en donde mostramos el funcionamiento más a fondo del detector de somnolencia y distracción.

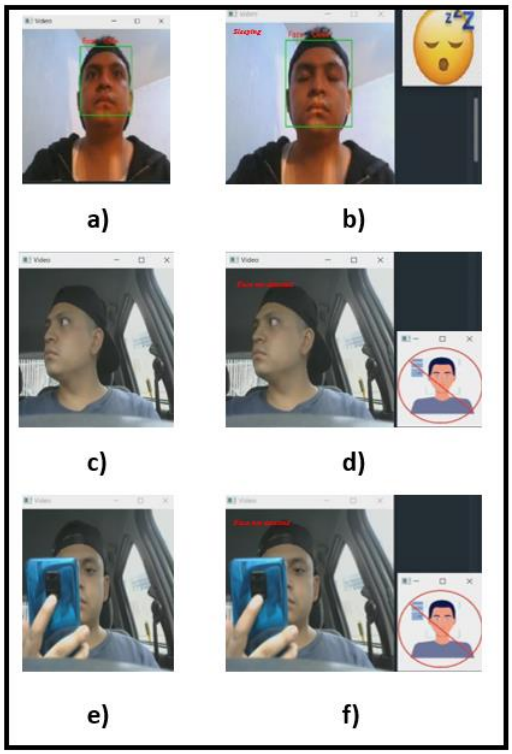


Figura 3. 42: Ejemplos operacionales del sistema de detección se somnolencia y distracción en tiempo real

Tabla 3. 12: Enlace de los videos de funcionamiento con una pequeña descripción

Enlace	Descripción
https://youtu.be/9rFOXpMPiGA	Demostración en el laboratorio en tiempo real.
https://youtu.be/Sjbj3Ae2q60	Demostración en tiempo real utilizando la base de datos NTHU-DDD.
https://youtu.be/f0K_lFp1QOc	Demostración bajo condiciones reales en tiempo real.

3.3 Mejoramiento del Sistema General

3.3.1 Mejoramiento del reconocimiento facial

El sistema propuesto tiene por objetivo ser universal, por tal motivo, se busca que pueda trabajar en condiciones no favorables y aun así mantenga la misma precisión en donde si son factibles, por esta razón se hicieron diversas pruebas de rendimiento en donde algunos conductores (en estado controlado) estuvieran realizando comportamientos exagerados a los normales para poder determinar que tan bien se comportaba en estos casos, el algoritmo de reconocimiento facial, la RNC-PP propuesta, el sistema en general (alarmas y determinación de somnolencia y distracción, etc.) fueron algunos

ejemplos que se analizaron; posteriormente se estudiaron los resultados y nos dimos cuenta que un elemento empezaba a no funcionar de la mejor manera en condiciones no tan anormales, este fue el algoritmo de reconocimiento facial; a continuación enlistamos los principales problemas que se presentaron con este algoritmo:

1) ***No detección de rostro:*** Si bien el algoritmo propuesto por Viola and Jones tiene elementos favorables como un consumo mínimo de procesamiento digital, así como el tiempo de reconocimiento es veloz, el algoritmo sólo es capaz de detectar rostros en condiciones sumamente controladas, esto quiere decir que si el sujeto o conductor mueve o rota el rostro tan sólo un poco (en promedio 15 grados a cualquier dirección), hay una alta probabilidad de que el reconocimiento facial falle y no detecte la región a estudiar hasta que se vuelva estar en una posición en específico (mayormente enfrente del sensor óptico) como se muestra en la Figura 3.43; esto en principio no era un problema, debido a que nos favorecía al momento de determinar la distracción del conductor de una manera muy simple, pero a lo largo de diversas pruebas, nos dimos cuenta que la precisión de reconocimiento facial era sumamente baja en condiciones reales, siendo el principal punto por lo que se optó mejor por buscar otro algoritmo que cupla con todas las características que nosotros requerimos, pero sin la limitante que se tiene utilizando V&J.

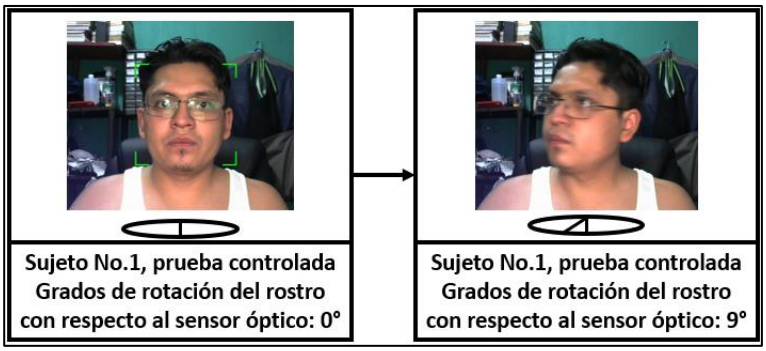


Figura 3. 43: Ejemplo de la perdida de región a estudiar por rotación del rostro

2) ***Falsos positivos:*** A lo largo de las pruebas controladas, el algoritmo de reconocimiento facial no tuvo muchos problemas de reconocer rostros estáticos, pero tuvo un gran problema, como se muestra en la Figura 3.44 a, el algoritmo no sólo detectaba la cara, si no que en ocasiones, detectaba las regiones faciales en lugares en donde no existía alguna; estas detecciones erróneas aumentaban cuando empleábamos el sistema en condiciones reales, como se muestra en la Figura 3.44 b.

El que el algoritmo de reconocimiento facial tuviera los falsos positivos se convirtió en un gran problema, principalmente porque nuestro sistema depende puramente de la región a analizar, por ende, cuando se le aplicaba la RNC-PP propuesta para detectar la somnolencia en los conductores y esta determinaba

probabilísticamente que la región del falso positivo tenía patrones de estado normal, el contador ***ec_count*** automáticamente se igualaba a cero y con ellos se desactivaban las alarmas, si es que estas estuvieran encendidas, o en todo caso el monitoreo se reiniciaba, inutilizando el sistema por unos segundos, lo que es grave, ya que esto es crucial para poder prevenir un accidente.

Por otra parte, la detección de distracción tenía el mismo problema, si el algoritmo de reconocimiento facial detectaba un falso positivo los contadores de distracción automáticamente se igualaban a cero y con ello se reducía la eficacia del monitoreo para este tipo de estado.

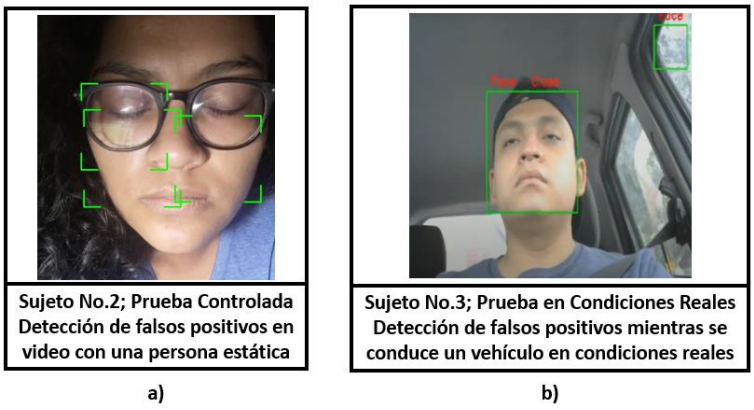


Figura 3. 44: Ejemplo de falsos positivos en los videos de prueba

3) ***Accesorios:*** Este error se presenta cuando el usuario trae prendas que reducen la visibilidad del rostro, un ejemplo de ello es cuando el conductor trae gorra o lentes muy voluminosos, en este caso, el algoritmo no reconoce la región facial del conductor, haciendo que el sistema lo detectara como distracción y naturalmente activara las alarmas, por ello, para que funcionara de la mejor manera, el sujeto no debía usar ningún objeto adicional y a su vez debía tener visible lo mejor posible la parte de la cara.

Lo anterior afectó aún más cuando se implementó el sistema en condiciones con nula iluminación como se muestra en la Figura 3.45 b.

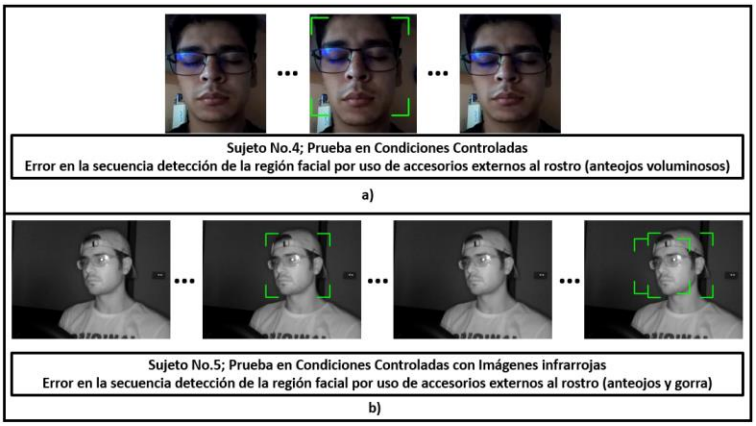


Figura 3. 45: Ejemplo de no reconocimiento facial por uso de accesorios externos al rostro

4) ***Posicionamiento del sensor óptico:*** El lugar en donde se encuentre la cámara será de suma importancia cuando se implementa el algoritmo propuesto por V&J, debido a que su precisión de reconocimiento es directamente proporcional a la posición que se coloque el sensor óptico, por ejemplo si ponemos el sensor enfrente del usuario, el sistema puede detectar de una mejor manera la región facial en comparación a una posición lateral o que no enfoque directamente al rostro del usuario, esto es un punto importante, porque si bien, es útil en sistemas que serán puramente implementados en laboratorios (corriendo el software en dispositivos estáticos como laptops, Workstation, pc, etc.), su uso en condiciones reales de conducción es imposible, esto debido a que por normativa general así como por seguridad propia, no debemos reducir la visibilidad del conductor.

En la Tabla 3.7, vemos una comparativa entre la precisión de reconocimiento facial con el sensor colocado directamente enfrente del rostro (Figura 3.46 a) y entre una posición lateral(Figura 3.46 b).

Tabla 3. 13: Tabla comparativa de precisión dependiendo la posición del sensor óptico

Número de videos en la prueba	Posición del sensor óptico	Precisión
20	Frontal (Figura 3.466 a)	98.46%
20	Lateral (Figura 3.46 b)	65.32%



Figura 3. 46: Ejemplo de errores ocasionados por el posicionamiento del sensor óptico

5) ***Acciones:*** Finalmente tenemos una problemática que no afecta en demasía en comparación con los primero 3 aspectos, pero se deben tomar en cuenta, cuando nos referimos a acciones, hacemos un enfoque a que cuando el usuario tiene la necesidad de tocarse el rostro de forma consciente o inconsciente ya sea por un bostezo, por picazón en alguna parte del rostro o simplemente una acción involuntaria de tocarse la cara, el algoritmo pierde inmediatamente la detección de la región facial como se muestra en la Figura 3.47.



Figura 3. 47: Ejemplo de no detección de la región a estudiar por acciones involuntarias del conductor

Los puntos comentados anteriormente son la principal causa por la cual decidimos cambiar el algoritmo de reconocimiento facial, por lo cual nos dimos a la tarea de investigar diferentes opciones y concluimos que la mejor elección es por mucho uno creado e impulsado por Google denominado ***MediaPipe Face Detection;*** este algoritmo de acuerdo con su documentación es una solución ultrarrápida a la detección facial enfocada a streaming, en donde se utilizan 6 puntos de referencia (Figura 3.48) los cuales son la pauta para generar el cuadro delimitador, esta solución está basada en la detección de rostros de BlazeFace[50], la cual está diseñada específicamente para la implementación móvil, esto es de gran ayuda, debido a que empata con lo requerido por nuestro sistema.

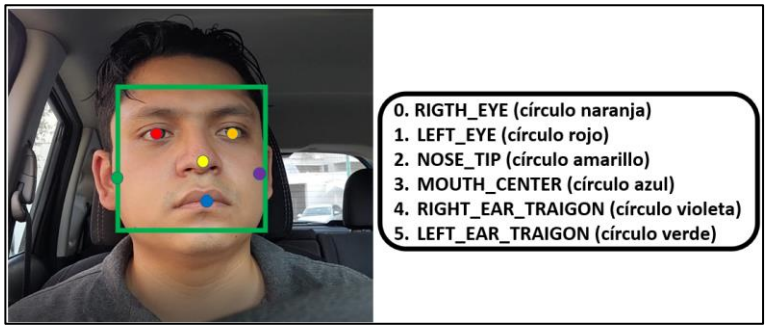


Figura 3. 48: Puntos de referencia para poder detectar un rostro de acuerdo con MediaPipe Face Detection

Las principales características de este tipo de reconocimiento facial son:

1. ***Extracción de características:*** BlazeFace es un algoritmo que toma como base la extracción de características livianas inspirada de MobileNetV1/V2[51], en la Tabla 3.14, podemos observar a la arquitectura de BlazeFace.
2. ***Detección de objetos:*** Para generar un modelo liviano, BlazeFace utiliza y modifica el esquema de anclaje de Single Shot MultiBox Detector (SSD) [52], que en resumen es un algoritmo de reconocimiento de objetos de un solo disparo el cual predice los puntajes de categorías y compensaciones de diversas regiones para que en conjunto se generen cuadros delimitadores, esto se realiza con la ayuda de pequeños filtros convolucionales aplicados a mapas de características.

Tabla 3. 14: Arquitectura de BlazeFace

Layer/block	Input size	Conv. kernel sizes
Convolution	128×128×3	5×5×3×24 (stride 2)
Single BlazeBlock	64×64×24	5×5×24×1 1×1×24×24
Single BlazeBlock	64×64×24	5×5×24×1 1×1×24×24
Single BlazeBlock	64×64×24	5×5×24×1 (stride 2) 1×1×24×48
Single BlazeBlock	32×32×48	5×5×48×1 1×1×48×48
Single BlazeBlock	32×32×48	5×5×48×1 1×1×48×48
Double BlazeBlock	32×32×48	5×5×48×1 (stride 2) 1×1×48×24 5×5×24×1 1×1×24×96
Double BlazeBlock	16×16×96	5×5×96×1 1×1×96×24 5×5×24×1 1×1×24×96

Double BlazeBlock	16×16×96	5×5×96×1 1×1×96×24 5×5×24×1 1×1×24×96
Double BlazeBlock	16×16×96	5×5×96×1 (stride 2) 1×1×96×24 5×5×24×1 1×1×24×96
Double BlazeBlock	8×8×96	5×5×96×1 1×1×96×24 5×5×24×1 1×1×24×96
Double BlazeBlock	8×8×96	5×5×96×1 1×1×96×24 5×5×24×1 1×1×24×96

En la Table 3.15 podemos observar una comparativa entre el algoritmo propuesto por V&J [15] y MediaPipe Face Detection [50], como se muestra, si existe una mejora de precisión y corrección de errores considerables en todos los parámetros los cuales nos causaban conflictos, adicionalmente observamos que la velocidad de reconocimiento se mantuvo o en todo caso se superó y más importante aún, el consumo físico de hardware se redujo.

Tabla 3. 15: Tabla comparativa de rendimiento y precisión entre el reconocimiento facial propuesto por V&J y MediaPipe Face Detection

Comparativa (40 pruebas)	Reconocimiento facial V&J		Reconocimiento facial MediaPipe Face Detection	
Detección de rostro (precisión promedio)	84.86%		100%	
Generación de falsos positivos (SI/NO)	Si		No	
Reconocimiento facial con el uso de accesorios (precisión promedio)	70.2%		97.58%	
Posicionamiento del sensor óptico (precisión promedio de reconocimiento facial)	Frontal	Lateral	Frontal	Lateral
	98.46%	65.32%	99.9%	85%

Velocidad de reconocimiento facial en video (sin implementar RNC-PP)	63 fps	65 fps
Velocidad de reconocimiento facial en tiempo real (sin implementar RNC-PP)	30 fps	30 fps
Uso de CPU ()	5%	14%
Uso de GPU ()	4%	10%

NOTA: Para las pruebas se utilizó un único dispositivo (3070TI GPU, 8GB Memory, 6144 cores, 1.770 MHz) así como para la prueba en tiempo real se utilizó la cámara "Cámara genérica con una resolución de 640×480 pixeles a 3 canales".

3.3.2 Mejoramiento de la RNC-PP propuesta

El tener un algoritmo sumamente eficaz de reconocimiento facial es bueno para nosotros, esto porque aumentamos el nivel de precisión al momento de prevenir la somnolencia en los conductores; pero a lo largo de varias pruebas nos hemos encontrado con la necesidad de aumentar de igual manera los parámetros de clasificación de estados, debido a que si bien, aumentar el número de rostros detectados en un video en tiempo real disminuye los errores y aumenta la seguridad, el algoritmo general propuesto está basado en las limitantes del algoritmo propuesto por V&J, más específicamente, el sistema ya no tiene la capacidad de detectar la distracción de una manera tan simple, debido a que con utilizando dicha propuesta su detección era tan simple que sólo utilizábamos al contador **"ec_countf"** el cual aumentaba cuando el sistema no detectaba el rostro, con MediaPipe Face Detection, esto ya no es posible en su gran mayoría, por ende, se tuvo que modificar la RNC-PP propuesta, aumentando un estado más a los dos que ya se tenían, teniendo en cuenta lo que se ha comentado y controlado hasta el momento (número de imágenes de entrenamiento, una configuración liviana, universalidad, entre otros), por lo tanto se tomó la siguiente acción.

1) ***Estados de predicción:*** Este punto es similar al de la RNC-PP (primera versión), los patrones escogidos para la nueva selección son iguales para poder clasificar un estado de conducción normal y uno con rasgos de somnolencia pero siendo más severos al momento de crear la base de datos, un parámetros que antes estaba y actualmente se ha evitado, es que si los sujetos tienen patrones de los estado descritos previamente, pero están volteando hacia algún lado o no es posible observar con claridad más del 70% del rostro, estos fotogramas no serán agregado al set individual de cada uno de los dos conjuntos de origen y se enviarán a una nueva etiqueta denominada ***"Distracción"***, a continuación observamos las diferencias y los patrones que se tomaron en cuenta para crear el nuevo dataset.

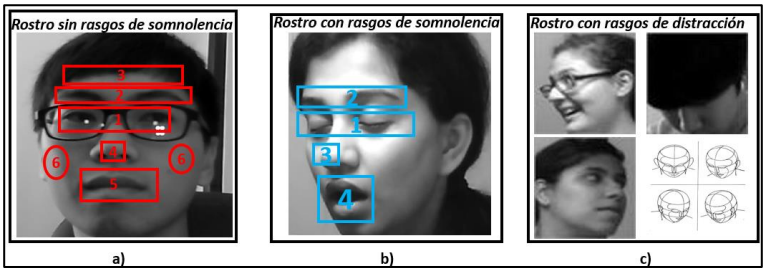


Figura 3. 49: Patrones escogidos para generar la nueva base de datos

- Rostro sin rasgos de somnolencia:
  1. ***Ojos (Figura 3.49 a 1)***
  2. ***Posición de las cejas(Figura 3.49 a 2):***
  3. ***Líneas de expresión (Figura 3.49 a 3 y a 6):***
  4. ***Posición del rostro (Figura 3.49 a 4):***
  5. ***Posición de la boca (Figura 3.49 a 5):***
- Rostro con rasgos de somnolencia:
  1. ***Ojos (Figura 3.49 b 1):***
  2. ***Posición de las cejas(Figura 3.49 b 2):***
  3. ***Posición del rostro (Figura 3.49 b 3):***
  4. ***Posición de la boca(Figura 3.49 b 4):***

Como observamos, los patrones para rasgos con y sin somnolencia son los mismos que para la primera RNC-PP propuesta, la única diferencia que tenemos, es en la posición del rostro, esto en ambos casos, se debe tener por lo menos el 75% de los rasgos característicos de un rostro visible, tomando como base los 6 puntos descritos por MediaPipe Face Detection (Ojo derecho, Ojo izquierdo, Punta de la nariz, Centro de la boca, Trago derecho de la oreja e izquierdo), de lo contrario, independientemente que se tenga los patrones de sueño o de conducción correcta, se clasificarán como distracción, por lo tanto:

- Rostro con rasgos de distracción:
  1. ***Posición del Rostro (Figura 3.49 c)***: Para poder determinar que el rostro tiene patrones de distracción, la región a analizar debe mostrar un valor menor del 75% de los patrones característicos de un rostro.
  2. ***Posición de los ojos (Figura 3.49 c):*** Con ayuda del posicionamiento, si el conductor presenta una deviación del rostro así como patrones de los ojos en donde se mire hacia la dirección inferior del sensor óptico, esto lo hemos etiquetado como un parámetro para determinar que el usuario está observando objetos innecesarios en la conducción así como una disminución de atención al momento de conducir.

     Teniendo en cuenta lo anterior y para finalizar este punto, los estados que se van a etiquetar y posteriormente a clasificar en el

entrenamiento de la RNC-PP propuesta (nueva versión) serán: ***"ojos abiertos"***, ***"ojos cerrados"*** y ***"distracción".***

2) ***Base de datos:*** La base de datos que vamos a utilizar para crear nuestro dataset es el mismo que se utilizó para la primera RNC-PP propuesta, el set de imágenes se manipulación libre creada por la National Tsing Hua University (NTHU) que es muy utilizada para este tipo de investigaciones, el nombre del set de datos es Driver Drowsiness Detection Dataset o comúnmente denominada ***"NTHU-DDD"*** (Figura 3.17).

De acuerdo con la nueva estructura de la base de datos que vamos a formar, se creó la siguiente distribución:

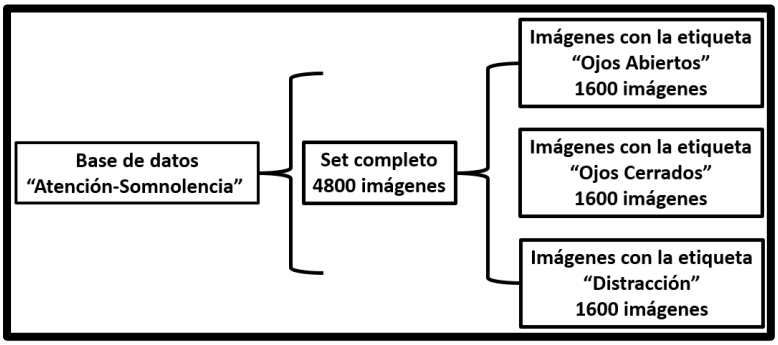


Figura 3. 50: Diagrama de distribución de las imágenes de entrenamiento y prueba

Como observamos en la Figura 3.50, se etiquetaron 4,800 imágenes las cuales cumplen la condición de no ser una base de datos sumamente grande que superen los 5000 fotogramas, además que se redujo el número de carpetas a una sola que contiene todo el set, esto debido a que la configuración que se planeó para entrenar el nuevo modelo de la RNC-PP, así requiere organizado los datos de entrada.

Como en la primera creación de la base de datos, aquí también debemos tener en cuenta características especiales de la imagen de entrenamiento, en donde se cuida aún más en comparación del primer date el tema de costo computacional, por lo tanto, las modificaciones que se hicieron a partir de esto son:

I. ***Normalización del tamaño de imágenes:*** En comparación del primer set de datos que tenían un tamaño normalizado de 150×150 pixeles, en esta base de datos de igual manera se utilizó una reducción dimensional de su imagen original utilizando interpolación por área, pero ahora con un tamaño de 64×64 pixeles por cada imagen de entrada.

II. ***Canales de la imagen de entrenamiento***: El número de canales de entrenamiento se queda en 3 canales en el orden: Canal Azul, Canal Verde y Canal Rojo.

3) ***Arquitectura de la RNC-PP propuesta nueva versión:*** La red neuronal que proponemos tiene la siguiente arquitectura de configuración:

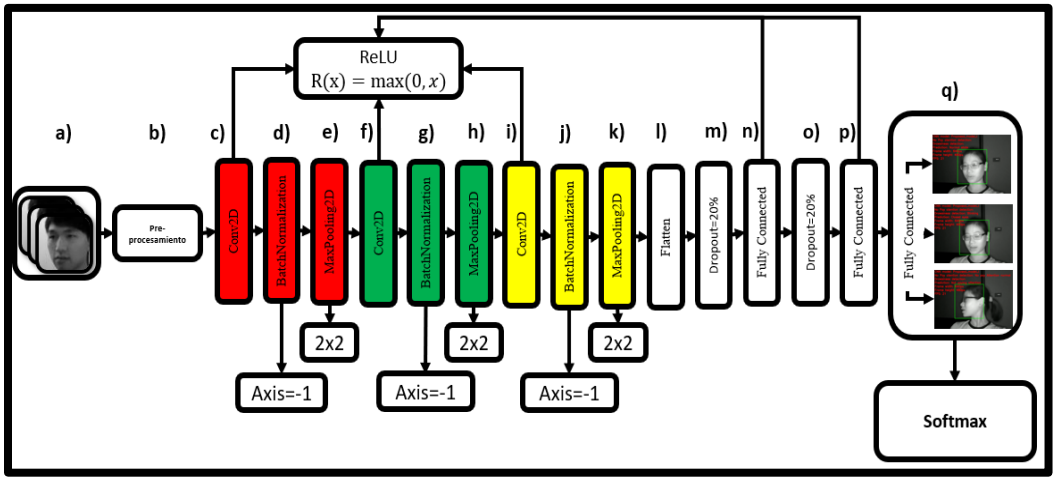


Figura 3. 51:Arquitectura de la nueva RNC-PP propuesta

I. ***Imágenes de entrada (Figura 3.51 a)***: Como en la primera arquitectura creada, las imágenes de entradas son las que previamente hemos etiquetado, normalizado y redimensionado, una característica diferente a la primera RNC-PP propuesta, es que aquí, las imágenes de entrada tendrán un tamaño reducido pasando de 150×150 pixeles a 64×64 pixeles, pero manteniendo el mismo nivel de canales (3)

II. ***Preprocesamiento (Figura 3.51 b):*** El procesamiento digital de las imágenes de entrenamiento que se realiza habitualmente en el entrenamiento de una red neuronal convolucional, es de igual manera, en principio los mismos preprocesamientos de la primera arquitectura:

1. ***Re-escalamiento [30]:*** 1/255.

2. ***Rango de corte [31]:*** 0.2 o 20%

3. ***Rango de zoom [31]***: 0.2 o 20%

Como observamos los 3 principales procesamientos se encuentran en esta nueva arquitectura, pero teniendo en cuenta que tenemos más clases que clasificar, menor número de imágenes por clasificación, así como una disminución de imágenes de entrada como una reducción de características por el redimensionamiento de las imágenes de entrada, nos dimos a la tarea de implementar unos nuevos mapas de extracción de características los cuales son:

4. ***Rango de cambio de anchura [31]:*** Lo que hacen este procesamiento, es redirigir los datos de una imagen a lo ancho, es decir, recorre de izquierda a derecha los valores de los pixeles del fotograma, como se muestra en la Figura 3.52.

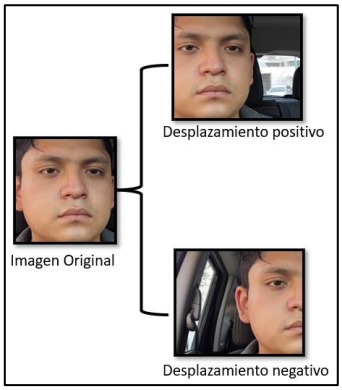


Figura 3. 52: Ejemplo de la implementación del rango de cambio de anchura

Los valores con los que se trabaja este tipo de procesamiento es de tipo flotante y <=1, el cual se toma como un porcentaje de movimiento horizontal, este puede ser positivo el cual se desplaza hacia la derecha y el porcentaje negativo se desplaza hacia la izquierda, nosotros pusimos el valor de 0.2 o 20%.

5. ***Rango de cambio de altura [31]:*** Al igual que rango de cambio de anchura, este redirige los datos de una imagen a lo largo, es decir, recorre de arriba hacia abajo los valores de los pixeles del fotograma, como se muestra en la Figura 3.53.

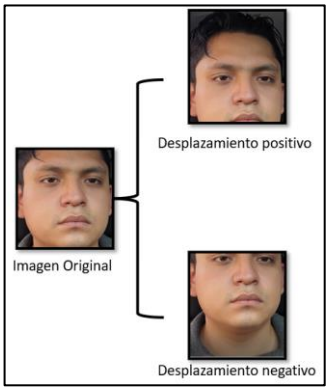


Figura 3. 53: Ejemplo de la implementación del rango de cambio de altura

Los valores con los que se trabaja este tipo de procesamiento es de tipo flotante y <=1, el cual se toma como un porcentaje de movimiento vertical, este puede ser positivo el cual desplaza hacia arriba y el porcentaje negativo se desplaza hacia abajo, nosotros pusimos el valor de 0.2 o 20%.

6. ***Rango de rotación [31]:*** Este procesamiento digital es fácil de entender, lo que se realiza es una revolución a un ángulo N de la

imagen de entrada en turno, tomando como centroide el píxel medio del fotograma y rellenando con una interpolación del vecino más cercano al momento de rotar, no exista un píxel de la imagen original, en la Figura 3.54 vemos un ejemplo de este procedimiento.

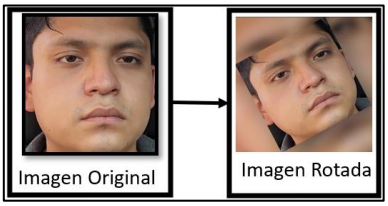


Figura 3. 54:Ejemplo de la implementación del rango de rotación

El valor N es el ángulo en cual la imagen se va a rotar, está dado en gados (º) y se escogió debido a que es una simulación de como el usuario puede rotar un poco el rostro y seguir siendo parte de la etiqueta en donde esta implementado la imagen en turno que se va a entrenar, el ángulo que seleccionamos es de N = 10º.

7. ***Volteado horizontal [31]:*** Lo que realiza este procesamiento digital es muy simple, crea (como todos los preprocesamientos digitales vistos hasta ahora) una copia espejo horizontalmente de la imagen original, en la Figura 3.55 vemos un ejemplo de esto.

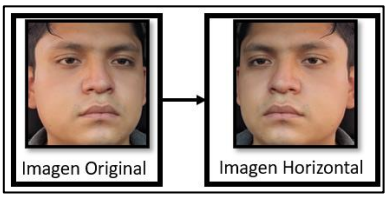


Figura 3. 55: Ejemplo de la implementación del volteado horizontal

Al igual que la anterior estructura, diversos preprocesamientos de las imágenes que hemos seleccionado son aleatorios y sumatorios.

III. Extracción y mapeo de características: El proceso de extracción de características en esta nueva arquitectura es muy similar a la primera versión de red neuronal creada, pero en esta haremos uso de diversas técnicas, las cuales aumentarán la precisión al momento de entrenar el modelo predictivo, esto debido a la cantidad tan baja de imágenes del dataset, por lo tanto el siguiente proceso es obligatorio.

1. ***Set de entrenamiento, validación y prueba:*** Comúnmente, cuando entrenamos una RN tenemos que tener dos carpetas en donde tengamos ya ordenado y clasificado cuantas imágenes serán de entrenamiento y cuantas de prueba, pero en esta ocasión, no requerimos la creación previa de dichos archivos, el único requisito

que vamos a necesitar es que el set total este ordenado y etiquetado como en la Figura 3.50, posteriormente con la codificación obtendremos esta agrupación de forma aleatoria y automática, por lo que ha quedado de la siguiente manera el set de entrenamiento así como el de prueba.

A. ***Número de imágenes de entrenamiento:*** Se propone un set de entrenamiento no mayor a 5000 imágenes, las cuales serán cuidadosamente seleccionadas y filtradas, tomando en cuenta cada patrón característico a cada etiqueta, así como también un número balanceado de imágenes promedio entre cada individuo (18 usuarios), por lo que quedo de la siguiente manera:

i. El set de entrenamiento contiene un total de 4800 imágenes en principio, posteriormente este se modificará y se reducirá naturalmente.

ii. El set de entrenamiento se dividió en tres sets secundarios, uno con la etiqueta ***"Ojos Abiertos"***, el otro con la etiqueta ***"Ojos Cerrados"*** y finalmente el tercero con la etiqueta ***"Distracción".***

B. ***Número de imágenes de prueba (Figura 3.50)***: A diferencia de la primera versión de RN, aquí no tendremos una carpeta en específico donde contengan imágenes preseleccionadas para tomarlas como de prueba, si no que del set completo de entrenamiento, tomaremos el 25% de cada set secundario de forma aleatoria, por lo tanto la configuración del set de prueba queda de la siguiente manera:

i. El set de prueba contiene un total de 1200 imágenes seleccionadas aleatoriamente del set total de imágenes para entrenamiento.

ii. El set de prueba, al igual que el de entrenamiento se dividió en tres sets secundarios (esto sólo de forma digital, debido a que los archivos originales en ningún momento se modificaron), etiquetados de la misma manera que el set completo de imágenes, ***"Ojos Abiertos"*** , ***"Ojos Cerrados" y "Distracción"***, los cuales en cada carpeta digital contienen 400 imágenes por cada clase.

2. ***Validación cruzada[53]:*** La validación cruzada es una técnica de aprendizaje automático en donde es utilizada para evaluar resultados estadísticos y garantizar el mejor nivel de aprendizaje fragmentando el set central de entrenamiento, esto se hace repitiendo en diversas

ocasiones el proceso de aprendizaje del modelo, variando constantemente los datos de entrenamiento y prueba.

Esta táctica de aprendizaje se utiliza cuando el set de datos es reducido, y se requiere el mayor nivel de predicción; un problema al usar esta técnica es que en la mayoría de los casos, obtendremos un valor alto de sobreajuste, el cual se puede reducir modificando los híper parámetros de la RNC o en todo caso, su configuración debe ser tal que se mantenga el sobre ajuste al mínimo.

- ***Validación cruzada por K iteraciones[54]:*** Este tipo de validación cruzada tiene como propiedad de que nosotros podemos decidir que tantas veces el algoritmo repetirá el proceso de aprendizaje al momento de entrenar la RN, K es el número de veces en el cual nuestro set completo re-entrenara el modelo, la forma que lo hace es la siguiente: el set original se fragmenta en porciones equivalentes de acuerdo al nivel de k, este se va modificando aleatoriamente por cada iteración hasta terminar el proceso, en la Figura 3.56 podemos ver un ejemplo de cómo va entrenando por cada iteración, otro punto importante de esta validación es que los pesos se van modificando de acuerdo con el cálculo del error de cada iteración, por lo tanto se hace uso de la suma aritmética dada por $Error = \frac{1}{k}\sum_{i=1}^{k} Error_i$

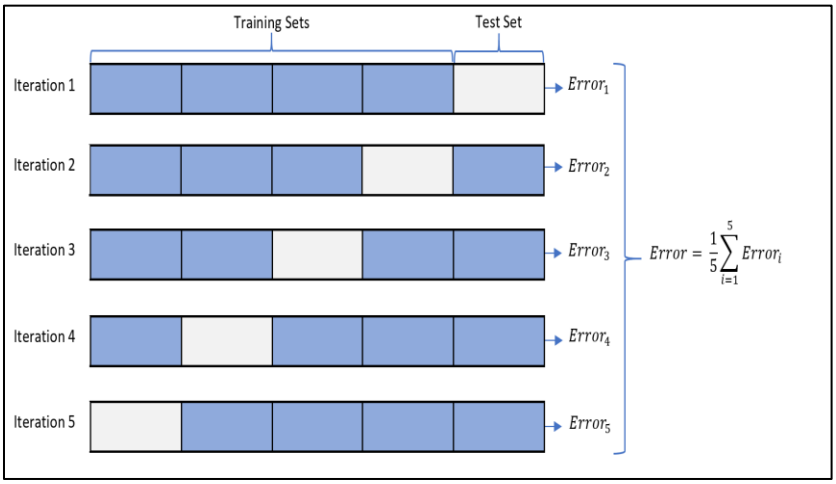


Figura 3. 56: Implementación de la validación cruzada propuesta[16]

[16] Figura tomada de "Aprendizaje automático (12) ejemplos de validación cruzada", programador clic, https://programmerclick.com/article/8295250019/

Nuestros parámetros iniciales que planteamos y teniendo en cuenta lo que se requiere para un buen entrenamiento, hemos decidido implementar una validación cruzada con un nivel de 5 iteraciones (Figura 3.56), las cuales, después de diversas pruebas (Tabla 3.16), fueron la mejor opción, tanto por el número de parámetros entrenados, así como su precisión final, en la Figura 3.57 vemos una representación gráfica del proceso de entrenamiento el cual se utilizó.

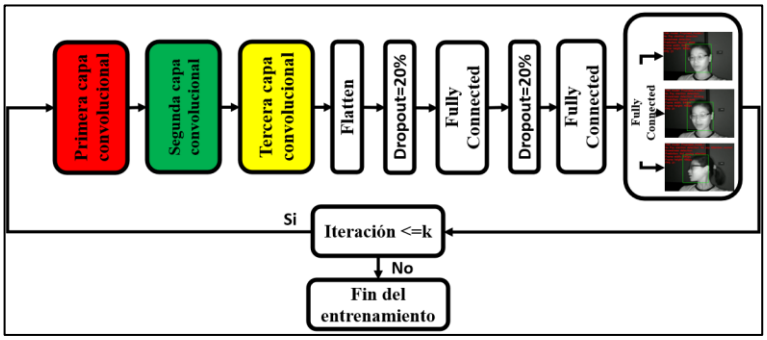


Figura 3. 57: Proceso de implementar la validación cruzada a la arquitectura convolucional y a la red neuronal propuesta

3. ***Red Neuronal Convolucional Poco Profunda (RNC-PP):*** La primera parte de la arquitectura de la nueva RNC-PP propuesta es muy parecida a la primera red, debido a que se utilizó como base para poder crear esta, a continuación vemos la configuración que se empleó en cada una de las 3 capas convolucionales.

Tabla 3. 16: Parámetros propuestos para la arquitectura de la nueva RNC-PP

No. de capa	Parámetro	Valor
Primera capa	Capa convolucional (No. de filtros)	32
	Tamaño de kernel	3×3
	Batch Normalization	Axis = -1
	Activación	ReLU
	MaxPooling	2×2
Segunda capa	Capa convolucional (No. de filtros)	32
	Tamaño de kernel	3×3
	Batch Normalization	Axis = -1
	Activación	ReLU
	MaxPooling	2×2
Tercera Capa	Capa convolucional (No. de filtros)	64
	Tamaño de kernel	3×3
	Batch Normalization	Axis = -1

	Activación	ReLU
	MaxPooling	2×2

I. ***Primera capa convolucional:*** La primera capa convolucional de nuestra arquitectura contiene 32 filtros con un Kernel de 3×3, el cual genera 32 imágenes con una reducción de 2 pixeles, en este caso, a diferencia de la primera arquitectura, la imagen de entrada ya no será de 150×150 pixeles, si no que tendremos una imágenes de 64×64 pixeles, con esto reducimos el espacio de memoria requerido para el entrenamiento de la red, así como la reducción en el tiempo de procesamiento; por ende, tomando en cuenta la reducción de la imagen, pasamos a tener un tamaño de imagen de entrada en turno de 62×62 pixeles, a continuación implementamos un nuevo parámetro a la red, el denominada Batch normalization. El Batch normalization es una normalización por lotes, donde regulariza los datos resultantes de la convolución.

La normalización por lotes tiene la característica principal en ayudar al entrenamiento, haciendo más fluido los datos y además que apoya a la red neuronal a trabajar de una mejor manera; se decidió tomar este parámetro para poder reducir y/o mantener el sobreajuste, debido a que como estamos utilizando validación cruzada, las normalizaciones previas se van perdiendo a lo largo de cada iteración.

En esta primera capa convolucional hemos utilizado la función de activación ReLU (Figura 3.51 c) $\phi(q)=\max(0,q)$ propia de la librería utilizada, así como también la reducción de dimensiones espaciales utilizada fue la implementación de la operación MaxPooling con una ventana de 2×2, creando 32 imágenes de salida con un tamaño final de 31×31 pixeles.

II. ***Segunda capa convolucional:*** En la segunda capa convolucional, implementamos 32 filtros a cada 1 de las 32 imágenes creadas con la primera capa convolucional, por lo que tenemos 32 imágenes resultantes por cada una de las 32 imágenes de la primera capa, con una dimensión reducida de 31×31 pixeles a 29×29 pixeles, posteriormente aplicamos la normalización por bloques y al igual que la primera capa, hemos decidido utilizar a ReLU ($\phi(q)=\max(0,q)$) como linealizador, así como también, la implementación de un MaxPooling con

una ventana de 2×2, teniendo como resultado 32 imágenes de salida por cada 32 de entrada con un tamaño final de 14×14 pixeles cada una, dando un resultado de 1024 imágenes nuevas.

III. Tercera capa convolucional: En esta última capa implementamos 64 diferentes filtros a cada una de las 1024 imágenes, por lo que tenemos un total de 65,536 imágenes resultantes, con una dimensión reducida de 14×14 pixeles a 12×12 pixeles, de igual manera implementamos la normalización por lotes y utilizamos al activador ReLU ($\phi(q)=\max(0,q)$), finalmente implementamos un MaxPoolig como reductor de dimensiones espaciales utilizando una ventana 2×2, teniendo como resultado 64 imágenes de salida por cada 1024 imágenes de entrada con un tamaño final de 6×6 pixeles cada una.

4. Aplanamiento: Al tener ya terminado el proceso de cada capa convolucional, aplicamos el redimensionamiento de matriz a vector para poder alimentar al perceptrón multicapa posterior.

5. Dropout [55]: El Dropout es una técnica de aprendizaje profundo el cual su función principal es el de desactivar N% de neuronas del entrenamiento de forma aleatoria, esto se hará por cada iteración en donde se lleve a cabo el proceso de aprendizaje, este método ayuda a reducir el sobreajuste generado normalmente, y además, que para la estructura de nuestra red neuronal, nos ayudará en gran medida para mantenerlo.

Para utilizar a Dropout, se debe escoger un valor entre 0 a1 en términos de flotantes, el cual, será el porcentaje de neuronas apagadas, en nuestro caso, para esta etapa, apagaremos el 20% de del total de neuronas utilizadas a continuación.

6. Completamente conectada: Terminado ya la primera etapa de la RNC, nos damos a la tarea de crear la arquitectura siguiente, la cual es la creación de las capas ocultas de neuronas que a continuación se explicará su configuración:

6.1 Primera capa oculta: Para la primera capa oculta utilizaremos 128 neuronas convencionales totalmente conectadas las cuales estarán acopladas directamente al vector de características resultante del aplanamiento, adicionalmente se utilizó una linealización de ReLU ($\phi(q)=\max(0,q)$).

6.2 Segunda capa oculta: La segunda capa oculta tendrá igualmente un número total de 128 neuronas conectadas entre

sí, con un Dropout del 20% y con un activador lineal ReLU ($\phi(q)=\max(0,q)$).

6.3 ***Tercera capa oculta:*** En esta última capa oculta hacemos uso de 3 neuronas, las cuales extraerán los últimos patrones resultantes de la capa anterior y los clasificará en tres estados probabilísticamente, como activador usaremos la función Softmax, la cual devuelve la distribución probabilística; al tener 3 estados, Softmax [56] calculará el nivel de porcentaje que se tiene de cada clasificación y nos arrojará el resultado en un vector de 3 componentes.

7. ***Optimizador:*** Como en la primera RNC-PP, el optimizador usado fue Adam, tomando los mismos hiper parámetros $\beta_1 = 0.9$, $\beta_2 = 0.999$, Tasa de aprendizaje $\boldsymbol{\alpha}$ = 0.001 y finalmente un valor de 64 el tamaño del lote

Para finalizar este apartado, la RNC-PP que proponemos nos arroja dos clasificaciones probabilísticas, las cuales en 3.3.5 haremos su análisis y su caracterización.

3.3.3 Análisis de resultados consecutivos

Como hemos visto a lo largo de todo este apartado, hemos cambiado el uso del algoritmo de reconocimiento facial propuesto de V&J por el impulsado por Google (MediaPipe Face Detection), por ende nos vemos en la necesidad de modificar los procesos de detección de distracción por falta de rostro (Figura 3.27) y el de somnolencia (Figura 3.32) descritos en la primera versión del sistema, pero como tal, el sistema no depende puramente del detector de la región a estudiar, si no que este puede ser modificado sin afectar los demás procesos, por lo tanto su modificación es mínima como se muestra en la Figura 3.58:

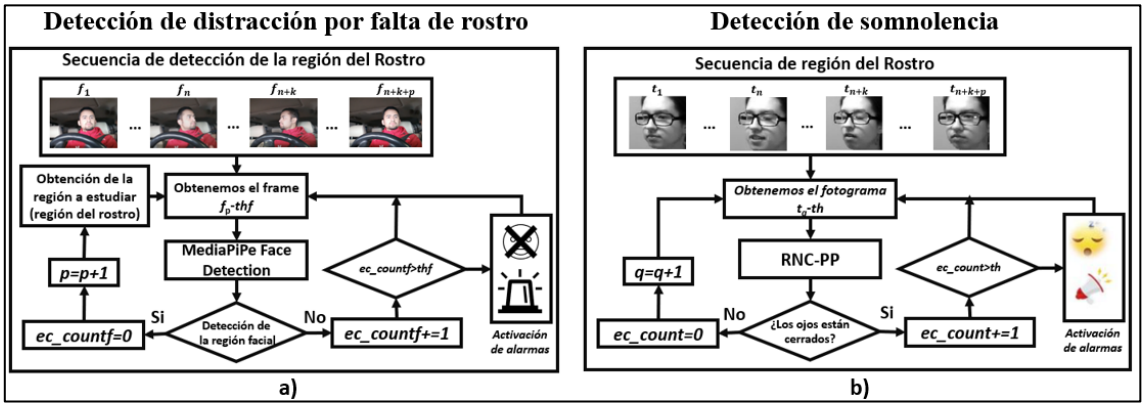


Figura 3. 58: Algoritmo para el reconocimiento de distracción (nueva versión).

Como observamos en la detección de distracción por falta de rostro (Figura 3.58 a), el modelo de reconocimiento facial ha sido sustituido, al igual que con el proceso de detección de somnolencia en donde la primera RNC-PP ha sido cambiada por la nueva arquitectura.

Observamos que en teoría siguen siendo los mismos algoritmos, esto debido a que la estructura de ambos se pensó primeramente para que los modelos de reconocimiento facial y el de clasificación se puedan cambiar constantemente sin la necesidad de estar creando constantemente nuevos sistemas generales.

Otro parámetro que se verá afectado por la mejora del reconocimiento facial, así como la aplicación de nuevos elementos clasificatorios, es el aumento de un algoritmo en paralelo que estará trabajando a la par con la detección de distracción por falta de rostro (Figura 3.58 a) y el de somnolencia (Figura 3.58 b), este lo hemos denominado ***detección de distracción por clasificación*** y funciona de la siguiente manera:

Tomamos en consideración los mismos parámetros y puntos referentes con la detección de somnolencia, aquí la diferencia es que primero haremos uso de una primera clasificación la cual es la siguiente:

- Si la ***RNC-PP*** estima que el usuario tiene patrones de una conducción normal o en todo caso, de patrones donde el usuario se esté quedando dormido, entonces su etiqueta en este proceso será de ***"atención".***
- Si la ***RNC-PP*** estima que el usuario tiene patrones de distracción, su etiqueta será de ***"distracción"***.

Teniendo la base de estas predicciones, hemos implementado el siguiente algoritmo para poder determinar los estados ***atención y distracción***:

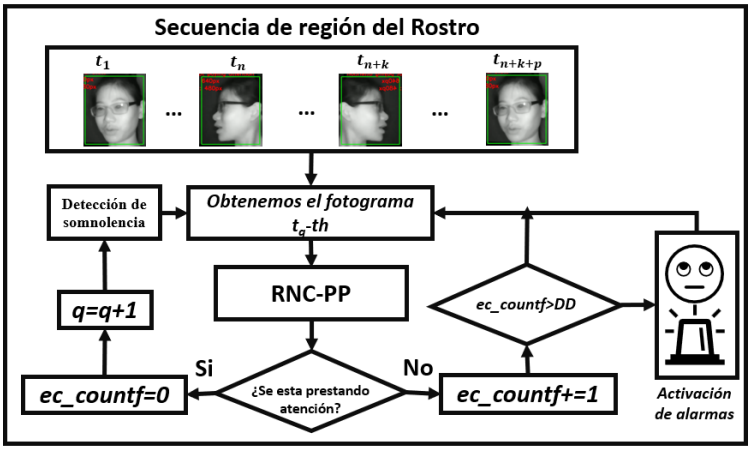


Figura 3. 59: Detección de distracción por clasificación

En la Figura 3.59, podemos observar el proceso general para la determinación de distracción utilizando la nueva arquitectura de RNC-PP, este procedimiento es una combinación de los dos procesos originales pero con las diferencias que a continuación se explicarán:

1) ***Estado de atención:*** Cuando la ***RNC-PP*** propuesta (nueva arquitectura) establece que la imagen t_q tiene patrones de un conductor despierto o en todo caso, patrones de somnolencia se clasifica como ***"atención"***, por lo tanto el contador que usamos para detectar la distracción en donde no se detecta en rostro que es ***ec_countf*** se iguala a 0 y abre paso al proceso de detección de somnolencia, posteriormente al resultado que se haya determinado con el procesos antes mencionado, termina el proceso de este estado y sigue analizando la siguiente imagen t_{q+1}.
2) ***Distracción:*** Si la RNC-PP (nueva arquitectura) clasifica que el conductor tiene características de distracción, ***"ec_countf"*** se incrementa en una unidad, posteriormente se analiza la siguiente imagen t_{q+1} y así consecutivamente, si ***"ec_countf"*** supera el umbral *thf* (que es el mismo de detección de distracción por falta de rostro) , se activan las alarmas pertinentes a este proceso (Punto 4).
3) ***Ruptura del ciclo de alarma:*** Las alarmas estarán activas siempre y cuando el ***ec_countf>thf***, por lo tanto, si el sistema en algún punto clasifica a t_{q+1} como **atención**, automáticamente ***"ec_countf"*** se reinicia (***ec_countf = 0***) desactivando las alertas consigo, debido a que el reconocimiento de somnolencia entra en juego y finalmente el sistema se cicla infinitamente o hasta que el sistema termine de monitorear al conductor.



Figura 3. 60: Ejemplo de las diferentes detecciones detectadas por SOMN_IA aplicando la nueva RNC-PP

3.3.4 Activación de alarmas pertinentes

El apartado de activación de las alarmas pertinentes en este caso no tuvo modificación alguna en dispositivos de laboratorio, en cambio, donde si hubo no sólo una modificación de secuencia de alertas, si no que un aumento de módulos y elementos en el SAS fue en SOMN_IA en donde se aumentó el cuidado del conductor.

1. ***Configuración de la alarma por distracción en SOMN_IA:*** Al igual que la primera versión de la configuración de **SAL** y **SAS,** SOMN_IA hará uso de dos de sus cuatro tiras de leds, así como de cuatros piezo eléctricos (que a diferencia de la primera versión, en esta ocasión aumentaron un total de tres elementos sonoros adicionales) los cuales tendrán la tarea de alertar al conductor que no está prestando atención al camino por lo tanto:

a) **SAL para alertar al conductor que no está prestando atención al camino:** Esta acción se activará cuando el algoritmo detección de distracción, ya sea por clasificación o por no reconocimiento del rostro del usuario sobrepasa el contador ***ec_countf*** al umbral ***thf,*** después de esto, se inicia un contador interno en el microcontrolador denominado "***SAMD_counter"*** (Subsistema de Alerta Múltiple por Distracción); ***SAMD_counter*** se incrementa cada 2 segundos una unidad; por cada unidad aumentada de ***SAMD_counter***, se va intensificando la secuencia de parpadeo de dos de las cuatros tiras de leds hasta llegar un punto el cual, los leds ya no generen el parpadeo por lo tanto estas quedan prendidas y es donde ***SAMD_counter*** es mayor que nuestro umbral ***SAMD_th*** (con valor de 5), si se llegase el caso de que el usuario no preste atención después de esto, se hará uso del proceso ***Rastreo*** (Punto 4). El ciclo se rompe en el momento que ***ec_countf*** sea igual a cero o sea menor a ***thf***, si ***SAMD_counter< SAMD_th*** las alarmas lumínicas sólo se apagarán y continuará el monitoreo de forma normal, pero si ***SAMD_counter>=SAMD_th***, las alertas lumínicas sólo se apagarán mediante un comando directo de la interfaz gráfica.

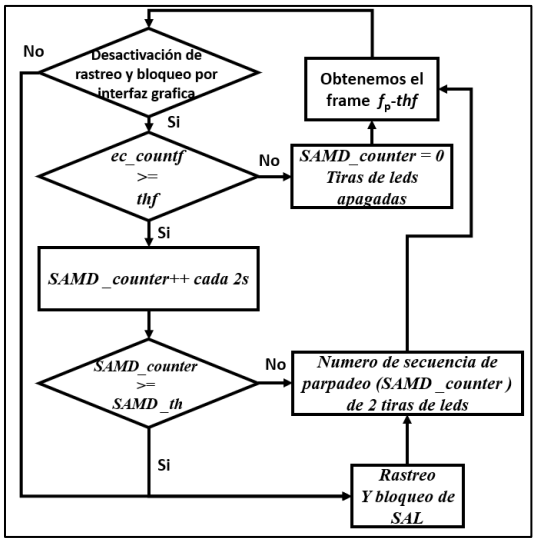


Figura 3. 61:Proceso de activación y desactivación del SAL modificada al momento de detectar la distracción

b) **SAS para alertar al conductor que no está prestando atención al camino:** Este proceso es idéntico al **SAL para alertar al conductor que no está prestando atención al camino**, lo único que cambia seria que en paralelo, se activa una secuencia de sonido interno, para avisar que el conductor no está prestando atención al camino; el sonido se va haciendo cada vez más constante, cuanto más tiempo no se preste atención a la carretera.

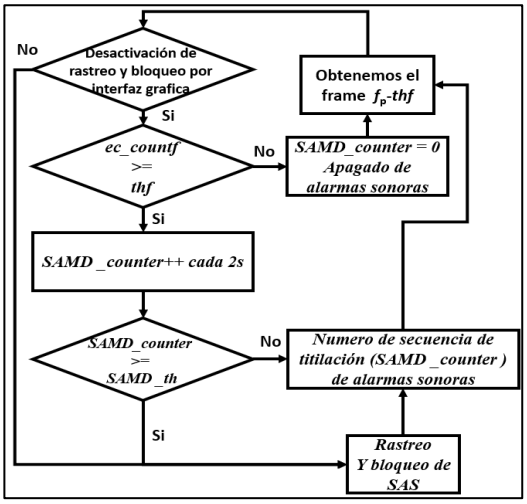


Figura 3. 62: Proceso de activación y desactivación del SAS modificada al momento de detectar la distracción

2. ***Configuración de la alarma por somnolencia en SOMN_IA:*** Al igual que la primera versión de la configuración de **SAL** y **SAS,** SOMN_IA hará uso de sus cuatro tiras de leds, así como de los cuatros piezo eléctrico internos, los cuales tendrán la tarea de despertar al conductor:

   a) **SAL para despertar al conductor:** Esta acción se activará cuando el algoritmo detección de somnolencia sobrepasa el contador ***ec_count*** al umbral ***th,*** después de esto, se inicia un contador interno en el microcontrolador al igual que en la configuración de detección de distracción, en esta ocasión se ha denominado "***SAMS_counter"*** (Subsistema de Alerta Múltiple por Somnolencia); ***SAMS_counter*** se incrementa cada segundo una unidad; por cada unidad aumentada de ***SAMS_counter***, se va intensificando la secuencia de parpadeo de las cuatros tiras de leds hasta llegar un punto el cual, los leds ya no generen el parpadeo por lo tanto estas quedan prendidas y es donde ***SAMS_counter*** es mayor que nuestro umbral ***SAMS_th*** (con valor de 5), si se llegase el caso de que el usuario no preste atención después de esto, se hará uso del proceso ***Rastreo*** (Punto 4). El ciclo se rompe en el momento que ***ec_count*** sea igual a cero o es menor a ***th***, si ***SAMS_counter< SAMS_th*** las alarmas lumínicas sólo se apagarán y continuará el monitoreo de forma normal, pero si ***SAMS_counter>= SAMS_th***, las alertas lumínicas sólo se apagarán mediante un comando directo de la interfaz gráfica.

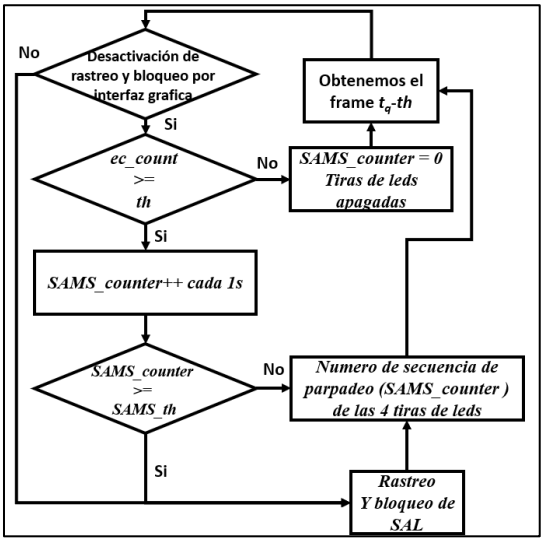


Figura 3. 63: Proceso de activación y desactivación del SAL modificada al momento de detectar la somnolencia

b) **SAS para despertar al conductor:** Este proceso es idéntico al **SAL para despertar al conductor:**, lo único que cambia seria que en paralelo, se activa una secuencia de sonido interno, para avisar que el conductor no está prestando atención al camino; el sonido se va haciendo cada vez más constante, cuanto más tiempo no se preste atención a la carretera.

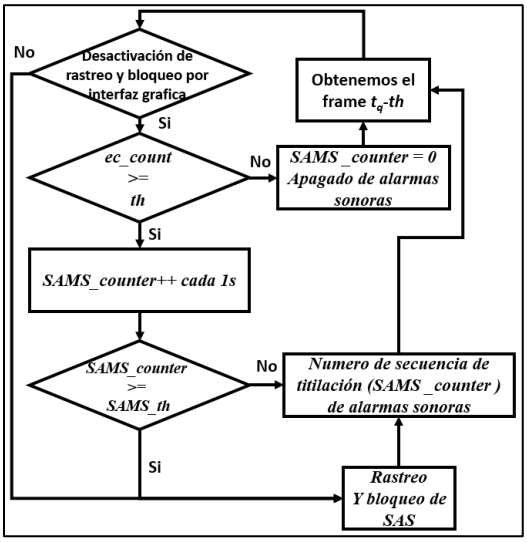


Figura 3. 64: Proceso de activación y desactivación del SAL modificada al momento de detectar la somnolencia

3. ***Alarma Informativa:*** La única modificación importante a este apartado fue que la leyenda **"*Face not detected*"** fue sustituido por **"*Distraction*",** de ahí en fuera, el proceso de activación y desactivación quedo de la misma manera.

4. ***Rastreo:*** La idea principal de este nuevo módulo, es que avisa a una gente externo (ya sea una persona de confianza del conductor, un área de monitoreo, una agencia, etc.) las condiciones actuales del conductor así como envía su ubicación en tiempo real, es decir, esta sólo se va a activar siempre y cuando el sistema sobrepase los umbrales ***SAMS_th*** y ***SAMD_th*** , en donde en cada proceso se hará lo siguiente:
   - ***a) Rastreo por distracción:*** Este subproceso se activará cuando ***SAMD_th*** sea superado por ***SAMD_counter***, el funcionamiento en idea es simple, se enviará un mensaje aun número telefónico dado por el conductor (por el momento el número telefónico escogido por el usurario se debe cargar desde la exportación del código general en el microcontrolador), dicho mensaje contendrá la siguiente información:
     I. Nombre del conductor.
     II. Vehículo que está conduciendo.
     III. Placas del vehículo.
     IV. Motivo del mensaje (En este caso, por falta de atención al camino).
     V. Coordenadas en tiempo real del vehículo.

     El mensaje con dicha información se enviará de forma permanente hasta que el usuario del vehículo digite un patrón en la interfaz gráfica.
   - b) ***Rastreo por somnolencia:*** Este subproceso se activará cuando ***SAMS_th*** sea superado por ***SAMS_counter***, el funcionamiento es el mismo que el ***Rastreo por distracción***, se enviará un mensaje aun número telefónico dado por el conductor, dicho mensaje contendrá la siguiente información:
     I. Nombre del conductor.
     II. Vehículo que está conduciendo.
     III. Placas del vehículo.
     IV. Motivo del mensaje (En este caso, por quedarse dormido mientras se conduce).
     V. Coordenadas en tiempo real del vehículo.

     El mensaje con dicha información se enviará de forma permanente hasta que el usuario del vehículo digite un patrón en la interfaz gráfica.

3.3.5 Resultados obtenidos del subcapítulo

1. **Análisis de resultados de la RNC-PP nueva versión:** Como se muestra en la Tabla 3.17, la configuración de la RNC-PP nueva versión tiene como base la arquitectura de la Tabla 3.7, pero en esta ocasión ya le hemos implementado una normalización por lotes en cada capa convolucional y adicionalmente se agregó una capa profunda con su respectivo Dropout; de igual manera se observa que únicamente requerimos poco más de 340,000 parámetros, para obtener una precisión del 95.77%.

Tabla 3. 17: Arquitectura general de la RNC-PP (nueva versión)

ESTRUCTURA PROPUESTA DE RNC-PP (nueva versión)		
Nombre de la capa	***Tamaño del mapa de características***	***Número de parámetros***
Conv2D	62 × 62 × 32	896
BatchNormalization	62 × 62 × 32	128
MaxPooling2D	31 × 31 × 32	0
Conv2D	29 × 29 × 32	9248
BatchNormalization	29 × 29 × 32	128
MaxPooling2D	14 × 14 × 32	0
Conv2D	12 × 12 × 64	18496
BatchNormalization	12 × 12 × 64	256
MaxPooling2D	6 × 6 × 16	0
Flatten	2304	0
Dropout	2304	0
FC	128	295040
Dropout	128	0
FC	128	16512
FC	3	387
Total de parámetros : 341,091 Parámetros entrenables: 340,835 Parámetros no entrenables: 256 Precisión= 95.77		

Como se muestra en la Tabla 3.18, la precisión en esta nueva versión de RNC-PP se redujo un 3.18 %, si bien, podría considerase como una desmejora, hay que tener en cuenta que se está clasificando un estado más en comparación a la primera versión.

Si bien, la precisión se redujo, de igual manera pudimos reducir tanto los parámetros entrenados, así como su peso en disco, haciendo mucho más liviano y acelerando el proceso de análisis al momento de implementar la red neuronal en los positivos móviles, entonces, es válido decir, que aun que la precisión es menor, las propiedades que conllevan a su creación y aplicación, son superiores y con esto, hemos cumplido con el objetivo de poder crear una RNC de bajo coste computacional teniendo el mejor rendimiento, además que el problema de los errores de predicciones dadas por nuestro modelo, se pueden corregir y de esto se aborda más a fondo en el punto **3.7 Corrección de predicción en tiempo real.**

Tabla 3. 18: Tabla comparativa entre las dos versiones de RNC-PP creadas

Modelo de RNC	Parámetros entrenables	Número de clasificaciones	Peso en disco (MB)	Precisión (%)
Primera versión	601,841	2	6.94	98.95
Segunda versión	340,835	3	3.99	95.77

En la Figura 3.65 mostramos los resultados obtenidos al momento de hacer diversas pruebas con diversos sujetos, teniendo diferentes condiciones de conducción, tales como:

I. **Conducción con luz normal y sin lentes (Figura 3.65 a):** En esta prueba podemos concluir que el sistema tuvo un rendimiento de predicción considerablemente bueno, ya que en general se consiguió en promedio un 96% de precisión, siendo la detección de patrones de somnolencia la que más se pudo predecir, esto en perspectiva es sumamente bueno, ya que evitar accidentes por somnolencia es uno de los principales puntos de esta investigación.

II. **Conducción con luz normal y con lentes (Figura 3.65 b):** Si bien, tener objetos que dificulten el análisis de los patrones de sueño es difícil de clasificar, la RNC tuvo una tasa de predicción promedio casi igual a estudiar rostros en condiciones normales sin usar ningún accesorio externo, esto es sumamente bueno, ya que podemos concluir que la RNC que proponeos no tiene mayor problema de determinar la somnolencia o la distracción en condiciones difíciles de estudio.

III. **Conducción con luz infrarroja sin lentes (Figura 3.65 c):** Las pruebas en condiciones de poca o nula iluminación siempre son y será un reto muy grande para los sistemas de monitoreo, por lo tanto tener una precisión promedio del 95.9% concluye que se puede estudiar a los conductores de una forma correcta y a su vez, el sistema que proponemos es capaz de poder monitorear al conductor las 24 horas del día sin ningún problema.

IV. **Conducción con luz infrarroja con lentes (Figura 3.65 d):** Finalmente, vemos como en esta prueba si se tuvo una precisión menor a los anteriores y por consiguiente a la general, ya que en este caso, se tuvo una precisión del 94%, pero esto no quiere decir que el sistemas sea malo, si no que en condiciones suma-mente no favorables, la RNC pude reconocer de forma eficaz 94 fotogramas de 100; los 6 fotogramas que por alguna razón no son posibles pronosticar de la mejor manera, esto se corrige con **el punto 3.7 Corrección de predicción en tiempo real**.

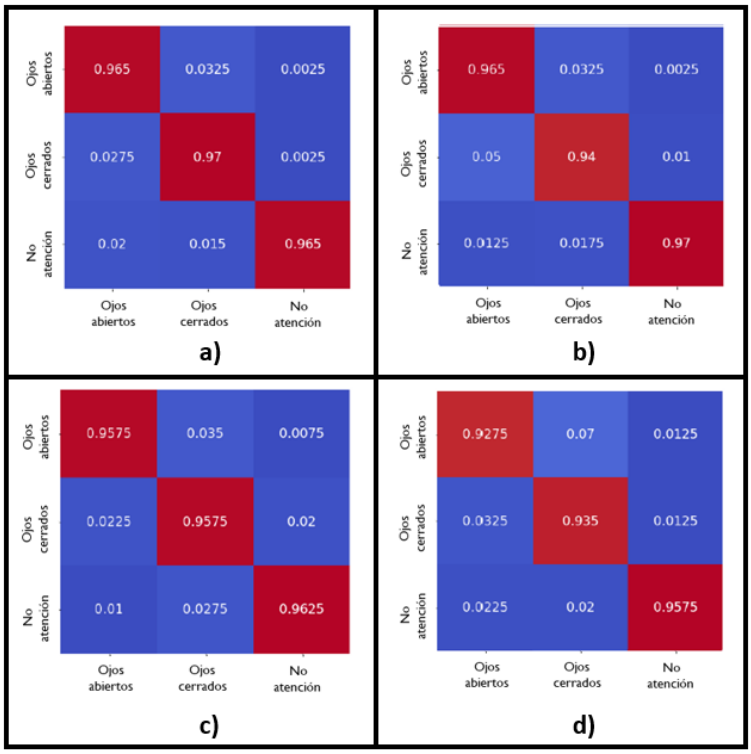


Figura 3. 65: Matrices de confusión [49] en las diferentes pruebas utilizando la base de datos NTHU-DDD

2. **Análisis de rendimiento del sistema propuesto:** Después del proceso de actualización y optimización que en este apartado se estuvo describiendo, se volvieron a replicar las pruebas de las Tablas 3.9 y 3.10, pero ahora implementando la RNC-PP propuesta (nueva versión), así como también el detector de rostros propuesto por Google y como se observa en la Tabla 3.19, nuestro sistema si tuvo un considerable aumento de velocidad de estudio, ya que el monitoreo en general aumentó casi el 80% en los dispositivos móviles empleados en estas pruebas, haciéndolos en este caso, factibles para su implementación en condiciones reales.

Tabla 3. 19: Tabla comparativa empleado el sistema general actualizado y optimizado de detección somnolencia y distracción en tiempo real en los diferentes dispositivos utilizados en esta investigación

Dispositivo	Velocidad de monitoreo en tiempo real a fotogramas por segundo(fps)			
	IMX219-77IR	Cámara Web HD C505	Cámara genérica	Video Cámara integrada Huawei D16
3070TI GPU, 8GB Memory,	No Posee Puerto CSI	22	23	----------

6144 cores, 1.770MHz.				
Huawei D16 Laptop, AMD Ryzen 5 4600H	No Posee Puerto CSI	20	21	31
Nvidia Jetson AGX Xavier	No Posee Puerto CSI	13.03	13	----------
Nvidia Jetson Nano 4GB	7	7.69	7.66	----------
SOMN_IA	7	7.69	7.66	----------

En la Tabla 3.20, podemos observar como el sistema general se mantuvo casi sin modificación al momento de compararlo con los resultados de la Tabla 3.10 hablando en el caso de los dispositivos del laboratorio; sin en cambio, donde si hubo un aumento considerable de aceleración al momento de monitorear la somnolencia y distracción en videos pregrabados, fue en los dispositivos móviles, haciendo que el sistema sea capaz de monitorear a los conductores, tanto en condiciones reales, así como en condiciones controladas, además que se puede concluir, que el método de aceleración, optimización y la implementación de otro detector de rostros con mayor precisión y la configuración propuesta de la nueva RNC-PP fue considerablemente un éxito.

Tabla 3. 20: Tabla comparativa en el proceso de fragmentación de videos pregrabados en los diferentes dispositivos utilizados en esta investigación

Dispositivo	Velocidad de reconocimiento en video a fotogramas por segundo(fps)			
	Video 1 Resolución: 1920 x1080	Video 2 Resolución: 1080 x 1920	Video 3 Resolución: 1280 x 720	Video 4 Resolución: 640 x 480
3070TI GPU, 8GB Memory, 6144 cores, 1.770MHz.	19.48	18.83	20.35	21.83
Huawei D16 Laptop, AMD Ryzen 5 4600H	18.89	18.22	20.12	20.98
Nvidia Jetson AGX Xavier	12.19	12.29	13.27	13.38

Nvidia Jetson Nano 4GB	7.61	7.06	7.53	7.02

3. **Ejemplos operacionales:** En la Tabla 3.21 dejamos el enlace de algunos videos en donde mostramos el funcionamiento y la comparación del sistema de detección de somnolencia y distracción primera y segunda versión

Tabla 3. 21: Enlace de los videos de funcionamiento con una pequeña descripción

Enlace	Descripción
https://youtu.be/tsZ9-kMBGtw	Demostración en tiempo real utilizando la base de datos NTHU-DDD.
https://youtu.be/tZyMdu6SZBY	Demostración bajo condiciones reales en tiempo real.

3.4 Red Neuronal Convolucional Propuesta vs Redes neuronales Convolucionales Populares

En el mundo de la IA existen diversos algoritmos que han sido diseñados de tal manera, que son tan buenos al grado que se han ido utilizando constantemente por mucho tiempo, de igual manera en el DL, existen modelos de redes neuronales diseñadas tan bien, que son arquitecturas que hasta el día de hoy se siguen utilizando, debido a su gran cantidad de parámetros entrenados, la versatilidad que se tienen y porque son capaces de poder detectar y clasifican muchas clases al mismo tiempo.

En el caso de la detección de objetos no es diferente, este tipo de redes son utilizadas para detectar elementos de diferentes naturalidades, y además que pueden clasificar imágenes de regiones específicas, esto ayuda mucho a sistemas de monitoreo, en específico para esta tesis nos importa mucho el área de la detección de somnolencia y distracción del conductor, algunas arquitecturas de RNC estándar o vulgarmente llamadas populares, como MobileNet [28], VGG16 [43] y ResNet50 [57], son utilizadas para la realización de dicha detección, ya que como vimos en el estado del arte, diversas investigaciones, patentes y prototipos tienen como base estos modelos.

Si bien tenemos como precedente las arquitecturas de dichas RNC populares, por si solas no pueden clasificar de la misma manera como el modelo de RNC-PP que nosotros proponemos, si no que tenemos que reentrenar estas redes con la base de datos que hemos creado, para que pueda reconocer los patrones que necesitamos clasificar, pero ¿Necesitamos crear nuestra propia versión del modelo de RNC popular tomando como base su estructura conforme a su documentación y así volver a repetir el proceso ya existente?, en definitiva no, porque si bien, la arquitectura es de libre acceso, ya existen diversos modelos ya creados por las bibliotecas en donde se genere aprendizaje profundo (como TensorFlow[58] o PyTorch [59]), y adicionalmente tienen varios

puntos a su favor debido a que estos algoritmos ya han sido codificados, optimizados y probados con anterioridad, por lo tanto, utilizar lo ya creado nos ha ahorrado mucho tiempo de escritura y nos ha evitado errores; cabe mencionar que en la utilización de estas estructuras podemos hacer uso de dos principales métodos los cuales son:

1. ***Entrenamiento de la RNC estándar con el puro set de imágenes:*** Este proceso es simple, lo único que necesitamos es tener clasificado y etiquetado los parámetros que queremos agregar a las arquitecturas hechas previamente por la librería, se agregan a la memoria y se reentrenan los modelos sin modificar ningún parámetro ni hiper parámetro, es decir, sólo agrandaremos los patrones de reconocimiento a la propia RNC sin afectar ningún dato de su arquitectura original, esto es factible si no requerimos una estructura especifica, si no que sólo un set de resultados que posteriormente se obtendrán; por la conformación de este tipo de investigación, el ***Entrenamiento de la RNC estándar con el puro set de imágenes*** no nos será de utilidad.
2. ***Entrenamiento de la RNC estándar por transferencia de aprendizaje[60]:*** En este proceso a diferencia de la anterior, podemos modificar muchos parámetros de la arquitectura original, no sólo la implementación de nuestro set de imágenes. La técnica de transferencia de aprendizaje ajusta los pesos de conexión desde la capa N hasta la última capa Completamente Conectada (FC) de la RNC estándar. Esta técnica es efectiva cuando el tamaño del conjunto de datos es relativamente pequeño y la tarea es una clasificación de imágenes, porque algunas capas congeladas anteriores que se dedican a extraer características elementales de las imágenes se pueden usar sin un proceso de entrenamiento.

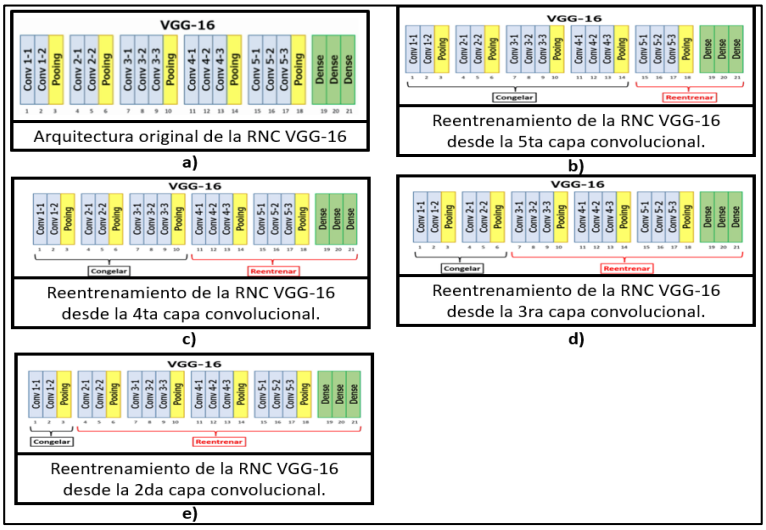


Figura 3. 66: Diferentes tipos de transferencia de aprendizaje utilizando la arquitectura VGG16 como ejemplo

En la Figura 3.66 vemos un ejemplo del proceso de transferencia de aprendizaje, tomando como ejemplo la arquitectura de la RNC estándar del Grupo

de Geometría Visual (VGG por sus siglas en inglés) con un número de capas convolucionales de 16.

- En la Figura 3.66 a vemos un ejemplo del proceso de entrenamiento sin que modifiquemos ningún parámetro de las primeras 5 capas convolucionales, posteriormente aplicamos nosotros nuestra propia arquitectura de Top model (FC) así como añadimos los valores del compilador del entrenamiento sin módulos modificado**s**, la cual estará unida a la arquitectura original, cabe mencionar que los parámetros de aplanamiento, capas completamente conectadas, optimizadores y demás de Top model, nosotros lo diseñaremos.
- En la Figura 3.66 b, podemos observar un ejemplo donde empezamos a modificar la configuración de la arquitectura original, vemos dos parámetros muy interesantes, el primero como se comentó anteriormente es un congelamiento de entrenamiento, lo que hace esto es no modificar ningún parámetro de la configuración original así como mantener los pesos originales, posteriormente se entrena la RNC con normalidad, pero en este caso, a partir de la 5ta capa, este proceso de congelamiento se termina y los pesos que no se modificaron anteriormente, aquí se modificarán debido a que en esta capa convolucional se ha cambiado diversos valores como el optimizador, la función de perdida, así como el número de épocas de validación, finalmente se prosigue con la unión de nuestra propia configuración Top model, así como el anexo de los parámetros del anterior rentrenamiento.
- En las figuras posteriores, se da un ejemplo de diversas transferencias de aprendizaje que se pueden hacer utilizando a VGG16; el proceso es el mismo que en el punto anterior, pero aquí hay que tener en cuenta sólo un punto importante, la primera capa no se recomienda modificar, debido a que esta es la que inicia los pesos originales de la configuración en turno de la RNC estándar, si reentrenamos esta capa, se modifica el 100% de la configuración original, por ende, la utilización de este tipo de técnica se hace inútil, debido a que se estaría entrenando una nueva red y no haríamos uso del aprendizaje obtenido previamente.

Retomando en punto principal de este apartado y teniendo ya un conocimiento general de cómo funciona la transferencia de aprendizaje (fine tuning), nosotros hemos optado por aplicarle fine tuning a cinco RNC populares, las cuales son: MobileNet-V2 [28], VGG16 [43], ResNet-50V2[58], InceptionV3 [61] y Xception [29].

- MobileNet-V2 fue diseñado para funcionar bien en los dispositivos móviles, reduciendo el número de pesos de conexión y manteniendo un buen rendimiento.
- El módulo de repeticiones VGG16 compuesto por dos 3×3 Conv2D y Pooling. La profundidad de esta RNC es relativamente pequeña; sin embargo, la cantidad de peso de conexión es enorme.
- ResNet-50V2 es una de las clases de ResNet, que se diseñó para evitar el problema de desaparición de gradientes en la RNC más profunda, introduciendo un módulo residual.
- El InceptionV3 está compuesto por varios módulos de inicio, que aplican diferentes tamaños de RNC para ingresar datos en paralelo.
- Xception tiene un concepto similar al de InceptionV3, pero el módulo de inicio de Xception contiene una mayor cantidad de RNC.

Para obtener un modelo óptimo de transferencia de aprendizaje de cada RNC, descongelamos gradualmente desde la última capa de FC hasta las primeras capas (capas cercanas a la capa de entrada). Por ejemplo, en MobileNet-V2, que contiene 17 módulos residuales invertidos descongelamos desde la capa FC final a los módulos residuales invertidos anteriores para obtener un modelo óptimo de clasificación de patrones de somnolencia y distracción.

3.4.1 Configuración de transferencia de aprendizaje en la arquitectura MobileNet-V2

A continuación veremos la configuración de aprendizaje de cada módulo que conforma a la arquitectura de MobileNet-V2, así como los parámetros que se tomaron por cada reentrenamiento.

- Características generales de la base de datos
  - Tamaño del set de entrenamiento: 3600 imágenes:
  - Tamaño del set de prueba: 1200 imágenes.
  - Clasificaciones o etiquetas: 3, “ojos abiertos”, “ojos cerrados”, “distracción”.
  - Tamaño de la imagen de entrada: 64×64 pixeles.
  - Número de canales de la imagen de entrada: 3 canales en el orden RGB.
  - Pesos pre-entrenados usados: Se utiliza los pesos del preentrenamiento estándar utilizando el set de datos ImageNET [62].
  - Preprocesamiento utilizado: Se utiliza el preprocesamiento ImageNET interno de la librería utilizada (TensorFlow 2.5).
- Top Model o arquitectura de FC y posteriores:
  - ***Preprocesamiento de nuestro set de imágenes***: Este preprocesamiento es el tratamiento que nosotros le daremos a nuestras imágenes.
    - Rango de rotación: 10º.

- Rango de cambio de anchura: 0,1.
- Rango de cambio de altura: 0,1.
- Rango de corte: 0,1.
- Rango de zoom: 0,1.
- Volteado horizontal: True.
- Relleno: "nearest".

- ***Arquitectura de FC:***
  - Aplanamiento.
  - Dropout: 25%.
  - Full Conect (FC) o la Capa Profunda Completamente Conectada:
    - Número de neuronas completamente conectadas: 64.
    - Activación o Linealización: ReLU.
  - Dropout: 25%.
  - Full Conect (FC) o la Capa Profunda Completamente Conectada:
    - Número de neuronas completamente conectadas: 64.
    - Activación o Linealización: ReLU.
  - Dropout: 25%.
  - Full Conect (FC) o la Capa Profunda Completamente Conectada:
    - Número de neuronas completamente conectadas: 3.
    - Activación o Linealización: Softmax.
- ***Compilador*:**
  - Optimizador: Adam.
  - β_1 : 0,9.
  - β_2 : 0,99999.
  - Tasa de aprendizaje α : 0,00001.
  - Función de pérdida: Categorical Crossentropy
  - Tamaño del lote: 32.

La arquitectura de MobileNet-V2 está compuesto por 17 módulos residuales invertidos, por lo tanto generamos 17 diferentes versiones de fine tuning, además de una adicional, en donde no utilizamos ninguna técnica de congelamiento si no que sólo fue la inclusión del Top model y el compilador:

- ***Rentrenamiento sin módulos modificados***: En este proceso dejamos que la RNC se entrenara sin la modificación de ningún parámetro por lo tanto congelamos todas las capas convolucionales y posteriormente se agregaron las configuraciones de FC e híper parámetros:
  - ***Top Model***.
  - ***Compilador del Rentrenamiento Sin Módulos Modificados (CRSMM):***
    - Optimizador: Propagación de raíz cuadrada media (RMSProp)[63].
    - β_1 : 0,9.

- β_2 : 0,999.
- Tasa de aprendizaje α : 0,001.
- Función de pérdida: Categorical Crossentropy [64].
- Tamaño del lote: 32.

- ***Primer módulo entrenado (capa 151):*** Este proceso es el primero en donde aplicamos transferencia de aprendizaje a las capas convolucionales de MobileNET V2, en esta Fine tuning congelamos las primeras capas hasta la 151, es a partir de esta que modificamos los siguientes parámetros:
  - ***Top Model***.
  - ***CRSMM.***
  - ***Compilador del Módulo 1:***
    - Optimizador: Gradiente descendiente Estocástico (SGD) [63].
    - Tasa de aprendizaje α : 0,001.
    - Función de pérdida: Categorical Crossentropy
    - Tamaño del lote: 32.

A partir de este módulo, los compiladores posteriores tiene la misma estructura, por lo tanto:

Tabla 3. 22:Diferentes FINE TUNING implementadas a la RNC MobileNet-V2

NIVEL DEL MÓDULO DE FINE-TUNING	Arquitectura
Rentrenamiento sin módulos modificados	Top Model. CRSMM.
MÓDULO 1 ENTRENADO (CAPA 143)	Top Model. CRSMM. Compilador del Módulo 1
MÓDULO 2 ENTRENADO (CAPA 143)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2
MÓDULO 3 ENTRENADO (CAPA 134)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3
MÓDULO 4 ENTRENADO (CAPA 125)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4
ccccccc	Top Model.

	CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5
MÓDULO 6 ENTRENADO (CAPA 107)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6
MÓDULO 7 ENTRENADO (CAPA 98)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7
MÓDULO 8 ENTRENADO (CAPA 90)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8
MÓDULO 9 ENTRENADO (CAPA 81)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8 Compilador del Módulo 9

MÓDULO 10 ENTRENADO (CAPA 72)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8 Compilador del Módulo 9 Compilador del Módulo 10
MÓDULO 11 ENTRENADO (CAPA 63)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8 Compilador del Módulo 9 Compilador del Módulo 10 Compilador del Módulo 11
MÓDULO 12 ENTRENADO (CAPA 54)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8 Compilador del Módulo 9 Compilador del Módulo 10 Compilador del Módulo 11 Compilador del Módulo 12
MÓDULO 13 ENTRENADO (CAPA 45)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4

	Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8 Compilador del Módulo 9 Compilador del Módulo 10 Compilador del Módulo 11 Compilador del Módulo 12 Compilador del Módulo 13
MÓDULO 14 ENTRENADO (CAPA 36)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8 Compilador del Módulo 9 Compilador del Módulo 10 Compilador del Módulo 11 Compilador del Módulo 12 Compilador del Módulo 13 Compilador del Módulo 14
MÓDULO 15 ENTRENADO (CAPA 27)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8 Compilador del Módulo 9 Compilador del Módulo 10 Compilador del Módulo 11 Compilador del Módulo 12 Compilador del Módulo 13 Compilador del Módulo 14 Compilador del Módulo 15
MÓDULO 16 ENTRENADO (CAPA 18)	Top Model. CRSMM. Compilador del Módulo 1

	Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8 Compilador del Módulo 9 Compilador del Módulo 10 Compilador del Módulo 11 Compilador del Módulo 12 Compilador del Módulo 13 Compilador del Módulo 14 Compilador del Módulo 15 Compilador del Módulo 16
MÓDULO 17 ENTRENADO (CAPA 8)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8 Compilador del Módulo 9 Compilador del Módulo 10 Compilador del Módulo 11 Compilador del Módulo 12 Compilador del Módulo 13 Compilador del Módulo 14 Compilador del Módulo 15 Compilador del Módulo 16 Compilador del Módulo 17

3.4.2 Configuración de transferencia de aprendizaje en la arquitectura VGG16

A continuación veremos la configuración de aprendizaje de cada módulo que conforma a la arquitectura de VGG 16, así como los parámetros que se tomaron por cada reentrenamiento.

- Características generales de la base de datos
  - Tamaño del set de entrenamiento: 3600 imágenes:
  - Tamaño del set de prueba: 1200 imágenes.
  - Clasificaciones o etiquetas: 3, "ojos abiertos", "ojos cerrados", "distracción".
  - Tamaño de la imagen de entrada: 64×64 pixeles.
  - Número de canales de la imagen de entrada: 3 canales em el orden RGB.
  - Pesos pre-entrenados usados: Se utiliza los pesos del preentrenamiento estándar utilizando el set de datos ImageNET.
  - Preprocesamiento utilizado: Se utiliza el preprocesamiento ImageNET interno de la librería utilizada (TensorFlow 2.5).
- Top Model o arquitectura de FC y posteriores:
  - ***Preprocesamiento de nuestro set de imágenes***:
    - Rango de rotación: 10º.
    - Rango de cambio de anchura: 0,1.
    - Rango de cambio de altura: 0,1.
    - Rango de corte: 0,1.
    - Rango de zoom: 0,1.
    - Volteado horizontal: True.
    - Relleno: "nearest".
  - ***Arquitectura de FC:***
    - Aplanamiento.
    - Dropout: 25%.
    - Full Conect (FC) o la Capa Profunda Completamente Conectada:
      - Número de neuronas completamente conectadas: 64.
      - Activación o Linealización: ReLU.
    - Dropout: 25%.
    - Full Conect (FC) o la Capa Profunda Completamente Conectada:
      - Número de neuronas completamente conectadas: 64.
      - Activación o Linealización: ReLU.
    - Dropout: 25%.
    - Full Conect (FC) o la Capa Profunda Completamente Conectada:
      - Número de neuronas completamente conectadas: 3.

- Activación o Linealización: Softmax.

- ***Compilador*:**
  - Optimizador: Adam.
  - β_1 : 0,9.
  - β_2 : 0,99999.
  - Tasa de aprendizaje α : 0,00001.
  - Función de pérdida: Categorical Crossentropy
  - Tamaño del lote: 32.

La arquitectura de VGG16 está compuesto por 4 módulos residuales invertidos, por lo tanto generamos 4 diferentes versiones de fine tuning, además de una adicional, en donde no utilizamos ninguna técnica de congelamiento si no que sólo fue la inclusión del Top model y el compilador:

- ***Rentrenamiento sin módulos modificados***: En este proceso dejamos que la RNC se entrenara sin la modificación de ningún parámetro por lo tanto congelamos todas las capas convolucionales y posteriormente se agregaron las configuraciones de FC e híper parámetros:
  - ***Top Model***.
  - ***Compilador del Rentrenamiento Sin Módulos Modificados (CRSMM):***
    - Optimizador: Propagación de raíz cuadrada media (RMSProp).
    - β_1 : 0,9.
    - β_2 : 0,999.
    - Tasa de aprendizaje α : 0,001.
    - Función de pérdida: Categorical Crossentropy
    - Tamaño del lote: 32.
- ***Primer módulo entrenado (capa 151):*** Este proceso es el primero en donde aplicamos transferencia de aprendizaje a las capas convolucionales de MobileNET V2, en esta Fine tuning congelamos las primeras capas hasta la 151, es a partir de esta que modificamos los siguientes parámetros:
  - ***Top Model***.
  - ***CRSMM.***
  - ***Compilador del Módulo 1:***
    - Optimizador: Gradiente descendiente Estocástico (SGD).
    - Tasa de aprendizaje α : 0,001.
    - Función de pérdida: Categorical Crossentropy
    - Tamaño del lote: 32.

A partir de este módulo, los compiladores posteriores tiene la misma estructura, por lo tanto:

Tabla 3. 23: Diferentes FINE TUNING implementadas a la RNC VGG16

NIVEL DEL MÓDULO DE FINE-TUNING	Arquitectura
Rentrenamiento sin módulos modificados	Top Model. CRSMM.
MÓDULO 1 ENTRENADO (CAPA 15)	Top Model. CRSMM. Compilador del Módulo 1
MÓDULO 2 ENTRENADO (CAPA 11)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2
MÓDULO 3 ENTRENADO (CAPA 7)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3
MÓDULO 4 ENTRENADO (CAPA 4)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4

3.4.3 Configuración de transferencia de aprendizaje en la arquitectura ResNet-50V2

A continuación veremos la configuración de aprendizaje de cada módulo que conforma a la arquitectura de ResNet-50V2, así como los parámetros que se tomaron por cada reentrenamiento.

- Características generales de la base de datos
  - Tamaño del set de entrenamiento: 3600 imágenes:
  - Tamaño del set de prueba: 1200 imágenes.
  - Clasificaciones o etiquetas: 3, “ojos abiertos”, “ojos cerrados”, “distracción”.
  - Tamaño de la imagen de entrada: 64×64 pixeles.
  - Número de canales de la imagen de entrada: 3 canales em el orden RGB.
  - Pesos pre-entrenados usados: Se utiliza los pesos del preentrenamiento estándar utilizando el set de datos ImageNET.
  - Preprocesamiento utilizado: Se utiliza el preprocesamiento ImageNET interno de la librería utilizada (TensorFlow 2.5).
- Top Model o arquitectura de FC y posteriores:

- ***Preprocesamiento de nuestro set de imágenes***:
  - Rango de rotación: 10º.
  - Rango de cambio de anchura: 0,1.
  - Rango de cambio de altura: 0,1.
  - Rango de corte: 0,1.
  - Rango de zoom: 0,1.
  - Volteado horizontal: True.
  - Relleno: "nearest".
- ***Arquitectura de FC:***
  - Aplanamiento.
  - Dropout: 25%.
  - Full Conect (FC) o la Capa Profunda Completamente Conectada:
    - Número de neuronas completamente conectadas: 64.
    - Activación o Linealización: ReLU.
  - Dropout: 25%.
  - Full Conect (FC) o la Capa Profunda Completamente Conectada:
    - Número de neuronas completamente conectadas: 64.
    - Activación o Linealización: ReLU.
  - Dropout: 25%.
  - Full Conect (FC) o la Capa Profunda Completamente Conectada:
    - Número de neuronas completamente conectadas: 3.
    - Activación o Linealización: Softmax.
- ***Compilador*:**
  - Optimizador: Adam.
  - β_1 : 0,9.
  - β_2 : 0,99999.
  - Tasa de aprendizaje α : 0,00001.
  - Función de pérdida: Categorical Crossentropy
  - Tamaño del lote: 32.

La arquitectura de ResNet-50V2 está compuesto por 4 módulos residuales invertidos, por lo tanto generamos 4 diferentes versiones de fine tuning, además de una adicional, en donde no utilizamos ninguna técnica de congelamiento si no que sólo fue la inclusión del Top model y el compilador:

- ***Rentrenamiento sin módulos modificados***: En este proceso dejamos que la RNC se entrenara sin la modificación de ningún parámetro por lo tanto congelamos todas las capas convolucionales y posteriormente se agregaron las configuraciones de FC e híper parámetros:
  - ***Top Model***.
  - ***Compilador del Rentrenamiento Sin Módulos Modificados (CRSMM):***
    - Optimizador: Propagación de raíz cuadrada media (RMSProp).
    - β_1 : 0,9.
    - β_2 : 0,999.
    - Tasa de aprendizaje α : 0,001.
    - Función de pérdida: Categorical Crossentropy

- Tamaño del lote: 32.

- ***Primer módulo entrenado (capa 143):*** Este proceso es el primero en donde aplicamos transferencia de aprendizaje a las capas convolucionales de ResNet-50V2, en esta Fine tuning congelamos las primeras capas hasta la 143, es a partir de esta que modificamos los siguientes parámetros:
  - ***Top Model***.
  - ***CRSMM.***
  - ***Compilador del Módulo 1:***
    - Optimizador: Gradiente descendiente Estocástico (SGD).
    - Tasa de aprendizaje α : 0,001.
    - Función de pérdida: Categorical Crossentropy
    - Tamaño del lote: 32.

A partir de este módulo, los compiladores posteriores tiene la misma estructura, por lo tanto:

Tabla 3. 24: Diferentes FINE TUNING implementadas a la RNC ResNet-50V2

NIVEL DEL MÓDULO DE FINE-TUNING	Arquitectura
Rentrenamiento sin módulos modificados	Top Model. CRSMM.
MÓDULO 1 ENTRENADO (CAPA 143)	Top Model. CRSMM. Compilador del Módulo 1
MÓDULO 2 ENTRENADO (CAPA 81)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2
MÓDULO 3 ENTRENADO (CAPA 39)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3
MÓDULO 4 ENTRENADO (CAPA 7)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4

3.4.4 Configuración de transferencia de aprendizaje en la arquitectura InceptionV3

A continuación veremos la configuración de aprendizaje de cada módulo que conforma a la arquitectura de InceptionV3, así como los parámetros que se tomaron por cada reentrenamiento.

- Características generales de la base de datos
  - Tamaño del set de entrenamiento: 3600 imágenes:
  - Tamaño del set de prueba: 1200 imágenes.
  - Clasificaciones o etiquetas: 3, "ojos abiertos", "ojos cerrados", "distracción".
  - Tamaño de la imagen de entrada: 75×75 pixeles.
  - Número de canales de la imagen de entrada: 3 canales em el orden RGB.
  - Pesos pre-entrenados usados: Se utiliza los pesos del preentrenamiento estándar.
  - Preprocesamiento utilizado: Se utiliza el preprocesamiento ImageNET interno de la librería utilizada (TensorFlow 2.5).
- Top Model o arquitectura de FC y posteriores:
  - ***Preprocesamiento de nuestro set de imágenes***:
    - Rango de rotación: 10º.
    - Rango de cambio de anchura: 0,1.
    - Rango de cambio de altura: 0,1.
    - Rango de corte: 0,1.
    - Rango de zoom: 0,1.
    - Volteado horizontal: True.
    - Relleno: "nearest".
  - ***Arquitectura de FC:***
    - Aplanamiento.
    - Dropout: 25%.
    - Full Conect (FC) o la Capa Profunda Completamente Conectada:
      - Número de neuronas completamente conectadas: 64.
      - Activación o Linealización: ReLU.
    - Dropout: 25%.
    - Full Conect (FC) o la Capa Profunda Completamente Conectada:
      - Número de neuronas completamente conectadas: 64.
      - Activación o Linealización: ReLU.
    - Dropout: 25%.
    - Full Conect (FC) o la Capa Profunda Completamente Conectada:
      - Número de neuronas completamente conectadas: 3.

- Activación o Linealización: Softmax.

- ***Compilador*:**
  - Optimizador: Adam.
  - β_1 : 0,9.
  - β_2 : 0,99999.
  - Tasa de aprendizaje α : 0,00001.
  - Función de pérdida: Categorical Crossentropy
  - Tamaño del lote: 32.

La arquitectura de InceptionV3 está compuesto por 7 módulos residuales invertidos, por lo tanto generamos 7 diferentes versiones de fine tuning, además de una adicional, en donde no utilizamos ninguna técnica de congelamiento si no que sólo fue la inclusión del Top model y el compilador:

- ***Rentrenamiento sin módulos modificados***: En este proceso dejamos que la RNC se entrenara sin la modificación de ningún parámetro por lo tanto congelamos todas las capas convolucionales y posteriormente se agregaron las configuraciones de FC e híper parámetros:
  - ***Top Model***.
  - ***Compilador del Rentrenamiento Sin Módulos Modificados (CRSMM):***
    - Optimizador: Propagación de raíz cuadrada media (RMSProp).
    - β_1 : 0,9.
    - β_2 : 0,999.
    - Tasa de aprendizaje α : 0,001.
    - Función de pérdida: Categorical Crossentropy
    - Tamaño del lote: 32.
- ***Primer módulo entrenado (capa 280):*** Este proceso es el primero en donde aplicamos transferencia de aprendizaje a las capas convolucionales de InceptionV3, en esta Fine tuning congelamos las primeras capas hasta la 280, es a partir de esta que modificamos los siguientes parámetros:
  - ***Top Model***.
  - ***CRSMM.***
  - ***Compilador del Módulo 1:***
    - Optimizador: Gradiente descendiente Estocástico (SGD).
    - Tasa de aprendizaje α : 0,001.
    - Función de pérdida: Categorical Crossentropy
    - Tamaño del lote: 32.

A partir de este módulo, los compiladores posteriores tiene la misma estructura, por lo tanto:

Tabla 3. 25: Diferentes FINE TUNING implementadas a la RNC InceptionV3

NIVEL DEL MÓDULO DE FINE-TUNING	Arquitectura
Rentrenamiento sin módulos modificados	Top Model. CRSMM.

MÓDULO 1 ENTRENADO (CAPA 280)	Top Model. CRSMM. Compilador del Módulo 1
MÓDULO 2 ENTRENADO (CAPA 249)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2
MÓDULO 3 ENTRENADO (CAPA 229)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3
MÓDULO 4 ENTRENADO (CAPA 197)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4
MÓDULO 5 ENTRENADO (CAPA 165)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5
MÓDULO 6 ENTRENADO (CAPA 133)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6
MÓDULO 7 ENTRENADO (CAPA 101)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7

3.4.5 Configuración de transferencia de aprendizaje en la arquitectura Xception

A continuación veremos la configuración de aprendizaje de cada módulo que conforma a la arquitectura de Xception, así como los parámetros que se tomaron por cada reentrenamiento.

- Características generales de la base de datos
  - Tamaño del set de entrenamiento: 3600 imágenes:
  - Tamaño del set de prueba: 1200 imágenes.
  - Clasificaciones o etiquetas: 3, "ojos abiertos", "ojos cerrados", "distracción".
  - Tamaño de la imagen de entrada: 64×64 pixeles.
  - Número de canales de la imagen de entrada: 3 canales em el orden RGB.
  - Pesos pre-entrenados usados: Se utiliza los pesos del preentrenamiento estándar.
  - Preprocesamiento utilizado: Se utiliza el preprocesamiento interno de la librería utilizada (TensorFlow 2.5).
- Top Model o arquitectura de FC y posteriores:
  - ***Preprocesamiento de nuestro set de imágenes***:
    - Rango de rotación: 10º.
    - Rango de cambio de anchura: 0,1.
    - Rango de cambio de altura: 0,1.
    - Rango de corte: 0,1.
    - Rango de zoom: 0,1.
    - Volteado horizontal: True.
    - Relleno: "nearest".
  - ***Arquitectura de FC:***
    - Aplanamiento.
    - Dropout: 25%.
    - Full Conect (FC) o la Capa Profunda Completamente Conectada:
      - Número de neuronas completamente conectadas: 64.
      - Activación o Linealización: ReLU.
    - Dropout: 25%.
    - Full Conect (FC) o la Capa Profunda Completamente Conectada:
      - Número de neuronas completamente conectadas: 64.
      - Activación o Linealización: ReLU.
    - Dropout: 25%.
    - Full Conect (FC) o la Capa Profunda Completamente Conectada:
      - Número de neuronas completamente conectadas: 3.

- Activación o Linealización: Softmax.

- ***Compilador*:**
  - Optimizador: Adam.
  - β_1 : 0,9.
  - β_2 : 0,99999.
  - Tasa de aprendizaje α : 0,00001.
  - Función de pérdida: Categorical Crossentropy
  - Tamaño del lote: 32.

La arquitectura de Xception está compuesto por 13 módulos residuales invertidos, por lo tanto generamos 13 diferentes versiones de fine tuning, además de una adicional, en donde no utilizamos ninguna técnica de congelamiento si no que sólo fue la inclusión del Top model y el compilador:

- ***Rentrenamiento sin módulos modificados***: En este proceso dejamos que la RNC se entrenara sin la modificación de ningún parámetro por lo tanto congelamos todas las capas convolucionales y posteriormente se agregaron las configuraciones de FC e híper parámetros:
  - ***Top Model***.
  - ***Compilador del Rentrenamiento Sin Módulos Modificados (CRSMM):***
    - Optimizador: Propagación de raíz cuadrada media (RMSProp).
    - β_1 : 0,9.
    - β_2 : 0,999.
    - Tasa de aprendizaje α : 0,001.
    - Función de pérdida: Categorical Crossentropy
    - Tamaño del lote: 32.
- ***Primer módulo entrenado (capa 126):*** Este proceso es el primero en donde aplicamos transferencia de aprendizaje a las capas convolucionales de 126, en esta Fine tuning congelamos las primeras capas hasta la 126, es a partir de esta que modificamos los siguientes parámetros:
  - ***Top Model***.
  - ***CRSMM.***
  - ***Compilador del Módulo 1:***
    - Optimizador: Gradiente descendiente Estocástico (SGD).
    - Tasa de aprendizaje α : 0,001.
    - Función de pérdida: Categorical Crossentropy
    - Tamaño del lote: 32.

A partir de este módulo, los compiladores posteriores tiene la misma estructura, por lo tanto:

Tabla 3. 26: Diferentes FINE TUNING implementadas a la RNC Xception

NIVEL DEL MÓDULO DE FINE-TUNING	Arquitectura
Rentrenamiento sin módulos modificados	Top Model. CRSMM.
MÓDULO 1 ENTRENADO (CAPA 126)	Top Model. CRSMM. Compilador del Módulo 1
MÓDULO 2 ENTRENADO (CAPA 116)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2
MÓDULO 3 ENTRENADO (CAPA 106)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3
MÓDULO 4 ENTRENADO (CAPA 96)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4
MÓDULO 5 ENTRENADO (CAPA 86)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5
MÓDULO 6 ENTRENADO (CAPA 76)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6
MÓDULO 7 ENTRENADO (CAPA 66)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5

	Compilador del Módulo 6 Compilador del Módulo 7
MÓDULO 8 ENTRENADO (CAPA 56)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8
MÓDULO 9 ENTRENADO (CAPA 46)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8 Compilador del Módulo 9
MÓDULO 10 ENTRENADO (CAPA 36)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8 Compilador del Módulo 9 Compilador del Módulo 10
MÓDULO 11 ENTRENADO (CAPA 26)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8

	Compilador del Módulo 9 Compilador del Módulo 10 Compilador del Módulo 11
MÓDULO 12 ENTRENADO (CAPA 16)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8 Compilador del Módulo 9 Compilador del Módulo 10 Compilador del Módulo 11 Compilador del Módulo 12
MÓDULO 13 ENTRENADO (CAPA 7)	Top Model. CRSMM. Compilador del Módulo 1 Compilador del Módulo 2 Compilador del Módulo 3 Compilador del Módulo 4 Compilador del Módulo 5 Compilador del Módulo 6 Compilador del Módulo 7 Compilador del Módulo 8 Compilador del Módulo 9 Compilador del Módulo 10 Compilador del Módulo 11 Compilador del Módulo 12 Compilador del Módulo 13

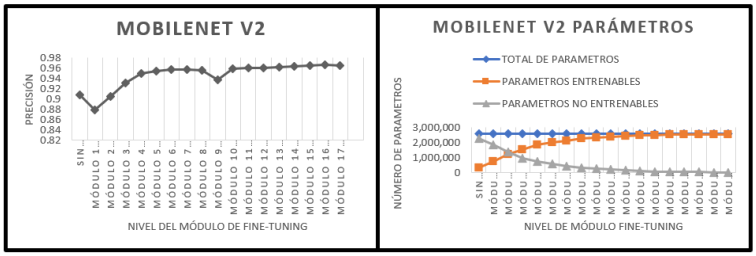
3.4.6 Resultados obtenidos del subcapítulo

1. Análisis de graficas por cada transferencia de aprendizaje.

I. **Mobilenet V2:** En la Figura 3.67 a podemos denotar como la precisión va en aumento por cada reentrenamiento de módulo, hasta llegar a un máximo de 96.75%, el cual, en este caso fue en la capa 18 o módulo 16 (en la Tabla 3.22 se muestra los parámetros utilizados en el proceso de transferencia de aprendizaje).

En la Figura 3.67 b se muestra como cada vez que se rentrena el modelo, se empiezan a necesitar más patrones de entrenamiento, los cuales

son necesarios para que la arquitectura pueda aprender, en este caso se necesitaron 2,549,011 patrones.

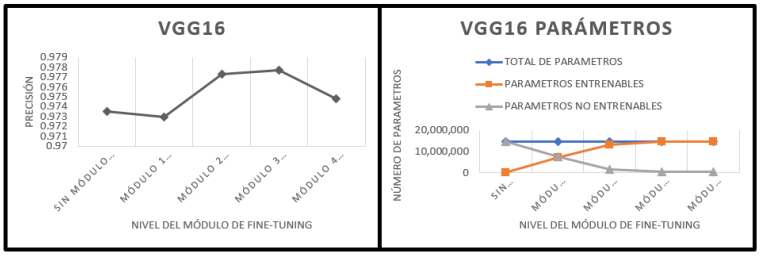


a) b)

Figura 3. 67: Gráficas de las capas reentrenadas de Mobilenet v2 con respecto a su precisión y el número de parámetros entrenados. Se realizaron 18 reentrenamientos.

II. **VGG16:** En la Figura 3.68 a podemos denotar como la precisión va en aumento por cada reentrenamiento de módulo, hasta llegar a un máximo de 97.77%, el cual, en este caso fue en la capa 7 o módulo 3 (en la Tabla 3.23 se muestra los parámetros utilizados en el proceso de transferencia de aprendizaje).

En la Figura 3.68 b se muestra como cada vez que se rentrena el modelo, se empiezan a necesitar más patrones de entrenamiento, los cuales son necesarios para que la arquitectura pueda aprender, en este caso se necesitaron 14,590,019 patrones.



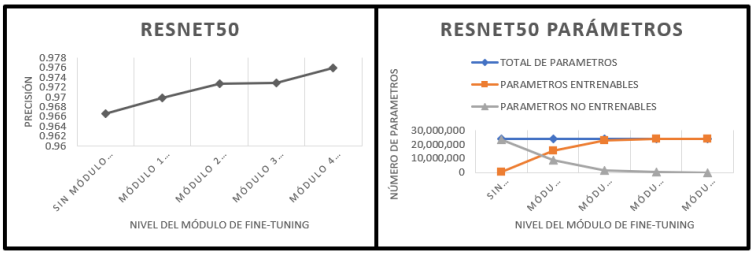
a) b)

Figura 3. 68: Gráficas de las capas reentrenadas de VGG16 con respecto a su precisión y el número de parámetros entrenados. Se realizaron 5 reentrenamientos.

III. **ResNet-50V2:** En la Figura 3.69 a podemos denotar como la precisión va en aumento por cada reentrenamiento de módulo, hasta llegar a un máximo de 97.6%, el cual, en este caso fue en la capa 7 o módulo 4 (en la Tabla 3.24 se muestra los parámetros utilizados en el proceso de transferencia de aprendizaje).

En la Figura 3.69 b se muestra como cada vez que se rentrena el modelo, se empiezan a necesitar más patrones de entrenamiento, los cuales

son necesarios para que la arquitectura pueda aprender, en este caso se necesitaron 24,053,699 patrones.

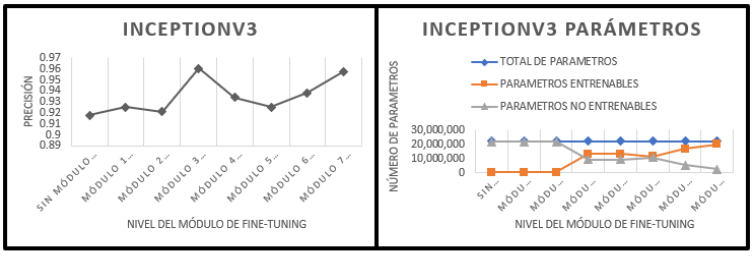


a) b)

Figura 3. 69: Gráficas de las capas reentrenadas de ResNet-50V2 con respecto a su precisión y el número de parámetros entrenados. Se realizaron 5 reentrenamientos.

IV. **InceptionV3:** En la Figura 3.70 a podemos denotar como la precisión va en aumento por cada reentrenamiento de módulo, hasta llegar a un máximo de 96%, el cual, en este caso fue en la capa 229 o módulo 3 (en la Tabla 3.25 se muestra los parámetros utilizados en el proceso de transferencia de aprendizaje).

En la Figura 3.70 b se muestra como cada vez que se rentrena el modelo, se empiezan a necesitar más patrones de entrenamiento, los cuales son necesarios para que la arquitectura pueda aprender, en este caso se necesitaron 12,947,395 patrones.



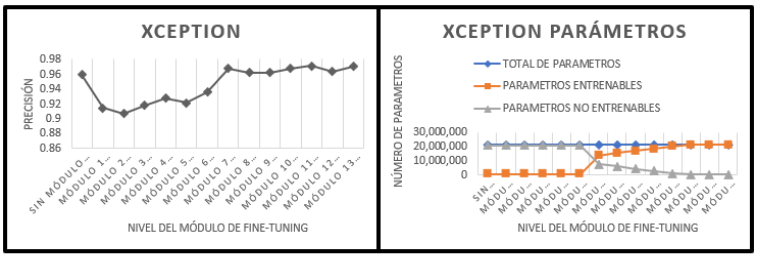
a) b)

Figura 3. 70: Gráficas de las capas reentrenadas de InceptionV3 con respecto a su precisión y el número de parámetros entrenados. Se realizaron 8 reentrenamientos.

V. **Xception:** En la Figura 3.71 a podemos denotar como la precisión va en aumento por cada reentrenamiento de módulo, hasta llegar a un máximo de 97.13%, el cual, en este caso fue en la capa 63 o módulo 11 (en la Tabla 3.26 se muestra los parámetros utilizados en el proceso de transferencia de aprendizaje).

En la Figura 3.71 b se muestra como cada vez que se rentrena el modelo, se empiezan a necesitar más patrones de entrenamiento, los cuales

son necesarios para que la arquitectura pueda aprender, en este caso se necesitaron 21,144,843 patrones.



a) b)

Figura 3. 71: Gráficas de las capas reentrenadas de Xception con respecto a su precisión y el número de parámetros entrenados. Se realizaron 14 reentrenamientos.

2. Análisis de rendimiento del sistema propuesto.

Tabla 3. 27: Tabla comparativa de los mejores modelos reentrenados de acuerdo con su mejor precisión.

Nombre	Parámetros entrenables	Mejor precisión (%)	Tamaño de modelo	Tamaño de imagen de entrada	Velocidad de operación	
					V&J	MediaPipe
RNC-PP (nueva versión)	340,835	95.77	3.98 MB	64x64	21	22
Mobilenet V2 Fine-Tuning 18	2,549,011	96.75	10.30 MB	64x64	17	16
VGG16 Fine-Tuning 7	14,590,019	97.77	56.8 MB	64x64	15	16
Resnet 50 Fine-Tuning 7	24,053,699	97.60	92.5 MB	64x64	14	13
Inceptionv3 Fine-Tuning 229	12,947,395	96	84.5 MB	75x75	13	12
Xception Fine-Tuning 26	21,144,843	97.13	81.9 MB	64x64	16	12

De la Tabla 3.27, el sistema propuesto puede operar 22 fotogramas por segundo utilizando el detector de rostros Mediapipe y 21 fotogramas por segundo utilizando el algoritmo del detector de rostros Viola & Jones. Esta rápida velocidad de operación es muy importante para la implementación en tiempo real, aunque la precisión de detección es ligeramente inferior, por ejemplo, un 0,03 % a InceptionV3 y un 2 % inferior a VGG16. En el espacio de memoria requerido para implementar en el dispositivo móvil, el sistema propuesto solo requiere 4Mbytes, que es aproximadamente 2,6 veces más pequeño que MobileNet-V2 y 23 veces más pequeño que ResNet-50V2. Esta especificación favorable del sistema propuesto hace posible una operación en tiempo real en los dispositivos móviles con espacio de memoria y poder de cómputo limitados.

3.5 Implementación del algoritmo detección de distracción y somnolencia en los teléfonos inteligentes con sistema operativo Android

Una característica especial de esta investigación es la implementación del protocolo general de detección de distracción y somnolencia en dispositivos móviles, tanto con sistema operativo Android[65] así como en tarjetas de desarrollo dedicadas; con esto demostramos que el sistema que proponemos es tan amigable que puede correrse en casi cualquier artefacto digital, así como también podremos comprobar que los análisis, las configuraciones y los métodos anteriores propuestos, son óptimos para trabajar en cualquier condición.

En este apartado en particular veremos los procedimientos que se llevaron a cabo para la implementación el sistema general en los teléfonos inteligentes con sistema operativo Android, donde una característica puntual, es que aquí tendremos todos los recursos limitados (memoria, energía, capacidad de procesamiento, sensores embebidos ,etc.)

3.5.1 Puntos importantes para tomar en cuenta

La implementación universal no existe y por ello, debemos conocer muy bien las limitantes que se tiene cuando se está desarrollando un proyecto. Hablando específicamente del mundo de la programación y sus derivados no es diferente, debido a que cada sistema operativo, cada lenguaje de programación y cada dispositivo es diferente, con ventajas y desventajas particulares o generales, por ellos necesitamos saber muy bien hasta donde se quiere llegar en cada uno de estos.

Si bien la implementación universal no existe, lo que si podemos generalizar es nuestra idea principal debido a que si bien, el proceso y la estructura interna será diferente, los resultados tienen que ser los mismo o en su defecto lo más parecido posible a lo esperado.

Entrando más en materia, anteriormente se ha estado trabajando y codificando todos los procesos, así como las implementaciones físicas como internas, utilizando un sólo lenguaje de programación el cual es Python; si bien, este lenguaje es sumamente amigable, y su estructura no es nada compleja en comparación a otros, Python tiene un enorme problema, y es que su performance es interpretado[66], como resultado su nivel de procesamiento es muy lento, además de que es sumamente pesado, por ende, su implementación actual de Python puro en dispositivos móviles (teléfonos inteligentes) no es posible.

Tomando lo anterior en cuenta, el lenguaje de programación que debemos utilizar como base el SO Android, es Java, que a diferencia de Python, este es sumamente liviano y genera un procesamiento superior, en otras palabras, es más rápido que Python.

Pero no todo es bueno, existen diversos puntos negativos y es que a diferencia de Python en donde no es necesario que sepamos todos los datos en cada proceso al momento de ejecutar ya se un análisis de imágenes o de que ejecutemos el modelo de RNC, en Java si es obligatorio que sepamos cada parámetro utilizado, desde la configuración de los bits de una imagen, hasta la normalización de lotes o contenedores de la RNC, por lo tanto hemos enlistado todos los puntos que son obligatorio saber y que se diferencian cuando se utiliza Python:

1. Librerías utilizadas: Esta demás decir que Python y Java no es lo mismo, pero comparten muchas características, así que es importante determinar que todas las librerías utilizadas principalmente en Python se encuentren desarrolladas en Java, o en todo caso bibliotecas con las mismas características, por eso sabemos que:
   - ***Procesamiento de imágenes***: La biblioteca enfocada al procesamiento de imágenes es la misma en ambos lenguajes de programación[67], con algunas diferencias, esta se ven en su documentación, pero el desarrollo es el mismo, esto nos facilita el trabajo de buscar una librería en paralelo que realice las mismas funciones tanto en Java como en Python.
   - ***Aprendizaje profundo***: La biblioteca enfocada al análisis y creación de aprendizaje automático así como su autor es el mismo autor en ambos lenguajes[58], pero su codificación es diferente.
2. Con referente a la implementación de la biblioteca de procesamiento de imágenes:
   - ***Búfer de bytes***: En Java no es tan fácil crear un proceso de análisis de imágenes en comparación con Python, debido a que aquí debemos hacer uso de contenedores de memoria los cuales alojan 4 bytes por cada píxel y por cada canal, por esta razón debemos saber exactamente cuántos pixeles se tienen en la imagen de entrada, así como en la imagen fragmentada la cual será clasificada por la RNC-PP posteriormente, además que debemos determinar que la cantidad de canales de entrada sea del mismo tamaño que la de la red.
   - ***Normalización a mapa de bytes***: La normalización de las imágenes en Java no es sólo dividir el valor de cada píxel por el rango que se desea limitar, si no que aquí, el proceso se hace a nivel de bit por píxel tomando en cuenta la dimensión del fotograma (número de canales).
   - ***Imagen de entrada***: Cuando utilizamos Python, la librería de procesamiento de imágenes, tiene una entrada normal de un fotograma con los canales BGR, pero con Java, este proceso es inverso, la imagen de entrada tienen los canales RGB, por lo tanto tenemos dos opciones al momento de implementar el sistema de monitoreo, la primera y más complicada, es que volvamos a configurar la base de datos a una imagen RGB y volver a hacer los procesos de entrenamiento de la RNC-PP tomando este procesamiento,

o bien, preprocesamos la imagen de entrada y la convertimos a una imagen BGR, este es el camino más sencillo, por ende, escogemos la permuta de canales.

- ***Redimensionamiento***: Cuando trabajamos en Java, debemos saber con qué datos o procesos estamos trabajando, uno de estos es el proceso de redimensionar las imágenes, debido a que este lenguaje de programación es muy susceptible a los cambios de dimensiones de las imágenes, por lo tanto es recomendable utilizar siempre las interpolaciones de reducción y ampliación específicos para cada acción, recordemos que nuestro principal problema, es la velocidad de monitoreo, escoger mal un algoritmo y que esto haga que nuestro sistema se alenté, será crucial al momento de la implementación del análisis en tiempo real.

3. Con referente a la implementación de la biblioteca de aprendizaje profundo:

- ***Modelos de RN***: Si bien, estamos utilizando la misma biblioteca que en Python, en la versión para Java, existen limitantes las cuales reducen el uso de dicha librería, una de las más importante, es que no todos los modelos de RN son compatibles con Java, es decir, que no todos los modelos de aprendizaje profundo como lo son: arquitecturas estándar, configuraciones específicas, modelos de cierto tamaño, entre otros no se podrán implementar en Android.
- ***Compresión del modelo entrenado***: El modelo utilizado en nuestro caso, de RNC, pasa por un proceso de compresión[68], lo cual hace que nuestra nueva RNC tenga un porcentaje menor de precisión, ya que es el costo por reducir el peso en disco, la ventaja de esto es que la nueva red es tan ligera que no tiene problema de correrse en dispositivos que tengan una memoria limitada.

  Previendo esta situación, la configuración que propusimos de RNC-PP no tuvo una gran pérdida de precisión por su arquitectura y su proceso de creación.
- ***Preprocesamiento***: Como se ha comentado a lo largo de todo este módulo, saber específicamente que hace cada proceso de nuestro sistema es crucial, esto incluye saber cómo funcionan los preprocesamientos que se hacen a las imágenes antes de aplicarle algún modelo de clasificación, por ende, la utilización de modelos creados a partir de Fine-tuning no son actualmente posibles su implementación en Java, ya que si bien podemos comprimir los modelos creados, como tiene un procesamiento específico no dicho por la documentación de TensorFlow, nos limitamos a utilizar sólo nuestro modelo propuesto.

4. En forma general, debemos recordar en que estamos trabajando en dispositivos con memoria limitada y con procesamiento limitado, por ende, se debe tener en cuenta que mientras mejor manejemos las variables declaradas, los bucles creados, la utilización de los mejores optimizadores y demás, mejoraremos el rendimiento y la velocidad de reconocimiento y de clasificación.
5. Finalmente, hay que tomar un punto en consideración, a partir de este momento, haremos uso de un dispositivo móvil, este es el Samsung S20 FE, el cual será nuestro dispositivo de prueba, en donde correremos los tests en las diversas apps generadas, además que, la mayoría de los resultados descritos a continuación, son con referencia a este gadget, posteriormente, el algoritmo final se implementó en diversos teléfonos móviles.
6.

3.5.2 Primeros parámetros para la implementación en teléfonos móviles

Como se ha comentado a lo largo de toda esta investigación, el algoritmo general se implementará en todos los dispositivos, con sus respectivas adecuaciones o mejoras, por ello, el desarrollo en teléfonos móviles quedará de la siguiente manera:

- ***Obtención de video en tiempo real:*** En el caso de los teléfonos móviles, la obtención de imágenes o videos en tiempo real se puede realizar de dos métodos diferentes, el primero sería utilizando las cámaras internas del dispositivo digital; por ende, para poder instalar la aplicación en el dispositivo Android es necesario que se tengan incorporadas tanto la cámara frontal (Figura 3.72 a)como la trasera (Figura 3.72 b), esto debido a que el sistema que se codificó en la app puede monitorear al conductor haciendo uso de cualquiera de estos sensores; la segunda es la utilización del puerto OTG de los smartphones, en el cual podemos instalar un tercer sensor óptico (Figura 3.72 c), si bien, esta podría ser una opción interesante, la app hasta el momento ha omitido esta opción.



Figura 3. 72:Diferentes sensores ópticos que se pueden o prima utilizar en la app SOMN_IA

- ***Separación del video de entrada a fotogramas:*** El procedimiento de la fragmentación del video en imágenes es en teoría el mismo que se utiliza cuando lo implementamos en Python, lo que cambia es lo siguiente:
  - ***Orden de canales:*** El orden de los canales de la imagen digital que se maneja en Java es RGB.



Figura 3. 73: Vista del fotograma en los diferentes lenguajes de programación

Si bien, como observamos en la Figura 3.73, la imagen mostrada por la interfaz gráfica de cada uno de los lenguajes de programación tienen las mismas características, se debe tener muy en cuenta, que internamente, el orden es diferente y eso se debe tomar muy en cuenta para los procesos posteriores.

3.5.3 Implementación del detector de rostros en los teléfonos móviles

El proceso siguiente, es la implementación del algoritmo de reconocimiento facial; en este caso haremos uso del propuesto por V&J, debido a su gran facilidad de implementación y a que no requiere parámetros y condiciones específicas de codificación, a comparación del respaldado por Google, que debe tener parámetros muy específicos, así como su desarrollo debe ser en SOs Linux, ya que no existen APIS estables en Windows, que es en donde estamos creando la app.

Al tener ya escogido el detector facial, procedemos a codificarlo, teniendo en cuenta todos los parámetros necesarios, así como las condiciones iniciales de las imágenes de entrada. Después de esto, nos dimos cuenta de un problema muy grande; resulta que al momento de implementar el algoritmo propuesto por V&J, la velocidad de monitoreo era demasiado lenta (en promedio a 4 fps), por tanto decidimos implementar la siguiente solución:

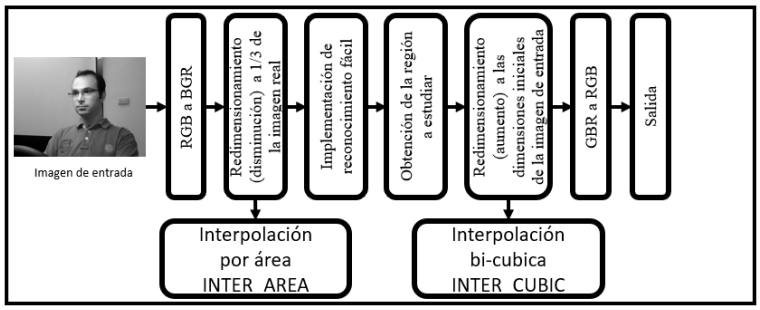


Figura 3. 74: Algoritmo de aceleración para reconocimiento facial con imagen de entrada y salida aceptable

- **Cambio de canales:** El primer proceso que debemos realizar es el cambio del orden de los canales de la imagen de entrada, esto si bien no afecta directamente al reconocimiento facial, es un parámetro muy importante cuando se implemente la RNC-PP, por ello aseguramos su preprocesamiento al inicio del monitoreo.

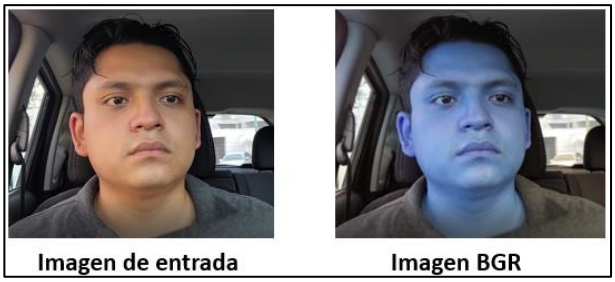


Figura 3. 75: Permuta de en los canales RGB a BGR de la imagen de entrada

- ***Redimensionamiento de la imagen de entrada***: Este proceso es el más importante de la aplicación del sistema en dispositivos móviles; originalmente la imagen de entrada tiene una dimensión de 864×1920 pixeles, lo que ocupa toda la pantalla del teléfono móvil, esto es importante, debido a que la calidad de imagen y la dimensión misma representan un coste computacional muy elevado para el gadget, por esta razón, el algoritmo de reconocimiento facial es tan lento, ya que debe estar analizando una región muy grande a nivel de información, dando como resultado su muy lento monitoreo, como se observa en la Figura 3.76.



Imagen de entrada completa y con una velocidad de reconocimiento promedio de 5 fps.

Figura 3. 76: Detección de rostros implementado en el teléfono móvil, con una velocidad de reconocimiento lento

Por ello, después de diversas pruebas, nos dimos cuenta de que al reducir la cantidad de información que se analiza, aumentamos considerablemente la velocidad de monitoreo, como resultado se decidió minimizar a la imagen de entrada a un tercio de sus dimensiones originales (Figura 3.77 b), esto hizo que la velocidad de reconocimiento aumentara casi tres veces en comparación a la primera prueba (Figura 3.77 a), para esto, se utilizó el redimensionamiento INTER_AREA o interpolación por área, esto gracias a que sus propiedades de disminución ayudan a disminuir el coste computacional y a acelerar la tasa de muestreo (en el inciso 3.2.5 Análisis de resultados consecutivos- ***Redimensionamiento de la imagen fragmentada*** se explica porque esta interpolación es específica para este tipo de tareas), en la Tabla 3.28 vemos un comparativa de diversos algoritmos y sus velocidad de muestreo.



Figura 3. 77: Implementación de procesamiento de disminución de la imagen de entrada

Otro punto a tomar en cuenta es la selección el factor de reducción de las dimensiones a un tercio de la imagen de entrada, esto fue debido a que se realizaron diversas pruebas dando como resultado que la máxima velocidad de reconocimiento se alcazaba con este valor, una reducción menor no aumentaba la velocidad pero si incrementaba la pérdida de información, como se muestra en Tabla 3.29

Tabla 3. 28: Tabla de velocidad utilizando diversos preprocesamientos de redimensionamiento de imágenes.

Algoritmo de redimensionamiento de imágenes.	Velocidad de muestreo (FPS)
Interpolación por área (INTER_AREA) [40]	13
Interpolación bi-cubica (NTER_CUBIC) [41]	8
Interpolación Lineal (INTER_LINEAR) [41]	10
Interpolación por el vecino más cercano (INTER_NEAR) [41]	11

Tabla 3. 29: En esta tabla representamos el nivel de perdida de información mientras más se reduce la imagen utilizando el redimensionamiento INTE_AREA

Factor de reducción de la imagen de entrada.	**Tamaño resúltate (Pixeles)**	**Velocidad media de muestreo (FPS)**	**Porcentaje de perdida de información**
1/1	864×1920	4	0%
1/2	432×960	9	50%
1/3	288×640	13	66%
1/4	216×480	13	75%
1/5	173×384	13	80%
1/6	144×320	13	83%

- ***Implementación del reconocimiento facial:*** Teniendo ya resuelto el problema del monitoreo lento, el siguiente paso es implementar el algoritmo de reconocimiento facial, este paso si bien vimos ejemplos en el punto anterior ya implementando este paso, esto fue sólo la forma de determinar que el sistema no era viable en principio, ya que la toma de video y la fragmentación por si solos no se tenía ningún problema de reducción de velocidad .

  Teniendo en cuenta lo anterior, procedemos a la implementación del detector de rostros propuesto por V&J, en este apartado se tomaron en consideración casi los mismos parámetros que se han ido trabajando desde el primer sistema propuesto, por lo tanto:

  - ***Imagen de entrada:*** Al igual que en el punto ***3.2.3 Detección de la región a analizar - Imagen de entrada***, la imagen de entrada al momento de aplicar el algoritmo entra con una configuración RGB, pero nosotros convertimos esta imagen a escala de grises para su mejor análisis y la disminución de coste computacional.
  - ***Factor de escalamiento:*** 1.1
  - ***Mínimo de vecinos:*** 5
  - ***Máximo y mínimo tamaño del cuadro delimitador:*** El tamaño máximo del cuadro delimitador que proponemos es el mismo que la primera versión, de

250×250px; aquí lo que difiere es la dimensión mínima, ya que no emplearemos la primera RNC-PP, si no que en esta ocasión, haremos uso de la nueva RNC-PP propuesta (nueva versión) y como sabemos, el tamaño que requiere este modelo es de 64×64px por cada imagen de entrada, por ende, este será el valor de nuestro cuadro delimitador.

- ***Redimensionamiento (aumento) a las dimensiones iniciales de la imagen de entrada:*** Como observamos en la Figura 3.77, la imagen en donde hacemos todo el procesamiento de reconocimiento facial, así como posteriormente, el protocolo de detección de distracción y somnolencia, es el fotograma reducido a un tercio de su valor inicial, esto por obvias razones no se puede mostrar en la interfaz gráfica, por ello decidimos utilizar un algoritmo que nos ayude a redimensionar la imagen XX (Figuras 3.78), en este caso hacemos uso de un la Interpolación bi-cubica o INTER_CUBIC [41], el cual esta optimizado y especialmente diseñado para aumentar el tamaño de las imágenes:

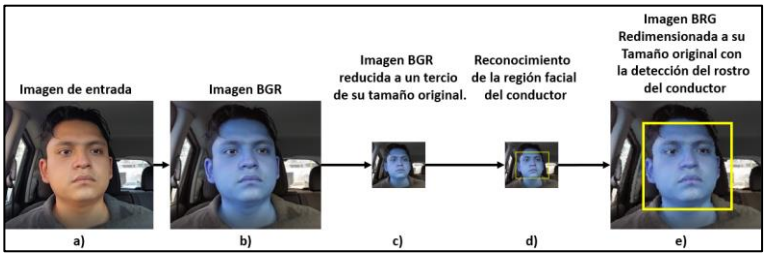


Figura 3. 78: Proceso de adecuación de la imagen analizada para su muestra en la GUI

Como observamos en la Figura 3.78 e, ya obtenemos la imagen que si es posible mostrar en la interfaz gráfica de la app, igual podemos observar un poco de ruido en la imagen de salida, esto como resultado de la perdida de información que se tuvo en los dos procesos de redimensionamiento previos.

- **Cambio de canales:** Para finalizar el proceso previo antes de la implementación del sistema detector de somnolencia y distracción y la aplicación del clasificador (RNC-PP), el único paso faltante es cambiar la posición de los canales que desde un principio se hizo para que en la interfaz gráfica, el usuario pueda ver de forma correcta el monitoreo en tiempo real que se está ejecutado, por ello cambiamos el orden de los canales de la Figura 3.78 e de BGR a una imagen final RGB.

En la Figura 3.79 podemos observar cómo nuestra configuración propuesta aumentó considerablemente la velocidad de monitoreo y reconocimiento facial.



Figura 3. 79:Comparación entre la primera velocidad de reconocimiento facial y el reconocimiento empleando el algoritmo de aceleración de monitoreo propuesto

3.5.4 Implementación del sistema detector de somnolencia y distracción en los conductores

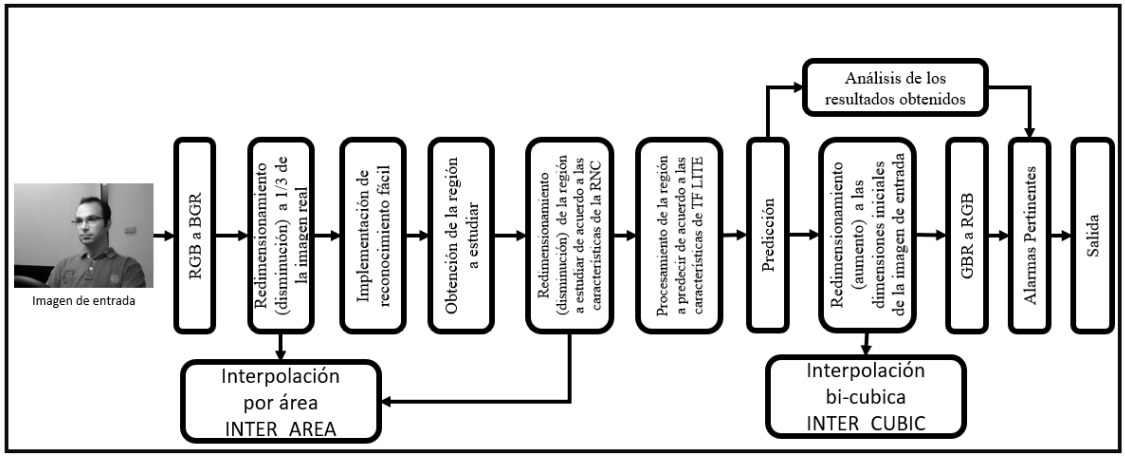


Figura 3. 80: Algoritmo general de reconocimiento de somnolencia y distracción implementando en teléfonos móviles

La implementación del sistema general para la detección de somnolencia y distracción consta de lo siguiente:

- ***Detección del área a analizar:*** Este punto hace referencia al proceso que se hizo en el punto 3.5.4, el cual contiene el procedimiento de la obtención de la imagen de entrada, el cambio de canal al que necesitamos (de RGB a BGR), la parte fundamental, que es el redimensionamiento de la imagen de entrada y finalmente la aplicación del algoritmo de reconocimiento facial).
- ***Redimensionamiento (disminución) de la región a estudiar de acuerdo con las características de la RNC:*** Al igual que el primero y segundo método de detección de somnolencia y distracción, teniendo ya ubicada la región a analizar (que en este caso es el rostro del conductor), extraemos el fragmento del rostro del conductor y posteriormente realizamos una reducción de tamaño de la imagen rf_n, utilizando la técnica de interpolación por área como se hace en cada proceso de disminución de dimensiones.

- ***Procesamiento de la región a predecir de acuerdo con las características de TF LITE:*** En este paso aplicaremos la RNC-PP propuesta (nueva versión), la cual contiene la clasificación de los tres estados que se han comentado hasta ahora, pero antes de este proceso, debemos tener en cuenta diversos parámetros y realizar diferentes procesamientos y procedimientos que anteriormente no eran necesarios:
  1. ***Conversión del modelo de H5 a modelo TFLite:*** Ese paso es fundamental para poder correr el modelo de red neuronal que creamos a partir de Python en Android; el proceso es muy simple, en un programa en Python seleccionamos el modelo en formato (H5) que queramos convertir a formato tflite; corremos el programa y listo, la conversión se ha completado.
  2. ***Normalización de los parámetros:*** A diferencia de Python, en donde sólo teníamos que igualar el tamaño de la imagen al tamaño requerido por la RNC, así como también tener normalizados los valores de los pixeles del fotograma y con ello ya podíamos obtener la clasificación, en Java es muy diferente, ya que aquí no sólo es cuidar estos aspectos si no que:
     - ***I. Conversión de la matriz (imagen del rostro fragmentada) a mapa de bits***: El primer proceso es convertir la matriz de entrada a un mapa de 32 bits con un valor de α según sea la opacidad de la imagen de entrada.
     - ***II. Normalización del mapa de bits a un búfer de bytes:*** En este proceso debemos tomar muy en cuenta la configuración del entrenamiento de la imagen de entrada, debido a que aquí enviaremos y normalizaremos los datos del mapa de bits a un contenedor o búfer para su clasificación, por esta razón, implementar las configuraciones de Mobilenet, RESNET, etc., no es son posibles, ya que tienen preprocesamientos internos específicos.

       Confirmamos que no existe una configuración especial de nuestra RNC-PP y proseguimos a normalizar cada canal empezando por el rojo, después por el azul y finalmente el verde.

       Posterior a la normalización del mapa de bits, estos resultados se cargarán directamente a un búfer que se creó específicamente para almacenar los valores resultantes de este procesamiento, el tamaño en este caso de este búfer es de 49152 bytes.
     - ***III. Características del búfer de bytes:*** Un búfer se puede entender como un espacio de memoria temporal en la memoria física, este espacio debe ser del mismo tamaño (en bytes) que los datos analizados por la RNC-PP, que en este caso es de 49152 bytes :

$$\boldsymbol{byteBuffer} = (\boldsymbol{4} * \boldsymbol{BT} * \boldsymbol{width} * \boldsymbol{height} * \boldsymbol{channel}) \quad (11)$$

Donde:

- El 4 es una constante dada por la librería.
- BT: es el valor de lotes que deseemos, en este caso lo igualaremos a 1.
- Width: Es el ancho de la imagen de entrada.
- Height: Es el largo de la imagen de entrada:
- Channel: Es el número de canales de la imagen.

Como resultado de esta opción no da que el tamaño del búfer de bytes es de un tamaño de 49152 bytes.

3. ***Predicción***: Teniendo ya internamente guardados los datos procesados en el búfer, el siguiente paso es aplicar la RNC-PP y obtener los resultados de la predicción dada de la utilización del modelo, esto es igual a los procesos anteriores.
4. ***Análisis de resultados:*** Los análisis de resultados tienen la misma configuración de aplicación que en ***3.3.3 Análisis de resultados***, lo que nos lleva a obviar este proceso pero sin antes puntualizar, que aunque se está utilizando un modelo que clasifica patrones de distracción así como también, se utiliza un algoritmo que como remarcamos, no tiene gran estabilidad de reconocimiento facial cuando se rota el rostro, ambos tienen la tarea de determinar si se está prestando o no atención al camino, por lo tanto, no hay problema en el uso de dicha RNC-PP, ya que la base de datos esta tan bien hecha, que los únicos datos que se detectará es cuando se tiene patrones de una conducción correcta o de somnolencia.

   Un punto importante antes de pasar al último proceso, si bien obviamos los procesos de análisis de datos, debido a que sería algo repetitivo volverlos a escribir, lo que si debemos dejar muy en claro que en este caso, los valores de ***thf*** y ***th***, se verán modificados, esto debido a que la velocidad de reconocimiento es diferente al primer dispositivo de prueba, por ellos, los umbrales han quedado igualados a lo siguiente:

   - th = 23
   - thf = 30
5. ***Presentación de resultados en interfaz gráfica:*** En este último proceso, haremos uso del ***Redimensionamiento (aumento) a las dimensiones iniciales de la imagen de entrada*** así como ***del cambio de canales final***, que en el punto 3.5.4 se explicó a fondo, además que también implementaremos un último subproceso, el cual es el que hemos denominado como ***"activación de alarmas pertinentes":***

   - ***Activación de alarmas pertinentes:*** En este último subproceso haremos uso de la configuración de la ***alarma informativa*** que se ha

estado trabajando con anterioridad, por ello obviamos esta parte (si desea más información en el punto ***3.2.6 Activación de alarmas pertinentes***, se encuentra explicado a más profundidad el funcionamiento de esta activación o desactivación de esta alarma.)

3.5.5 Resultados obtenidos del subcapítulo

1. Análisis de rendimiento del sistema propuesto.

En la Tabla 3.30 podemos observar las diferentes velocidades finales de monitoreo en tiempo real, utilizando como base una sola cámara, por lo tanto, esto de puede interpolar hacia las demás cámaras que se pueden utilizar en el sistema.

Tabla 3. 30: Tabla de rendimiento en los diversos dispositivos de prueba

Dispositivo	***FPS Utilizando reconocimiento de rostro V&J***	***FPS Utilizando reconocimiento de rostro MediaPipe Face***	***Video cámara utilizada***
3070TI GPU, 8GB Memory, 6144 cores, 1.770MHz.	21	22	Cámara Web HD C505
Huawei D16 Laptop, AMD Ryzen 5 4600H	20	21	Cámara Web HD C505
Nvidia Jetson AGX Xavier	10.67	13.03	Cámara Web HD C505
Nvidia Jetson Nano 4GB	4	7.69	Cámara Web HD C505
Motorola One, Cámara frontal 8 Mp, ROM 64 GB. RAM 4 GB, Pantalla resolución 1440 x 720, Android 8.0 Oreo	21	-	Cámara frontal interna 600 x 800 x 3
Samsung s20 fe, Cámara frontal 32 MP, f/2.0, wide, HDR, 2160p, 1080p, ROM 128GB, RAM 6GB, Resolución 1080 x 2400 pixels, Android 12 Snow Cone	14	-	Cámara frontal interna 864 x 1920 x 3

2. **Ejemplos operacionales:** En la Tabla 3.31 dejamos el enlace de las diversas pruebas implementando el algoritmo de detección de somnolencia y distracción en diversos smartphones con sistema operativo Android.

Tabla 3. 31: Enlace de los videos de funcionamiento

Enlace	Descripción
https://youtu.be/3GeBGMi_pCw	Prueba de rendimiento en el teléfono inteligente Samsung A20 Fe

3.6 Automatización de umbral para la detección de Somnolencia y Distracción

Al momento de querer implementar el algoritmo de detección de somnolencia y distracción en diversos dispositivos, tales como Workstation, Laptops, PC, SOMN_IA, tarjetas de desarrollo enfocadas, así como finalmente en dispositivos móviles con SO Android y adicionalmente al hacer pruebas en entornos de conducción controladas y no controladas, en donde se puso a prueba el rendimiento en condiciones reales de manejo los diversos dispositivos antes mencionados se detectó un gran inconveniente.

El problema se enfoca en la implementación en masa del sistema, en donde las variaciones de las capacidades de los dispositivos con limitaciones de hardware así como los de laboratorio, juegan un papel vital, estas características tales como la velocidad de procesamiento y la resolución de la imagen de entrada causan una gran variación en las velocidades de muestreo o de monitoreo en tiempo real, trayendo consigo que los umbrales propuestos *"thf"* y *"th"*, se estén constantemente modificando de forma manual.

Como observamos en la Tabla 3.30, de los dispositivos en los cuales se ha implementado el algoritmo de detección de somnolencia y distracción, las velocidades de monitoreo así como las calidades de imágenes son muy inestables y no siguen un patrón; por ende, hacer un código para cada terminal no es factible; por esta razón, se ha desarrollado el siguiente algoritmo universal, en donde automatizamos los umbrales *"thf"* y *"th*".

3.6.1 Algoritmo Universal

El sistema universal tiene como idea principal, eliminar los umbrales específicos, los cuales se modifican de forma manual cada vez que existe una mejora o se migra a dispositivos diferentes (*thf* y *th* mencionados en el apartado anterior), por tal motivo hacemos uso de una nueva variable a la cual hemos denominado drowsiness detection threshold ***"DDT"*** (12), que es el producto entre el umbral de somnolencia (*th*) y la velocidad el cual es sistema se está corriendo (fps), a lo cual observamos que ***"DDT"*** es directamente proporcional a la velocidad de muestreo del dispositivo en donde se corre el sistema de detección de somnolencia y distracción.

$$\boldsymbol{DDT = th * fps} \quad (12)$$

"DDT" es sólo utilizado para diferenciar entre somnolencia o micro sueños y parpadeos normales y se debe tomar en consideración un punto muy importante al momento de elegir "*th*". "*th*" si bien es un umbral especifico, se debe tomar como unidades de tiempo (s), esto quiere decir que "*th*" deja de ser utilizado como un umbral de fotogramas y a partir de este momento será utilizado como el umbral en tiempo en que los ojos se mantienen cerrados, por lo tanto, como se comentó anteriormente (***3.3.3 Análisis de resultados consecutivos - Algoritmo propuesto para el reconocimiento de somnolencia en el conductor***) se recomienda que "*th*" sea mayor a 0.750, el cual, es el lapso máximo que se puede considerar un parpadeo normal en un conductor adulto.

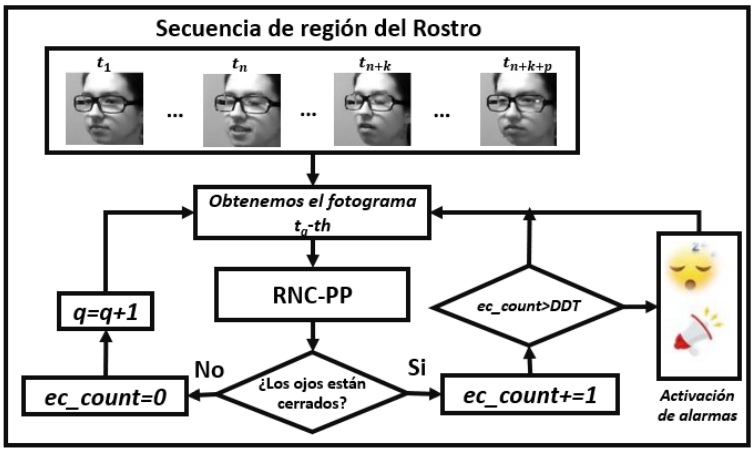


Figura 3. 81: Algoritmo del reconocimiento de somnolencia implementado la automatización de umbral auto generativa

Para poder diferenciar entre distracción (o no detección de rostro) y atención al momento de conducir, utilizamos otra nueva variable, la cual la hemos denominado distraction detection threshold ***"DD"*** (13), que es el producto entre el umbral de distracción (*thf*) y la velocidad el cual es sistema se está corriendo (fps), al igual que ***DDT***, ***DD*** es directamente proporcional a la velocidad de muestreo del dispositivo.

$$\boldsymbol{DD = thf * fps} \quad (13)$$

DD (13) es muy parecida a ***DDT*** (12), la diferencia radica a que *thf* (que igual que *th*, deja de ser utilizado como un umbral de fotogramas y a partir de este momento será utilizado como el umbral en tiempo) es la adición de *th* + *N*, donde *N* es un valor constante en tiempo (s), el cual es criterio del desarrollador, se recomienda que la suma *thf* no supere los 2s (como se comentó en ***3.3.3 Análisis de resultados consecutivos - Algoritmo para el reconocimiento de distracción*** ; en nuestro caso N lo igualamos a 0.5s, dándonos buenos resultados al momento de implementarlo en el sistema.

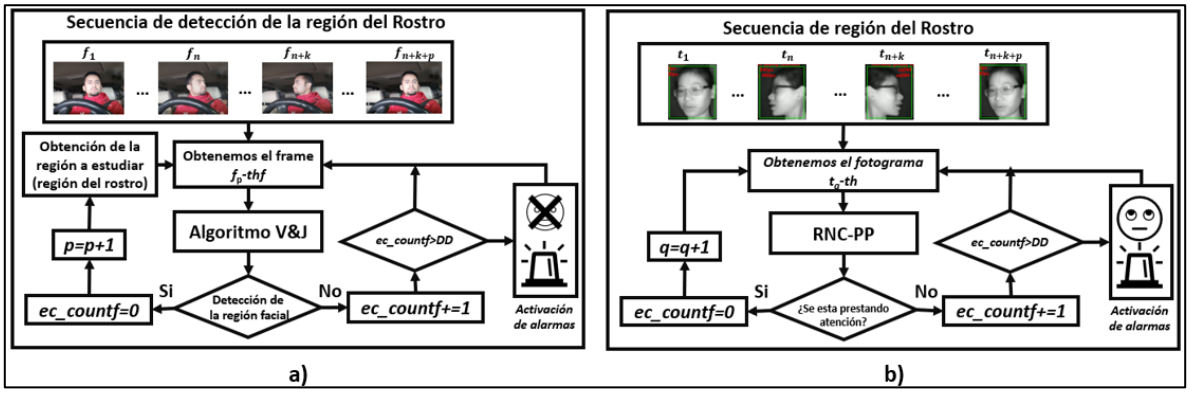


Figura 3. 82: Algoritmos para el reconocimiento de distracción implementado la automatización de umbral auto generativa

En la Figura 3. 83 a y b podemos observar cómo quedaría el algoritmo general de detección de somnolencia y distracción, implementando las automatizaciones de umbrales auto generativas (*DDT* y *DD*).

3.7 Corrección de predicción en tiempo real

Cuando se utiliza cualquier algoritmo de aprendizaje automático, incluido los clasificadores por aprendizaje profundo, tienden a no ser siempre exactos, ya que por ello existe en todo momento un porcentaje de error, esto afecta cuando se quiere implementar en sistemas en tiempo real, dando como resultado que en dispositivos los cuales se exigen la mayor precisión muchas veces fallen, por el error de un sólo fotograma (hablando específicamente de sistemas enfocados en el monitoreo) y esto puede ser la diferencia entre la vida y la muerte.

Como se mencionó anteriormente, el sistema proporciona aproximadamente una tasa de error del 4%, confundiendo, por ejemplo, el estado de "ojos cerrados" con el estado de "ojos abiertos", debido a diversas condiciones en los fotogramas, como variación de iluminación, posicionamiento del sensor óptico, etc. En la Figura 3.83 se muestra un ejemplo claro de este tipo de error debido a que en la Figura 3.83 a vemos claramente que el conductor tiene los ojos cerrados y podemos observar como ya ha superado el valor de ***DDT***, debido a que la alarma por sueño se han activado, en Figura 3.83 b se puede observar como la imagen a simple vista no ha cambiado, sigue siendo la misma al ojo humano pero la RNC-PP la clasificó por alguna razón como si el conductor tuviera patrones de una conducción normal, con este error de predicción, ***ec_count*** se igualó a cero, desactivando naturalmente todas las alarmas que habían activado hasta ese momento, en el posterior cuadro (Figura 3.83 c) la clasificación siguió siendo correcta, pero ahora el contador ***ec_count*** volvió a incrementarse en una unidad partiendo desde cero, haciendo que se tenga un lapso de tiempo muerto crucial para el conductor, que si no llegase a despertar por sí sólo, las probabilidades de que exista un siniestro mortal, empieza a aumentar.

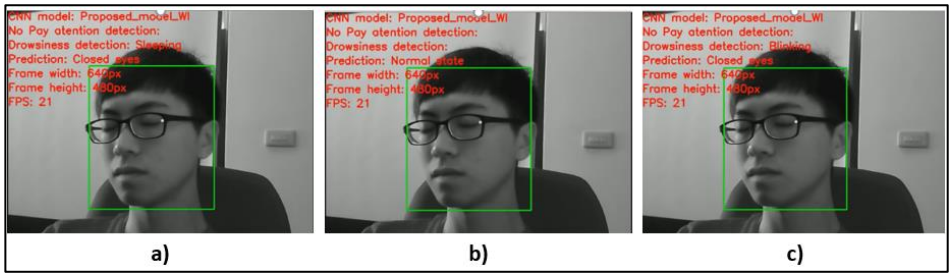


Figura 3. 84: Error de predicción en fotogramas secuenciales

Tomando como base el problema del error de predicción mostrado anteriormente, se introduce en la versión modificada del sistema una nueva variable denominada "Corrección de Error de Predicción" (PEC por sus siglas en inglés) dada por (14). La PEC se usa como un valor de umbral para corregir los errores aislados que ocurren durante el monitoreo en tiempo real fotograma por fotograma, como se muestra en la Figura 3.83.

$$PEC = \frac{fps}{4} \tag{14}$$

Donde:

- fps : es la velocidad de operación del dispositivo.

La PEC se utiliza para discriminar entre el parpadeo normal y el error aislado causado por la mala clasificación de la RNC-PP. De acuerdo con [11], la duración de un parpadeo normal está entre 290 y 750 milisegundos, por lo que menos de un cuarto de segundo (250 ms) indicado por [] puede considerarse como un error aislado, mientras que un número mayor de fotogramas consecutivos se es considerado como un cambio de estado normal

La Figura 3.84 muestra el diagrama de bloques de este proceso.

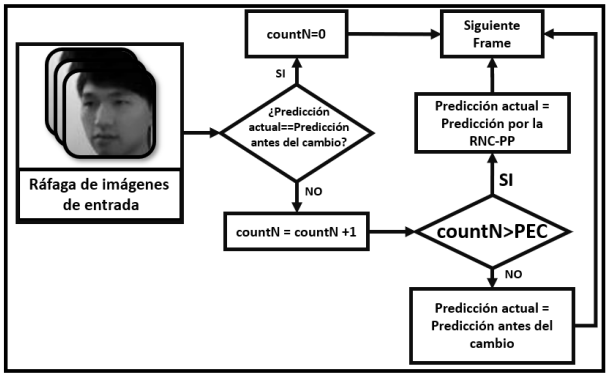


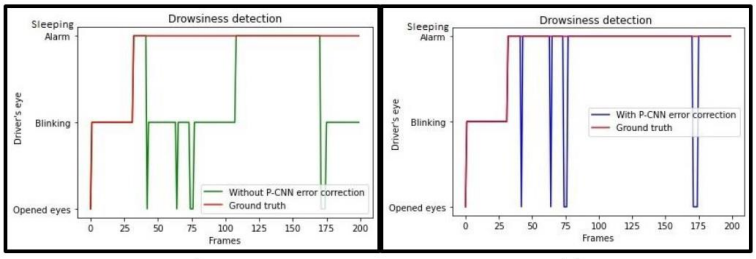
Figura 3. 85: Algoritmo corrección de error de predicción

1. Primera condición ***"Estado sin cambio"***: Si en el monitoreo, la RNC-PP no determina ningún cambio en el estudio de imagen por imagen, el sistema trabaja de manera normal y ***"countN"*** se iguala a cero sin pasar por la tercera condición.

2. Segunda condición ***"Estado de cambio normal"***: Si repentinamente existe un cambio de estado en el monitoreo de imagen por imagen la variable ***"countN"*** aumenta en una unidad cada vez que la RNC-PP haga una clasificación diferente al del estado previo del cambio imprevisto, si este cambio tiene una duración superior a PEC, entonces el cambio de estado es un proceso normal, por lo tanto el sistema modifica el valor del estado previo del cambio imprevisto, continuando así el monitoreo de forma normal.
3. Tercera condición ***"Corrección del error de predicción"***: Si existe un cambio repentino de estado en el monitoreo de imagen por imagen la variable ***"countN"*** aumenta en una unidad cada vez que la RNC-PP haga una clasificación diferente al del estado previo del cambio imprevisto, si este cambio tiene una duración inferior a PEC, entonces aquí existe un error al momento de clasificar, dando como resultado, que el sistema cambie automáticamente el valor de ***ec_count*** o ***ec_countf*** (según sea el caso) al valor previo antes del cambio de estado imprevisto y por consiguiente, las acciones anteriores a este error vuelvan a como estaban, evitando así, los tiempos muertos que ponen en riesgo la salud física del conductor, posteriormente el sistema continua así el monitoreo desde donde se quedó antes de esta mala predicción.

3.7.1 Resultados obtenidos del subcapítulo

Para obtener los resultados de la corrección del error de predicción realizamos 50 pruebas en donde ya se está aplicando de igual manera el proceso de automatización descrito en el punto **3.6 Automatización de umbral para la detección de Somnolencia y Distracción**, de los cuales a continuación ejemplificaremos el proceso que se tomó para poder obtener los resultados de la Tabla 3.27.



a) b)

Figura 3. 86: Comparación de rendimiento entre el sistema anterior (a) y la versión modificada (b) en la detección de somnolencia del conductor

Como se observa en la Fig. 3.85, un conductor comienza a sentirse somnoliento en el cuadro 0 y sigue durmiendo durante más de 200 cuadros, por lo tanto, sistema considera que los primeros fotogramas que no superan a DDT son de "parpadeo normal" y luego

de este punto, el usuario sobrepasa el umbral DDT, por cual el sistema automáticamente pasa de tener la etiqueta "***Blinking***" a "***Sleeping***".

Como se muestra en la Figura 3.85 a, en esta prueba no se está aplicando la **Corrección de predicción en tiempo real** por lo tanto, cada que ocurre el error de clasificación dada por la RNC-PP propuesta, se desactiva la alarma un mínimo de 3.3 s; tomando lo anterior en cuenta en este ejemplo el error es causado en la mala predicción de la RNC-PP ya que, de acuerdo con la Figura 3.85 a, el fotograma 46 se clasificó como "ojo cerrado" (cabe mencionar que están activadas las alarmas), en el siguiente frame (47) que en teoría se debería predecirse como "ojo cerrado" (de acuerdo con el Ground Truth) pero por algún factor la RNC-PP lo predijo como "ojo abierto" y finalmente en el frame 49 se predijo correctamente como "ojo cerrado", causando que la alarma se desactivara de forma automática, ya que se iguala el contador "ec_count" a cero; en consecuencia por este error, el sistema se hace inservible por poco más de 3,3s.

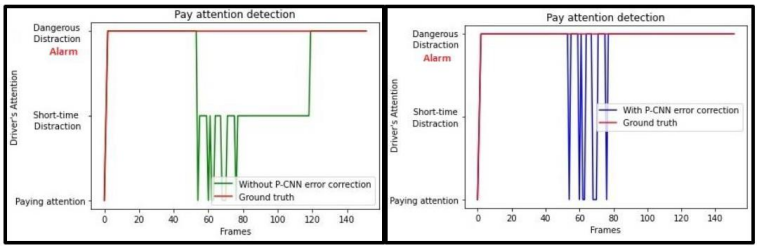


Figura 3. 87: Comparación de rendimiento entre el sistema anterior (a) y la versión modificada (b) en la detección de distracciones del conductor

Ahora bien, en la Figura 3.85 b, donde ya está aplicado la **Corrección de predicción en tiempo real** podemos observar que si bien, los errores propios de la predicción dada por la RNC-PP aún siguen apareciendo, la corrección redujo el tiempo de desactivación de las alarmas a un tiempo inferior de 0,24s. Vale la pena señalar que esta breve interrupción de la alarma no es perceptible para el oído humano.

De la misma manera, la Figura 3.86 se muestra una mejora con la utilización de la **Corrección de predicción en tiempo real** bajo situaciones de peligro real por distracción del conductor, eliminando errores de predicción aislados causados por la RNC-PP. En este caso, la distracción del conductor comienza en el cuadro 0 y se mantiene en el mismo estado durante al menos 200 cuadros. El sistema anterior interrumpía la alarma durante más de 3s bajo el estado de distracción continua del conductor como se muestra en la Figura 3.86 a, mientras que en la versión modificada, la interrupción de la alarma ocurre durante solo 0.24s como se muestra en la Figura 3.86 b, evitando un posible accidente.

Después de la realización de las diversas pruebas mencionadas anteriormente, se evaluaron las dos versiones necesarias para aplicar una comparativa, por lo tanto se hizo la comparativa entre el sistema aplicando la **Corrección de predicción en tiempo real**

y una versión sin su implementación. La métrica de evaluación denominada $\boldsymbol{AR_{\text{danger}}}$ se basa en el número de cuadros con alarma proporcionados por los sistemas de detección anteriormente mencionados, durante la situación peligrosa para el conductor, como "seguir durmiendo" y "seguir distrayendo". Los cuadros de video con situación peligrosa corresponden a Ground Truth (GT) que se muestran con líneas rojas en las Figuras 3.85 y 3.86.

La métrica de evaluación viene dada por (15).

$$\boldsymbol{AR_{danger}} = \frac{\mathbf{NF_{alarm}}}{\mathbf{GT_{danger}}} \tag{15}$$

donde GT_{danger} y NF_{alarm} son el número de fotogramas con alarma (Ground Truth) y el número de fotogramas con alarma proporcionados por el sistema de detección de somnolencia y distracción. La Tabla 3.32 muestra la comparación entre el sistema sin la implementación de la **Corrección de predicción en tiempo real** y la versión modificada propuesta. Vale la pena señalar que la precisión de detección que mostramos en la Tabla 3.32 es sin la implementación SAS, SAL Y SR debido a que solo se requiere obtener el porcentaje de mejoramiento de predicción al momento del monitoreo en tiempo real y la implementación del sistema. La precisión global de RNC-PP en ambos sistemas es la misma (95.77%).

Tabla 3. 32: Comparación de rendimiento entre el sistema anterior y la versión modificada propuesta

Patrón de detección	**Corrección de predicción en tiempo real**	
	Sin	con
Somnolencia	70%	96%
Distracción	78%	95%

3.8 Conclusiones del capítulo

A lo largo de toda esta síntesis, se explica a detalle el sistema que proponemos para la de detección de distracción y somnolencia en los conductores basado en una red neuronal tipo convolucional, donde su principal característica es que no sea tan compleja y por ende, que no sea tan profunda y con ello cumplir el objetivo principal, el cual es que SOMN_IA y sus derivados puedan operar en tiempo real en un dispositivo móvil o compacto con limitación de espacio de memoria y poder computacional, así como también, que contenga la característica de poder adaptarse a terminales que no posean estas limitaciones. Para ello, seleccionamos dos algoritmos de detección de rostros con la complejidad computacional más baja, que son el detector de rostros propuesto por

Viola-Jones y el detector de rostros MediaPipe y a su vez se propuso un clasificador de tres estados una RNC.

Después de varias evaluaciones, seleccionamos la arquitectura de RNC más apropiada desde los siguientes parámetros, que para nuestro punto de vista es la mejor opción:

- Precisión de detección
- Velocidad de operación
- Espacio de memoria requerido.

El sistema basado en RNC propuesto se evalúa utilizando el conjunto de datos NTHU-DDD, obteniendo un 95,77 % de precisión global (en la segunda y última versión). Los resultados experimentales mostraron que el sistema propuesto se comporta muy bien en las diferentes condiciones de iluminación y uso de accesorios externos o no. La diferencia entre la mejor precisión (96,67 %) con iluminación normal sin gafas y la peor (94,67 %) con iluminación IR con gafas es sólo el 2%.

El sistema propuesto se comparó con cinco RNC disponibles comercialmente, que son MobileNet-V2, VGG16, ResNet-50V2, InceptionV3 y Xception. Aunque la precisión de estas RNC's es ligeramente mejor que la propuesta, esta diferencia no es crítica, ya que en la operación de video se obtienen pequeñas diferencias de precisión en cada cuadro que pueden ser absorbidas en el rendimiento total. Además de eso, estas RNC's requieren un gran espacio de memoria para operar y la velocidad de operación disminuye un 23-40% en comparación con la propuesta. Por ejemplo, la velocidad de funcionamiento es de 16 fps en Xception frente a los 22 fps del sistema propuesto (Esto tomando muy en claro que solo se hicieron pruebas en la Workstation, debido a que su implementación en dispositivos móviles no es posible).

Por otra parte, al momento de poder generalizar el algoritmo general de SOMN_IA, nos vimos en la necesidad de mejorar el sistema de detección de somnolencia y distracción propuesto previamente por nosotros. En la versión modificada, propusimos dos puntos esenciales: la automatización de umbrales y la corrección de errores de predicción mediante secuencia de video.

Debido a la gran versatilidad de los dispositivos móviles, los umbrales constantes utilizados en los primeros pasos de esta investigación no son posibles unificarlos en todos las terminales en las cuales es posible su implementación (software). La automatización de umbrales tiene la propiedad de adaptar dos valores de umbral al dispositivo móvil utilizado para la prueba en turno. El primer umbral diferencia entre "Sleeping" y "Blimking" y el segundo umbral en donde la diferencia radica entre poder distinguir la "distracción peligrosa" y "distracción de corta duración" que ocurre con frecuencia durante la conducción normal.

Como casi todos los clasificadores basados en aprendizaje automático y por consiguiente en la RNC-PP propuesta proporciona algunos errores de predicción en la

fase de prueba, debido a las imperfecciones naturales de los entornos no controlados, como la variación de iluminación y la variación de distancias entre la cámara web y el controlador, etc. En los sistemas anteriores, estos errores provocan una inicialización de los contadores del sistema y la alarma se interrumpe durante una situación de peligro del conductor. La corrección de estos errores de predicción garantiza que la alarma suene constantemente mientras el conductor siga durmiendo o el conductor se distraiga durante largos periodos de tiempo anormales para evitar un posible accidente. La versión modificada propuesta proporciona un mejor rendimiento, desde el punto de vista de la alarma mientras continúa la situación de peligro del conductor.

El sistema con las modificaciones propuestas se evalúa en condiciones de tiempo real, mostrando un buen desempeño utilizando dispositivos móviles portátiles. La evaluación del sistema propuesto en una amplia gama de dispositivos móviles (además de los ya existentes) es por consiguiente, nuestro trabajo futuro.

3.9 Referencia del capítulo

[1] Aragonés-Ferrero J y Luján -Mora S, "Imagen digital", Universitat d'Alacant, 2002-2008.
[2] AGElectrocnica, SKU16755: "Camara Imx219-77ir De Visión Nocturna Por Infrarrojo De 8mp Para Jetson Nano", 2021.
[3] Anónimo, "Dispositivos y circuitos ópticos", PP 28-29, 2010.
[4] Anónimo, "Espectro visible", PP 1-4, 2013
[5] Anónimo, "Teoría de la visión nocturna", [en línea] http://www.sapiensman.com/tecnoficio/optica/tecnologia_de_vision_nocturna.php
[6] Anónimo, "Interfaz USB", 2012
[7] WIkikipedia, CSI (bus serie), [en línea] https://es.wikipedia.org/wiki/CSI_(bus_serie)
[8] Rosales-Mayor E, Rey-de-Castro-Mujica J, "Somnolencia: Qué es, qué la causa y cómo se mide", PP 1-3, 2010
[9] Gelfand JM, Douglas VC. Fatiga. In: Kasper D, Fauci A, Hauser S, Longo D, Jameson J, Loscalzo J. eds. Harrison. Principios de Medicina Interna, 19e. McGraw Hill; Capitulo 19, 2019.
[10] Logitech, "CÁMARA WEB HD C505", [en línea] https://www.logitech.com/es-mx/products/webcams/c505-hd-webcam.960-001363.html
[11] Amazon, "Cámara web1080p cámara de vídeo Full HD USB", [en línea] https://www.amazon.com.mx/TXG-C%C3%A1mara-Ajustable-web1080p-c%C3%A1mara/
[12] HUAWEI, "HUAWEI MateBook D 16 AMD R5 4600H 8GB+512GB", [en línea] https://consumer.huawei.com/mx/laptops/matebook-d-16-amd/buy/
[13] Estrada R, "¿Cómo funciona una cámara de video?". PREMIERE, 2021, [en línea] https://www.cinepremiere.com.mx/como-funciona-una-camara-de-video.html#:~:text=El%20proceso%20de%20captura%20de,del%20otro%20lado%20del%20prisma.
[14] Intel OpenVino, OpenCV, [en línea] https://opencv.org/

[15] P. Viola and M. Jones, "Robust real-time object detection", In Proc. 2nd Int. workshop on Statistical and Computation Theories of Vision – Modeling, Learning, Computing and Sampling, p. 25, 2001.

[16] Intel OpenVino, cv::CascadeClassifier Class Reference, [en línea] https://docs.opencv.org/4.x/d1/de5/classcv_1_1CascadeClassifier.html#aaf8181cb63968136476ec4204ffca498

[17] SGMA, "Inteligencia Artificial", PP 10-15, 2011

[18] Hinestroza-Ramírez D, "El Machine Learning A Través De Los Tiempos, Y Los Aportes A La Humanidad", 2018

[19] Hussain I, "Clustering In Machine Learning", 2011

[20] Stack Overflow contributors, "Aprendizaje machine-learning", PP 16-21, 2019

[21] Stack Overflow contributors, "Aprendizaje machine-learning", PP 21-25, 2019

[22] Stack Overflow contributors, "Aprendizaje machine-learning", PP 67-69, 2019

[23] Stack Overflow contributors, "Aprendizaje machine-learning", PP 61-63, 2019

[24] Alba-Centeno F, "Deep Learning", PP 3-5, 2019

[25] Alba-Centeno F, "Deep Learning", PP 5-20, 2019

[26] García-Sánchez E, "Introducción a las redes neuronales de convolución. Aplicación a la visión por ordenador", PP 17-25, 2019

[27] C. H. Weng, Y. H. Lai and S. H. Lai, "Driver drowsiness detection via a hierarchical temporal deep belief network", Proc Asian Conference on Computer Vision, PP 117-133, 2016

[28] Sandler M, Howard A, Zhu M, Zhmoginov A, Chen L, "MobileNetV2: Inverted residu-als and linear bottlenecks" In: Proc. the IEEE Conference on Computer Vision and Pattern Recognition" PP 4510-4520, 2018

[29] Chollet F, "Xception: Deep Learning with Depthwise Separable Convolutions", PP 1800-1807, 2017

[30] TensorFlow. tf.keras.layers.Rescaling. Mayo 2022. [en línea] https://www.tensorflow.org/api_docs/python/tf/keras/layers/Rescaling#args

[31] TensorFlow. tf.keras.preprocessing.image.ImageDataGenerator. Mayo 2022. [en línea] https://www.tensorflow.org/api_docs/python/tf/keras/preprocessing/image/ImageDataGenerator

[32] García-Sánchez E. Introducción a las redes neuronales de convolución. Aplicación a la visión por ordenador. Septiembre 2019. PP: 18-22.

[33] Loncomilla P. Deep learnig: Redes convolucionales. Junio 2016. PP: 20-25.

[34] Wu J. Introduction to Convolutional Neural Networks. Mayo 2017. PP 12-15, 22

[35] Ghosh A, Sufian A, Sultana F, Chakrabarti A, DE D. Fundamental Concepts of Convolutional Neural Network. Enero 2020. PP: 6, 12, 33,

[36] Gu J, Wang Z, Kuen J, Ma L, Shahroudy A, Shuai B, Liu T, Wang X, Wang L, Wang G, Cai J, Chen T. Recent Advances in Convolutional Neural Networks. Octubre 2017, PP: 1-4

[37] TesnorFlow. tf.nn.relu. Mayo 2022. [en línea] https://www.tensorflow.org/api_docs/python/tf/nn/relu

[38] Gholamalinezhad H, Khosravi H. Pooling Methods in Deep Neural Networks, a Review. Septiembre 2019. PP:11-12.

[39] Kingma DP, Ba J. Adam: A Method for Stochastic Optimization. 2014. [en línea] Adam: A Method for Stochastic Optimization

[40] Dong W. What is OpenCV's INTER_AREA Actually Doing?, Junio 2018. [en línea] https://medium.com/@wenrudong/what-is-opencvs-inter-area-actually-doing-282a626a09b3#:~:text=However%2C%20INTER_AREA%20is%20relatively%20mysterious,similar%20to%20the%20INTER_NEAREST%20method.

[41] OpenCV. Geometric Image Transformations. [en línea] https://docs.opencv.org/3.4/da/d54/group__imgproc__transform.html

[42] Kwon KA, Shipley R, Edirisinghe M, Ezra DG, Rose G, Best SM, Cameron RE. High-speed camera characterization of voluntary eye blinking kinematics. J. of the Royal Society Interface. Agosto 2013; PP: 1-6.

[43] Simonyan K., Zisserman A.: Very deep convolutional networks for large-scale image recognition. In: International Conference on Learning Representations (ICLR), San Diego, (2015).

[44] Elite. Overfitting in Machine Learning: What It Is and How to Prevent It. [en línea] https://elitedatascience.com/overfitting-in-machine-learning

[45] Ayachi R, Afif M, Said Y, Abdelali AB. Drivers Fatigue Detection Using EfficientDet In Advanced Driver Assistance Systems. 2021, PP: 738-742.

[46] Hashemi M, Mirrashid A, Shirazi AB. Driver Safety Development: Real-time driver drowsiness detection system based on convolutional neural networks. 2020. PP: 1-289.

[47] Jabbar R, Al-Khalifa K, Kharbeche M, Alhajyaseen R, Jafari M, Jiang S. Real-time Driver Drowsiness Detection for Android Application using Deep Neural Networks Techniques. 2018. PP: 400-407.

[48] Lei F, Liu C, Dai Q, Ling BW-K. Shallow Convolutional Neural Network For Image Classifcation. Diciembre 2019, PP: 1-3.

[49] Powers D. Evaluation: From precision, recall and F-measure to ROC, informedness, markedness & correlation. Enero 2011. PP 3.

[50] Bazarevsky V, Kartynnik Y, Vakunov A, Raveendran K, Grundmann M. BlazeFace: Sub-millisecond Neural Face Detection on Mobile GPUs. Google Research. Julio 2019. PP:1-4.

[51] Sanler M, Howard A. MobileNetV2: The Next Generation of On-Device Computer Vision Networks. Abril 2018. [en línea] https://ai.googleblog.com/2018/04/mobilenetv2-next-generation-of-on.html

[52] Liu W, Anguelov D, Erhan D, Szegedy C, Reed S, Fu C-Y, Berg A-C. SSD: Single Shot MultiBox Detector. Diciembre 2016. PP:1-17.

[53] Berrar D. Cross-validation. Enero 2018, PP: 1-9.

[54] Programador clic. Aprendizaje automático (12) ejemplos de validación cruzada. [en línea] https://programmerclick.com/article/8295250019/.

[55] Srivastava N, Hinton G, Krizhevsky A, Sutskever I, Salakhutdinov R. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. Junio 2014. PP: 6-7.

[56] Berzal F. Softmax. Febrero 2018. [en línea] https://elvex.ugr.es/decsai/deep-learning/slides/NN8%20Softmax.pdf

[57] He. K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition.2016. PP. 770-778.

[58] TensorFlow. [en línea] https://www.tensorflow.org/

[59] PyTorch.[en línea] https://pytorch.org/

[60] TensorFlow. Transferencia de aprendizaje y ajuste. [en línea] https://www.tensorflow.org/tutorials/images/transfer_learning

[61] Szegedy C, Liu W, Jia Y, Sermanet P, Reed S, Anguelov D, Erhan D, Vanhouckel V, Rabinovich A. Going Deeper with Convolutions. 2015. PP: 1-9.

[62] IMAGENET. Marzo 2021. [en línea] https://www.image-net.org/update-mar-11-2021.php

[63] Ruder S. An Overview Of Gradient Descent Optimization Algorithms. Junio 2017. PP: 2,7

[64] TensorFlow. tf.keras.losses.CategoricalCrossentropy. [en línea] https://www.tensorflow.org/api_docs/python/tf/keras/losses/CategoricalCrossentropy

[65] Wikipedia. Android. [en línea] https://es.wikipedia.org/wiki/Android

[66] Santander Universidades. Python: qué es y por qué deberías aprender a utilizarlo. Abril 2021. [en línea] https://www.becas-santander.com/es/blog/python-que-es.html

[67] OpenCV. [en línea] https://opencv.org/.

[68] TensorFlow. Deploy machine learning models on mobile and edge devices. [en línea] https://www.tensorflow.org/lite?hl=es-419

Capítulo 4: SOMN_IA

4.1 Introducción

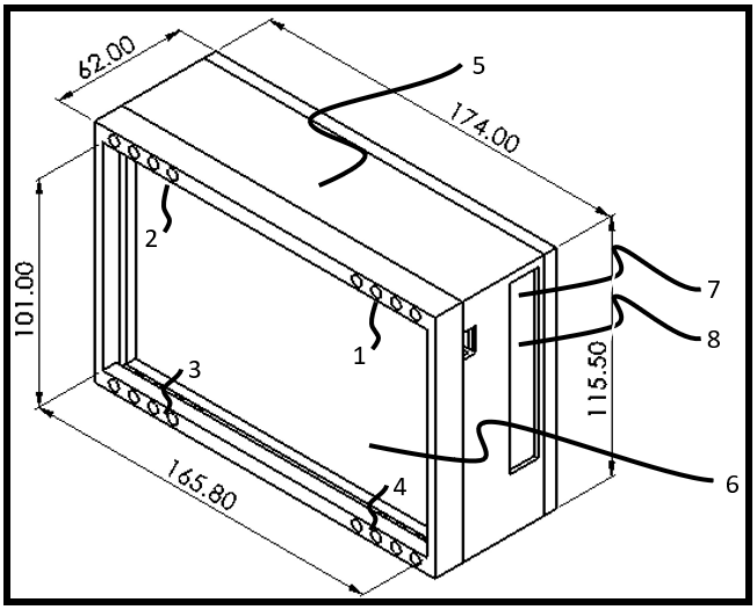
SOMN_IA es un prototipo inalámbrico el cual ha sido desarrollado específicamente para detectar la somnolencia y la distracción en los conductores empleando toda la investigación que en esta tesis se ha abordado, por ello, veremos a continuación como está conformado, su implementación en el vehículo, algunas pruebas de rendimiento y demás.

4.2 Construcción de hardware externo

La principal característica que tiene SOMN_IA como sistema, es que está diseñado para ser un dispositivo móvil que se pueda ubicar en cualquier vehículo, ya que su instalación es muy simple, a continuación enlistamos los componentes generales que lo conforman.

1. ***Primera alarma lumínica:*** En esta posición encontramos la primera alarma lumínica, la cual está conformada a su vez por 4 diodos emisores de luz ultrabrillantes los cuales se activarán siempre y cuando se trate de alertar al conductor que se está quedando dormido, su ubicación especifica es: Esquina superior derecha.
2. ***Segunda alarma lumínica:*** En esta posición encontramos la segunda alarma lumínica, que de igual forma que la primera a alarma está conformada a su vez por 4 diodos emisores de luz ultrabrillantes los cuales se activarán siempre y cuando se trate de alertar al conductor que se está quedando dormido, su ubicación especifica es: Esquina superior izquierda.
3. ***Tercera alarma lumínica:*** En esta posición encontramos la tercera alarma lumínica conformada por 4 diodos emisores de luz ultrabrillantes los cuales se activarán siempre y cuando se trate de alertar al conductor que se está quedando dormido o bien, alertará al conductor que no está prestando atención al camino, su ubicación especifica es: Esquina inferir izquierda
4. ***Cuarta alarma lumínica:*** En esta posición se encuentra la cuarta y última alarma lumínica, conformada por 4 diodos emisores de luz ultrabrillantes, que al igual que el tercer sensor, este se puede activar para alertar que el usuario se está quedando dormido, o en todo caso para alertarle al conductor que no está prestando atención al camino, la ubicación real de este sensor es: Esquina inferir derecha
5. ***Carcasa:*** Es el contenedor general del sistema, está hecho de plástico PLA (ácido poliláctico) reciclado, el método por el que se creo es por impresión 3D.
6. ***Display 7":*** Este componente es una pantalla digital LCD de 7" con una resolución de 1024×600 pixeles IPS, es utilizado como interfaz gráfica y es el que hace posible la interacción entre SOMN_IA y el usuario.

7. ***Terminal de corriente:*** SOMN_IA tiene dos formas de funcionamiento, local y móvil.
   - ***I. Local:*** En el caso de que se requiera implementar en forma local, se puede conectar un eliminador de corriente con una entrada de 120v/220v de CA y una salida de 5v a 4A mínimo de CC. En la Figura 4.5 podemos ver el diagrama de conexión, el tipo de conector utilizado es un Jack macho de 5.5×2.1mm.
   - ***II. Móvil:*** En el caso de que se requiera implementar de forma móvil, se puede conectar un convertidor DC-DC de entrada 12v-24v de CC y de salida de 5v a 4A mínimo. En la Figura 4.6 podemos ver el diagrama de conexión, el tipo de conexión utilizado es un Jack macho de 5.5×2.1mm.
8. ***Periférico de cámara WEB:*** SOMN_IA tiene la versatilidad de poder trabajar con dos cámaras (no al mismo tiempo), por lo tanto, si se quiere trabajar directamente con imágenes de espectro visible, el sistema tiene un puerto donde se puede conectar de una manera sencilla, en las Figuras 4.5 y 4.7 podemos ver un diagrama de conexión



Vista isométrica de SOMN_IA

Figura 4. 1: Vista isométrica del dispositivo SOMN_IA

A continuación veremos los elementos que conforman a SOMN_IA en la parte trasera:

1. ***Antena GSM:*** En esta ranura va conectada la antena GSM en el periférico SMA designado, esta parte debe ir libre, debido que es la encargada de recibir señal de la compañía telefónica, para que posteriormente, si sea el caso, SOMNI_IA pueda mandar mensajes con la información pre-registrada, así como la ubicación en tiempo real del conductor.

2. ***Disipador:*** Es un disipador de aluminio conectado en la tarjeta madre el cual hace todo el proceso de análisis y detección de somnolencia y distracción.

   NOTA: Esta parte debe estar bien ventilada y fuera del alcance del conductor debido a que se llega a calentar a temperaturas mayores de 50°C.
3. ***Conector de cámara IRS por ISP:*** Como se comentó anteriormente, SOMN_IA es capaz de trabajar con dos tipos de cámaras digitales, ya sea la cámara web normal o la cámara que toma imágenes infrarrojas para poder monitorear en tiempo real en condiciones con poca o nula iluminación.
4. ***Modulo GSM:*** Este módulo interno en el sistema es el encargado de conectarse a la red celular de la compañía telefónica designada (en este caso Telcel) y con esto poder enviar mensajes codificados previamente.
5. ***Módulo de control:*** Este módulo es el segundo más importante, es el encargado de monitorear los resultados dados por la tarjeta principal y poder así activar o desactivar las alarmas sonoras y lumínicas, además que esta está en constante comunicación entre el módulo de GSM y GPS. En este módulo se han codificado los mensajes prediseñados, así como es el encargado de decodificar la dirección GPS, para que posteriormente, mande el mensaje al número precargado, con ayuda del módulo GSM.
6. ***Tarjeta madre:*** Este es el módulo más importante, debido que en este apartado se está implementado el software general de SOMN_IA, aquí se realiza todo el proceso de monitoreo del conductor, la aplicación de los algoritmos de inteligencia, además que se realizan los comandos de alarmas, se manda la información a la interfaz gráfica y es la fuente de distribución de energía general.
7. ***Módulo GPS:*** Es un pequeño módulo encargado de recibir los datos en tiempo real de la ubicación del sistema con ayuda de su antena, posteriormente, se comunica con el módulo de control para poder proporcionar la información necesaria la cual se decodificará y se mandará con ayuda del módulo GSM al número precargado escogido por el usuario.
8. ***Antena GPS:*** Es la receptora de las coordenadas de la ubicación en tiempo real.
9. ***Tornillería:*** Estos son los encargados de unir a la tapa con carcasa.
10. **Tapa del sistema.**

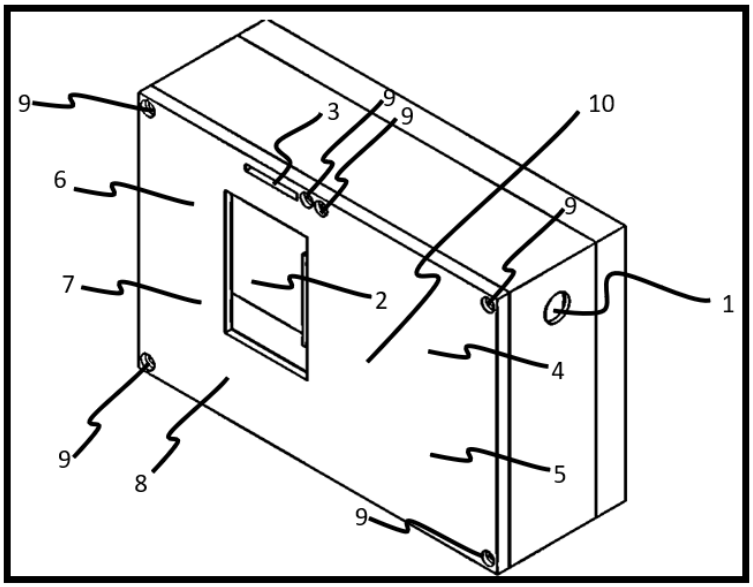


Figura 4. 2: Vista isométrica-trasera de SOMN_IA

Finalmente veremos la parte inferior del sistema

1. ***Antena GPS:*** Es la receptora de las coordenadas de la ubicación en tiempo real.

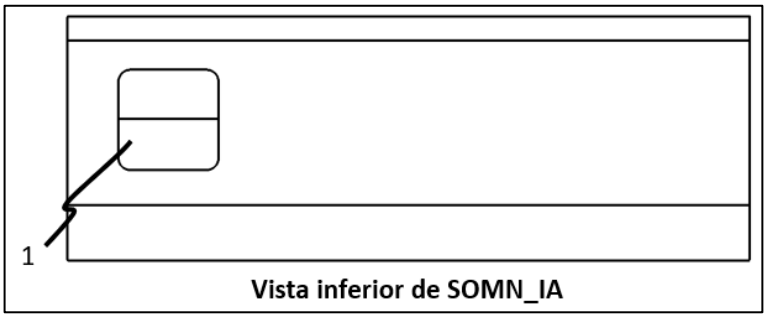


Figura 4. 3: Vista inferior de SOMN_IA

4.3 Composición interna de SOMN_IA

1. ***Módulo de control:*** Este módulo es el segundo más importante, es el encargado de monitorear los resultados dados por la tarjeta principal y poder así activar o desactivar las alarmas sonoras y lumínicas, además que está en constante comunicación entre el módulo de GSM y GPS. En este módulo se han codificado los mensajes prediseñados, así como es el encargado de decodificar la dirección GPS, para que posteriormente, mande el mensaje al número precargado, con ayuda del módulo GSM.
2. ***Tarjeta madre:*** Este es el módulo el más importante, debido que en este apartado se está implementado el software general de SOMN_IA, aquí se realiza todo el proceso de monitoreo del conductor, la aplicación de los algoritmos de

inteligencia, además que se realizan los comandos de alarmas y finalmente se manda la información a la interfaz gráfica.

3. ***Módulo GPS:*** Es un pequeño módulo encargado de recibir los datos en tiempo real de la ubicación del sistema con ayuda de su antena, posteriormente, se comunica con el módulo de control para poder proporcionar la información necesaria la cual se decodificará y se mandará con ayuda del módulo GSM al número precargado escogido por el usuario.
4. ***Módulo de energía:*** Este módulo es el encargado de distribuir todas las corrientes a los diferentes módulos que conforman a SOMN_IA; la característica principal de esta parte del sistema es que obtiene los 12v de cc que salen del encendedor de auto y lo reduce a 5v-5A cc de forma continua, lo que asegura el buen funcionamiento del sistema, ya que para que SOMN_IA trabaje sin limitaciones, se debe mantener una alimentación constante de 20w.

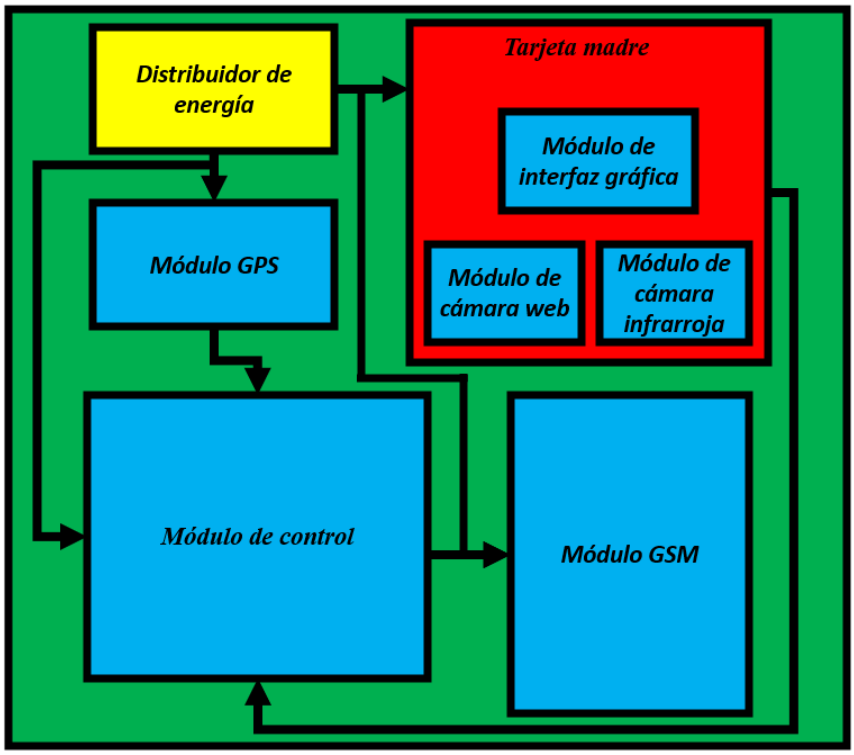


Figura 4. 4: Diagrama a bloques de la intercomunicación de los módulos internos de SOMN_IA

4.4 Diagrama de conexión

1. Conexión para trabajo local.
   a) Utilizando cámara WEB.

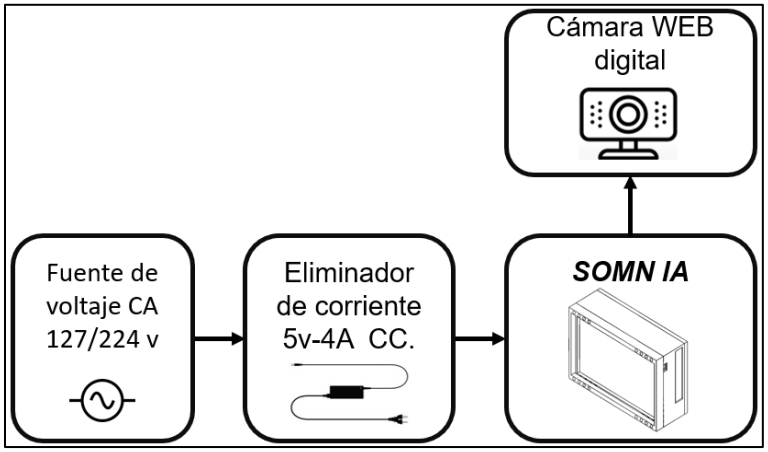


Figura 4. 5: Diagrama de conexión local utilizando la cámara web

   b) Utilizando cámara IR.

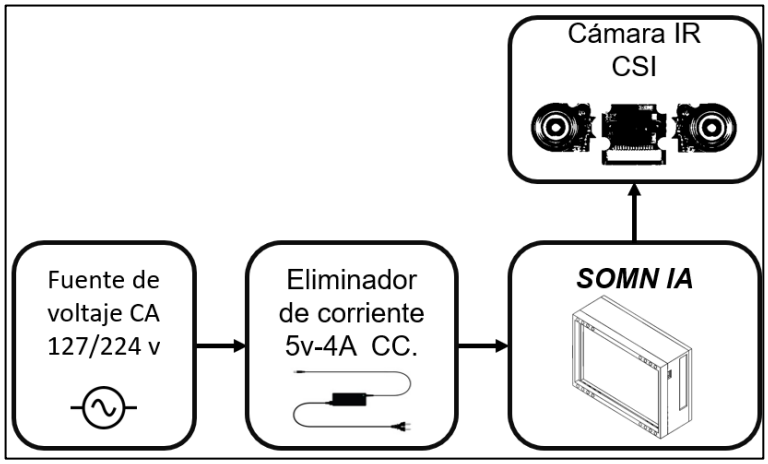


Figura 4. 6: Diagrama de conexión local utilizando la cámara IR

2. Conexión para trabajo móvil.
   a) Utilizando cámara WEB.

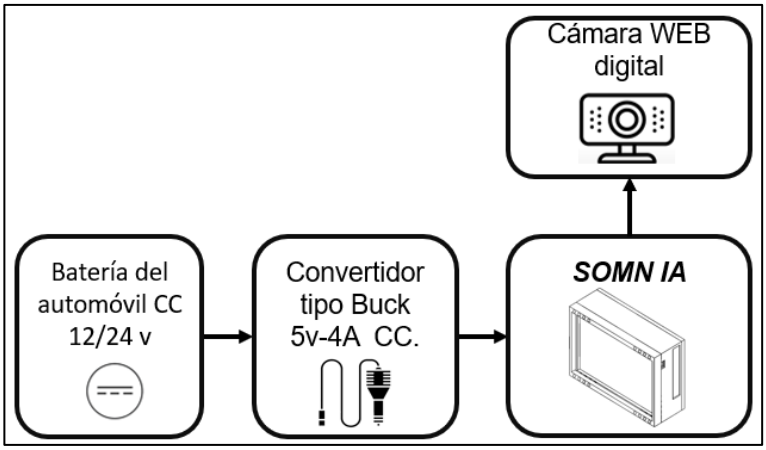


Figura 4. 7: Diagrama de conexión móvil utilizando la cámara WEB

   b) Utilizando cámara IR.

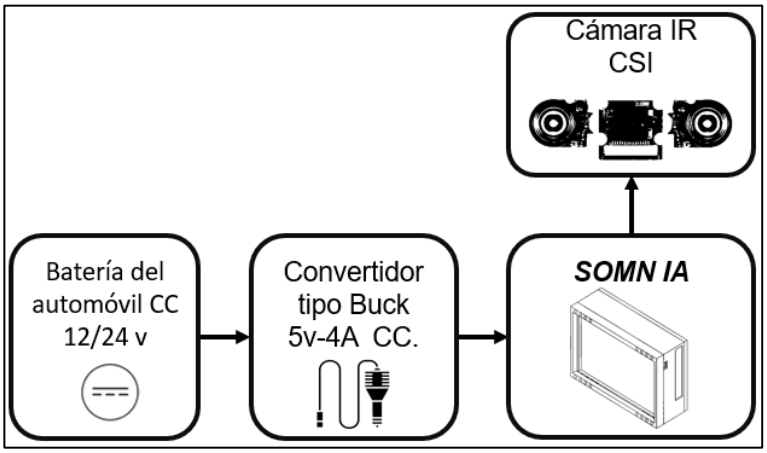


Figura 4. 8: Diagrama de conexión móvil utilizando la cámara IR

4.5 Prototipo en físico

1) Vista isométrica.



Figura 4. 9: Vista isométrica en físico del prototipo de SOMN_IA

2) Vista Frontal mostrando las dos cámaras utilizadas.



Figura 4. 10: Vista Frontal del prototipo de SOMN_IA mostrando las dos cámaras seleccionadas

3) Vista isométrica trasera.



Figura 4. 11: Vista isométrica trasera del prototipo de SOMN_IA

4) Vista inferior.



Figura 4. 12: Vista inferior del prototipo de SOMN_IA

4.6 Ejemplos de funcionamiento

1) Detección de somnolencia y distracción en conductores utilizando la cámara WEB.

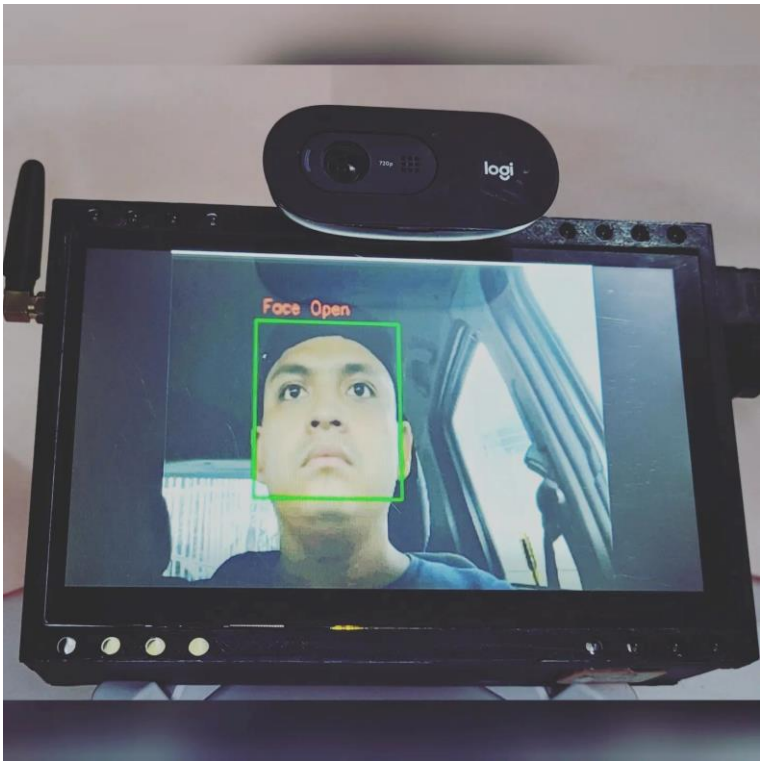


Figura 4. 13: Detección de somnolencia y distracción en conductores utilizando la cámara WEB.

2) Detección de somnolencia y distracción del conductor utilizando la cámara IR (Versión con iluminación).

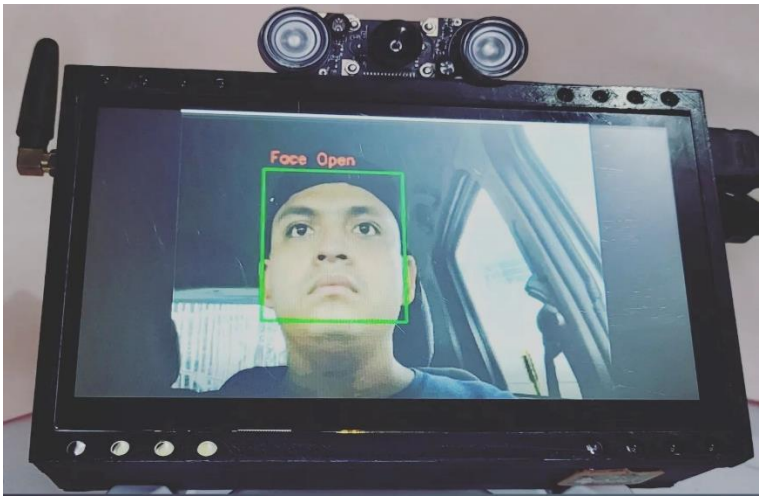


Figura 4. 14: Detección de somnolencia y distracción del conductor utilizando la cámara IR (Versión con iluminación)

3) Detección de somnolencia y distracción del conductor utilizando la cámara IR (Versión con poca o nula iluminación).
4) Detección de distracción y activación de las alarmas pertinentes.

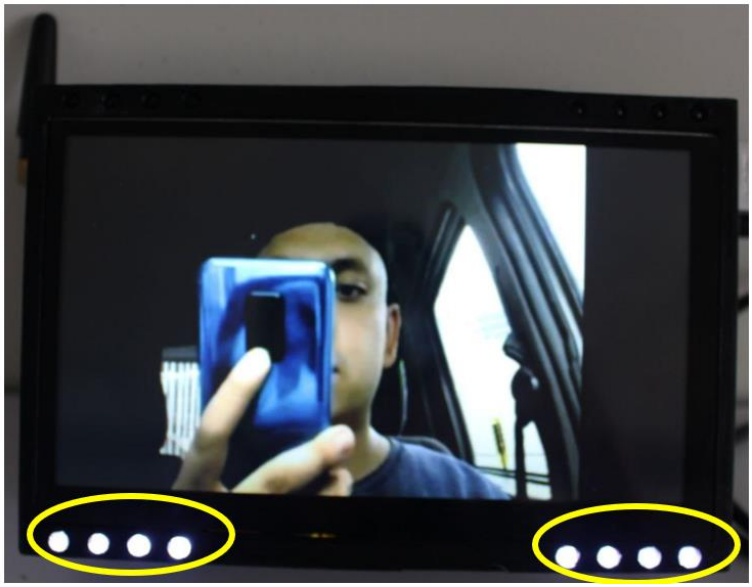


Figura 4. 15: Detección de distracción y activación de las alarmas pertinentes.

5) Detección de somnolencia y activación de las alarmas pertinentes.

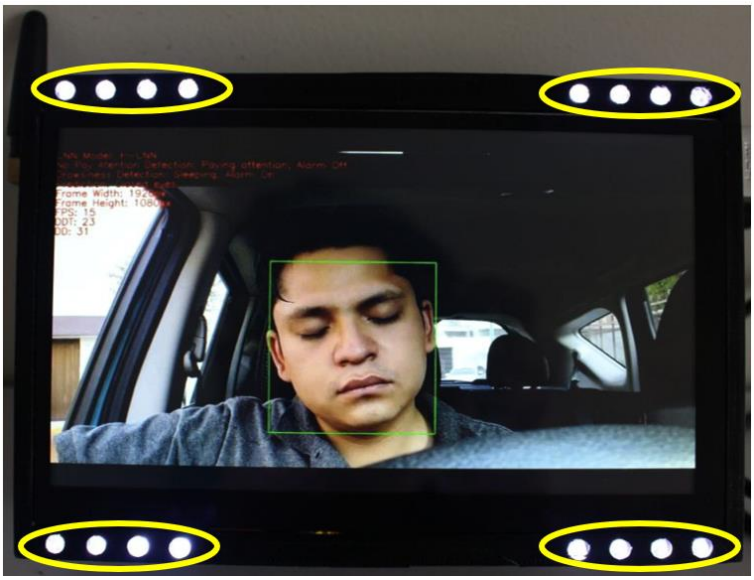


Figura 4. 16: Detección de somnolencia y activación de las alarmas pertinentes.

6) Ejemplo del monitoreo en tiempo real.

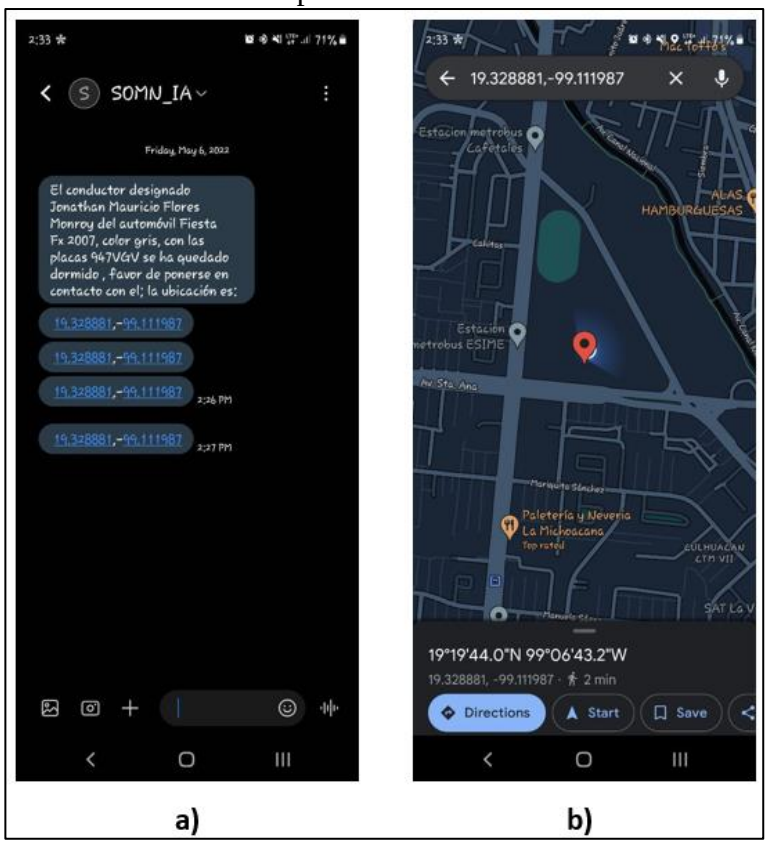


Figura 4. 17: Ejemplo del monitoreo en tiempo real.

7) Ejemplo del bloqueo por GUI:

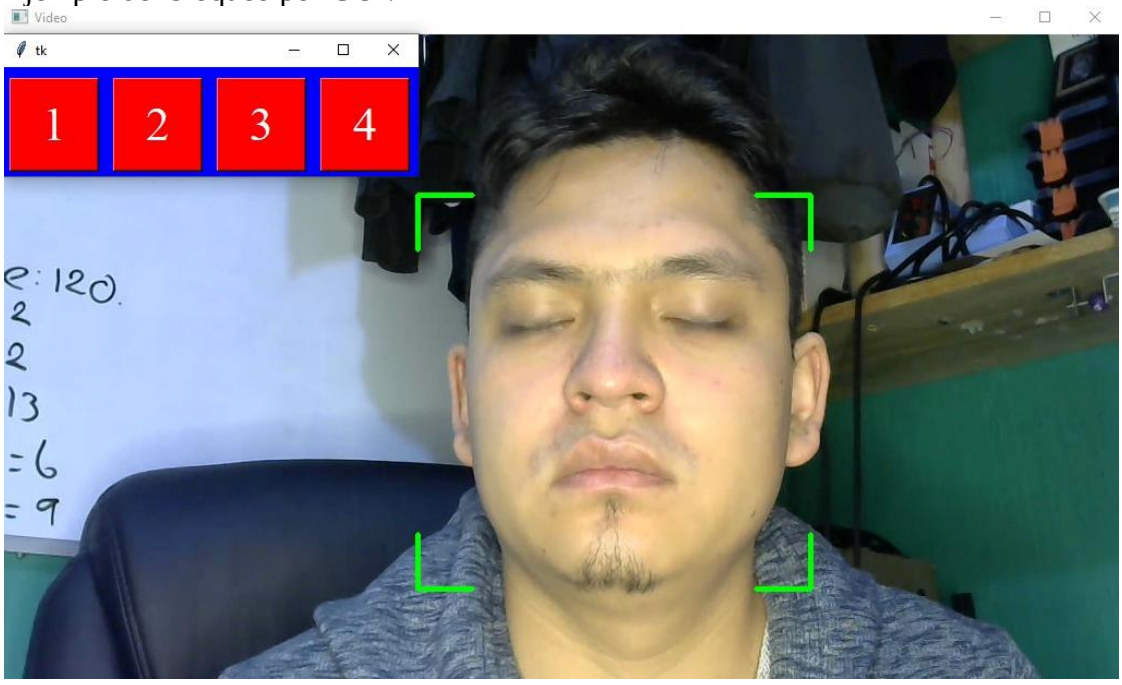


Figura 4. 18:Bloqueo de SOMN_IA e GUI para su desbloqueo

4.7 Implementación del software en los diferentes SO.

Como se comentó a lo largo de toda esta investigación, SOMN_IA esta pensado para poder implementarse en diferentes dispositivos, por lo tanto, se ha dejado un Manual de usuario en el siguiente enlace https://1drv.ms/w/s!AhFUixwJjva6khLEr9oVSH6rOTNe?e=GlsPuu, en el cual, están todos los pasos detallados para su instalación correcta.

4.8 Conclusiones del capítulo.

SOMN_IA es la culminación de toda la investigación que se hizo en esta tesis, por ellos, su diseño, su funcionamiento y su implementación está especialmente diseñada para poder ser funcional, en muchas condiciones de uso, ya que puede usarse tanto en situaciones sumamente controladas, al grado de monitorear al usuario dentro de sistemas controlados, donde las condiciones y la iluminación no tenga variaciones algunas y sean de la mejor calidad, hasta poder funcionar sin ningún problema en escenarios donde el conductor tenga poca o nula intensidad lumínicas se esté utilizando elementos adicionales a un rostro normal; además al ser un dispositivo inalámbrico, no tendrá problema alguno de poder situar en cualquier automóvil, además que, al ser tan simple su instalación, no se necesario un tercero para que pueda funcionar.

Las alarmas que integran a SOMN_IA están pensadas y distribuidas, a tal grado que es casi imposible no prevenir a los usuarios que están conduciendo de una manera incorrecta; pero en tal caso de que las elementos preventivos que están integrado, no sean suficientes para poder despertar o de advertir al piloto, el sistema de bloqueo y rastreo que se tiene acoplado en SOMN_IA como último recursos asegura su sistema se seguridad y con eso podemos concluir que si bien, acabar con los accidentes causados por estas causas no es posible en estos momentos, estamos dando un paso muy grande para por lo menos reducirlos y con ello, disminuir la mortalidad y los gasto generados anualmente por este tipo de siniestros.

Finalmente podemos concluir que SOMN_IA si es una alternativa económica, general y funcionar para combatir los accidentes terrestres causados por la somnolencia y la distracción.

Capítulo 5: CONCLUSIONES GENERALES Y TRABAJOS FUTUROS

5.1 Conclusiones generales.

Los accidentes vehiculares son una enorme problemática para la calidad de vida de muchas personas, debido a que gracias a estos, muchos individuos han sufrido secuelas que los han marcado de por vida o por el otro lado, hay quienes ni siquiera pudieron sobrevivir a este tipo de peripecias. En México particularmente, es un enorme problema, ya que se ocupa el séptimo lugar a nivel mundial por incidentes relacionados con vehículos terrestres, lo peor de todo, es que más del 90% de estos accidentes se pueden evitar, debido a que estos suceden por errores o negligencias de los conductores.

Del 90% de los accidentes comentados anteriormente, más del 60% son causados por personas que se han quedado dormidas tras el volante o bien, se han distraído por algún motivo mientras se va conduciendo.

Actualmente, se ha trabajado mucho para poder reducir los accidentes terrestres, en donde el área puramente tocológica no ha sido la única participe, ya que muchas ramas de ingeniera, investigación o bien sociales sean pronunciado y han tomado medidas sobre este tipo de problemáticas, y esto va desde la creación de autopistas cada vez mejores, tomando en cuenta puntos esenciales en su construcción, hasta la creación de movimientos sociales de concientización al momento de conducir, así como en el ámbito específicamente automotriz, en donde los mejoramientos de los sistema internos del automóvil, cada vez son mejores y con ellos se reduce el estrés y cansancio, así como también, ayudan al conductor a no tener distracciones mientras se conduce, pero todo esto a pesar del gran esfuerzo que se hace, no es suficiente para reducir los accidentes causados por las negligencias de los conductores; por ellos y con ayuda de todos las bases teóricas, hemos creado a SOMN_IA, el cual es un dispositivo tecnológico enfocado puramente en el monitoreo permanente y la detección de los dos estados que más accidentes provocan anualmente, esto son la somnolencia y la distracción en los conductores.

SOMN_IA es un dispositivo que como se vio a lo largo de toda esta investigación, es capaz de detectar y prevenir la somnolencia y la distracción en los conductores, utilizando protocolos de monitoreo tan certeros que cumple con la misión de cuidar al usuario utilizando a su vez, nuevas tecnologías que aseguran el buen monitoreo en tiempo y condiciones reales .

Las propiedades que emplea SOMN_IA como dispositivo móvil, así como de software, ayuda seguir innovando y mejorando cada vez más los sistemas de monitoreo, ya que actualmente, es uno de los pocos que contemplan estos dos estados en una sola terminal.

Los sistemas de alarmas que integran a SOMN_IA, si bien, no son nada complejos, son sumamente funcionales, ya que están tan bien distribuidos y se tiene un patrón de encendido pensado en alertar cuando sea necesario y de despertar si llegase el

caso extremo, todo esto en tiempo real y en condiciones reales, hacen que SOMN_IA sea un dispositivo altamente eficaz.

Una de las propiedades que más diferencian SOMN_IA, además de la implementación de las clasificaciones y prevenciones de los dos estados que causan más daños a los conductores, es su subsistema de rastreo en tiempo real, ya que, con esto aseguramos aún más la integridad física, psicológica y monetaria del usuario así como también de terceros,

Finalmente, SOMN_IA, es un dispositivo pensado en la universalidad y en la generalidad, por lo tanto, tiene una propiedad insignia, el cual es, que su software es capaz de poder correr en diversas terminales, tanto con restricciones de memoria y de poder computacional, así como también en módulos que no tengan ningún inconveniente con estos elementos, esto ayuda de igual manera a seguir investigando, innovando y mejorando cada vez más, tanto el sistema mismos, como su código general, además que, hemos dado la pauta para la creación de nuevos sistemas, los cuales tengan como base, resguardar la integridad física de los cotutor.

5.2 Trabajos futuros.

A la fecha de este término de trabajo de investigación, se ha finalizado de la mejor manera la detección de somnolencia y distracción, donde además de ser implementado en un dispositivo móvil específico para este trabajo, se ha creado un software capaz de correrse en diversos SOs, por lo tanto hemos dado el primer paso para un sistema que no sólo detecte estos estados, sino que también tenga las condiciones para su predicción.

Actualmente se está trabajando en la implementación de sistemas predictivos utilizando redes neuronales recurrentes y así mismo, no sólo estamos prediciendo los estados bases descritos en toda esta tesis, sino que también se ha desarrollado los principios para la diferenciación de estados emocionales al momento de conducir y eso sumado a lo ya desarrollado, tiene como finalidad poder no solo evitar accidentes, sino que también, generar un sistema tan completo que pueda a su vez mejorar las condiciones de manejo del usuario y ser un sistema tanto de seguridad al momento de conducir ser por ende, un respaldo el cual pueda detectar otros tipos de amenazas, como enfermedades o en todo caso evitar el acceso a la delincuencia en medio de la conducción.

ANEXO